

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Российский экономический университет им. Г.В. Плеханова»
Московский приборостроительный техникум

Курсовой проект

ПМ 01 Разработка модулей программного обеспечения для
компьютерных систем

МДК 01.01 Разработка программных модулей

Специальность 09.02.07 «Информационные системы и
программирование»

Квалификация: Программист

Тема: «Разработка мобильного приложения и веб-панели управления для
интернет-магазина продуктов «Нектар»»

Пояснительная записка

Листов: 46

Руководитель

_____ / Е.М. Парамонова
«____» _____ 2026 год

Исполнитель

_____ / И.Н. Веников
«____» _____ 2026 год

«УТВЕРЖДАЮ»

Заместитель директора по учебной работе

_____ Д.А. Клопов

« ____ » _____ 2026 г.

ЗАДАНИЕ

на выполнение курсового проекта (курсовой работы)

Веникову Ивану Николаевичу

(фамилия, имя, отчество студента — полностью)

студенту группы П-7-23 специальности 09.02.07 «Информационные системы и программирование»
по МДК 01.01 «Разработка программных модулей»

1. Исходные данные к проекту (работе):
 - 1.1. Тема: «Разработка программного комплекса для интернет-магазина продуктов «Нектар»».
 - 1.2. Состав курсового проекта:
 - 1.2.1. Задание КП
 - 1.2.2. Пояснительная записка
 - 1.2.3. Программа (исходные данные) на электронном носителе
 - 1.2.4. Презентация и инсталляционный пакет программы на электронном носителе
 - 1.3. Содержание пояснительной записки:

ВВЕДЕНИЕ

 1. **ОБЩАЯ ЧАСТЬ**
 - 1.1. Цель разработки
 - 1.2. Средства разработки
 2. **СПЕЦИАЛЬНАЯ ЧАСТЬ**
 - 2.1. Постановка задачи
 - 2.1.1. Входные данные
 - 2.1.2. Выходные данные
 - 2.1.3. Подробные требования к проекту
 - 2.2. Внешняя спецификация
 - 2.2.1. Описание задачи
 - 2.2.2. Входные и выходные данные
 - 2.2.3. Методы
 - 2.2.4. Тесты
 - 2.2.5. Контроль целостности данных
 - 2.3. Проектирование
 - 2.3.1. Схема архитектуры приложения
 - 2.3.2. Логическая схема данных
 - 2.3.3. Физическая схема данных
 - 2.3.4. Диаграмма классов
 - 2.3.5. Схема тестирования
 - 2.3.6. Структурная схема
 - 2.3.7. Функциональная схема
 - 2.3.8. Схема пользовательского интерфейса
 - 2.4. Результат работы программы
 3. **ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ**
 - 3.1. Инструментальные средства
 - 3.2. Отладка программы
 - 3.3. Защитное программирование
 - 3.4. Характеристики программы

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗУЕМЫХ МАТЕРИАЛОВ

ПРИЛОЖЕНИЕ А. Техническое задание

ПРИЛОЖЕНИЕ Б. Сценарий и результаты тестовых испытаний

ПРИЛОЖЕНИЕ В. Руководство пользователя
 ПРИЛОЖЕНИЕ Г. Текст программы
 ПРИЛОЖЕНИЕ Д. Скрипт базы данных
 ПРИЛОЖЕНИЕ Е. Входные и выходные данные
 ПРИЛОЖЕНИЕ Ж. Диаграмма классов

2. Содержание задания по проекту (работе) — перечень вопросов, подлежащих разработке

	Разрабатываемый вопрос	Объем от всего задания, %	Срок выполнения
А	Описательная часть проекта (введение, общее описание и т. д.)	5	17.01.26
1.	Введение	-	17.01.26
2.	Цель разработки	-	17.01.26
3.	Средства разработки	-	17.01.26
Б	Анализ задачи и её постановка	15	31.01.26
1.	Определение требований к программе	-	24.01.26
2.	Спецификация программы (описание задачи, описание входных и выходных данных, методы)	-	31.01.26
3.	Тесты, контроль целостности данных	-	31.01.26
В	Проектирование и реализация	55	28.03.26
1.	Схемы проекта (схема архитектуры, логическая схема данных, физическая схема данных, функциональная и структурная схемы, диаграмма классов, схема тестирования, схема пользовательского интерфейса)	-	28.02.26
2.	Реализация в инструментальной среде	-	28.03.26
Г	Технологическая часть проекта	5	10.04.26
1.	Инструментальные средства разработки	-	04.04.26
2.	Отладка программа	-	04.04.26
3.	Защитное программирование	-	10.04.26
4.	Характеристика программы	-	10.04.26
Д	Программная документация	10	18.04.26
1.	Приложение А. Техническое задание	-	11.04.26
2.	Приложение Б. Сценарий и результаты тестовых испытаний	-	11.04.26
3.	Приложение В. Руководство пользователя	-	11.04.26
4.	Приложение Г. Текст программы	-	18.04.26
5.	Приложение Д. Скрипт базы данных	-	18.04.26
6.	Приложение Е. Входные и выходные данные	-	18.04.26
7.	Приложение Ж. Диаграмма классов	-	18.04.26
Е	Экспериментальная часть проекта	10	25.04.26
1.	Программа на машинном носителе. Информация на носителе разбита на разделы: эксплуатационный пакет, тексты программы, документация.	-	25.04.26

Руководитель курсового проекта (работы) Парамонова Елизавета Михайловна, преподаватель
 «02» мая 2026 года _____ / Е.М. Парамонова /

Дата выдачи курсового задания «17» января 2026 года
 Срок сдачи законченного проекта (работы) «02» мая 2026 года

Задание принял к исполнению
 «17» января 2026 года _____ / И.Н. Веников /

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1. ОБЩАЯ ЧАСТЬ.....	6
1.1 Цель разработки.....	6
1.2 Средства разработки.....	6
2. СПЕЦИАЛЬНАЯ ЧАСТЬ.....	8
2.1 Постановка задачи.....	8
2.1.1 Входные данные предметной области.....	8
2.1.2 Выходные данные предметной области.....	8
2.1.3 Требования к проекту.....	9
2.2 Внешняя спецификация.....	11
2.2.1 Описание задачи.....	11
2.2.2 Входные и выходные данные.....	14
2.2.3 Методы.....	14
2.2.4 Тесты.....	19
2.2.5 Контроль целостности данных.....	19
2.3 Проектирование.....	22
2.3.1 Схема архитектуры приложения.....	22
2.3.2 Логическая схема данных.....	22
2.3.3 Физическая схема данных.....	24
2.3.4 Диаграмма классов.....	24
2.3.5 Схема тестирования.....	24

2.3.6	Структурная схема.....	25
2.3.7	Функциональная схема.....	27
2.3.8	Схема пользовательского интерфейса.....	27
2.4	Результат работы программы.....	32
3.	ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ.....	35
3.1	Инструментальные средства.....	35
3.2	Отладка программы.....	36
3.3	Защитное программирование.....	37
3.4	Характеристики программы.....	40
	ЗАКЛЮЧЕНИЕ.....	42
	СПИСОК ИСПОЛЬЗОВАННЫХ МАТЕРИАЛОВ.....	45
	ПРИЛОЖЕНИЕ А. Техническое задание.	
	ПРИЛОЖЕНИЕ Б. Сценарий и результаты тестовых испытаний.	
	ПРИЛОЖЕНИЕ В. Руководство пользователя.	
	ПРИЛОЖЕНИЕ Г. Текст программы.	
	ПРИЛОЖЕНИЕ Д. Скрипт базы данных.	
	ПРИЛОЖЕНИЕ Е. Входные и выходные данные.	
	ПРИЛОЖЕНИЕ Ж. Диаграмма классов.	

ВВЕДЕНИЕ

Сегодня сложно представить торговлю без мобильных технологий: люди всё чаще делают покупки прямо со смартфонов. Для бизнеса это значит, что наличие своего приложения – это уже не просто приятный бонус, а реальная необходимость.

Особенно это касается продажи продуктов питания с доставкой на дом. В этой нише покупателям важны скорость и удобство: нужно быстро найти нужный товар, проверить его наличие и свежесть и оформить заказ, не тратя время на поход в магазин. Мобильные версии сайтов часто неудобно отображаются на экране телефона, поэтому отдельное приложение справляется с этой задачей гораздо лучше.

Цель данной курсовой работы – разработать программное решение для интернет-магазина продуктов «Нектар». В самом приложении покупатели смогут просматривать каталог, добавлять товары в корзину, оформлять заказы и следить за их выполнением, а курьеры – брать заказы в работу и подтверждать доставку.

Кроме того, проект включает создание веб-панели для администраторов и менеджеров. Это нужно для того, чтобы сотрудники магазина могли в одном месте управлять ассортиментом и остатками товаров, справочниками и промокодами, обрабатывать заказы, назначать курьеров и управлять пользователями, не тратя время на лишнюю рутину.

В конечном итоге должна получиться полноценная система, которая свяжет покупателей, курьеров и сотрудников магазина. Внедрение такого решения поможет «Нектар» повысить лояльность аудитории за счет современного сервиса, а также навести порядок во внутренних процессах учета остатков и продаж.

1. ОБЩАЯ ЧАСТЬ

1.1. Цель разработки

Основной целью разработки является автоматизация процессов розничной торговли продуктами питания, учета товарных позиций и информационного сопровождения клиентов интернет-магазина «Нектар». Внедрение разрабатываемого программного обеспечения должно обеспечить удобное взаимодействие между магазином, покупателем и курьером посредством мобильного устройства, а также предоставить сотрудникам магазина инструменты для эффективного управления каталогом, остатками, заказами и пользователями через веб-интерфейс.

1.2. Средства разработки

Для реализации программного комплекса использовался современный стек технологий, включающий клиентскую часть и облачную платформу Google Firebase, заменяющую собственную серверную часть и базу данных. В качестве программных средств разработки использовалось ПО, представленное в таблице 1.

Таблица 1 – Программные средства разработки

№	Тип средства	Название средства	Назначение
1	Операционная система	Windows / macOS / Linux	Организация взаимодействия программ и пользователя, среда разработки
2	Интегрированная среда разработки (IDE)	Visual Studio Code / Android Studio	Написание и отладка исходного кода мобильного приложения и веб-панели
3	Кроссплатформенный фреймворк	Flutter 3 (Dart 3.9)	Создание пользовательского интерфейса мобильного приложения
4	Платформа Firebase	Authentication, Cloud Firestore, Hosting	Авторизация пользователей, хранение данных, размещение веб-панели
5	Язык веб-панели	JavaScript (ES-модули), Firebase Web SDK 10	Разработка административной веб-панели без сборщика проекта

6	Среда выполнения	Node.js 20+, Firebase CLI, Java	Запуск локальных эмуляторов Firebase и автоматических тестов
7	Система контроля версий	Git / GitHub	Хранение исходного кода проекта

В качестве средств вычислительной техники при разработке ПО использовался персональный компьютер разработчика. Характеристики представлены в таблице 2.

Таблица 2 – Характеристики вычислительной техники

№	Тип средства	Название средства
Персональный компьютер разработчика		
1	Процессор:	Тактовая частота не ниже 2.0 ГГц, 64-разрядный
2	Оперативная память:	Не менее 8 ГБ
3	Накопитель:	SSD, свободно не менее 10 ГБ
4	Монитор:	Разрешение не менее 1920x1080
5	Мобильное устройство:	Эмулятор Android или смартфон с Android 8.0+ / iOS 15.0+

2. СПЕЦИАЛЬНАЯ ЧАСТЬ

2.1. Постановка задачи

Разработать информационную систему, обеспечивающую автоматизацию процессов розничной торговли продуктами питания с доставкой. Разрабатываемое программное решение должно состоять из мобильного приложения для покупателей и курьеров и административной веб-панели для сотрудников магазина.

2.1.1. Входные данные предметной области

Входными данными информационной системы является информация о пользователях (логин, имя, email, пароль, номер телефона, аватар профиля, адреса доставки, маски банковских карт), ролях пользователей (администратор, менеджер, курьер, покупатель), товарах (название, цена, фасовка, остаток на складе, категория, производитель, поставщик, изображение, пищевая ценность, состав, описание, срок годности), категориях, производителях и поставщиках, промокодах (код, размер скидки, срок действия), заказах (адрес, способ оплаты, интервал доставки, промокод, состав), действиях курьера (взятие, возврат, начало и подтверждение доставки), а также поисковых запросах, параметрах фильтрации и сортировки, настройках языка и уведомлений.

2.1.2. Выходные данные предметной области

Выходными данными информационной системы являются данные о пользователях, ролях, категориях, производителях, поставщиках, промокодах, каталоге товаров и остатках, избранных товарах, корзине покупателя с расчетом суммы, скидки и доставки, заказах и их составе, статусах заказов, уведомлениях, пулах заказов и истории доставок курьера, а также показатели дашборда (количество покупателей, активные заказы, товары с низким остатком, выручка, продажи по месяцам, ТОП-5 товаров).

2.1.3. Требования к проекту

Использовать облачную документоориентированную базу данных Cloud Firestore для хранения и обработки информации; права доступа к данным описать правилами безопасности базы данных.

Спроектировать и разработать программное решение, которое позволит покупателям ознакомиться с доступным ассортиментом магазина, при необходимости воспользовавшись поиском товара по названию, а также сортировкой по необходимому параметру (по умолчанию, сначала свежие, по возрастанию и убыванию цены) и фильтрацией товаров по производителям и диапазону цены.

Интерфейс мобильного приложения должен быть простым и понятным рядовому пользователю, выполненным в светлой цветовой теме с зелеными акцентами, с поддержкой русского и английского языков. Также должен быть предусмотрен простой механизм перехода по разделам приложения через нижнее меню.

Для получения доступа к возможностям приложения в соответствии с ролями пользователь должен пройти процедуру авторизации. Покупатели могут пройти процедуру регистрации, указав логин, имя, телефон, email и пароль; на указанную почту приходит письмо для восстановления пароля. Учетные записи курьеров, менеджеров и администраторов создаются назначением роли администратором.

Программное решение «Нектар» должно иметь следующие функциональные возможности:

- Возможность регистрации новой учетной записи покупателя;
- Возможность авторизации по логину или email и паролю, а также через аккаунт Google;
- Возможность восстановления утерянного пароля через электронную почту;

- Возможность выхода из учетной записи и удаления аккаунта;
- Отображение главного экрана с акционным баннером, популярными товарами и товарами по категориям;
- Отображение каталога товаров по категориям;
- Возможность поиска товаров по названию;
- Возможность фильтрации товаров по производителям и диапазону цен;
- Возможность сортировки списка товаров;
- Отображение карточки товара с описанием, характеристиками и пищевой ценностью;
- Возможность добавления товаров в список «Избранное» и их удаления;
- Возможность добавления товаров в корзину с контролем остатков;
- Возможность изменения количества товаров в корзине и их удаления;
- Возможность ввода и проверки промокода на скидку;
- Автоматический расчет итоговой стоимости заказа с учетом скидки и доставки;
- Возможность оформления заказа с выбором сохраненного адреса, интервала доставки и способа оплаты;
- Обеспечение сохранения заказа в базе данных с одновременным списанием остатков;
- Отображение истории заказов, их состава и хода выполнения;
- Возможность подтверждения получения заказа;
- Отображение уведомлений об изменении статуса заказа;
- Возможность редактирования профиля, аватара, адресов доставки и банковских карт;
- Возможность смены пароля;

- Возможность выбора языка интерфейса;
- Возможность просмотра пула свободных заказов курьером, взятия заказа в работу и возврата его в пул;
- Возможность начала и подтверждения доставки курьером;
- Возможность авторизации сотрудника в веб-панели;
- Отображение дашборда с показателями и диаграммами;
- Возможность создания, изменения, скрытия и восстановления карточек товаров и управления остатками;
- Возможность управления категориями, производителями и поставщиками;
- Возможность создания, изменения и удаления промокодов;
- Отображение списка заказов, изменение статуса заказа и назначение курьера;
- Возможность просмотра пользователей, смены ролей, отключения и восстановления учетных записей.

2.2. Внешняя спецификация

2.2.1. Описание задачи

Для перевода процессов товарооборота магазина в электронный вид разрабатывается решение на основе облачной платформы Firebase. Мобильное приложение и веб-панель обмениваются данными с базой данных Cloud Firestore непосредственно через Firebase SDK, а права доступа проверяются правилами безопасности базы данных.

На этапе анализа требований были определены пять ролей пользователей: «Неавторизованный пользователь», «Покупатель», «Курьер», «Менеджер» и «Администратор». Функциональные возможности системы строго разграничены в зависимости от роли.

Пользователь с ролью «Неавторизованный пользователь» обладает следующими возможностями:

- Отображение экрана загрузки и экрана приветствия;

- Возможность выбора языка интерфейса;
- Возможность перехода к экрану регистрации;
- Возможность перехода к экрану авторизации;
- Возможность перехода к экрану восстановления пароля.

Пользователь с ролью «Покупатель» обладает следующими возможностями:

- Возможности неавторизованного пользователя;
- Отображение главного экрана, каталога и списков товаров категорий;
- Поиск товаров по названию;
- Фильтрация товаров по производителям и диапазону цен, сортировка списка товаров;
- Отображение карточки товара с характеристиками и пищевой ценностью;
- Возможность добавления товара в список «Избранное» и удаления из него;
- Возможность добавления товара в корзину с выбором количества;
- Возможность изменения количества товаров в корзине и их удаления;
- Возможность ввода и применения промокода на скидку;
- Отображение стоимости заказа с автоматическим расчетом товаров, скидки и доставки;
- Возможность оформления заказа с выбором адреса, интервала доставки и способа оплаты («Онлайн» или «Наличными»);
- Отображение списка заказов, их состава и хода выполнения;
- Возможность подтверждения получения заказа и скрывания завершенных заказов;
- Отображение уведомлений и настройка уведомлений;

- Возможность изменения имени, телефона и аватара;
 - Возможность добавления и удаления адресов доставки и банковских карт;
 - Возможность изменения пароля учетной записи;
 - Возможность выбора языка интерфейса (RU/EN);
 - Возможность выхода из учетной записи и удаления аккаунта.
- Пользователь с ролью «Курьер» обладает следующими возможностями:

- Отображение списка свободных заказов (без курьера, в статусах «Новый» и «В сборке»);
- Возможность взять заказ в работу и вернуть его в пул до начала доставки;
- Отображение активных заказов с данными клиента, составом заказа и суммой к получению наличными;
- Возможность копирования телефона клиента;
- Возможность начать доставку и подтвердить доставку заказа;
- Отображение истории доставок;
- Возможность выбора языка и выхода из учетной записи.

Пользователь с ролью «Менеджер» обладает следующими возможностями:

- Возможность авторизации в веб-панели;
- Отображение дашборда;
- Возможность добавления, изменения, скрытия и восстановления товаров, управления остатками;
- Возможность управления категориями, производителями и промокодами;
- Отображение списка поставщиков (только просмотр);
- Отображение списка заказов, изменение статуса заказа и назначение курьера;

- Отображение списка пользователей (только просмотр);
- Возможность смены пароля и выхода из панели.

Пользователь с ролью «Администратор» обладает следующими возможностями:

- Все возможности менеджера;
- Возможность добавления, изменения и удаления поставщиков;
- Возможность полного удаления товаров;
- Возможность смены роли пользователей (администратор, менеджер, курьер, покупатель);
- Возможность отключения и восстановления учетных записей пользователей;
- Возможность перезаливки стартового каталога товаров.

2.2.2. Входные и выходные данные

Входные и выходные данные информационной системы приведены в Приложении Е «Входные и выходные данные»

2.2.3. Методы

При разработке программного решения использовались современные технологии: язык программирования Dart и фреймворк Flutter для мобильного приложения, язык JavaScript для веб-панели и облачная платформа Google Firebase для хранения данных и авторизации.

Данное приложение было написано с использованием основных парадигм объектно-ориентированного программирования, таких как: инкапсуляция (используется для скрытия внутренних состояний компонентов программы и ограничения прямого доступа к бизнес-логике), наследование (необходимо для повторного использования кода и построения иерархий классов) и полиморфизм (позволяет по-разному реализовывать одинаковые методы в разных классах).

Пример инкапсуляции представлен на рисунке 1.

```

class CartProvider with ChangeNotifier {
  static const int maxQuantityPerItem = 99;

  final Map<String, CartItem> _items = {};
  String? _appliedPromoCode;
  double _discountPercent = 0;

  Map<String, CartItem> get items => Map.unmodifiable(_items);

  String? get appliedPromoCode => _appliedPromoCode;

  double get discountPercent => _discountPercent;

  int get itemCount => _items.length;

  int get totalQuantity => _items.values.fold(0, (sum, item) => sum + item.quantity);

  bool get isEmpty => _items.isEmpty;
  // ...
}

```

Рисунок 1 – Инкапсуляция

Пример наследования представлен на рисунке 2.

```

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  // ...
}

class DatabaseOperationException implements Exception {
  final String message;

  const DatabaseOperationException(this.message);

  @override
  String toString() => message;
}

```

Рисунок 2 – Наследование

Пример полиморфизма представлен на рисунке 3.

```

class CartScreen extends StatelessWidget {
  const CartScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // ...
  }
}

class SplashScreen extends StatefulWidget {
  const SplashScreen({super.key, required this.onFinished});

  final VoidCallback onFinished;

  @override
  State<SplashScreen> createState() => _SplashScreenState();
}

enum ProductSort {
  none,
  freshFirst,
  priceDescending,
  priceAscending;

  String label(AppLocalizations l10n) {
    switch (this) {
      case ProductSort.none:
        return l10n.sortDefault;
      case ProductSort.freshFirst:
        return l10n.sortFreshFirst;
      case ProductSort.priceDescending:
        return l10n.sortPriceDesc;
      case ProductSort.priceAscending:
        return l10n.sortPriceAsc;
    }
  }
}

```

Рисунок 3 – Полиморфизм

Для управления состоянием мобильного приложения применялся паттерн Provider (ChangeNotifier). На рисунке 4 изображена работа паттерна.

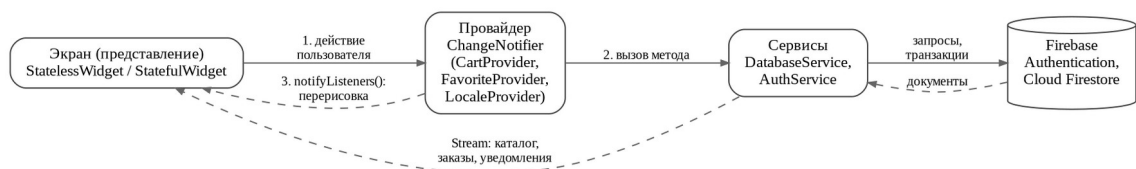


Рисунок 4 – Работа паттерна проектирования Provider

Экран (представление) читает данные из провайдера и подписывается на его изменения. При действии пользователя экран вызывает метод провайдера или сервиса. Провайдер изменяет своё состояние или обращается к сервису, который выполняет запрос к

Firebase. После изменения состояния вызывается `notifyListeners()`, и экран автоматически перерисовывается. Данные каталога, заказов и уведомлений экраны получают потоками (Stream) непосредственно от сервиса `DatabaseService`.

На рисунках 5–7 представлены модель данных, представление и логический контроллер (провайдер) приложения.

На рисунке 5 представлена модель данных.

```
final double calories;
final double proteins;
final double fats;
final double carbs;
final DateTime? expiryDate;
final int popularity;

bool get hasNutrition => calories > 0 || proteins > 0 || fats > 0 || carbs > 0;

bool get isOutOfStock => stock <= 0;

String nameFor({required bool english}) => _pick(english, name, nameEn);

String descriptionFor({required bool english}) => _pick(english, description, descriptionEn);

String compositionFor({required bool english}) => _pick(english, composition, compositionEn);

String categoryFor({required bool english}) => _pick(english, categoryName, categoryNameEn);

String qtyFor({required bool english}) => localizeQty(qty, english: english);

String countryFor({required bool english}) {
  return english ? (countryTranslations[country] ?? country) : country;
}

static String _pick(bool english, String russian, String englishValue) {
  return english && englishValue.trim().isEmpty ? englishValue : russian;
}
```

Рисунок 5 – Модель

На рисунке 6 представлена часть представления страницы.

```

class CartScreen extends StatelessWidget {
  const CartScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final cart = context.watch<CartProvider>();
    final cartItems = cart.items.values.toList();

    return Scaffold(
      backgroundColor: AppPalette.background,
      appBar: AppBar(title: Text(context.l10n.cartTitle)),
      body: Column(
        children: [
          const Divider(thickness: 1, height: 1, color: Color(0xFFE2E2E2)),
          Expanded(
            child: cartItems.isEmpty
              ? CenteredMessage(context.l10n.cartEmpty, icon: Icons.shopping_cart_outlined)
              : ListView.separated(
                  itemCount: cartItems.length,
                  separatorBuilder: (_, __) =>
                    const Divider(thickness: 1, color: Color(0xFFE2E2E2), indent: 20, endIndent: 20),
                  itemBuilder: (context, index) => _CartRow(item: cartItems[index]),
                ),
          if (cartItems.isNotEmpty) _CheckoutPanel(cart: cart),
        ],
      ),
    );
    // ...
  }
}

```

Рисунок 6 – Представление

На рисунке 7 представлен контроллер логики.

```

/// Adds one unit of [product]. Returns false when the stock limit is reached.
bool addProduct(Product product) {
  final existingItem = _items[product.id];
  final maxAllowed = math.min(product.stock, maxQuantityPerItem);
  final nextQuantity = (existingItem?.quantity ?? 0) + 1;

  if (product.isOutOfStock || nextQuantity > maxAllowed) {
    return false;
  }

  _items[product.id] = existingItem?.copyWith(
    quantity: nextQuantity,
    stockQuantity: product.stock,
  ) ??
  CartItem(
    id: product.id,
    name: product.name,
    quantity: nextQuantity,
    price: product.price,
    imageUrl: product.imageUrl,
    qty: product.qty,
    stockQuantity: product.stock,
    nameEn: product.nameEn,
  );
  notifyListeners();
  return true;
}

```

Рисунок 7 – Контроллер

2.2.4. Тесты

Сценарий и результаты тестовых испытаний приведены в Приложении Б.

2.2.5. Контроль целостности данных

В процессе разработки были реализованы методы валидации входных данных в информационной системе, которые обеспечивают целостность этих входных данных.

Контроль целостности данных, описывающий ситуации и реакции приложения на выполнение задач, связанных с сохранением, изменением или удалением данных информационной системы, представлен в таблице 3.

Таблица 3 – Контроль целостности данных

№	Ситуация	Аномалия	Реакция программы
1	Регистрация пользователя	Введен логин короче 3 символов или с недопустимыми символами	Вывод сообщения «Длина логина от 3 до 50 символов» / «Только латиница, цифры и символы . _ -»
2	Регистрация пользователя	Введен существующий логин	Вывод сообщения «Пользователь с таким логином уже существует.»
3	Регистрация пользователя	Введен некорректный Email	Вывод сообщения «Неверный формат email»
4	Регистрация пользователя	Введен существующий Email	Вывод сообщения «Этот email уже используется.»
5	Регистрация пользователя	Введен пароль короче 6 символов, без цифры или с пробелом	Вывод сообщения о несоответствии пароля требованиям
6	Регистрация пользователя	Введено некорректное имя пользователя	Вывод сообщения «Поле не может быть пустым» / «Только буквы, пробелы и дефисы»
7	Авторизация в системе	Введен логин, который не зарегистрирован	Вывод сообщения «Пользователь с таким логином не найден.»
8	Авторизация в системе	Введен неверный пароль или email	Вывод сообщения «Неверный email или пароль.»
9	Авторизация в системе	Учетная запись отключена	Вывод сообщения «Этот аккаунт был удален.», выход из системы
10	Авторизация в	Вход администратора или	Вывод сообщения о

	мобильном приложении	менеджера	необходимости использовать веб-панель
11	Восстановление пароля	Введен пустой или некорректный Email	Вывод сообщения «Введите email» / «Неверный формат email»
12	Изменение личных данных профиля	Введен номер телефона неверного формата	Вывод сообщения «Неверный формат номера телефона»
13	Изменение личных данных профиля	Загружается файл аватара неподдерживаемого формата или слишком большого размера	Вывод сообщения «Разрешены только изображения JPEG и PNG.» / «Изображение слишком большое»
14	Изменение пароля аккаунта	Введен неверный текущий пароль	Вывод сообщения «Неверный текущий пароль.»
15	Изменение пароля аккаунта	Пароль в поле подтверждения не совпадает с новым паролем	Вывод сообщения «Пароли не совпадают»
16	Добавление адреса и карты	Пустой адрес, адрес длиннее 300 символов, номер карты не из 16 цифр	Вывод сообщений «Адрес не может быть пустым.», «Адрес не должен превышать 300 символов.», «Номер карты должен содержать 16 цифр.»
17	Добавление товара в корзину	Количество превышает остаток на складе	Добавляется доступный остаток, выводится сообщение «Больше нет в наличии»
18	Изменение количества товара в корзине	Количество уменьшено до нуля	Товар удаляется из корзины
19	Применение промокода	Введен несуществующий, неактивный или просроченный промокод	Вывод сообщения «Неверный или просроченный промокод.»
20	Оформление заказа	Не выбран адрес доставки	Вывод сообщения «Добавьте адрес в профиле» / «Выберите адрес доставки.»
21	Оформление заказа	Способ оплаты «Онлайн» без выбранной карты	Вывод сообщения «Для онлайн-оплаты выберите сохраненную карту.»
22	Оформление заказа	Товара недостаточно на складе или он скрыт	Заказ не создается, выводится «Недостаточно товара «...» на складе.»
23	Взятие заказа курьером	Заказ уже взял другой курьер	Вывод сообщения «Этот заказ уже взял другой курьер.»

24	Возврат заказа в пул	Заказ уже в пути или назначен другому курьеру	Вывод сообщения «Заказ нельзя вернуть: он уже в пути или назначен не вам.»
25	Добавление/изменение товара (сотрудник)	Пустое название или не выбраны категория/производитель	Вывод сообщений «Поле «Название» обязательно.» / «Выберите категорию товара.»
26	Добавление/изменение товара (сотрудник)	Цена не больше нуля или количество вне диапазона 0–9999	Вывод сообщений «Цена должна быть больше нуля.» / «Поле «Количество» должно быть от 0 до 9999.»
27	Изменение заказа (сотрудник)	Статус «Передан курьеру», «В пути» или «Доставлен» без курьера	Вывод сообщения «Для этого статуса назначьте курьера.»
28	Изменение заказа (сотрудник)	Попытка изменить завершённый или отменённый заказ	Элементы блокируются; сообщение «Завершённый или отменённый заказ изменить нельзя.»
29	Управление справочниками (сотрудник)	Создание дубликата (категория, производитель, поставщик, промокод)	Вывод сообщения о нарушении уникальности
30	Управление справочниками (сотрудник)	Удаление категории или производителя с привязанными товарами; промокода в активных заказах	Вывод сообщения «Нельзя удалить ... привязаны продукты» / «... используется в незавершённых заказах»
31	Управление промокодами (сотрудник)	Скидка вне диапазона 1–99, недопустимые символы в коде, срок в прошлом	Вывод сообщений «Скидка должна быть целым числом от 1 до 99.», «Код промокода: только латинские буквы и цифры.», «Срок действия промокода уже истек.»
32	Управление пользователями (администратор)	Смена роли или отключение себя, последнего администратора либо пользователя с незавершёнными заказами	Вывод сообщений «Нельзя изменить собственную роль.», «Нельзя снять роль с последнего администратора.», «У пользователя есть незавершённые заказы...»
33	Любое обращение к базе данных	Пользователь пытается прочитать или изменить данные, недоступные его роли	Операция отклоняется правилами безопасности Firestore

2.3. Проектирование

2.3.1. Схема архитектуры приложения

Программное решение построено на основе облачной платформы Google Firebase и не содержит собственной серверной части. Мобильное приложение и веб-панель подключаются к сервисам Firebase через Firebase SDK: Firebase Authentication выполняет регистрацию и вход, а Cloud Firestore хранит данные интернет-магазина. Каждое обращение к базе данных проверяется правилами безопасности, которые учитывают роль пользователя.

Файлы веб-панели размещаются на Firebase Hosting и открываются в браузере по адресу проекта. Для разработки и автоматического тестирования используются локальные эмуляторы Auth и Firestore, не затрагивающие боевую базу данных.

На рисунке 8 представлена архитектурная схема приложения

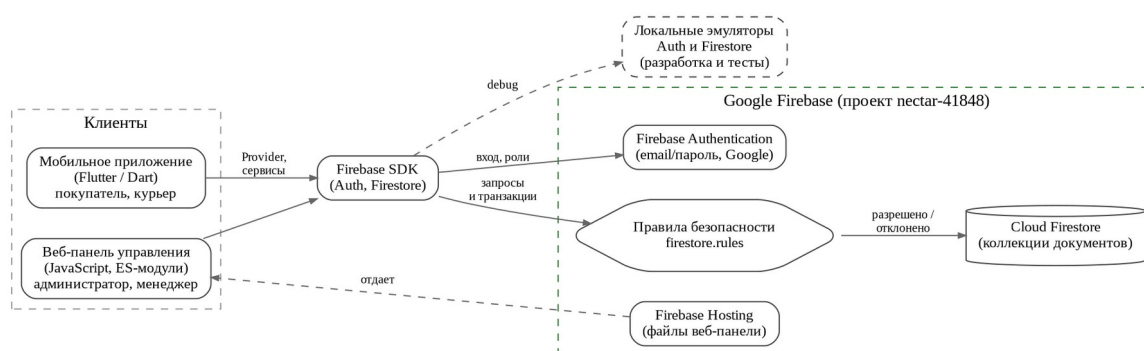


Рисунок 8 – Архитектурная схема приложения

2.3.2. Логическая схема данных

Перед разработкой на основе проведенного анализа предметной области и выделения основных бизнес-процессов была построена логическая схема данных, представленная на рисунке 9



Рисунок 9 – Логическая модель данных

2.3.3. Физическая схема данных

На основе построенной ранее логической модели данных было произведено построение физической модели данных.

База данных Cloud Firestore состоит из 11 коллекций документов, которые имеют определённый набор полей для записи информации. Каждый документ имеет идентификатор, по которому формируются связи между коллекциями; позиции заказа хранятся вложенным массивом внутри документа заказа.

На рисунке 10 представлена физическая модель данных

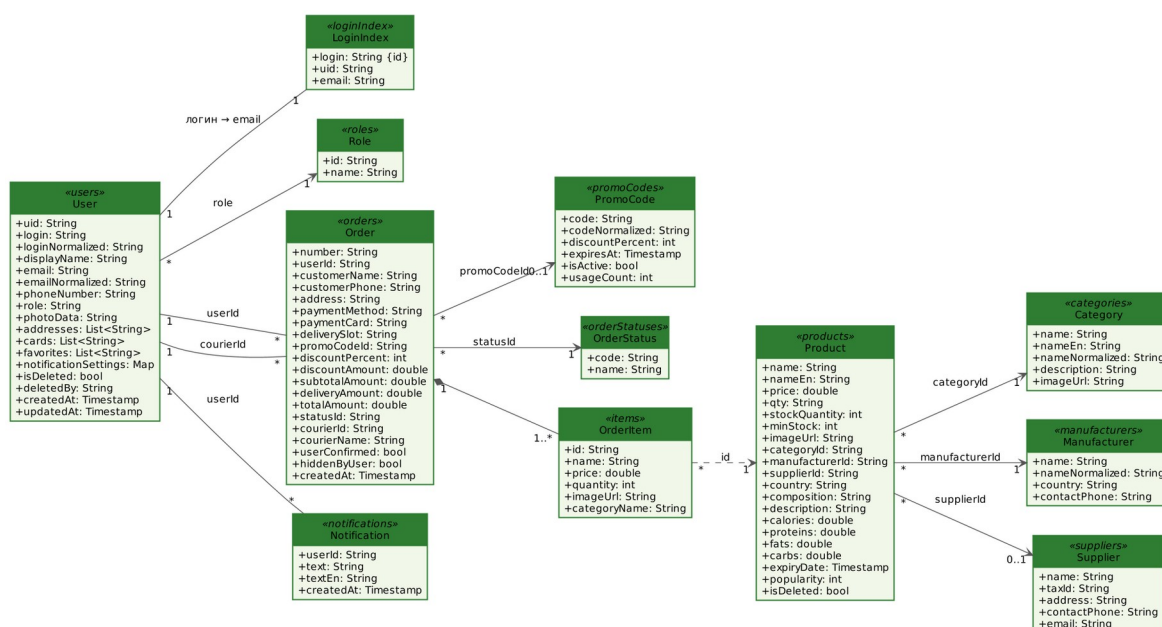


Рисунок 10 – Физическая модель данных

Словарь данных приведен в Приложении Д «Скрипт базы данных».

2.3.4. Диаграмма классов

Диаграмма классов приведена в Приложение Ж «Диаграмма классов».

2.3.5. Схема тестирования

Схема тестирования приведена в Приложении Б «Сценарий и результаты тестовых испытаний».

2.3.6. Структурная схема

На Рисунках 11, 12 и 13 представлены структурные схемы программного решения, в которых графически показано внутреннее устройство мобильного приложения и веб-панели.

Структурная схема мобильного приложения (Рисунок 11) отражает разделение исходного кода на слои: экраны и виджеты (пользовательский интерфейс), провайдеры состояния, сервисы работы с Firebase, модели данных и вспомогательные модули. Экраны обращаются к провайдерам и сервисам, а все обращения к базе данных сосредоточены в классе DatabaseService.

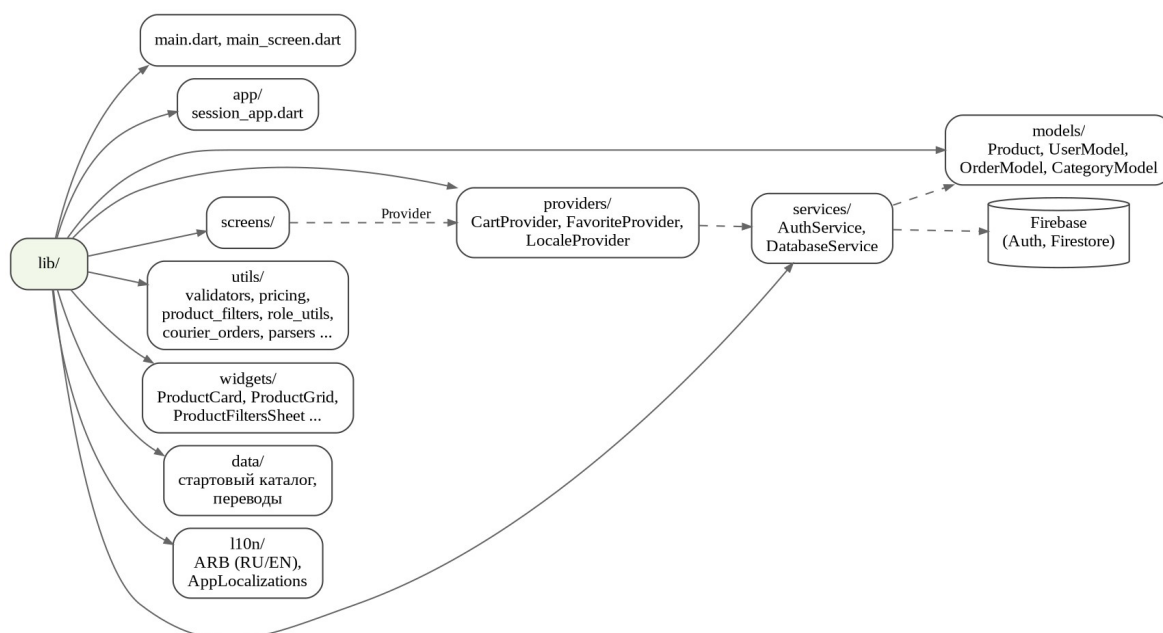


Рисунок 11 – Структурная схема мобильного приложения (часть 1)

Экраны приложения сгруппированы по разделам (Рисунок 12): авторизация, магазин, каталог, избранное, корзина, профиль и курьер.

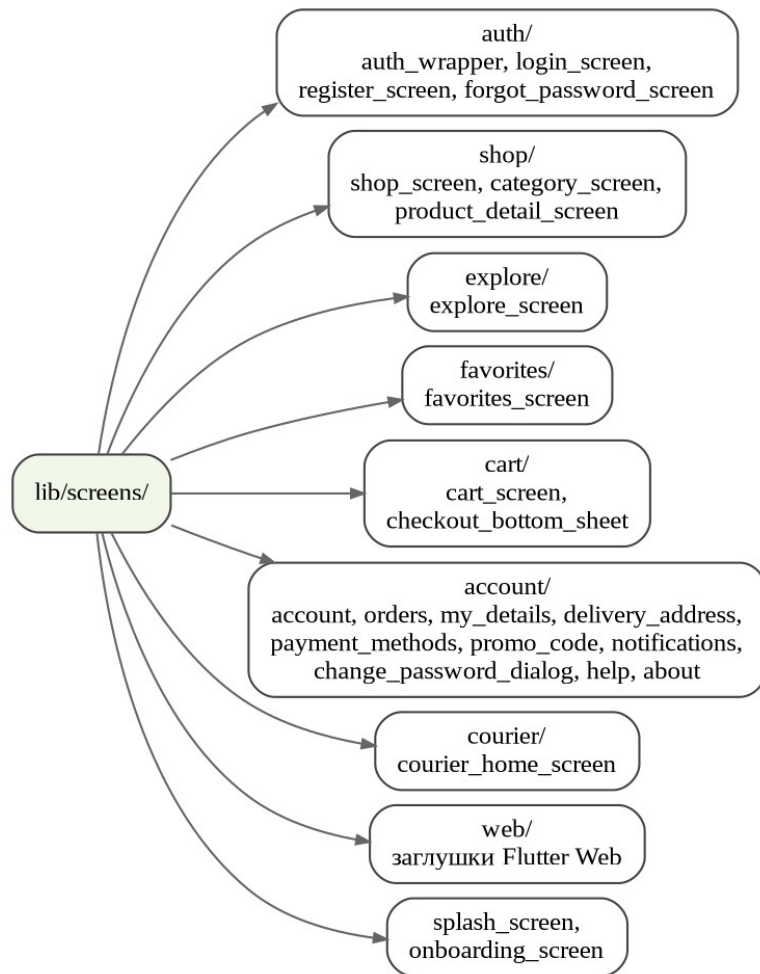


Рисунок 12 – Структурная схема мобильного приложения (часть 2)

Веб-панель (Рисунок 13) состоит из страницы `index.html`, модуля `admin.js` с логикой разделов, модуля `validation.js` с проверкой форм и расчетами, а также модулей локализации.

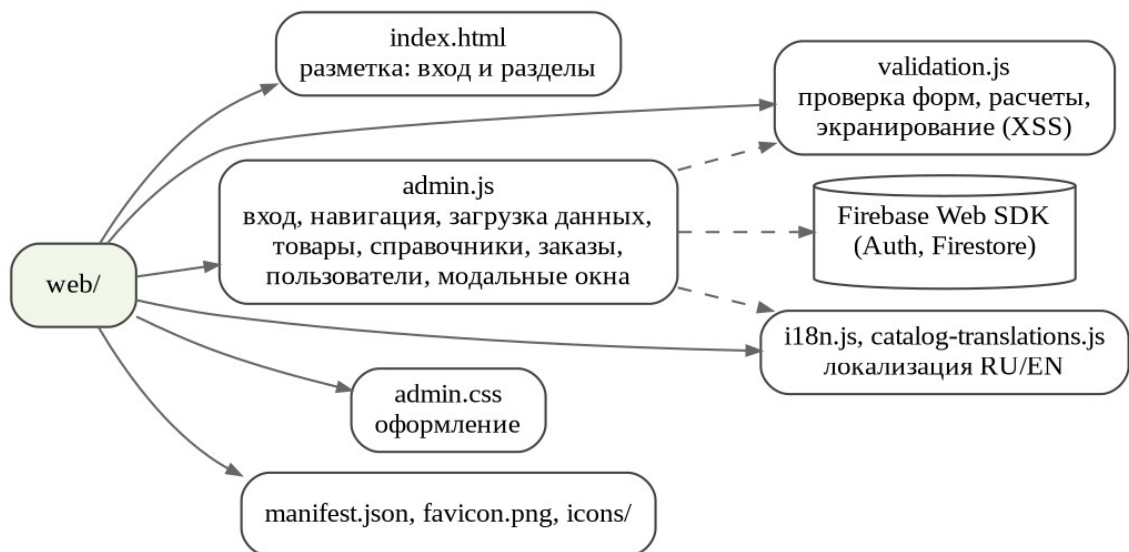


Рисунок 13 – Структурная схема веб-панели

Описание структурных схем и модулей представлено в Приложении Г «Текст программы».

2.3.7. Функциональная схема

На Рисунке 14 представлена функциональная схема программного продукта, на которой графически изображены возможности пользователей различных ролей в процессе эксплуатации программного решения

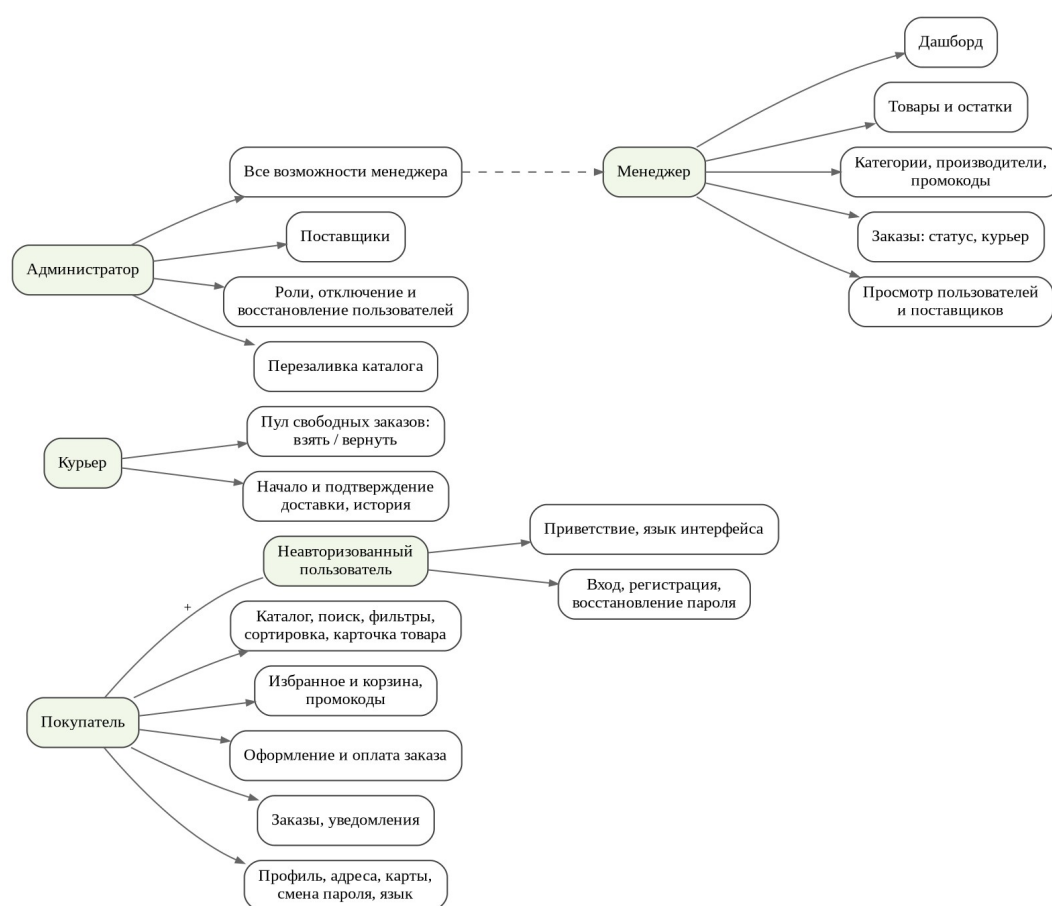


Рисунок 14 – Функциональная схема

2.3.8. Схема пользовательского интерфейса

На рисунках 15–17 представлены схемы пользовательского интерфейса программного решения

Согласно схеме на рисунке 15, взаимодействие пользователя с мобильным приложением начинается с «Экрана загрузки». Если сохраненного сеанса нет, пользователь направляется на «Экран приветствия», после чего переходит на «Экран входа»; если сеанс

сохранен, экран приветствия и вход пропускаются. С экрана входа доступны экраны регистрации и восстановления пароля. После успешного входа определяется роль: покупатель попадает на основной экран с нижней навигацией, курьер – на экран «Доставка», администратор и менеджер видят сообщение о необходимости использовать веб-панель.

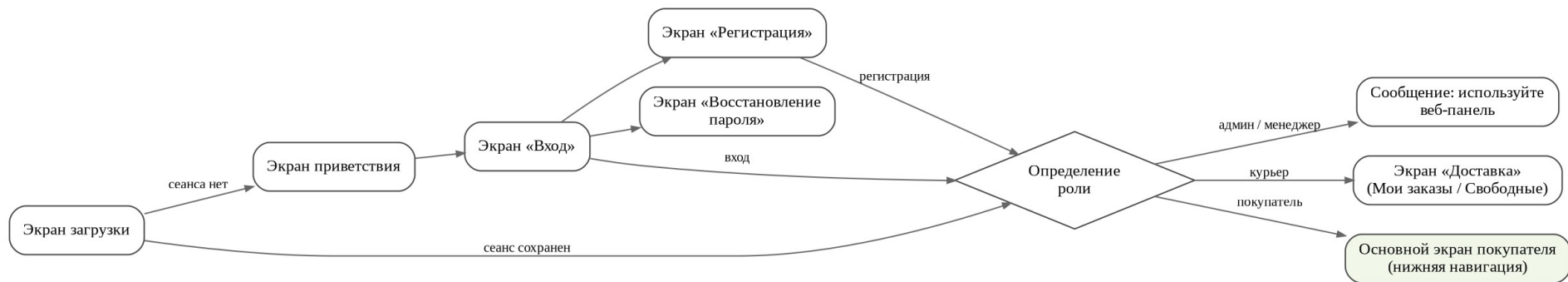


Рисунок 15 – Схема пользовательского интерфейса мобильного приложения (запуск и авторизация)

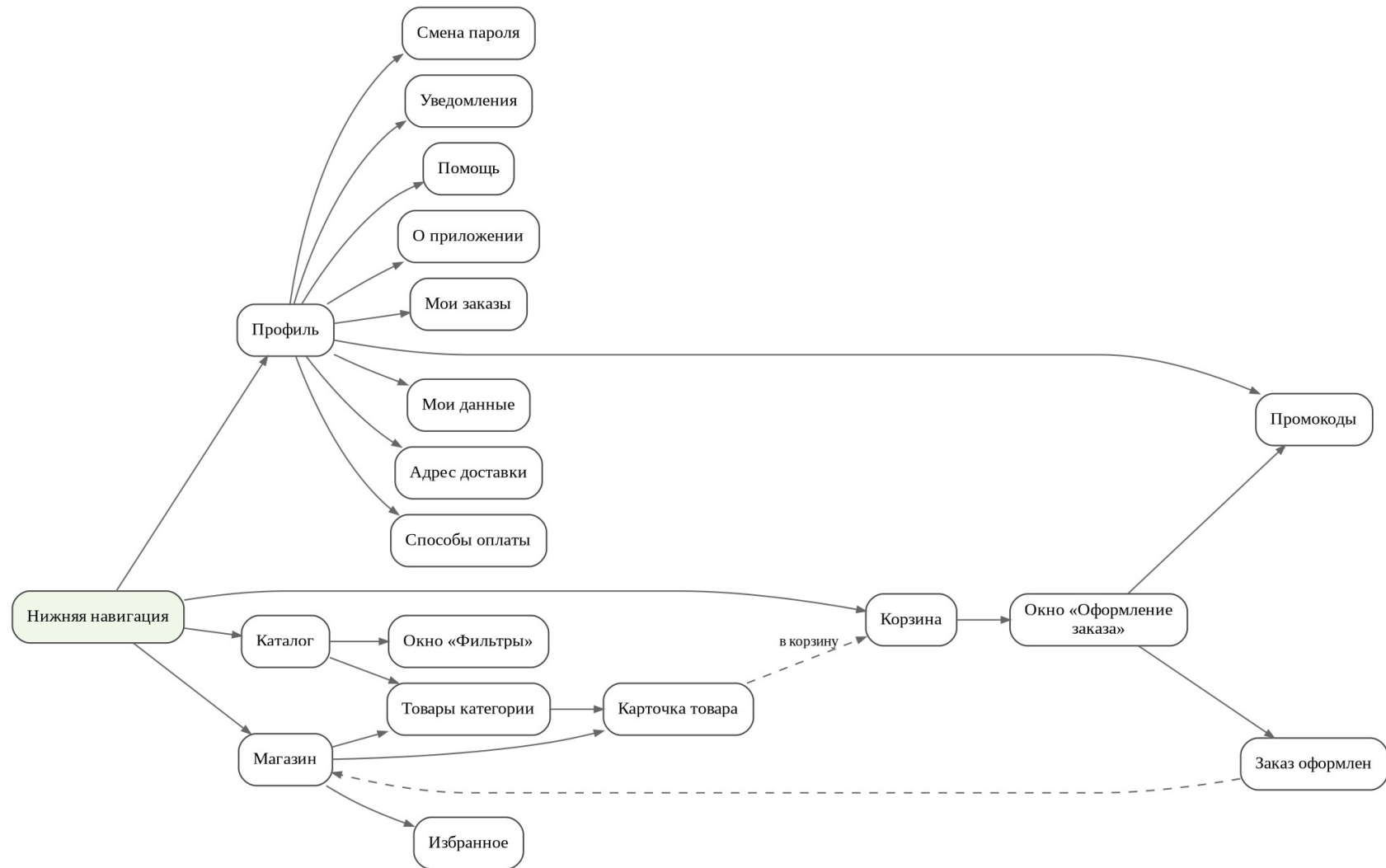


Рисунок 16 – Схема пользовательского интерфейса мобильного приложения (основные разделы)

Нижняя навигационная панель является главной навигационной системой покупателя и предоставляет доступ к четырем разделам: «Магазин», «Каталог», «Корзина» и «Профиль» (Рисунок 16).

Навигация внутри приложения имеет следующие особенности:

- Из разделов «Магазин» и «Каталог» осуществляется переход на экран товаров категории и на экран «Карточка товара», откуда товар можно добавить в корзину.
- Значок «сердце» на экране «Магазин» открывает экран «Избранное».
- Сценарий покупки линейен: из «Корзины» пользователь переходит в окно «Оформление заказа» (с возможностью выбора промокода), а после завершения – в окно «Заказ успешно оформлен!», откуда по кнопке «Вернуться в магазин» возвращается на экран «Магазин».
- В разделе «Профиль» доступны экраны «Заказы», «Мои данные», «Адрес доставки», «Способы оплаты», «Промокоды», «Смена пароля», «Уведомления», «Помощь» и «О приложении».

Схема переходов административной веб-панели имеет иерархическую структуру, которая представлена на рисунке 17.

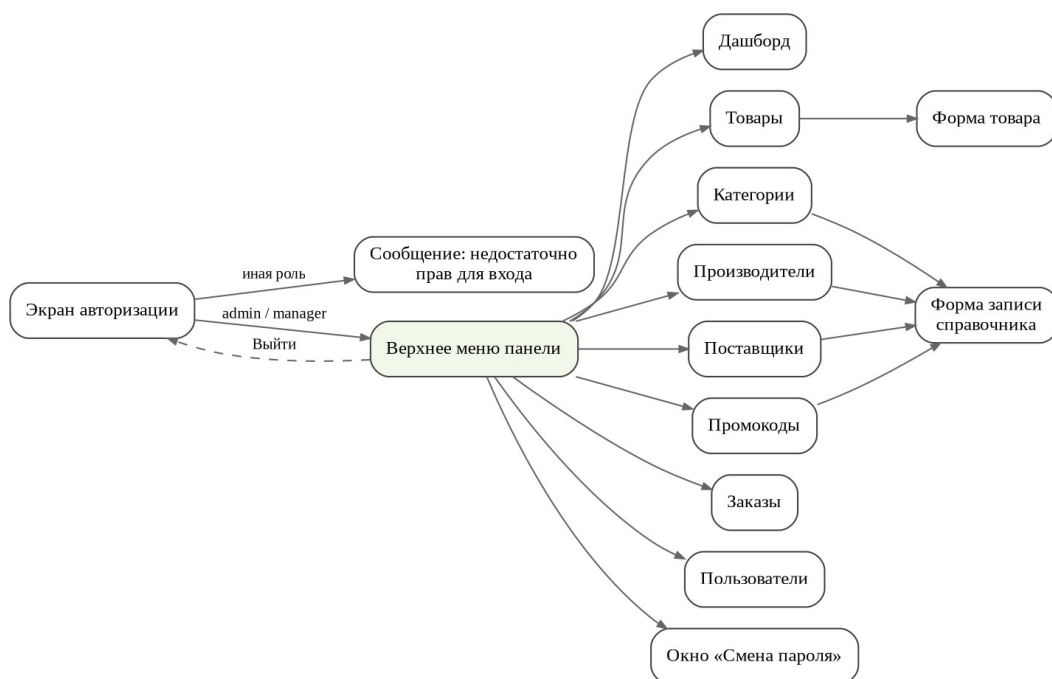


Рисунок 17 – Схема пользовательского интерфейса веб-интерфейса

Согласно схеме на рисунке 17, работа сотрудника начинается с «Экрана авторизации». В случае успешной проверки роли пользователь попадает в панель, откуда получает доступ к разделам: «Дашборд», «Товары», «Категории», «Производители», «Поставщики», «Промокоды», «Заказы» и «Пользователи», а также к смене пароля. Формы товаров и справочников открываются в модальных окнах.

2.4. Результат работы программы

В результате выполнения поставленной задачи было разработано требуемое программное решение, которое представляет собой комплекс «Интернет-магазин продуктов «Нектар», состоящий из мобильного приложения и веб-панели управления на платформе Firebase.

Данное решение реализует строгое разграничение полномочий и функциональных возможностей по ролям.

На рисунках 18-20 представлен результат работы программы. Более развернуто результат работы программы описан в Приложении В «Руководство пользователя».

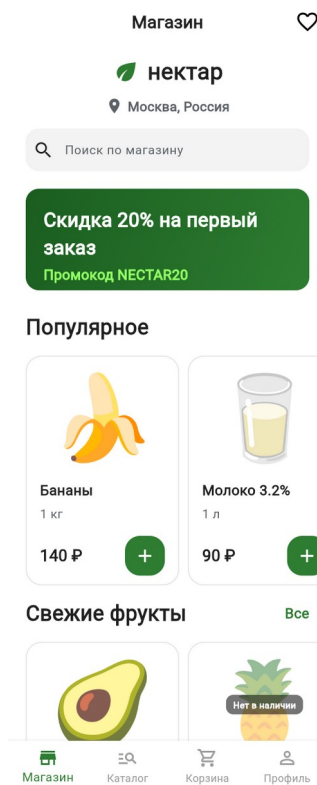


Рисунок 18 – Главный экран

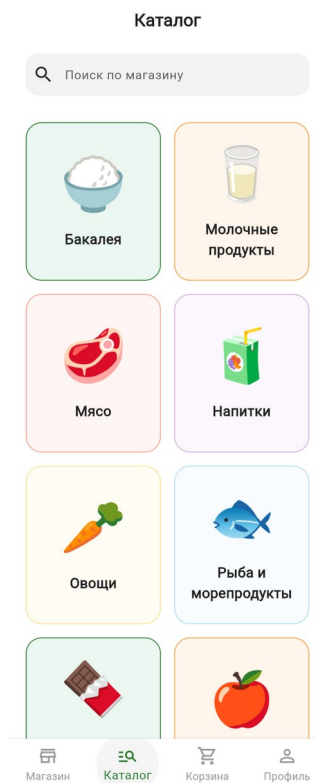


Рисунок 19 – Экран каталога

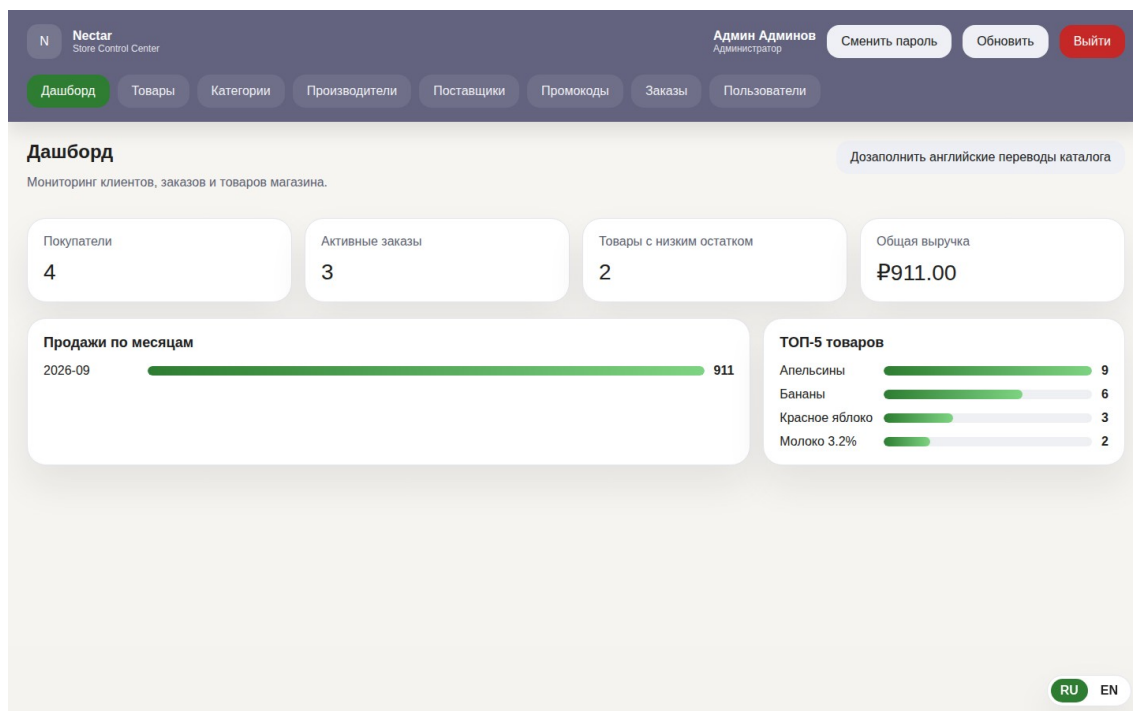


Рисунок 20 – Страница панели управления

3. ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

3.1. Инструментальные средства

Для хранения данных в проекте выбрана облачная документоориентированная база данных Cloud Firestore. Она подходит для данного проекта благодаря отсутствию необходимости в собственном сервере, обновлению данных в реальном времени, поддержке транзакций и гибким правилам безопасности. Администрирование данных и ролей выполнялось через консоль Firebase и веб-панель проекта.

Авторизация реализована средствами Firebase Authentication: регистрация и вход по email и паролю, вход через Google, отправка писем для восстановления пароля. Так как отдельного серверного кода нет, вся защита данных сосредоточена в правилах безопасности Firestore (файл `firestore.rules`).

Мобильное приложение создано на базе кроссплатформенного фреймворка Flutter на языке Dart 3.9. Этот выбор позволил получить приложения для Android и iOS из единой кодовой базы, сохранив при этом плавность интерфейса. Использованы пакеты `firebase_core`, `firebase_auth`, `cloud_firestore` и `google_sign_in` для работы с Firebase, `provider` для управления состоянием, `intl` и файлы ARB для локализации на русский и английский языки, `shared_preferences` для сохранения языка, `image_picker` для выбора аватара и `share_plus` для отправки информации о товаре.

Административная веб-панель написана на JavaScript с использованием ES-модулей и Firebase Web SDK без сборщика проекта и размещается на Firebase Hosting. Для локальной отладки и тестов применялись Firebase Emulator Suite, среда Node.js и библиотека `@firebase/rules-unit-testing`.

3.2. Отладка программы

Отладка программы выполнялась с помощью статического анализатора Dart, модульных тестов Flutter, локальных эмуляторов Firebase и инструментов разработчика браузера для веб-панели. Анализатор кода (команда `flutter analyze`) проверяет соответствие правилам `flutter_lints`; модульные тесты проверяют валидаторы, корзину, расчет доставки, модели, работу сервиса базы данных на имитации Firestore, логику курьера и локализацию.

На рисунке 21 показан результат запуска анализатора и модульных тестов приложения: замечаний нет, все 78 тестов пройдены.

```
$ flutter analyze
Analyzing nectar...

No issues found! (ran in 22.1s)

$ flutter test
00:00 +0: loading test/arb_text_test.dart
...
00:06 +75: test/courier_orders_test.dart: пустой список
00:06 +76: test/switch_theme_test.dart: включенный переключатель: зеленая дорожка и белый бегунок
00:06 +77: test/switch_theme_test.dart: SwitchListTile не переопределяет цвет бегунка зеленым
00:07 +78: All tests passed!
```

Рисунок 21 – Результат запуска анализатора и тестов приложения

Правила безопасности проверялись на локальном эмуляторе Firestore: 38 тестов правил и 21 тест валидации веб-панели (рисунок 22) подтвердили корректное разграничение прав ролей.

```
$ cd tool/rules-test && npm test

> node --test validation.test.mjs
# tests 21
# suites 9
# pass 21
# fail 0

> firebase emulators:exec --only firestore --project demo-nectar "node --test firestore.rules.test.mjs"
# tests 38
# suites 9
# pass 38
# fail 0
✓ Script exited successfully (code 0)
```

Рисунок 22 – Результат запуска тестов правил безопасности и веб-панели

Наиболее распространенными ошибками при разработке были ошибки авторизации и недоступности сети. Данные ошибки

перехватываются в сервисе AuthService, преобразуются в понятные пользователю сообщения и выводятся в нижней части экрана (рисунок 23).

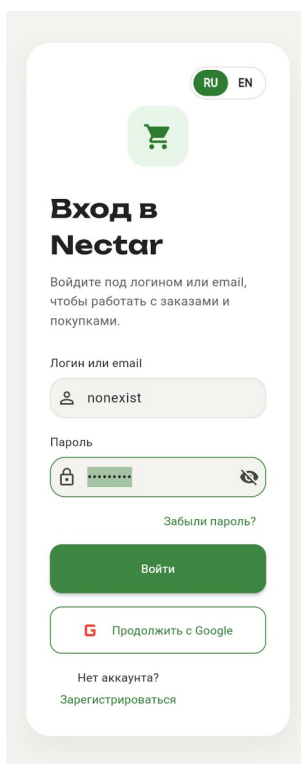


Рисунок 23 – Сообщение об ошибке авторизации

3.3. Защитное программирование

В программном коде предусмотрены механизмы защиты от ошибок и некорректных данных. Обработка исключительных ситуаций реализована с использованием конструкции `try...catch`, позволяющей перехватывать ошибки сети и базы данных без аварийного завершения приложения; пользовательские исключения `DatabaseOperationException` несут локализованное сообщение об ошибке.

Пример обработки исключений при проверке профиля после входа представлен на рисунке 24.

```
Future<void> _rejectIfDeleted(User user) async {
  // Сначала читаем профиль: удаленному пользователю запись в профиль запрещена
  // правилами, а сбой чтения не должен пропускать его в приложение.
  Map<String, dynamic>? profile;
  try {
    profile = await _dbService.getUserProfile(user.uid);
  } catch (error) {
    debugPrint('Profile check after sign-in failed: $error');
  }

  if (profile?['isDeleted'] == true) {
    await logout();
    throw DatabaseOperationException(AppLocale.strings.authAccountDeleted);
  }

  try {
    await _dbService.ensureUserProfileFromAuth(user);
  } catch (error) {
    debugPrint('Profile sync after sign-in failed: $error');
  }
}
```

Рисунок 24 – Блок try...catch в коде сервиса авторизации

Все данные, вводимые пользователем, проверяются классом Validators до отправки в базу данных: проверяются длина, допустимые символы, формат email и телефона, сложность пароля, формат номера карты и промокода. Аналогичная проверка выполняется в модуле validation.js веб-панели, а вывод данных в веб-панели экранируется для защиты от внедрения скриптов.

Пример валидаторов приведен на рисунке 25.

```
static String? validatePassword(String? value) {
  if (value == null || value.trim().isEmpty) return l10n.valPasswordRequired;
  if (value.contains(' ')) return l10n.valPasswordNoSpaces;
  if (value.length < minPasswordLength) return l10n.valPasswordMinLength;
  if (!RegExp(r'\d').hasMatch(value)) return l10n.valPasswordDigit;
  if (value.length > 128) return l10n.valPasswordMaxLength;
  return null;
}

static String? validateEmail(String? value) {
  if (value == null || value.trim().isEmpty) return l10n.valEmailRequired;
  final trimmed = value.trim().toLowerCase();
  if (trimmed.length > 100) return l10n.valEmailLength;
  if (!RegExp(r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]+$').hasMatch(trimmed)) return l10n.valEmailFormat;
  return null;
}
```

Рисунок 25 – Валидаторы полей форм

Целостность данных при оформлении заказа обеспечивается транзакцией Firestore: остатки проверяются и списываются, заказ создается, а счетчик промокода увеличивается как единая операция. Цены берутся из базы данных, а не от клиента.

Фрагмент транзакции оформления заказа представлен на рисунке 26.

```
final orderDocumentReference = _ordersCollection.doc();

final createdOrder = await _database.runTransaction<Map<String, dynamic>>((transaction) async {
  // Firestore требует, чтобы все чтения шли до записей.
  final productDocs = <String, DocumentSnapshot<Map<String, dynamic>>>{};
  for (final orderItem in normalizedOrderItems) {
    final productId = _stringValue(orderItem['id']);
    productDocs[productId] = await transaction.get(_productsCollection.doc(productId));
  }

  double subtotalAmount = 0;
  final normalizedItems = <Map<String, dynamic>>[];
  final stockUpdates = <DocumentReference<Map<String, dynamic>>, int>{};

  for (final orderItem in normalizedOrderItems) {
    final productId = _stringValue(orderItem['id']);
    final productQuantity = _intValue(orderItem['quantity']);
    if (productQuantity <= 0) {
      throw DatabaseOperationException(AppLocale.strings.errQuantityPositive);
    }

    final productDocument = productDocs[productId]!;
    final productData = productDocument.data();

    if (productData == null || productData['isDeleted'] == true) {
      throw DatabaseOperationException(AppLocale.strings.errProductUnavailable(orderItem['name'].toString()));
    }
    // ...
  }
});
```

Рисунок 26 – Фрагмент транзакции оформления заказа

Аутентификация пользователей реализована средствами Firebase Authentication. Права доступа определяются ролью, записанной в профиле пользователя, и проверяются правилами безопасности на стороне сервера Firebase при каждом обращении: покупатель видит только свои заказы, курьер – только свои активные заказы и пул свободных, а роли и блокировку меняет только администратор. Отключенный пользователь не может войти в приложение, а его данные закрыты правилами.

Фрагмент правил безопасности представлен на рисунке 27.

```

function validProduct(data) {
  return data.get('name', 'x') is string
    && data.get('name', 'x').size() > 0
    && data.get('name', 'x').size() <= 255
    && data.get('price', 1) is number
    && data.get('price', 1) > 0
    && data.get('stockQuantity', 0) is number
    && data.get('stockQuantity', 0) >= 0;
}

allow create: if isBuyer() &&
  request.resource.data.userId == request.auth.uid &&
  request.resource.data.statusId == 'new' &&
  request.resource.data.get('courierId', null) == null &&
  request.resource.data.totalAmount is number &&
  request.resource.data.totalAmount >= 0 &&
  request.resource.data.items is list &&
  request.resource.data.items.size() > 0 &&
  request.resource.data.items.size() <= 100 &&
  validOrderDiscount(request.resource.data);

```

Рисунок 27 – Фрагмент правил безопасности Firestore

3.4. Характеристики программы

Разработанный комплекс «Интернет-магазин продуктов «Нектар» состоит из мобильного приложения и веб-панели, работающих с облачной платформой Firebase. Мобильное приложение может запускаться на устройствах с операционными системами Android (версии 8.0 и выше) и iOS (версии 15.0 и выше). Основным критерием для запуска является наличие постоянного подключения к сети Интернет.

Административная веб-панель может запускаться в браузерах разных настольных операционных систем. Для ее работы версия веб-браузера должна быть новой или обновленной (рекомендуются современные версии Google Chrome, Safari, Firefox).

Отдельная серверная часть не требуется: хранение данных и авторизацию обеспечивает облачная платформа Google Firebase. Для разработки и тестирования необходимы Flutter SDK, Node.js версии 20 и выше, Firebase CLI и Java для запуска эмуляторов.

Характеристики модулей программы представлены в Приложении Г «Текст программы».

ЗАКЛЮЧЕНИЕ

За время выполнения курсового проекта был разработан программный комплекс «Интернет-магазин продуктов «Нектар».

В процессе написания курсового проекта проведены изучение и анализ предметной области, а именно процессов розничной торговли продуктами питания и информационного сопровождения клиентов онлайн-магазина.

В процессе анализа была выполнена работа по постановке задачи, определены входные и выходные данные, а также выполнено описание задачи с разграничением функциональных возможностей по пяти ролям пользователей (неавторизованный пользователь, покупатель, курьер, менеджер и администратор).

Выполнены работы по проектированию, которые включали в себя построение различных схем, необходимых перед началом разработки. В процессе проектирования были построены логическая и физическая модели базы данных, архитектурная схема приложения, структурные и функциональная схемы, а также схемы пользовательского интерфейса.

Перед началом разработки был произведен подготовительный этап, связанный с настройкой сред разработки, созданием проекта Firebase, подключением и настройкой необходимых пакетов: Flutter для кроссплатформенного мобильного клиента и Firebase SDK для работы с базой данных и авторизацией.

В процессе разработки был реализован весь функционал программного комплекса в соответствии с требованиями. В результате было разработано решение, позволяющее покупателям просматривать каталог, пользоваться поиском и фильтрами, оформлять заказы с промокодами, курьерам – брать заказы и подтверждать доставку, а сотрудникам – управлять каталогом, заказами и пользователями.

На этапе тестирования было выполнено тестирование функциональных возможностей всех ролей, а также автоматические проверки: 78 модульных тестов приложения, 21 тест веб-панели и 38 тестов правил безопасности успешно пройдены.

В ходе выполнения курсового проекта были закреплены ранее полученные практические навыки проектирования баз данных и разработки программных модулей. Также в качестве новых знаний были получены навыки кроссплатформенной мобильной разработки на языке Dart, управления состоянием приложения с помощью паттерна Provider, работы с облачной платформой Firebase и написания правил безопасности Cloud Firestore. Результат выполнения курсового проекта полностью соответствует поставленным задачам.

Разработанный программный комплекс удачно сочетает в себе удобный интерфейс, функциональность и простоту использования: доступная навигация, структурированный каталог по категориям, фильтры и сортировка, а также поддержка русского и английского языков.

В ходе использования комплекса видно, что сценарий программного изделия работает без сбоев благодаря использованию различных методов защитного программирования, таких как обработка исключений, проверка вводимых данных, транзакции и правила безопасности базы данных.

Программный комплекс «Интернет-магазин продуктов «Нектар» поможет организации, желающей продавать продукты питания через Интернет, расширить свою клиентскую базу и увеличить прибыль за счёт:

- круглосуточного и удобного мобильного канала сбыта товаров;
- современного сервиса для потребителей, позволяющего быстро совершать заказы и отслеживать их выполнение;

- автоматизации внутренних бизнес-процессов магазина и минимизации рутинных операций для персонала при учете остатков, заказов и доставки.

СПИСОК ИСПОЛЬЗОВАННЫХ МАТЕРИАЛОВ

Бэнкс, Р. Dart в действии / Р. Бэнкс, Э. Шабан. – Москва: ДМК Пресс, 2021. – 382 с. [Электронный ресурс]. – URL: <https://dmkpress.com/> (дата обращения: 03.04.2026).

Гамма, Э. Паттерны объектно-ориентированного проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. – Санкт-Петербург: Питер, 2022. – 368 с. [Электронный ресурс]. – URL: <https://piter.com/> (дата обращения: 05.04.2026).

Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. – 8-е изд. – Москва: Вильямс, 2018. – 1328 с. [Электронный ресурс]. – URL: <https://ibooks.ru/> (дата обращения: 06.04.2026).

Документация Cloud Firestore [Электронный ресурс] // Firebase Documentation. – URL: <https://firebase.google.com/docs/firestore> (дата обращения: 04.04.2026).

Документация Firebase Authentication [Электронный ресурс] // Firebase Documentation. – URL: <https://firebase.google.com/docs/auth> (дата обращения: 07.04.2026).

Документация Firebase Emulator Suite [Электронный ресурс] // Firebase Documentation. – URL: <https://firebase.google.com/docs/emulator-suite> (дата обращения: 12.04.2026).

Документация Firebase Hosting [Электронный ресурс] // Firebase Documentation. – URL: <https://firebase.google.com/docs/hosting> (дата обращения: 09.04.2026).

Документация по правилам безопасности Cloud Firestore [Электронный ресурс] // Firebase Documentation. – URL: <https://firebase.google.com/docs/firestore/security/get-started> (дата обращения: 10.04.2026).

Документация фреймворка Flutter [Электронный ресурс] // Официальный сайт Flutter. – URL: <https://docs.flutter.dev/> (дата обращения: 20.04.2026).

Макконнелл, С. Совершенный код. Мастер-класс / С. Макконнелл. – Санкт-Петербург: Русская Редакция, 2019. – 896 с. [Электронный ресурс]. – URL: <https://ibooks.ru/> (дата обращения: 09.04.2026).

Официальная документация языка Dart [Электронный ресурс] // Dart.dev. – URL: <https://dart.dev/guides> (дата обращения: 18.04.2026).

Пакет provider для управления состоянием [Электронный ресурс] // Pub.dev – официальный репозиторий пакетов Dart. – URL: <https://pub.dev/packages/provider> (дата обращения: 22.04.2026).

Пейн, Р. Flutter на практике. Основы создания кроссплатформенных мобильных приложений / Р. Пейн. – Москва: ДМК Пресс, 2021. – 414 с. [Электронный ресурс]. – URL: <https://dmkpress.com/> (дата обращения: 13.04.2026).

Руководство по JavaScript [Электронный ресурс] // MDN Web Docs. – URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript> (дата обращения: 15.04.2026).

Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл. – 10-е изд. – Москва: Вильямс, 2020. – 864 с. [Электронный ресурс]. – URL: <https://ibooks.ru/> (дата обращения: 19.04.2026).

АННОТАЦИЯ

В данном программном документе приведено техническое задание на разработку программы для интернет-магазина продуктов «Нектар».

В данном программном документе, в разделе «Введение» указано наименование, краткая характеристика области применения программы (программного изделия).

В разделе «Основания для разработки» указаны документы, на основании которых ведется разработка, наименование и условное обозначение темы разработки.

В данном программном документе, в разделе «Назначение разработки» указано функциональное и эксплуатационное назначение программы (программного изделия).

Раздел «Требования к программе» содержит следующие подразделы:

- требования к функциональным характеристикам;
- требования к надежности;
- условия эксплуатации;
- требования к составу и параметрам технических средств;
- требования к информационной и программной совместимости;
- специальные требования.

В данном программном документе, в разделе «Требования к программной документации» указаны предварительный состав программной документации и специальные требования к ней.

В разделе «Технико-экономические показатели» указаны: ориентировочная экономическая эффективность, предполагаемая годовая потребность, экономические преимущества разработки.

В данном программном документе, в разделе «Стадии и этапы

разработки» установлены необходимые стадии разработки, этапы и содержание работ.

В разделе «Порядок контроля и приемки» должны быть указаны виды испытаний и общие требования к приемке работы.

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ.....	5
1.1 Наименование программы.....	5
1.2 Краткая характеристика области применения программы	5
2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ.....	6
2.1 Основания для проведения разработки.....	6
2.2 Наименование условного обозначения для разработки.....	6
3. НАЗНАЧЕНИЕ РАЗРАБОТКИ.....	7
3.1 Функциональное назначение программы.....	7
3.2 Эксплуатационное назначение программы.....	7
4. ТРЕБОВАНИЯ К ПРОГРАММЕ.....	8
4.1 Требования к функциональным характеристикам.....	8
4.2 Требования к надежности.....	23
4.3 Условия эксплуатации.....	24
4.4 Требования к составу и параметрам технических средств	25
4.5 Требования к информационной и программной совместимости.....	26
4.6 Специальные требования.....	27
5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ.....	28
5.1 Предварительный состав программной документации.....	28
5.2 Специальные требования к программной документации	28
6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ.....	30
6.1 Ориентировочная экономическая эффективность.....	30

6.2	Предполагаемая потребность.....	30
6.3	Экономические преимущества.....	30
7.	СТАДИИ И ЭТАПЫ РАБОТКИ.....	31
7.1	Создание.....	31
7.2	Разработка.....	31
7.3	Содержание работ.....	31
7.4	Исполнители.....	32
8.	ПОРЯДОК КОНТРОЛЯ ПРИЕМКИ.....	33
8.1	Порядок.....	33
8.2	Поддержка.....	33

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование - «Интернет-магазин продуктов «Нектар».

1.2. Краткая характеристика области применения программы

Приложение предназначено к применению в сфере электронной коммерции для автоматизации процесса розничной торговли продуктами питания с доставкой. Система обеспечивает взаимодействие между магазином, покупателем и курьером через мобильное устройство, а также предоставляет сотрудникам магазина инструменты управления каталогом, заказами и пользователями через веб-панель.

2. ОСНОВАНИЕ ДЛЯ РАЗРАБОТКИ

2.1. Основание для проведения разработки

Основанием для проведения разработки является задание по курсовому проекту по МДК 01.01 «Разработка программных модулей».

2.2. Наименование и условное обозначение для разработки

Наименование темы разработки - «Разработка мобильного приложения и веб-панели управления для интернет-магазина продуктов "Нектар"»

3. НАЗНАЧЕНИЕ РАЗРАБОТКИ

3.1. Функциональное назначение программы

Функциональным назначением программы является учет товарных позиций и их остатков, автоматизация оформления и выполнения заказов и информационное сопровождение клиентов.

3.2. Эксплуатационное назначение программы

Программа должна эксплуатироваться в подразделениях:

- Веб-версия (веб-панель): в отделе продаж и логистики магазина «Нектар». Сотрудники используют её для управления каталогом товаров и остатками, обработки входящих заказов, назначения курьеров, управления промокодами и пользователями.
- Мобильная версия: предназначена для использования широким кругом лиц – покупателями для ознакомления с ассортиментом и совершения покупок, а также курьерами магазина для выполнения доставок.

Конечным пользователем программы будут клиенты магазина и персонал магазина – администраторы, менеджеры и курьеры.

4. ТРЕБОВАНИЯ К ПРОГРАММЕ

4.1. Требования к функциональным характеристикам

4.1.1. Требования к составу выполняемых функций

Программа должна обеспечивать выполнение следующих функций: регистрация новой учетной записи покупателя с вводом логина, имени, телефона, email и пароля, авторизация пользователя по логину или email и паролю, вход через аккаунт Google, восстановление утерянного пароля через электронную почту, просмотр главного экрана с акционным баннером, популярными товарами и товарами по категориям, просмотр каталога по категориям, поиск товаров по названию, фильтрация товаров по производителям и диапазону цен, сортировка списка товаров, просмотр карточки товара с описанием, характеристиками и пищевой ценностью, добавление и удаление товаров в списке «Избранное», добавление товаров в корзину с контролем остатков, изменение количества товаров в корзине, удаление товаров из корзины, ввод и проверка промокода на скидку, автоматический расчет стоимости заказа с учетом скидки и доставки, оформление заказа с выбором сохраненного адреса, интервала доставки и способа оплаты (онлайн-картой или наличными), сохранение заказа в базе данных с одновременным списанием остатков, отображение истории заказов и хода их выполнения, подтверждение получения заказа, получение уведомлений об изменении статуса заказа, редактирование профиля и аватара, управление адресами доставки и банковскими картами (хранятся только маски), смена пароля, переключение языка интерфейса (русский и английский), удаление аккаунта, выход из учетной записи.

В части курьера программа должна обеспечивать: просмотр пула свободных заказов, взятие заказа в работу (два курьера не могут взять один заказ), возврат заказа в пул до начала доставки, просмотр активных заказов и истории доставок, копирование телефона клиента, начало доставки и подтверждение доставки, получение уведомлений о назначенных заказах.

В административной части программы (веб-панель) должны выполняться функции: авторизация администратора и менеджера, просмотр дашборда (покупатели, активные заказы, товары с низким остатком, выручка, продажи по месяцам, ТОП-5 товаров), управление каталогом товаров (создание, изменение, скрытие и восстановление карточек, управление остатками), управление категориями, производителями, поставщиками и промокодами, просмотр списка заказов, смена статуса заказа и назначение курьера (с уведомлением покупателя и курьера и возвратом товаров на склад при отмене), управление пользователями (просмотр, смена роли, отключение и восстановление – только для администратора), смена пароля, переключение языка интерфейса.

4.1.2. Требования к переменным

Наименования переменных, полей и функций в клиентской части (Dart) должны соответствовать стилю `lowerCamelCase`, классов – `UpperCamelCase`, файлов – `snake_case`. Наименования переменных веб-панели (JavaScript) должны соответствовать стилю `camelCase`. Переменные должны иметь осмысленные названия, отражающие их содержимое.

4.1.3. Требования к функциям приложения (стиль оформления функций в коде)

Функции должны быть именованы глагольными словосочетаниями, отражающими выполняемое действие.

Для Dart: стиль `lowerCamelCase`. Соблюдение рекомендаций Effective Dart и правил пакета `flutter_lints`.

Для JavaScript: стиль `camelCase`, использование ES-модулей.

4.1.4. Требования к окнам приложения

Мобильное приложение (Клиентская часть):

- Экран загрузки: название «нектар» и подпись «онлайн магазин» по центру на зеленом фоне.
- Экран приветствия: три слайда с кратким описанием

преимуществ («Гарантия свежести», «Доставка за час», «Выгодные покупки»); переключатель языка RU/EN; кнопки «Далее» и «Пропустить»; кнопка «Начать» на последнем слайде.

- Экран «Вход»: поля «Логин или email» и «Пароль» (скрытый ввод), ссылка «Забыли пароль?», кнопки «Войти» и «Продолжить с Google», ссылка «Зарегистрироваться».

- Экран «Регистрация»: поля «Логин», «Имя», «Телефон», «Email», «Пароль», «Подтверждение пароля», кнопка «Зарегистрироваться».

- Экран «Восстановление пароля»: поле «Ваш Email», кнопка «Отправить письмо», окно «Письмо отправлено!».

- Экран «Магазин»: заголовок, значок избранного, строка поиска, акционный баннер, блок «Популярное», товары по категориям с кнопкой «Все», нижняя панель навигации.

- Экран «Каталог»: строка поиска, плитки категорий; при вводе запроса – результаты поиска и кнопка фильтров.

- Окно «Фильтры»: ползунок диапазона цены, выбор производителей, выбор сортировки, кнопки «Сбросить» и «Применить».

- Экран «Карточка товара»: фото, название, фасовка, остаток, цена, описание, характеристики, пищевая ценность, выбор количества, значки «Избранное» и «Поделиться», кнопка «В корзину».

- Экран «Избранное»: сетка избранных товаров, кнопка «В корзину всё».

- Экран «Моя корзина»: список позиций (фото, название, фасовка, количество, стоимость), кнопки «+», «-» и «×», блок расчета (подытог, скидка, доставка), кнопка «Оформить заказ».

- Окно «Оформление заказа»: адрес доставки, время доставки, способ оплаты («Онлайн» с выбором карты или «Наличными»), промокод, итоговые суммы, кнопка «Разместить заказ».

- Окно «Заказ успешно оформлен!»: номер заказа, кнопка «Вернуться в магазин».

- Экран «Промокоды»: поле ввода, кнопка «Применить», список доступных промокодов.
- Экран «Мои заказы»: список заказов с шагами выполнения, раскрывающиеся карточки с подробностями, кнопки «Подтвердить, что всё хорошо» и «Скрыть из списка».
- Экран «Профиль»: аватар, имя, email, роль, меню разделов, переключатель языка, кнопки «Выйти» и «Удалить аккаунт».
- Экраны «Мои данные», «Адрес доставки», «Способы оплаты», «Уведомления», «Помощь», «О приложении», окно «Смена пароля».

Курьерская часть мобильного приложения:

- Экран «Доставка»: вкладки «Мои заказы» (активные заказы и история доставок) и «Свободные» (пул заказов); кнопки «Взять заказ», «Вернуть в пул», «Начать доставку», «Подтвердить доставку».

Веб-панель (Административная часть):

- Экран авторизации: поля «Email» и «Пароль», кнопка «Войти».
- Верхняя панель: разделы «Дашборд», «Товары», «Категории», «Производители», «Поставщики», «Промокоды», «Заказы», «Пользователи», кнопки «Сменить пароль», «Обновить», «Выйти», переключатель языка RU/EN.
- Раздел «Дашборд»: показатели и диаграммы «Продажи по месяцам», «ТОП-5 товаров».
- Раздел «Товары»: таблица товаров, строка поиска, кнопка «Добавить товар», кнопки «Редактировать», «Удалить» (скрыть), «Восстановить».
- Форма товара: поля названия, цены, количества, единицы, категории, производителя, поставщика, изображения, страны, срока годности, пищевой ценности, состава и описания; кнопка «Сохранить».
- Разделы справочников «Категории», «Производители», «Поставщики», «Промокоды»: таблицы, кнопки «Добавить»,

«Редактировать», «Удалить».

- Раздел «Заказы»: таблица заказов, списки выбора статуса и курьера, кнопка «Сохранить».
- Раздел «Пользователи»: таблица пользователей, выбор роли, кнопки «Сохранить роль», «Отключить», «Восстановить», флажок «Показывать отключенных».

4.1.5. Требования к ролям пользователей

Неавторизованный пользователь

Это пользователь, который установил мобильное приложение, но не прошел авторизацию.

Права и возможности:

- Просмотр экрана загрузки и приветствия, переключение языка интерфейса.
- Доступ к экранам авторизации, регистрации и восстановления пароля.

Покупатель

Это основной пользователь мобильного приложения, прошедший авторизацию. Обладает всеми правами неавторизованного пользователя, а также расширенным функционалом.

Права и возможности:

- Просмотр каталога, поиск, фильтрация и сортировка товаров, просмотр карточки товара.
- Добавление и удаление товаров в разделе «Избранное».
- Добавление товаров в корзину, изменение количества, применение промокодов, выбор адреса, времени доставки и способа оплаты, оформление заказа.
- Просмотр истории своих заказов и уведомлений, подтверждение получения заказа.
- Редактирование профиля, аватара, адресов и карт, смена пароля, смена языка, удаление аккаунта, выход из аккаунта.

Курьер

Это пользователь мобильного приложения, отвечающий за доставку заказов.

Права и возможности:

- Просмотр пула свободных заказов, взятие заказа в работу и возврат его в пул до начала доставки.
- Просмотр своих активных заказов и истории, перевод заказа в статусы «В пути» и «Доставлен».
- Смена языка, выход из аккаунта.

Менеджер

Это пользователь, имеющий доступ к веб-панели. Роль предназначена для ведения каталога и обработки заказов.

Права и возможности:

- Создание и изменение карточек товаров, управление остатками, скрытие и восстановление товаров.
- Управление категориями, производителями и промокодами; просмотр поставщиков.
- Просмотр заказов, смена статуса заказа, назначение курьера.
- Просмотр списка пользователей, смена пароля, выход из аккаунта.

Администратор

Это пользователь, имеющий полный доступ к веб-панели. Роль предназначена для управления контентом, пользователями и бизнес-процессами магазина.

Права и возможности:

- Все возможности менеджера.
- Полное удаление товаров и управление поставщиками.
- Смена роли пользователей, отключение и восстановление учетных записей.
- Перезаливка стартового каталога товаров.

4.1.6. Требования к регистрации новой учетной записи

Для регистрации пользователю необходимо на экране входа нажать

«Зарегистрироваться» и заполнить следующие поля, к которым предъявляются правила валидации:

Поле «Логин»: текстовое поле. Разрешены латинские буквы, цифры и символы «.», «_», «-». Длина от 3 до 50 символов. Логин должен быть уникальным.

Поле «Имя»: текстовое поле. Разрешены буквы русского и латинского алфавитов, пробел и дефис. Ввод цифр и специальных символов запрещен. Длина от 2 до 50 символов.

Поле «Телефон»: текстовое поле. Разрешены цифры, знак «+», пробелы, скобки и дефис; количество цифр от 10 до 15.

Поле «Email»: текстовое поле. Строка должна содержать один символ «@» и корректное доменное имя, состоять из латинских букв, цифр и допустимых символов («.», «_», «%», «+», «-»), длиной не более 100 символов. Пробелы и кириллица не допускаются. Email должен быть уникальным.

Поле «Пароль»: текстовое поле со скрытым вводом. Минимальная длина – 6, максимальная – 128 символов, пробелы запрещены, обязательна хотя бы одна цифра.

Поле «Подтверждение пароля»: значение должно совпадать с полем «Пароль».

После корректного заполнения полей пользователю необходимо нажать кнопку «Зарегистрироваться». В случае успешной проверки программа должна создать учетную запись, сохранить профиль с ролью «Покупатель» в базе данных и автоматически выполнить вход.

4.1.7. Требования к авторизации

Для входа в мобильное приложение пользователю необходимо заполнить поля «Логин или email» и «Пароль» и нажать кнопку «Войти». В случае успеха открывается главный экран приложения согласно роли пользователя (покупатель – «Магазин», курьер – «Доставка»).

Для входа в веб-панель сотрудник открывает веб-страницу в браузере, вводит email и пароль (со скрытым вводом, не более 128

символов) и нажимает кнопку «Войти» мышью или клавишей Enter. Вход разрешен только ролям «Администратор» и «Менеджер».

4.1.8. Требования к восстановлению утерянного пароля

Для восстановления пароля пользователю необходимо на экране авторизации нажать на ссылку «Забыли пароль?», ввести свой email (латинские буквы, цифры и допустимые символы, обязательное наличие «@» и доменной зоны) в текстовое поле и нажать кнопку «Отправить письмо». На почту должна прийти ссылка для создания нового пароля.

4.1.9. Требования к поиску товаров

Для поиска товара пользователю необходимо нажать на строку поиска на экране «Магазин» или «Каталог» и ввести с клавиатуры часть названия (минимум 1 символ). Список товаров обновляется автоматически по мере ввода.

4.1.10. Требования к фильтрации товаров

Для настройки фильтров пользователю необходимо ввести запрос в строку поиска каталога и нажать кнопку с иконкой фильтра (либо иконку фильтра на экране категории). В открывшемся окне пользователь передвигает ползунок диапазона цены и выбирает производителей нажатием на соответствующие кнопки. Для подтверждения действий нажимается кнопка «Применить», для отмены – «Сбросить».

4.1.11. Требования к сортировке списка товаров

Для сортировки пользователю необходимо в окне фильтров выбрать один из вариантов: «По умолчанию», «Сначала свежие», «По убыванию цены» или «По возрастанию цены» и нажать «Применить».

4.1.12. Требования к добавлению и удалению товаров в списке «Избранное»

Для добавления товара в избранное пользователю необходимо нажать на иконку «сердце» в карточке товара. Иконка должна изменить цвет. Для удаления товара необходимо повторно нажать на закрашенную иконку «сердце». Просмотр списка выполняется по иконке «сердце» на экране «Магазин».

4.1.13. Требования к добавлению товаров в корзину

Для добавления в корзину пользователю необходимо в карточке товара выбрать количество кнопками «+» и «-» и нажать кнопку «В корзину» либо нажать кнопку «+» на карточке товара в списке. Количество одной позиции не может превышать остаток товара и 99 единиц.

4.1.14. Требования к изменению количества и удалению товаров из корзины

Для изменения количества пользователю необходимо на экране корзины нажать на кнопку «+» или «-» рядом с товаром. Для удаления товара необходимо нажать на кнопку «×» или уменьшить количество до нуля.

4.1.15. Требования к вводу промокода на скидку

Для применения скидки пользователю необходимо в окне оформления заказа нажать «Выбрать» в строке «Промокод», ввести с клавиатуры код (латинские буквы и цифры) либо выбрать его из списка и нажать кнопку «Применить». В случае успеха выводится сообщение о размере скидки (от 1 до 99 %), итоговая сумма пересчитывается. Просроченный или неактивный промокод не принимается.

4.1.16. Требования к расчету стоимости заказа

Стоимость рассчитывается автоматически: подытог – сумма цен товаров, скидка – процент от подытога, доставка – 149 ₽ (бесплатно, если сумма после скидки не меньше 1500 ₽), итог – подытог за вычетом скидки плюс доставка. Пользователь считывает значения в блоке расчета на экранах «Корзина» и «Оформление заказа».

4.1.17. Требования к оформлению заказа

Для перехода к оформлению пользователю необходимо нажать кнопку «Оформить заказ» в корзине. В открывшемся окне пользователь выбирает сохраненный ранее адрес доставки (адрес добавляется в разделе «Профиль» → «Адрес доставки», длина от 1 до 300 символов), интервал доставки («Как можно скорее (до 60 минут)» или двухчасовой интервал с

09:00 до 21:00) и способ оплаты: «Онлайн» – с выбором сохраненной карты (номер карты состоит из 16 цифр, в базе хранится только маска) или «Наличными» курьеру. Затем нажимается кнопка «Разместить заказ». Программа проверяет остатки, списывает товары со склада, создает заказ со статусом «Новый», очищает корзину и выводит номер заказа.

4.1.18. Требования к просмотру истории заказов

Для просмотра истории пользователю необходимо перейти в раздел «Профиль» и выбрать пункт «Заказы». Для каждого заказа отображаются номер, дата и ход выполнения. При нажатии на заказ карточка раскрывается и показывает адрес, время доставки, способ оплаты, курьера, итоговую сумму и состав заказа. Завершенные и отмененные заказы можно скрыть из списка.

4.1.19. Требования к подтверждению получения заказа

Пока заказ находится у курьера или доставлен, в раскрытой карточке заказа отображается кнопка «Подтвердить, что всё хорошо». После нажатия отображается сообщение о подтверждении получения.

4.1.20. Требования к просмотру уведомлений

Для просмотра уведомлений пользователю необходимо перейти в «Профиль» → «Уведомления». Здесь отображаются сообщения об изменении статуса заказа и назначении курьера, а также переключатели Push-, SMS- и Email-уведомлений. Кнопка «Очистить все» удаляет историю уведомлений.

4.1.21. Требования к редактированию профиля

Для изменения данных пользователю необходимо в разделе «Профиль» выбрать «Мои данные». Пользователь изменяет поля «Полное имя» (только буквы, пробелы и дефисы, от 2 до 50 символов) и «Номер телефона» (от 10 до 15 цифр) и нажимает кнопку «Сохранить». Email изменить нельзя. Для смены аватара пользователь нажимает на текущее фото и выбирает изображение из галереи устройства (разрешены форматы JPEG и PNG, после сжатия размер не более 150 КБ).

4.1.22. Требования к смене пароля

Для смены пароля пользователю необходимо выбрать пункт «Сменить пароль», ввести текущий пароль, новый пароль (не менее 6 символов, без пробелов, минимум одна цифра, не совпадает с текущим) и его подтверждение, после чего нажать кнопку «Сохранить».

4.1.23. Требования к настройке языка

Для смены языка пользователь нажимает на переключатель «RU/EN» в профиле (или на экранах приветствия и входа). Весь интерфейс приложения переключается на выбранный язык, выбор сохраняется на устройстве.

4.1.24. Требования к выходу из учетной записи и удалению аккаунта

Для выхода пользователю необходимо в разделе «Профиль» нажать кнопку «Выйти». Для удаления аккаунта необходимо нажать «Удалить аккаунт» и подтвердить действие; удаление невозможно при наличии незавершенных заказов.

4.1.25. Требования к работе курьера с заказами

Курьер на вкладке «Свободные» нажимает «Взять заказ», после чего заказ получает статус «Передан курьеру». До начала доставки заказ можно вернуть в пул кнопкой «Вернуть в пул» с подтверждением. Кнопка «Начать доставку» переводит заказ в статус «В пути». Кнопка «Подтвердить доставку» после подтверждения переводит заказ в статус «Доставлен»; отмена действия невозможна.

4.1.26. Требования к просмотру и изменению статуса заказа в веб-панели

Для работы с заказами сотруднику необходимо перейти в раздел «Заказы». Для изменения статуса в строке заказа выбираются новый статус (из списка: «Новый», «В сборке», «Передан курьеру», «В пути», «Доставлен», «Отменен») и курьер, после чего нажимается кнопка «Сохранить». Для статусов «Передан курьеру», «В пути» и «Доставлен» курьер обязателен. При отмене заказа (после подтверждения) товары возвращаются на склад. Завершенные и отмененные заказы изменять нельзя.

4.1.27. Требования к управлению каталогом товаров и остатками

Для добавления товара сотруднику необходимо нажать кнопку «Добавить товар» и заполнить поля: название (от 1 до 255 символов), цена (значение больше 0, не более 1 000 000, допускаются сотые доли), количество на складе (целое от 0 до 9999), единица (до 50 символов), категория и производитель (обязательный выбор из справочника), поставщик (необязательно), ссылка на изображение (<http://> или <https://>), страна, срок годности, пищевая ценность (от 0 до 10 000), состав и описание (до 2000 символов) на русском и английском языках. В конце необходимо нажать «Сохранить». Для скрытия товара сотрудник нажимает кнопку «Удалить» и подтверждает действие; скрытый товар можно восстановить.

4.1.28. Требования к управлению справочниками и промокодами

Категории (название до 100 символов, описание до 500), производители (название, страна, телефон) и поставщики (название, ИНН из 10 или 12 цифр, адрес до 300 символов, телефон, email) создаются и изменяются в соответствующих разделах кнопками «Добавить» и «Редактировать». Названия должны быть уникальны; запись, к которой привязаны товары, удалить нельзя. Промокод состоит из латинских букв и цифр (до 30 символов), имеет скидку от 1 до 99 % и срок действия не раньше текущей даты; промокод, используемый в незавершенных заказах, удалить нельзя.

4.1.29. Требования к управлению пользователями

Администратор в разделе «Пользователи» выбирает роль в списке и нажимает «Сохранить роль» либо нажимает «Отключить» и подтверждает действие. Нельзя изменить свою роль, отключить себя и последнего администратора, а также изменить роль или отключить пользователя с незавершенными заказами. Отключенных пользователей можно показать флажком «Показывать отключенных» и восстановить кнопкой «Восстановить».

4.1.30. Требование к письмам на почту

Программа должна обеспечивать отправку на электронную почту пользователя письма со ссылкой для создания нового пароля сразу после нажатия кнопки «Отправить письмо» на экране восстановления пароля. Письмо формируется сервисом Firebase Authentication на языке интерфейса приложения.

4.1.31. Требования к подключению базы данных

База данных: программа должна использовать облачную документоориентированную базу данных Google Cloud Firestore. База данных отвечает за постоянное хранение информации о пользователях, каталоге товаров, справочниках, промокодах, заказах и уведомлениях. Отдельный серверный код и REST API не разрабатываются: мобильное приложение и веб-панель обращаются к базе данных напрямую через Firebase SDK, а права доступа контролируются правилами безопасности Firestore (файл `firestore.rules`).

Аутентификация: регистрация и вход выполняются средствами Firebase Authentication (email и пароль, Google Sign-In).

4.1.32. Требования к дизайну приложения

Дизайн мобильного приложения должен быть выполнен в светлой цветовой теме с зелеными акцентами.

- Основной фон: светло-бежевый (#F5F3EF), поверхности карточек – белые.
- Основной цвет: зеленый (#2E7D32), темный оттенок – #1B5E20. Цвет применяется для кнопок целевых действий («В корзину», «Разместить заказ», «Войти»), активных иконок и акцентов. Кнопки опасных действий выделяются красным (#C62828).
- Шрифты: заголовки – Unbounded, основной текст – шрифт без засечек (Sans-serif) для обеспечения высокой читаемости. Все сочетания цвета текста и фона должны соответствовать контрасту не ниже 4.5:1 (WCAG AA).

4.1.33. Требования к таблицам базы данных

Специальные требования к архитектуре БД не предъявляются,

однако все коллекции и поля документов должны быть названы на английском языке в стиле camelCase. Примеры названий коллекций: users, products, categories, manufacturers, suppliers, promoCodes, orders, orderStatuses, notifications, loginIndex.

4.1.34. Требования к организации входных данных

Входные данные поступают в систему от пользователя через экранные формы (ввод текста с клавиатуры, переключатели, выбор из списков и файлов).

1. Окна «Вход», «Регистрация» и «Восстановление пароля»:

- Логин: тип String. Требования: 3–50 символов, латиница, цифры и символы «.», «_», «-».
- Имя: тип String. Требования: обязательное поле, только буквы, пробелы и дефисы, длина от 2 до 50 символов.
- Email: тип String. Требования: обязательное поле, наличие символа «@» и домена (формат *@*.*), не более 100 символов.
- Пароль: тип String. Требования: обязательное поле, скрытый ввод, от 6 до 128 символов, без пробелов, минимум одна цифра.

2. Окна «Магазин», «Каталог» и «Фильтры»:

- Строка поиска: тип String. Требования: запуск поиска при длине от 1 символа.
- Сортировка: тип Enum (по умолчанию, сначала свежие, по убыванию цены, по возрастанию цены).
- Выбор производителей: тип Set of Strings (множество выбранных значений).
- Диапазон цены (от и до): тип Double. Требования: значение «до» $\geq$ «от», ≥ 0 .

3. Окна «Корзина» и «Оформление заказа»:

- Количество товара (+ / -): тип Integer. Требования: от 1 до min(остаток, 99).
- Промокод: тип String. Требования: латинские буквы и цифры, без спецсимволов.

- Адрес доставки: тип String. Требования: выбор из сохраненных адресов.

- Способ оплаты: тип String (Онлайн или Наличными); карта: выбор из сохраненных.

4. Окна «Профиль» и настройки:

- Номер телефона: тип String. Требования: цифры и знаки «+», «-», «(», «)», пробел; 10–15 цифр.

- Аватар пользователя: тип File (форматы JPEG, PNG; после сжатия до 150 КБ).

- Смена пароля: поля «Текущий пароль», «Новый пароль», «Повтор нового пароля» (тип String, скрытый ввод).

- Уведомления Push/SMS/Email: тип Boolean (переключатель On/Off).

- Язык интерфейса: тип String (выбор из списка: RU, EN).

5. Окна курьера и административной веб-панели:

- Действие курьера: кнопки «Взять заказ», «Вернуть в пул», «Начать доставку», «Подтвердить доставку».

- Управление товаром: название, описание (тип String), цена (Double, > 0), количество (Integer, 0–9999), ссылка на изображение (String).

- Управление справочниками и промокодами: строковые и числовые поля согласно разделу 4.1.

- Управление заказами: изменение статуса заказа – тип Enum (выбор из выпадающего списка), курьер – выбор из списка.

- Управление пользователями: роль – тип Enum (администратор, менеджер, курьер, покупатель).

4.1.35. Требования к организации выходных данных

Выходные данные формируются базой данных и визуализируются в интерфейсе приложения или веб-панели.

1. Экраны «Магазин» и «Каталог»:

- Акционный баннер: строка (String) с описанием скидки и

промокодом.

- Карточки товаров: главное фото (Image), название (String), фасовка (String), цена (Double с символом валюты), остаток.
- Детали товара: описание (String), характеристики, пищевая ценность (Double), остаток (Integer).

2. Экраны «Избранное» и «Корзина»:

- Список избранного: карточки товаров, сохраненных пользователем (фото, название, цена).
- Содержимое корзины: миниатюра товара (Image), название (String), фасовка, количество (Integer).
- Финансовые расчеты: подытог, скидка, доставка, итог (тип Double).

3. Экраны «Заказы», «Профиль» и «Доставка»:

- Профиль: имя, email, роль (String), аватар (Image).
- Список заказов: номер (String), дата (DateTime), ход выполнения, итоговая сумма (Double).
- Детализация заказа: адрес, время и способ оплаты, курьер и его телефон, перечень товаров с количеством.
- Заказы курьера: адрес, время, клиент и телефон, состав, сумма и сумма к получению наличными.

4. Системные уведомления (Оба приложения):

- Сообщения об ошибках / успехах (SnackBar / Toast): текстовые сообщения (String), например: «Неверный email или пароль.», «Промокод SALE10 применен. Скидка 10%.», «Заказ взят в работу».

5. Окна административной веб-панели:

- Дашборд: показатели (Integer, Double) и столбчатые диаграммы.
- Таблицы товаров, справочников, заказов и пользователей: форматированный вывод данных.

4.2. Требования к надежности

4.2.1. Требования к обеспечению надежного (устойчивого)

функционирования программы

Надежное (устойчивое) функционирование программы должно быть обеспечено организацией бесперебойного питания технических средств и устойчивого подключения к сети Интернет. Целостность данных при оформлении заказа обеспечивается выполнением проверки остатков, списания и создания заказа в одной транзакции базы данных.

4.2.2. Время восстановления после отказа

Время восстановления после отказа, вызванного сбоем электропитания технических средств не должно превышать времени, необходимого на перезагрузку операционной системы и запуск программы, при условии соблюдения условий эксплуатации технических и программных средств.

Время восстановления после отказа, вызванного неисправностью технических средств, не должно превышать времени, требуемого на устранение неисправностей технических средств и переустановки программных средств.

4.2.3. Отказы из-за некорректных действий оператора

Отказы программы возможны вследствие некорректных действий пользователя при взаимодействии с операционной и информационной системой.

Во избежание возникновения отказов программы по указанной выше причине следует провести инструктаж с конечным пользователем о правильной эксплуатации информационной системы и убедиться в том, что конечный пользователь имеет опыт использования ЭВМ.

4.3. Условия эксплуатации

4.3.1. Климатические условия эксплуатации

Климатические условия эксплуатации, при которых должны обеспечиваться заданные характеристики, должны удовлетворять требованиям, предъявляемым к техническим средствам в части условий их эксплуатации.

4.3.2. Требования к видам обслуживания

Требования к видам обслуживания не предъявляются.

4.3.3. Требования к численности и квалификации персонала

Сотрудники должны иметь базовое умение работы с персональным компьютером и веб-браузером.

4.4. Требования к составу и параметрам технических средств

Клиентская часть (Мобильное устройство):

В состав технических средств должен входить смартфон или планшетный компьютер, включающий в себя:

- а) процессор с архитектурой ARM, тактовой частотой 1.5 ГГц и выше;
- б) оперативную память объемом не менее 2 ГБ;
- в) свободное место во встроенной памяти устройства объемом не менее 100 МБ;
- г) сенсорный экран (дисплей) с поддержкой технологии Multi-touch;
- д) модуль беспроводной связи (Wi-Fi или 3G/4G/LTE) для обеспечения постоянного доступа к сети Интернет.

Административная часть (Рабочее место сотрудника / разработчика):

В состав технических средств должен входить IBM-совместимый персональный компьютер (ПЭВМ) или ноутбук, включающий в себя:

- а) процессор с тактовой частотой не менее 2.0 ГГц;
- б) оперативную память объемом не менее 8 ГБ (необходимо для комфортного запуска сред разработки, эмулятора смартфона и эмуляторов Firebase);
- в) свободное место на жестком диске (HDD) или твердотельном накопителе (SSD) объемом не менее 10 ГБ;
- г) устройство ввода-вывода (клавиатура) и оптический манипулятор типа «мышь»;
- д) монитор с разрешением экрана не менее 1920x1080 пикселей (Full HD).

4.5. Требования к информационной и программной совместимости

4.5.1. Требования к информационным структурам и методам решения

Требования к информационным структурам на входе и выходе, а также к методам решения не предъявляются.

4.5.2. Требование к исходному коду и языкам программирования

Клиентская часть (Мобильное приложение): Исходные коды мобильного приложения должны быть реализованы на языке программирования Dart с использованием кроссплатформенного фреймворка Flutter (версии 3 и выше, Dart 3.9 и выше). В качестве интегрированной среды разработки (IDE) программы должна быть использована среда Visual Studio Code или Android Studio.

Административная часть: Исходные коды веб-панели управления должны быть реализованы на языке JavaScript (ES-модули) с использованием Firebase Web SDK, без сборщика проекта. Правила безопасности базы данных должны быть описаны на языке правил Cloud Firestore. В качестве среды разработки должна быть использована среда Visual Studio Code.

4.5.3. Требования к программным средствам, используемым программой

Для обеспечения бесперебойного функционирования программы требуются следующие программные средства:

а) На клиентском мобильном устройстве: операционная система Android (версии 8.0 и выше) или операционная система iOS (версии 15.0 и выше).

б) На рабочем месте сотрудника: операционная система семейства Windows 10/11, macOS или Linux; современный веб-браузер (Google Chrome, Safari, Mozilla Firefox или Yandex Browser актуальных версий) для доступа к веб-панели.

в) Облачные сервисы: проект Google Firebase с включенными Authentication (провайдеры «Электронная почта/пароль» и Google), Cloud Firestore и Hosting.

г) Для разработки и тестирования: Flutter SDK, Node.js версии 20 и

выше, Firebase CLI, Java (для запуска локальных эмуляторов Firebase).

4.5.4. Требования к защите информации и программ

Права доступа к данным должны определяться ролями пользователей (администратор, менеджер, курьер, покупатель) и контролироваться правилами безопасности Cloud Firestore: покупатель видит только свой профиль и заказы, курьер – только свои активные заказы и пул свободных, изменение ролей и отключение учетных записей доступно только администратору, заказы физически не удаляются. Пароли хранятся и проверяются сервисом Firebase Authentication, номера банковских карт в базе данных не хранятся (сохраняется только маска). Веб-панель должна экранировать выводимые данные (защита от XSS).

4.6. Специальные требования

Специальные требования не предъявляются.

5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

5.1. Предварительный состав программной документации

Состав программной документации должен включать в себя:

- Приложение А. Техническое задание;
- Пояснительная записка;
- Приложение Б. Сценарий и результаты тестовых испытаний;
- Приложение В. Руководство пользователя;
- Приложение Г. Текст программы;
- Приложение Д. Скрипт базы данных;
- Приложение Е. Входные и выходные данные;
- Приложение Ж. Диаграмма классов.

5.2. Специальные требования к программной документации

Техническое задание оформлялось по ГОСТам: 19.201-78 и 19.106-78.

Поля макета технической записки:

правое поле – 20 мм,

левое поле – 35 мм

верхнее поле – 25 мм,

нижнее поле – 20 мм.

Текст должен быть отпечатан машинным способом на принтере на одной стороне листа.

Шрифт должен быть с "засечками". Рекомендуется Times New Roman. Цвет шрифта должен быть черным.

Размер шрифта (его высота в кеглях):

- для основного текста: 14,
- для заголовка 14,
- для таблиц, схем и приложений 12,
- для колонтитулов (к каждому листу ПЗ) 12.

Межстрочный интервал равен:

- для основного текста 1,5;
- для таблиц, схем и приложений 1.

Текст содержит отступ каждого абзаца - 1,25 см.

Основной текст работы (проекта) печатается 1,5 междустрочным интервалом компьютерного набора.

Выравнивание текста устанавливается «По ширине страницы».

6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

6.1. Ориентировочная экономическая эффективность

Ориентировочная экономическая эффективность не рассчитывается.

6.2. Предполагаемая годовая потребность

Предполагаемая годовая потребность не рассчитывается.

6.3. Экономические преимущества разработки

Экономические преимущества разработки не рассчитываются.

7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

7.1. Стадии разработки

Разработка должна быть проведена в соответствии со следующими стадиями:

- Анализ предметной области. Разработка технического задания;
- Проектирование. Раздел 2. Специальная часть ПЗ;
- Рабочий проект;
- Тестирование. Разработка сценария и результатов тестирования.
- Внедрение. Разработка Руководства пользователя и Текста программы.

7.2. Этапы разработки

Сроки сдачи этапов разработки формируются исходя из требований заказчика.

Окончательный срок сдачи всего проекта – 25.04.2026.

7.3. Содержание работ по этапам

На этапе разработки технического задания должны быть выполнены перечисленные ниже работы:

- Постановка задачи;
- Определение и уточнение требований к техническим средствам;
- Определение требований к программе;
- Определение стадий, этапов и сроков разработки программы и документации на неё;
- Выбор языков программирования;
- согласование и утверждение технического задания.

На этапе проектирования должны быть выполнены перечисленные ниже работы:

- Предварительная разработка входных и выходных данных;
- Уточнение методов решения задачи;

- Разработка общего описания алгоритма решения задач;
- Согласование и утверждения схем проектирования.

На этапе подготовки готового программного проекта должна быть выполнена работа по подготовке и передаче программы и программной документации в эксплуатацию.

На этапе разработки пояснительной записки должны быть выполнены перечисленные ниже работы:

- Общая часть;
- Специальная часть;
- Технологическая часть;
- Заключение.

На этапе разработки результатов тестовых испытаний должны быть выполнены перечисленные ниже виды работ:

- Разработка, согласование и утверждение программы и методики испытаний;
- Проведение приемо-сдаточных испытаний;
- Корректировка программы и программной документации по результатам испытаний.

На этапе разработки руководства пользователя должны быть описаны следующие пункты:

- Описание установки информационной системы;
- Описание эксплуатации информационной системы;
- Описание деинсталляции информационной системы.

На этапе подготовки скрипта базы данных и текста программы необходимо выполнить работу по созданию соответствующего приложения.

7.4. Исполнители

Руководитель разработки

Парамонова Е.М.

Исполнитель

Веников И. Н.

8. ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ

8.1. Порядок контроля

Контроль выполнения работ проводится исполнителем проекта и заказчиком проекта в момент завершения каждого этапа /стадии.

Стадии разработки проекта указаны в разделе «7.1 Стадии разработки».

8.2. Порядок приемки

Исполнитель передает заказчику техническую документацию и разработанный проект в электронном виде.

Все документы должны быть подписаны исполнителем проекта и утверждены заказчиком.

Состав документации определен в разделе 5.1. «Предварительный состав программной документации».

ПРИЛОЖЕНИЕ Б. СЦЕНАРИЙ И РЕЗУЛЬТАТЫ ТЕСТОВЫХ ИСПЫТАНИЙ

АННОТАЦИЯ

В данном программном документе приведены сценарии тестовых испытаний и результаты тестовых испытаний.

В разделе «Цель испытаний» указана цель проведения испытаний.

В разделе «Объект испытаний» указаны наименование работы и цель её выполнения.

В разделе «Требования к тестированию» указаны функциональные возможности, подлежащие проверке во время испытаний и заданные в пояснительной записке на программу.

В разделе «Средства и порядок испытаний» указаны технические и программные средства, используемые во время испытаний, а также порядок проведения испытаний.

В разделе «Методы испытаний» приведено описание используемых методов испытаний.

В разделе «Тестовые примеры и результаты» приведены таблицы с тестовыми сценариями и результатами тестовых испытаний.

СОДЕРЖАНИЕ

1. ЦЕЛЬ ИСПЫТАНИЙ.....	3
2. ОБЪЕКТ ИСПЫТАНИЙ.....	3
2.1 Наименование объекта.....	3
2.2 Область применения объекта.....	3
2.3 Обозначение испытываемой программы.....	3
3. ТРЕБОВАНИЯ К ТЕСТИРОВАНИЮ.....	3
3.1 Схема тестирования.....	3
3.2 Требования, подлежащие проверке.....	4
4. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ.....	5
5. МЕТОДЫ ИСПЫТАНИЙ.....	6
6. ТЕСТОВЫЕ ПРИМЕРЫ И РЕЗУЛЬТАТЫ.....	7
6.1 Возможность переключения между страницами в соответствии с ролями доступа.....	7
6.2 Возможность авторизации, регистрации и управления доступом	9
6.3 Клиентские функциональные возможности.....	12
6.4 Функциональные возможности курьера.....	16
6.5 Функциональные возможности администратора и менеджера...	17

1. ЦЕЛЬ ИСПЫТАНИЙ

Целью проведения испытаний является проверка соответствия разработанного программного изделия требованиям, изложенным в документе «Пояснительная записка» для разработки мобильного приложения и веб-панели управления «Интернет-магазин продуктов «Нектар»

2. ОБЪЕКТ ИСПЫТАНИЙ

2.1. Наименование объекта

Наименование – мобильное приложение и веб-панель управления «Интернет-магазин продуктов «Нектар».

2.2. Область применения объекта

Программа предназначена для использования в организации, осуществляющей розничную продажу продуктов питания с доставкой, путем перевода процессов товарооборота и взаимодействия с покупателями в электронный вид.

2.3. Обозначение испытываемой программы

Обозначение – «Интернет-магазин продуктов «Нектар».

3. ТРЕБОВАНИЯ К ТЕСТИРОВАНИЮ

3.1. Схема тестирования

Тестирование разработанного программного решения проводилось по схеме тестирования, представленной на рисунке 1.

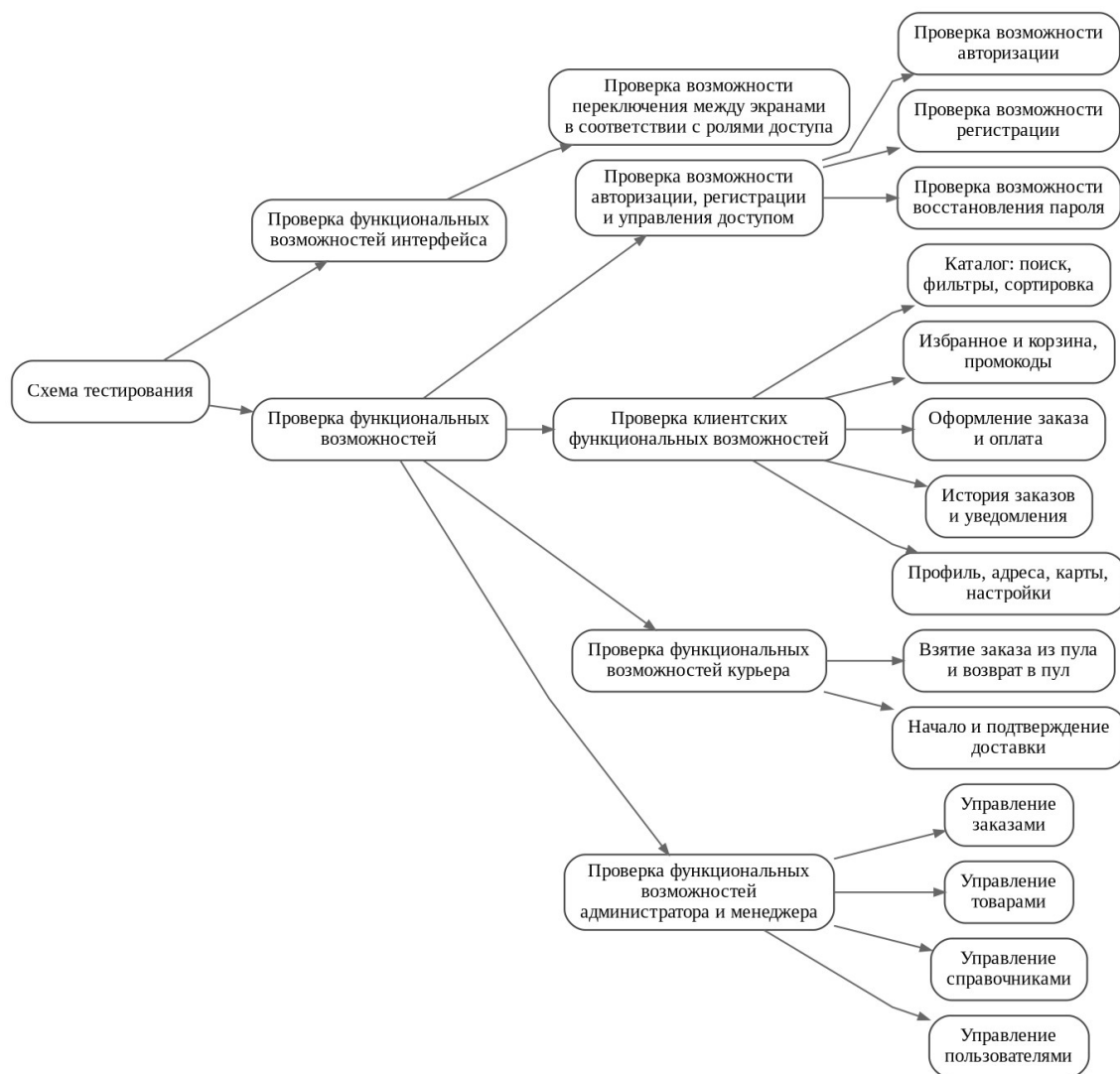


Рисунок 1 – Схема тестирования

3.2. Требования, подлежащие проверке

Функциональные возможности, подлежащие проверке при проведении тестирования:

- Функциональные возможности интерфейса: проверка возможности навигации по экранам приложения и разделам веб-панели в зависимости от ролей доступа, поиск товаров, фильтрация по производителям и ценам, сортировка товаров.
- Функциональные возможности авторизации и регистрации: проверка регистрации учетной записи с валидацией, авторизации по логину или email и паролю, восстановления и смены пароля.
- Функциональные возможности клиента: добавление товара в корзину и в избранное, применение промокода, оформление заказа с

выбором адреса, времени доставки и способа оплаты, просмотр истории заказов и уведомлений, редактирование профиля, адресов и карт, изменение языка интерфейса.

- Функциональные возможности курьера: просмотр свободных заказов, взятие заказа и возврат его в пул, начало и подтверждение доставки.

- Функциональные возможности администратора и менеджера: авторизация в веб-панели, просмотр показателей, управление заказами клиентов (изменение статусов, назначение курьера), управление каталогом товаров и остатками, управление категориями, производителями, поставщиками и промокодами, управление пользователями и их ролями.

4. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

В качестве средств вычислительной техники при разработке и тестировании ПО использовался персональный компьютер разработчика. Технические и программные средства испытаний представлены в таблице 1.

Таблица 1 – Технические и программные средства испытаний

№	Тип средства	Название средства
Персональный компьютер разработчика		
1	Процессор:	Тактовая частота не ниже 2.0 ГГц, 64-разрядный
2	Оперативная память:	Не менее 8 ГБ
3	Накопитель:	SSD, свободно не менее 10 ГБ
Программные средства		
4	Мобильная платформа:	Flutter 3 (Dart 3.9), эмулятор Android
5	Веб-браузер:	Google Chrome (веб-панель управления)
6	Серверная часть:	Google Firebase (Authentication, Cloud Firestore, Hosting), локальные эмуляторы Firebase
7	Автоматические тесты:	flutter_test, fake_cloud_firestore, Node.js (node:test), @firebase/rules-unit-testing

Испытания проводятся поэтапно согласно разделу «Требования к тестированию» настоящего документа. Тестовые учетные записи (пароль

Test1234): admin@nectar.test, manager@nectar.test, courier@nectar.test, buyer@nectar.test.

5. МЕТОДЫ ИСПЫТАНИЙ

По формальности тестирования.

Тестирование по тестам – тестирование по предварительно написанным тест-кейсам.

По исполнению кода.

Динамическое тестирование - во время тестирования код выполняется.

По уровню тестирования.

Системное – проверка работы всей системы на соответствие заявленным требованиям к программному продукту.

По целям.

Функциональное тестирование направлено на проверку того, какие функции ПО реализованы, и того, насколько верно они реализованы.

По степени автоматизации.

Ручное – без использования дополнительных программных средств автоматизации.

Автоматизированное – выполнение модульных тестов Dart, тестов веб-панели и тестов правил безопасности Firestore на локальном эмуляторе.

По позитивности сценария.

Позитивное – проверка ПО на соответствие ожидаемому поведению.

Негативное – проверяет, будет ли ПО работать в случае, когда поведение пользователя отличается от ожидаемого (ошибки ввода).

По знанию системы.

Тестирование «черного ящика» – тестирование без доступа к коду продукта в момент прохождения тест-кейса.

По разработке тестовых испытаний.

На основе требований – требование было определено до начала тестирования.

6. ТЕСТОВЫЕ ПРИМЕРЫ И РЕЗУЛЬТАТЫ

6.1. Возможность переключения между страницами в соответствии с ролями доступа

Таблица 2 – Тест перехода по страницам в соответствии с ролями доступа

№ п/п	Страница / Экран	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
1	Экран «Заставка»	Отображается при запуске мобильного приложения любому пользователю	Соответствует ожидаемому. Тест пройден.
2	Экран «Онбординг» (приветствие)	Отображается неавторизованному пользователю при запуске приложения	Соответствует ожидаемому. Тест пройден.
3	Экран «Вход»	Доступен только неавторизованным пользователям	Соответствует ожидаемому. Тест пройден.
4	Экран «Регистрация»	Доступен только неавторизованным пользователям	Соответствует ожидаемому. Тест пройден.
5	Экран «Восстановление пароля»	Доступен только неавторизованным пользователям	Соответствует ожидаемому. Тест пройден.
6	Экран «Магазин» (популярное, товары по категориям)	Доступен только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
7	Экран «Каталог»	Доступен только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
8	Экран «Товары категории»	Доступен только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
9	Экран «Карточка товара»	Доступен только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
10	Экран «Избранное»	Доступен только авторизованным	Соответствует ожидаемому. Тест

№ п/п	Страница / Экран	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
		покупателям	пройден.
11	Экран «Корзина»	Доступен только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
12	Окно «Оформление заказа»	Доступно только авторизованным покупателям при непустой корзине	Соответствует ожидаемому. Тест пройден.
13	Экран «Профиль»	Доступен только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
14	Экран «Мои заказы»	Доступен только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
15	Экран «Мои данные»	Доступен только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
16	Экран «Адрес доставки»	Доступен только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
17	Экран «Способы оплаты»	Доступен только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
18	Экран «Промокоды»	Доступен только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
19	Окно «Смена пароля»	Доступно только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
20	Экран «Уведомления»	Доступен только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
21	Экраны «Помощь» и «О приложении»	Доступны только авторизованным покупателям	Соответствует ожидаемому. Тест пройден.
22	Экран «Доставка» (мои заказы, свободные)	Доступен только курьерам	Соответствует ожидаемому. Тест пройден.
23	Мобильное приложение при входе администратора или менеджера	Вывод сообщения о необходимости использовать веб-панель	Соответствует ожидаемому. Тест пройден.
24	Веб-панель. Форма входа	Доступна всем; вход разрешен только ролям «Администратор» и «Менеджер»	Соответствует ожидаемому. Тест пройден.
25	Веб-панель при входе	Вывод сообщения	Соответствует

№ п/п	Страница / Экран	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
	покупателя или курьера	«Недостаточно прав для входа в web-панель», выход из системы	ожидаемому. Тест пройден.
26	Веб-панель. Разделы «Дашборд», «Товары», «Категории», «Производители», «Промокоды», «Заказы»	Доступны администраторам и менеджерам	Соответствует ожидаемому. Тест пройден.
27	Веб-панель. Раздел «Поставщики»	Менеджер только просматривает, изменяет администратор	Соответствует ожидаемому. Тест пройден.
28	Веб-панель. Раздел «Пользователи»	Менеджер только просматривает, роли и отключение меняет администратор	Соответствует ожидаемому. Тест пройден.

6.2. Возможность авторизации, регистрации и управления доступом

Таблица 3 – Возможность авторизации, регистрации и управления доступом

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
Авторизация и выход			
1	Логин: nonexistent, Пароль: Password1	Вывод сообщения об ошибке: «Пользователь с таким логином не найден.»	Соответствует ожидаемому. Тест пройден.
2	Логин или email: buyer@nectar.test, Пароль: WrongPass1	Вывод сообщения: «Неверный email или пароль.»	Соответствует ожидаемому. Тест пройден.
3	Логин или email: пустота, Пароль: пустота	Вывод в полях сообщений: «Введите логин или email», «Введите пароль»	Соответствует ожидаемому. Тест пройден.
4	Логин или email: bad@, Пароль: Test1234	Вывод сообщения: «Логин или email указан некорректно»	Соответствует ожидаемому. Тест пройден.
5	Email: blocked@nectar.test (отключенный пользователь), Пароль: Test1234	Вывод сообщения: «Этот аккаунт был удален.», выход из системы	Соответствует ожидаемому. Тест пройден.
6	Email: buyer@nectar.test, Пароль: Test1234	Успешная авторизация, переход на экран «Магазин»	Соответствует ожидаемому. Тест пройден.
7	Логин: ivan_venikov, Пароль: Strong12 (после регистрации)	Успешная авторизация по логину (email определяется по индексу логинов)	Соответствует ожидаемому. Тест пройден.

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
8	Email: courier@nectar.test, Пароль: Test1234	Успешная авторизация, открывается экран «Доставка»	Соответствует ожидаемому. Тест пройден.
9	Email: admin@nectar.test, Пароль: Test1234 (мобильное приложение)	Вывод сообщения: «Роль «Администратор» не может работать в мобильном приложении», кнопка «Выйти из аккаунта»	Соответствует ожидаемому. Тест пройден.
10	Нажатие кнопки «Выйти» в профиле	Сессия завершается, пользователь перенаправляется на экран входа, корзина очищается	Соответствует ожидаемому. Тест пройден.
Регистрация			
11	Логин: buyer (уже существует)	Вывод сообщения: «Пользователь с таким логином уже существует.»	Соответствует ожидаемому. Тест пройден.
12	Email: buyer@nectar.test (уже существует)	Вывод сообщения: «Этот email уже используется.»	Соответствует ожидаемому. Тест пройден.
13	Все поля: пустота	Вывод сообщений валидации об обязательности заполнения полей	Соответствует ожидаемому. Тест пройден.
14	Логин: ab	Вывод сообщения: «Длина логина от 3 до 50 символов»	Соответствует ожидаемому. Тест пройден.
15	Логин: иван	Вывод сообщения: «Только латиница, цифры и символы . _ -»	Соответствует ожидаемому. Тест пройден.
16	Имя: А (1 символ)	Вывод сообщения: «Длина от 2 до 50 символов»	Соответствует ожидаемому. Тест пройден.
17	Имя: Иван123	Вывод сообщения: «Только буквы, пробелы и дефисы»	Соответствует ожидаемому. Тест пройден.
18	Телефон: 12345	Вывод сообщения: «Неверный формат номера телефона»	Соответствует ожидаемому. Тест пройден.
19	Email: login_login	Вывод сообщения: «Неверный формат email»	Соответствует ожидаемому. Тест пройден.
20	Пароль: pas	Вывод сообщения: «Минимум 6 символов»	Соответствует ожидаемому. Тест пройден.
21	Пароль: password (без цифр)	Вывод сообщения: «Пароль должен содержать минимум одну цифру»	Соответствует ожидаемому. Тест пройден.

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
22	Пароль: pass word1 (с пробелом)	Вывод сообщения: «Пароль не должен содержать пробелы»	Соответствует ожидаемому. Тест пройден.
23	Подтверждение пароля не совпадает с паролем	Вывод сообщения: «Пароли не совпадают»	Соответствует ожидаемому. Тест пройден.
24	Логин: ivan_venikov, Имя: Иван, Телефон: +7 900 123-45-67, Email: ivan@mail.ru, Пароль: Strong12	Успешная регистрация, назначение роли «Покупатель», автоматический вход в приложение	Соответствует ожидаемому. Тест пройден.
Восстановление и смена пароля			
25	Email: пустота	Вывод сообщения: «Введите email»	Соответствует ожидаемому. Тест пройден.
26	Email: incorrect	Вывод сообщения: «Неверный формат email»	Соответствует ожидаемому. Тест пройден.
27	Email: ivan@mail.ru	Вывод окна «Письмо отправлено!»; на почту отправляется письмо со ссылкой для создания нового пароля	Соответствует ожидаемому. Тест пройден.
28	Смена пароля из профиля: текущий пароль неверный	Вывод сообщения: «Неверный текущий пароль.»	Соответствует ожидаемому. Тест пройден.
29	Смена пароля: новый пароль совпадает с текущим	Вывод сообщения: «Новый пароль совпадает с текущим»	Соответствует ожидаемому. Тест пройден.
30	Смена пароля: текущий Strong12, новый NewPass123, подтверждение NewPass123	Вывод сообщения: «Пароль успешно изменен.»; старый пароль больше недействителен	Соответствует ожидаемому. Тест пройден.
Безопасность и сессии			
31	Перезапуск приложения при активной сессии	Сессия восстанавливается, пользователь попадает на экран своей роли без повторного ввода пароля	Соответствует ожидаемому. Тест пройден.
32	Отключение пользователя администратором во время сессии	При следующем входе выводится сообщение об удаленном аккаунте, доступ к данным закрыт правилами безопасности	Соответствует ожидаемому. Тест пройден.
33	Попытка покупателя прочитать чужой заказ или профиль (правила	Операция отклонена правилами безопасности (автотесты правил)	Соответствует ожидаемому. Тест пройден.

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
	Firestore)		

6.3. Клиентские функциональные возможности

Таблица 4 – Клиентские функциональные возможности

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
Магазин, каталог, поиск и фильтрация			
1	Открытие экрана «Магазин»	Отображаются баннер со скидкой, строка поиска, блок «Популярное» и товары по категориям	Соответствует ожидаемому. Тест пройден.
2	Ввод в поиск экрана «Магазин»: Молоко	Отображаются только товары, в названии которых есть «Молоко»	Соответствует ожидаемому. Тест пройден.
3	Ввод в поиск: несуществующий_товар	Отображается сообщение «По вашему запросу ничего не найдено»	Соответствует ожидаемому. Тест пройден.
4	Открытие экрана «Каталог»	Отображаются плитки категорий товаров	Соответствует ожидаемому. Тест пройден.
5	Выбор категории в каталоге	Открывается список товаров выбранной категории	Соответствует ожидаемому. Тест пройден.
6	Открытие фильтров, выбор производителя	Отображаются только товары выбранного производителя	Соответствует ожидаемому. Тест пройден.
7	Фильтр по цене (ползунок «от – до»)	В списке остаются только товары в заданном ценовом диапазоне	Соответствует ожидаемому. Тест пройден.
8	Сортировка: По возрастанию цены	Товары выстраиваются в порядке возрастания цены	Соответствует ожидаемому. Тест пройден.
9	Сортировка: По убыванию цены	Товары выстраиваются в порядке убывания цены	Соответствует ожидаемому. Тест пройден.
10	Сортировка: Сначала свежие	Товары выстраиваются по сроку годности (позднейший срок первым)	Соответствует ожидаемому. Тест пройден.
11	Фильтры, под которые нет товаров	Отображается сообщение «Нет товаров, подходящих под выбранные фильтры»	Соответствует ожидаемому. Тест пройден.
12	Кнопка «Сбросить» в фильтрах	Фильтры и сортировка сбрасываются, отображается	Соответствует ожидаемому. Тест

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
		полный список	пройден.
13	Открытие карточки товара	Отображаются фото, название, цена, остаток, описание, характеристики и пищевая ценность	Соответствует ожидаемому. Тест пройден.
Избранное и корзина			
14	Переход в «Избранное» при отсутствии товаров	Отображается сообщение «У вас пока нет любимых товаров»	Соответствует ожидаемому. Тест пройден.
15	Нажатие на значок «сердце» в карточке товара	Товар добавляется в избранное текущего пользователя (массив favorites профиля)	Соответствует ожидаемому. Тест пройден.
16	Повторное нажатие на значок «сердце»	Товар удаляется из избранного	Соответствует ожидаемому. Тест пройден.
17	Кнопка «В корзину всё» в избранном	Товары в наличии добавляются в корзину, выводится сообщение с количеством	Соответствует ожидаемому. Тест пройден.
18	Переход в «Корзину» при отсутствии товаров	Отображается сообщение «Ваша корзина пуста»	Соответствует ожидаемому. Тест пройден.
19	Выбор количества 2 и нажатие «В корзину · цена»	Выводится сообщение «Товар × 2 добавлен в корзину», позиция появляется в корзине	Соответствует ожидаемому. Тест пройден.
20	Добавление товара больше остатка на складе	Добавляется весь доступный остаток: «Добавлено N шт. — это весь доступный остаток»	Соответствует ожидаемому. Тест пройден.
21	Товар с нулевым остатком	Кнопка недоступна, отображается «Распродано» / «Нет в наличии»	Соответствует ожидаемому. Тест пройден.
22	В корзине: увеличение количества (+)	Количество увеличивается, итоговая сумма пересчитывается; при достижении остатка вывод «Больше нет в наличии»	Соответствует ожидаемому. Тест пройден.
23	В корзине: уменьшение количества до 0 (–) или нажатие «×»	Товар удаляется из корзины	Соответствует ожидаемому. Тест пройден.
24	Проверка автоматического расчета стоимости (подытог 370 Р)	Итог = подытог 370 Р + доставка 149 Р = 519 Р (доставка бесплатна от 1500 Р)	Соответствует ожидаемому. Тест пройден.

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
25	Ввод промокода: INVALID	Вывод уведомления: «Неверный или просроченный промокод.»	Соответствует ожидаемому. Тест пройден.
26	Ввод промокода: пустота	Вывод сообщения: «Введите промокод»	Соответствует ожидаемому. Тест пройден.
27	Ввод промокода: SALE10 (10 %)	Промокод применен, отображается «Скидка 10 %», итоговая сумма пересчитывается	Соответствует ожидаемому. Тест пройден.
Оформление заказа			
28	Нажатие «Оформить заказ» без сохраненного адреса	Вывод сообщения: «Добавьте адрес в профиле»; оформление невозможно	Соответствует ожидаемому. Тест пройден.
29	Способ оплаты «Онлайн» без выбранной карты	Вывод сообщения: «Для онлайн-оплаты выберите сохраненную карту.»	Соответствует ожидаемому. Тест пройден.
30	Выбор адреса, интервала доставки и оплаты «Наличными»	Заказ оформляется без выбора карты	Соответствует ожидаемому. Тест пройден.
31	Выбор адреса, времени, оплаты «Онлайн» и сохраненной карты, нажатие «Разместить заказ»	Создается заказ со статусом «Новый», остатки товаров списываются, корзина очищается, отображается «Заказ успешно оформлен!» с номером	Соответствует ожидаемому. Тест пройден.
32	Оформление заказа с промокодом	Скидка сохраняется в заказе, счетчик использований промокода увеличивается	Соответствует ожидаемому. Тест пройден.
33	Оформление заказа на количество больше остатка (остаток изменился)	Заказ не создается, выводится «Недостаточно товара на складе»	Соответствует ожидаемому. Тест пройден.
История заказов, уведомления			
34	Переход на экран «Мои заказы» без заказов	Отображается «Пока нет заказов» и кнопка «За покупками»	Соответствует ожидаемому. Тест пройден.
35	Переход на экран «Мои заказы»	Отображается история заказов: номер, дата, статус, шаги «Создан – У курьера – Доставлен»	Соответствует ожидаемому. Тест пройден.
36	Раскрытие карточки заказа	Отображаются адрес, время доставки, оплата, курьер и его телефон, состав заказа и итоговая сумма	Соответствует ожидаемому. Тест пройден.
37	Заказ со статусом	Отображается пометка	Соответствует

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
	«Отменен»	«Заказ отменен»	ожидаемому. Тест пройден.
38	Нажатие «Подтвердить, что всё хорошо» у заказа у курьера	Выводится «Вы подтвердили, что всё хорошо!», кнопка заменяется надписью «Вы подтвердили получение. Ожидаем менеджера.»	Соответствует ожидаемому. Тест пройден.
39	Нажатие «Скрыть из списка» у доставленного или отмененного заказа	Заказ скрывается из списка покупателя (в базе остается)	Соответствует ожидаемому. Тест пройден.
40	Изменение статуса заказа менеджером	В разделе «Уведомления» появляется сообщение об изменении статуса заказа	Соответствует ожидаемому. Тест пройден.
41	Кнопка «Очистить все» в уведомлениях	История уведомлений очищается, выводится «Все уведомления очищены.»	Соответствует ожидаемому. Тест пройден.
Профиль и настройки			
42	Просмотр экрана «Профиль»	Отображаются аватар, имя, email и роль «Покупатель», меню разделов	Соответствует ожидаемому. Тест пройден.
43	«Мои данные»: смена имени и телефона	Выводится «Данные успешно сохранены», новые значения сохраняются в базе; email недоступен для изменения	Соответствует ожидаемому. Тест пройден.
44	«Мои данные»: имя из 1 символа	Вывод сообщения: «Длина от 2 до 50 символов»	Соответствует ожидаемому. Тест пройден.
45	Загрузка нового аватара (JPEG/PNG)	Изображение сжимается, сохраняется в профиле, выводится «Аватар успешно обновлен.»	Соответствует ожидаемому. Тест пройден.
46	Загрузка изображения неподдерживаемого формата	Вывод сообщения: «Разрешены только изображения JPEG и PNG.»	Соответствует ожидаемому. Тест пройден.
47	«Адрес доставки»: добавление адреса «Москва, Тверская 1»	Адрес сохраняется и предлагается при оформлении заказа	Соответствует ожидаемому. Тест пройден.
48	«Адрес доставки»: пустой адрес	Вывод сообщения об обязательности поля	Соответствует ожидаемому. Тест пройден.
49	«Способы оплаты»: добавление карты 1111 2222 3333 4444	Сохраняется маска «**** **** 4444» (полный номер не хранится)	Соответствует ожидаемому. Тест пройден.
50	«Способы оплаты»: номер из 15 цифр	Вывод сообщения: «Номер карты должен содержать 16	Соответствует ожидаемому. Тест

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
		цифр»	пройден.
51	«Промокоды»: просмотр	Отображаются доступные промокоды со скидкой и сроком действия	Соответствует ожидаемому. Тест пройден.
52	«Уведомления»: переключение Push/SMS/Email	Настройка сохраняется в профиле	Соответствует ожидаемому. Тест пройден.
53	Переключение языка на EN (переключатель RU/EN)	Все надписи приложения заменяются английскими, выбор сохраняется на устройстве	Соответствует ожидаемому. Тест пройден.
54	«Удалить аккаунт» при наличии незавершенных заказов	Вывод сообщения: «Нельзя удалить пользователя с незавершенными заказами.»	Соответствует ожидаемому. Тест пройден.
55	«Удалить аккаунт» без незавершенных заказов, подтверждение	Аккаунт отключается, выполняется выход из системы	Соответствует ожидаемому. Тест пройден.

6.4. Функциональные возможности курьера

Таблица 5 – Функциональные возможности курьера

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
Вход и просмотр заказов			
1	Вход курьера, вкладка «Свободные»	Отображаются заказы без курьера со статусом «Новый» или «В сборке»: адрес, время, состав, оплата и сумма	Соответствует ожидаемому. Тест пройден.
2	Вкладка «Свободные» при отсутствии заказов	Сообщение «Свободных заказов нет. Новые заказы появятся здесь сами.»	Соответствует ожидаемому. Тест пройден.
3	Вкладка «Мои заказы»	Отображаются блоки «Активные заказы» и «История доставок»	Соответствует ожидаемому. Тест пройден.
4	Вкладка «Мои заказы» без назначенных заказов	Сообщение «Активных заказов нет. Новые назначения появятся здесь.»	Соответствует ожидаемому. Тест пройден.
Работа с заказом			
5	Нажатие «Взять заказ»	Заказ становится «Передан курьеру», назначается на текущего курьера, выводится «Заказ взят в работу»	Соответствует ожидаемому. Тест пройден.

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
6	Два курьера берут один заказ одновременно	Заказ получает первый; второму выводится «Этот заказ уже взял другой курьер.»	Соответствует ожидаемому. Тест пройден.
7	Нажатие «Вернуть в пул» и подтверждение	Заказ возвращается в статус «В сборке» без курьера, выводится «Заказ возвращен в пул»	Соответствует ожидаемому. Тест пройден.
8	Возврат заказа, который уже в пути	Кнопка недоступна; при попытке выводится «Заказ нельзя вернуть: он уже в пути или назначен не вам.»	Соответствует ожидаемому. Тест пройден.
9	Нажатие на телефон клиента	Телефон копируется в буфер обмена, выводится «Телефон скопирован»	Соответствует ожидаемому. Тест пройден.
10	Заказ с оплатой «Наличными»	Отображается «К оплате наличными: сумма»	Соответствует ожидаемому. Тест пройден.
11	Нажатие «Начать доставку»	Статус заказа меняется на «В пути», покупателю отправляется уведомление	Соответствует ожидаемому. Тест пройден.
12	Нажатие «Подтвердить доставку» и подтверждение	Статус меняется на «Доставлен», заказ переходит в «История доставок»	Соответствует ожидаемому. Тест пройден.
13	Отмена подтверждения доставки	Статус заказа не изменяется	Соответствует ожидаемому. Тест пройден.
14	Отмененный менеджером заказ	Отображается пометка «Заказ отменен, доставка не нужна.»	Соответствует ожидаемому. Тест пройден.

6.5. Функциональные возможности администратора и менеджера

Таблица 6 – Функциональные возможности администратора и менеджера

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
Авторизация и выход			
1	Вход администратора: admin@nectar.test / Test1234	Успешный вход, отображаются разделы панели и роль «Администратор»	Соответствует ожидаемому. Тест пройден.
2	Вход менеджера: manager@nectar.test /	Успешный вход, отображается роль	Соответствует ожидаемому. Тест

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
	Test1234	«Менеджер»	пройден.
3	Пустые поля email и пароль	Вывод сообщения: «Введите email и пароль.»	Соответствует ожидаемому. Тест пройден.
4	Неверный пароль	Вывод сообщения: «Не удалось выполнить вход. Проверьте email и пароль.»	Соответствует ожидаемому. Тест пройден.
5	Вход покупателя buyer@nectar.test в веб- панель	Вывод сообщения: «Недостаточно прав для входа в web-панель.», выход из системы	Соответствует ожидаемому. Тест пройден.
6	Нажатие кнопки «Выйти»	Завершение сессии, отображается форма входа	Соответствует ожидаемому. Тест пройден.
7	Смена пароля: неверный текущий пароль	Вывод сообщения: «Неверный текущий пароль.»	Соответствует ожидаемому. Тест пройден.
8	Смена пароля: корректные значения	Вывод сообщения: «Пароль успешно изменен.»	Соответствует ожидаемому. Тест пройден.
Дашборд			
9	Открытие раздела «Дашборд»	Отображаются показатели: покупатели, активные заказы, товары с низким остатком, общая выручка	Соответствует ожидаемому. Тест пройден.
10	Просмотр графиков	Отображаются «Продажи по месяцам» и «ТОП-5 товаров»	Соответствует ожидаемому. Тест пройден.
Управление заказами			
11	Просмотр раздела «Заказы»	Выводится таблица: номер, клиент, сумма, статус, курьер, действия	Соответствует ожидаемому. Тест пройден.
12	Смена статуса «Новый» на «В сборке» и нажатие «Сохранить»	Статус сохраняется, покупателю создается уведомление, выводится «Заказ обновлен.»	Соответствует ожидаемому. Тест пройден.
13	Статус «Передан курьеру» / «В пути» без курьера	Вывод сообщения: «Для этого статуса назначьте курьера.»	Соответствует ожидаемому. Тест пройден.
14	Назначение курьера и статус «Передан курьеру»	Курьер видит заказ в «Мои заказы» и получает уведомление «Вам назначен заказ»	Соответствует ожидаемому. Тест пройден.
15	Отмена заказа (с подтверждением)	Статус «Отменен», товары возвращаются на склад, повторный возврат невозможен	Соответствует ожидаемому. Тест пройден.

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
16	Попытка изменить доставленный или отмененный заказ	Элементы блокируются, отображается «Заказ завершен»	Соответствует ожидаемому. Тест пройден.
Управление товарами			
17	Просмотр раздела «Товары», поиск по названию	Отображаются товары; при поиске остаются подходящие по названию	Соответствует ожидаемому. Тест пройден.
18	Создание товара: корректные данные	Выводится «Товар сохранен.», товар доступен в мобильном приложении	Соответствует ожидаемому. Тест пройден.
19	Создание товара: цена 0	Вывод сообщения: «Цена должна быть больше нуля.»	Соответствует ожидаемому. Тест пройден.
20	Создание товара: количество 10000	Вывод сообщения: «Поле «Количество» должно быть от 0 до 9999.»	Соответствует ожидаемому. Тест пройден.
21	Создание товара: ссылка на изображение без http(s)	Вывод сообщения: «Ссылка на изображение должна начинаться с http:// или https://.»	Соответствует ожидаемому. Тест пройден.
22	Изменение цены и остатка товара	Новые значения сохраняются и отображаются в приложении	Соответствует ожидаемому. Тест пройден.
23	Нажатие «Скрыть» и подтверждение	Товар скрывается из каталога приложения, в таблице помечается как скрытый	Соответствует ожидаемому. Тест пройден.
24	Нажатие «Восстановить»	Товар снова доступен в каталоге	Соответствует ожидаемому. Тест пройден.
Справочники и промокоды			
25	Добавление категории: пустое название	Вывод сообщения: «Поле «Название» обязательно.»	Соответствует ожидаемому. Тест пройден.
26	Добавление, изменение категории	Запись сохраняется («Изменения сохранены.»); при переименовании название обновляется у товаров	Соответствует ожидаемому. Тест пройден.
27	Добавление категории с существующим названием	Вывод сообщения: «Категория с таким названием уже существует.»	Соответствует ожидаемому. Тест пройден.
28	Удаление категории, к которой привязаны товары	Вывод сообщения: «Нельзя удалить категорию, к которой привязаны продукты.»	Соответствует ожидаемому. Тест пройден.
29	Просмотр, добавление,	Запись корректно	Соответствует

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
	изменение, удаление производителя	сохраняется, обновляется и удаляется (если нет привязанных товаров)	ожидаемому. Тест пройден.
30	Поставщик: ИНН из 9 цифр	Вывод сообщения: «ИНН должен содержать 10 или 12 цифр.»	Соответствует ожидаемому. Тест пройден.
31	Поставщики: вход менеджером	Записи доступны только для просмотра, кнопки изменения отсутствуют	Соответствует ожидаемому. Тест пройден.
32	Добавление промокода: SALE-10	Вывод сообщения: «Код промокода: только латинские буквы и цифры.»	Соответствует ожидаемому. Тест пройден.
33	Добавление промокода: скидка 100 %	Вывод сообщения: «Скидка должна быть целым числом от 1 до 99.»	Соответствует ожидаемому. Тест пройден.
34	Добавление промокода со сроком в прошлом	Вывод сообщения: «Срок действия промокода уже истек.»	Соответствует ожидаемому. Тест пройден.
35	Добавление промокода: SPRING15, 15 %, срок в будущем	Промокод сохраняется и применяется в корзине покупателя	Соответствует ожидаемому. Тест пройден.
36	Удаление промокода, применённого в незавершенных заказах	Вывод сообщения: «Нельзя удалить промокод: он используется в незавершенных заказах.»	Соответствует ожидаемому. Тест пройден.
Управление пользователями			
37	Просмотр раздела «Пользователи»	Выводятся имя, логин, email, телефон, роль и действия	Соответствует ожидаемому. Тест пройден.
38	Смена роли покупателя на «Курьер» администратором	Роль обновляется («Роль пользователя обновлена.»), пользователь попадает в список курьеров	Соответствует ожидаемому. Тест пройден.
39	Смена роли пользователю с незавершенными заказами	Вывод сообщения: «У пользователя есть незавершенные заказы: смена роли сейчас невозможна.»	Соответствует ожидаемому. Тест пройден.
40	Смена собственной роли или роли последнего администратора	Вывод сообщения: «Нельзя изменить собственную роль.» / «Нельзя снять роль с последнего администратора.»	Соответствует ожидаемому. Тест пройден.
41	Нажатие «Отключить» и подтверждение	Пользователь отключается («Пользователь отключен.») и не может войти в приложение	Соответствует ожидаемому. Тест пройден.

№ п/п	Действие (входные данные)	Ожидаемый результат	Фактический результат. Статус теста
1	2	3	4
42	Флажок «Показывать отключенных», нажатие «Восстановить»	Отключенный пользователь снова получает доступ	Соответствует ожидаемому. Тест пройден.
43	Просмотр раздела «Пользователи» менеджером	Отображается «Только просмотр», изменение ролей недоступно	Соответствует ожидаемому. Тест пройден.

ПРИЛОЖЕНИЕ В. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

АННОТАЦИЯ

В данном программном документе приведено руководство пользователя на разработанную информационную систему интернет-магазина продуктов «Нектар».

В разделе «Условия использования программы» описаны рекомендуемая и минимальная конфигурация устройств, а также перечень программных средств, необходимых для эксплуатации мобильного приложения и веб-панели управления.

В разделе «Выполнение программы» содержатся подразделы:

1. Действия для запуска программы – описывает порядок установки и запуска мобильного приложения и веб-интерфейса.

2. Выполнение программы с описанием функций – содержит подробное иллюстрированное описание пользовательских сценариев для покупателя, курьера, администратора и менеджера системы.

В разделе «Сообщения оператору» приведены тексты сообщений, выдаваемых программой, их содержание и действия пользователя.

СОДЕРЖАНИЕ

1. УСЛОВИЯ ИСПОЛЬЗОВАНИЯ ПРОГРАММЫ.....	3
2. ВЫПОЛНЕНИЕ ПРОГРАММЫ.....	4
2.1 Действия для запуска программы.....	4
2.2 Выполнение программы с описанием функций.....	4
2.2.1 Выполнение программы «Неавторизованный пользователь».....	4
2.2.2 Выполнение программы «Авторизация, регистрация и восстановление пароля».....	6
2.2.3 Выполнение программы «Покупатель интернет-магазина».....	11
2.2.4 Выполнение программы «Курьер».....	30
2.2.5 Выполнение программы «Администратор и менеджер».....	34
3. СООБЩЕНИЯ ОПЕРАТОРУ.....	41

1. УСЛОВИЯ ИСПОЛЬЗОВАНИЯ ПРОГРАММЫ

В таблице 1 представлены рекомендуемые и минимальные технические требования к аппаратному обеспечению для использования разработанного программного комплекса

Таблица 1 – Технические требования к аппаратному обеспечению

№	Тип оборудования	Наименование оборудования
1	2	3
Требования для ПК администратора и менеджера		
1	Разрядность	x64 (64-bit)
2	Процессор	Тактовая частота процессора – от 2.0 ГГц (Intel Core i3 или аналог)
3	Оперативная память	4 GB
4	Накопитель	10 GB свободного дискового пространства
Требования для смартфона покупателя и курьера		
1	Операционная система	Android 8.0+ или iOS 15.0+
2	Процессор	4-ядерный процессор от 1.4 ГГц
3	Оперативная память [RAM]	2 GB
4	Накопитель	100 MB свободного пространства
5	Сеть	Wi-Fi или мобильный интернет (постоянный доступ)

В Таблице 2 представлены программные средства, необходимые для использования системы.

Таблица 2 – Программные средства

№	Тип средства	Название средства	Назначение
1	2	3	4
1	Операционная система мобильного устройства	Android, iOS	Запуск мобильного приложения покупателя и курьера
2	Операционная система ПК	Windows, Linux, macOS	Организация рабочего места администратора и менеджера
3	Веб-браузер	Google Chrome, Safari, Yandex Browser	Обеспечение доступа к веб-панели управления
4	Облачная платформа	Google Firebase (Authentication, Cloud Firestore, Hosting)	Хранение данных и авторизация (предоставляется сервисом, установки не требует)

2. ВЫПОЛНЕНИЕ ПРОГРАММЫ

2.1. Действия для запуска программы

Для запуска мобильного приложения пользователю необходимо скачать и установить «.apk» (для Android) или «.ipa» (для iOS) файл на свое устройство, после чего нажать на иконку приложения «nectar» на рабочем столе устройства.

Для запуска веб-панели управления необходимо открыть веб-браузер и перейти по адресу <https://nectar-41848.web.app>. Для входа используются учетные записи с ролями «Администратор» и «Менеджер».

2.2. Выполнение программы с описанием функций

2.2.1. Выполнение программы «Неавторизованный пользователь»

После запуска мобильного приложения на несколько секунд отображается экран загрузки с названием магазина (рисунок 1).



Рисунок 1 – Экран загрузки

При запуске приложения неавторизованному пользователю показывается экран приветствия из трех слайдов: «Гарантия свежести», «Доставка за час» и «Выгодные покупки». Переключатель RU/EN в левом

верхнем углу меняет язык приложения. Для перехода к следующему слайду используется кнопка «Далее», для пропуска приветствия – кнопка «Пропустить» (рисунок 2).

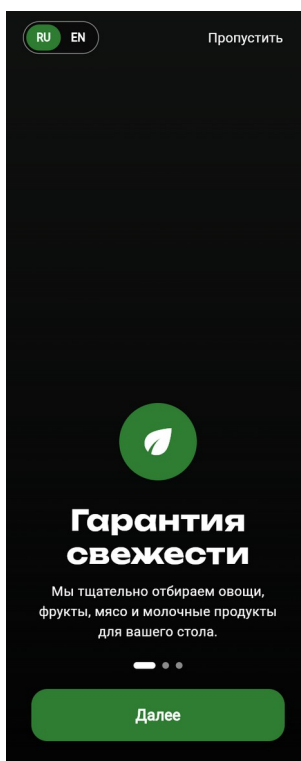


Рисунок 2 – Экран приветствия

На последнем слайде вместо кнопки «Далее» отображается кнопка «Начать», которая открывает экран входа (рисунок 3). Каталог товаров, корзина и профиль доступны только после авторизации.

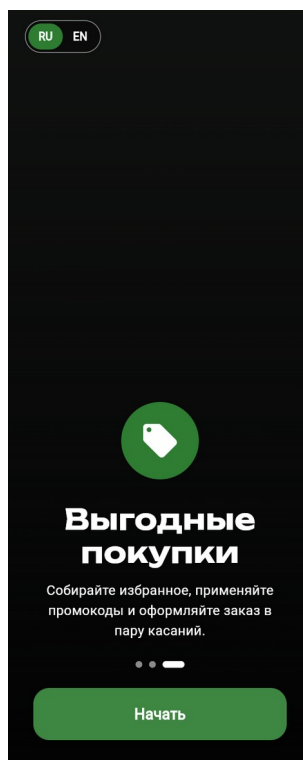


Рисунок 3 – Последний слайд приветствия

2.2.2. Выполнение программы «Авторизация, регистрация и восстановление пароля»

Для получения доступа к функционалу интернет-магазина пользователю необходимо пройти процедуру авторизации, введя логин или email и пароль. Значок «глаз» в поле пароля показывает или скрывает вводимые символы. Также предусмотрен вход через аккаунт Google (кнопка «Продолжить с Google») (рисунок 4).

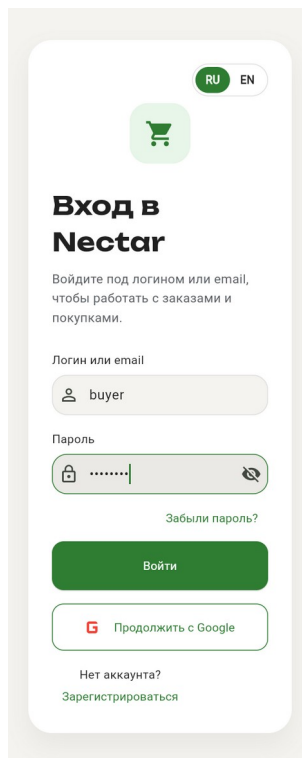


Рисунок 4 – Экран авторизации

При нажатии кнопки «Войти» с пустыми полями под ними выводятся сообщения об ошибках (рисунок 5).

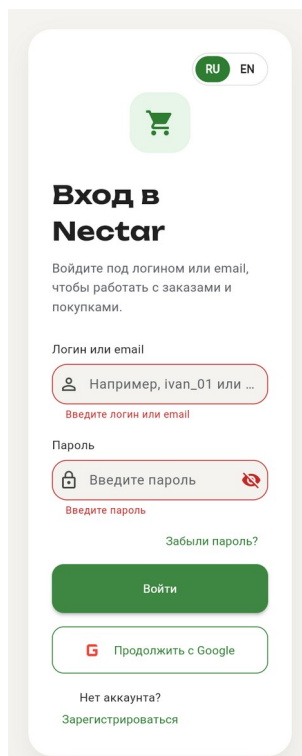


Рисунок 5 – Проверка полей формы авторизации

Если указанный логин не найден или пароль неверен, в нижней части экрана выводится сообщение об ошибке (рисунок 6).

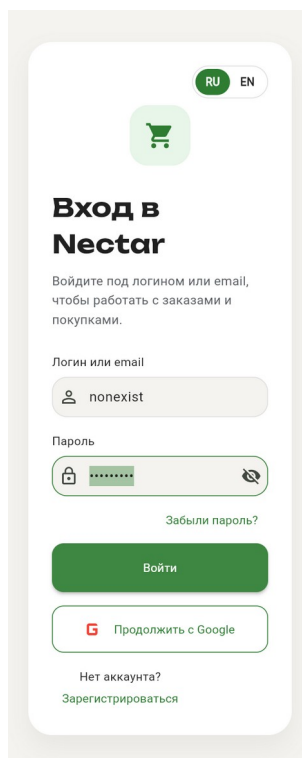


Рисунок 6 – Сообщение об ошибке авторизации

В случае отсутствия аккаунта пользователь нажимает «Зарегистрироваться» и заполняет форму: логин (латиница, цифры и символы . _ -, 3–50 символов), имя, телефон, email, пароль (не менее 6 символов, без пробелов, минимум одна цифра) и подтверждение пароля. После регистрации автоматически назначается роль «Покупатель», и выполняется вход (рисунок 7).

← Регистрация

Новый аккаунт покупателя

После регистрации вам
автоматически будет назначена
роль «Покупатель».

Логин

@ ivan_venikov

Имя

Иван

Телефон

+7 900 123-45-67

Email

ivan@mail.ru

Пароль

.....

Подтверждение пароля

.....

Зарегистрироваться

Уже есть аккаунт? Войти

Рисунок 7 – Экран регистрации

Поля, заполненные некорректно, подсвечиваются красным, под ними выводится описание ошибки (рисунок 8).

← Регистрация

Новый аккаунт покупателя

После регистрации вам
автоматически будет назначена
роль «Покупатель».

Логин

@ Например, ivan_venikov

Введите логин

Имя

Ваше имя

Поле не может быть пустым

Телефон

+7 900 123-45-67

Введите номер телефона

Email

example@mail.com

Введите email

Пароль

Минимум 6 символ...

Введите пароль

Подтверждение пароля

Повторите пароль

Подтвердите пароль

Зарегистрироваться

Уже есть аккаунт? Войти

Рисунок 8 – Проверка полей формы регистрации

При утрате пароля пользователь нажимает «Забыли пароль?» и вводит свой email (рисунок 9).

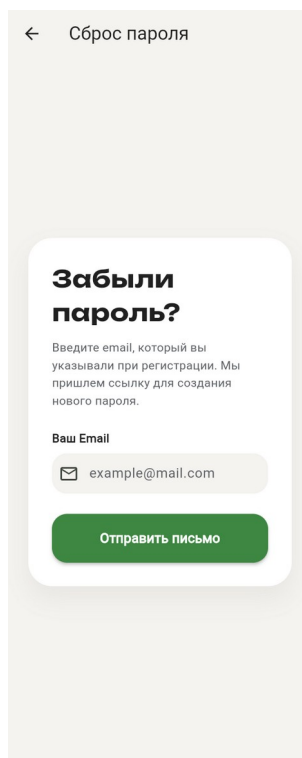


Рисунок 9 – Экран восстановления пароля

После нажатия «Отправить письмо» выводится сообщение об отправке инструкции по сбросу пароля на указанную почту (рисунок 10).

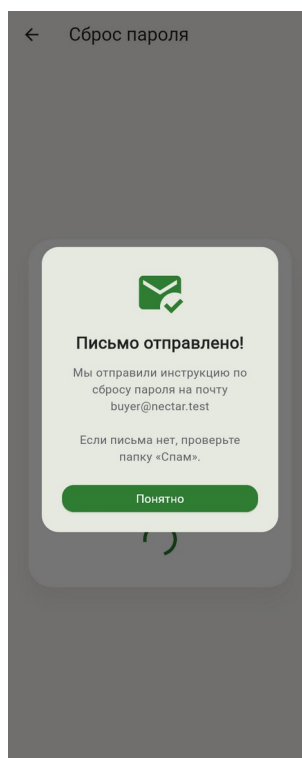


Рисунок 10 – Сообщение об отправке письма для сброса пароля

2.2.3. Выполнение программы «Покупатель интернет-магазина»

После успешной авторизации покупатель попадает на экран «Магазин». Здесь отображаются акционный баннер, строка поиска, блок «Популярное» и товары, сгруппированные по категориям. Кнопка «Все» открывает полный список категории, значок «сердце» в правом верхнем углу – раздел «Избранное». Нижнее меню содержит разделы «Магазин», «Каталог», «Корзина» и «Профиль» (рисунок 11).

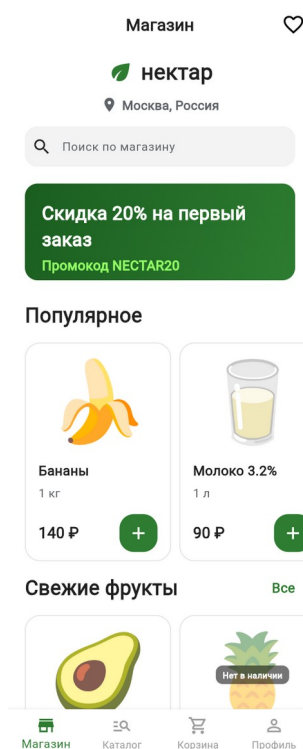


Рисунок 11 – Экран «Магазин»

Строка поиска на экране «Магазин» сразу отбирает товары, в названии которых есть введенный текст (рисунок 12). Кнопка «×» очищает запрос.

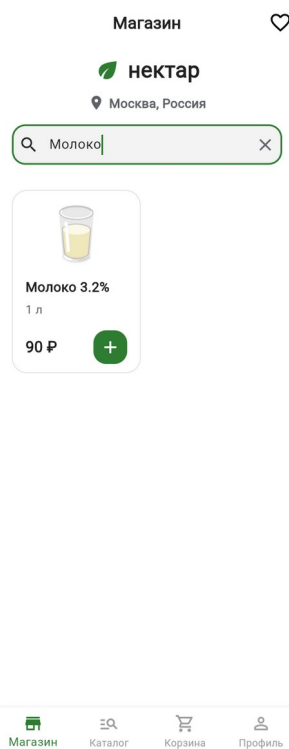


Рисунок 12 – Поиск товаров на экране «Магазин»

Раздел «Каталог» содержит плитки категорий товаров (рисунок 13).

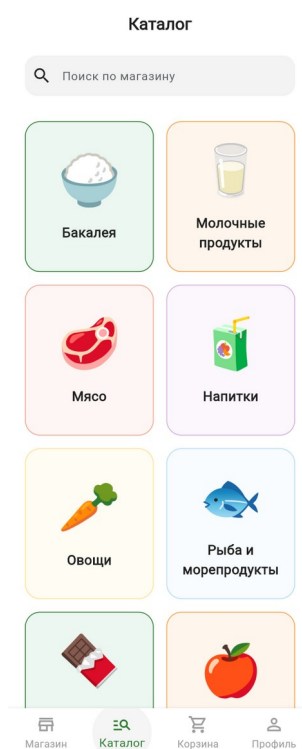


Рисунок 13 – Экран «Каталог»

При вводе текста в строку поиска каталога отображаются подходящие товары, а рядом с полем появляется значок фильтра (рисунок 14).

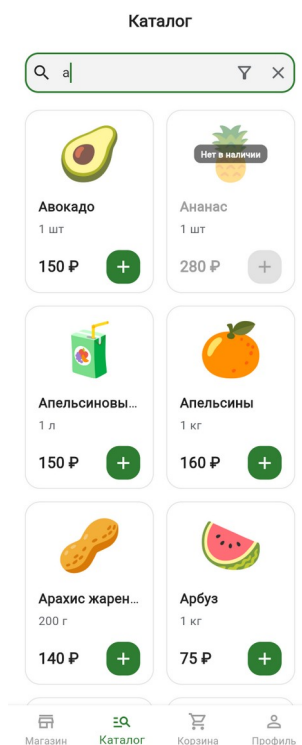


Рисунок 14 – Поиск товаров в каталоге

Во всплывающем окне «Фильтры» можно ограничить цену ползунком, выбрать производителей и способ сортировки: «По умолчанию», «Сначала свежие», «По убыванию цены», «По возрастанию цены» (рисунок 15). Кнопка «Сбросить» возвращает исходные значения, кнопка «Применить» применяет фильтры.

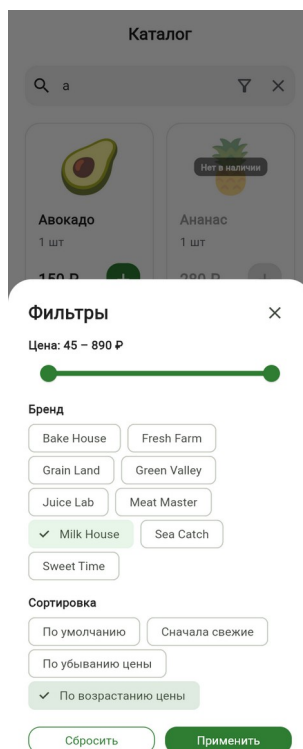


Рисунок 15 – Всплывающее окно фильтров

После применения фильтров в списке остаются только подходящие товары, значок фильтра подсвечивается (рисунок 16).

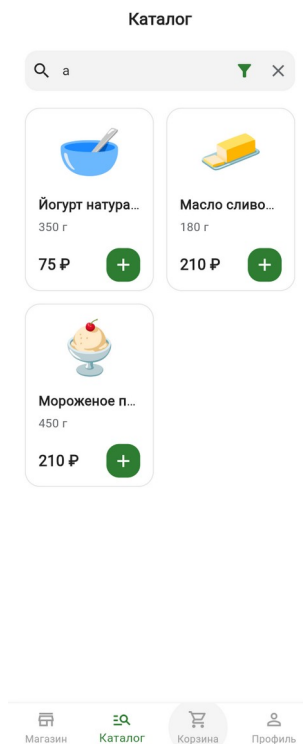


Рисунок 16 – Каталог с примененными фильтрами

При выборе плитки категории открывается список товаров этой категории. В нем также доступны фильтры (рисунок 17). Кнопка «+» на

карточке добавляет одну единицу товара в корзину; если товара нет в наличии, кнопка неактивна.

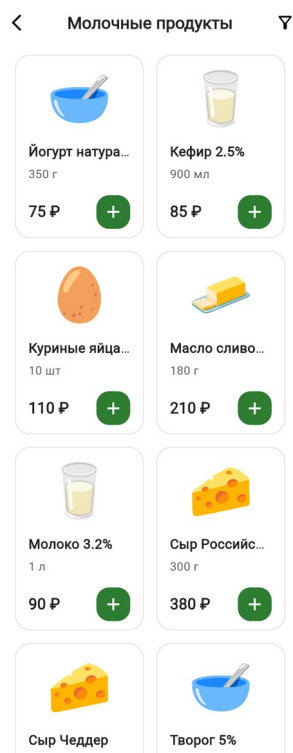


Рисунок 17 – Список товаров категории

Выбрав интересующий товар, покупатель открывает его карточку: фото, название, фасовка, остаток на складе, цена, описание, характеристики (производитель, страна происхождения, срок годности, состав) и пищевая ценность на 100 г (рисунок 18). Значок «сердце» добавляет товар в избранное, значок «Поделиться» формирует текст с названием и ценой для отправки.

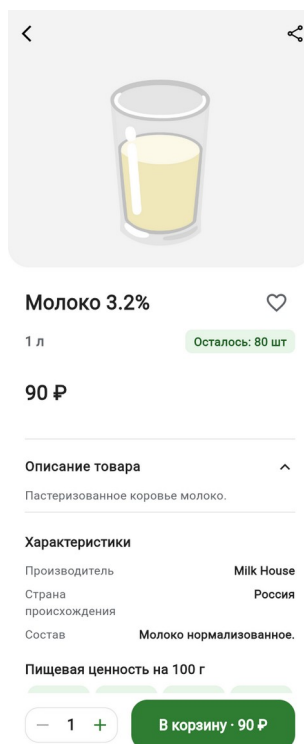


Рисунок 18 – Карточка товара

Кнопками «-» и «+» выбирается количество, после чего нажимается кнопка «В корзину». Если запрошено больше остатка, добавляется весь доступный остаток, а на экране выводится соответствующее сообщение (рисунок 19).

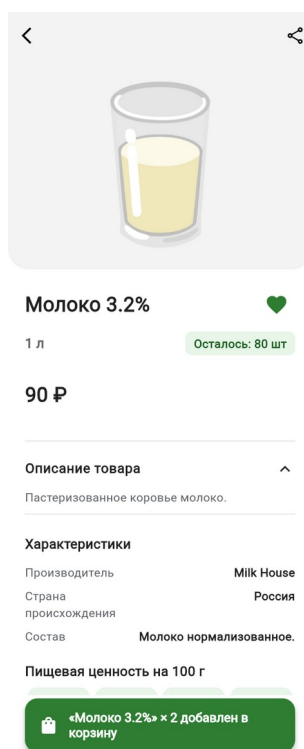


Рисунок 19 – Добавление товара в корзину

В разделе «Избранное» отображаются отмеченные товары. Повторное нажатие на «сердце» удаляет товар из избранного, кнопка «В корзину всё» добавляет в корзину все товары, имеющиеся в наличии (рисунок 20).

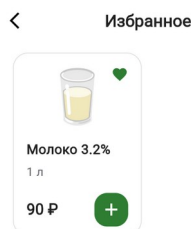


Рисунок 20 – Экран «Избранное»

На экране «Моя корзина» отображается список товаров с возможностью изменить количество кнопками «-» и «+» или удалить позицию значком «×». Внизу автоматически рассчитываются подытог, скидка, доставка (149 ₽, бесплатно при сумме заказа от 1500 ₽) и итоговая сумма (рисунок 21).

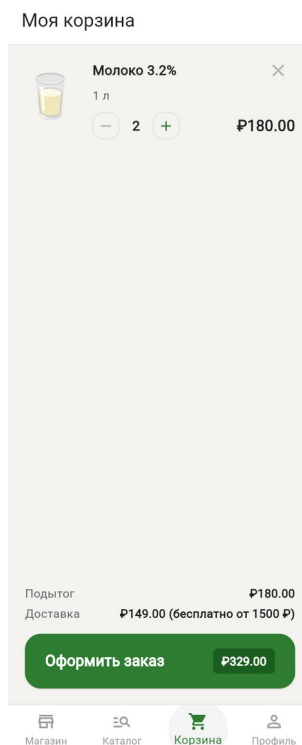


Рисунок 21 – Экран корзины

При нажатии кнопки «Оформить заказ» открывается окно оформления, где необходимо выбрать адрес доставки, интервал доставки («Как можно скорее» или один из двухчасовых интервалов) и способ оплаты: «Онлайн» (с выбором сохраненной карты) либо «Наличными» курьеру (рисунок 22).

Моя корзина

Молоко 3.2%

Оформление заказа

Адрес доставки Москва, Тверская 1

Время доставки Как можно скорее (...)

Способ оплаты

☒ Онлайн

Карта **** * 1111

☐ Наличными

Промокод Не выбран [Выбрать](#)

Сумма товаров	₽180.00
Скидка	-₽0.00
Доставка	₽149.00
Итого	₽329.00

Разместить заказ

Рисунок 22 – Оформление заказа

Ссылка «Выбрать» в строке «Промокод» открывает экран промокодов, где код вводится вручную или выбирается из списка доступных (рисунок 23).

← Промокоды

Введите промокод [Применить](#)

Доступные промокоды

SALE10
Скидка 10% до 29.10.2026 [Выбрать](#)

Рисунок 23 – Выбор промокода

При вводе действующего промокода и нажатии «Применить» выводится сообщение о размере скидки (рисунок 24). Кнопка «Сбросить» отменяет промокод.

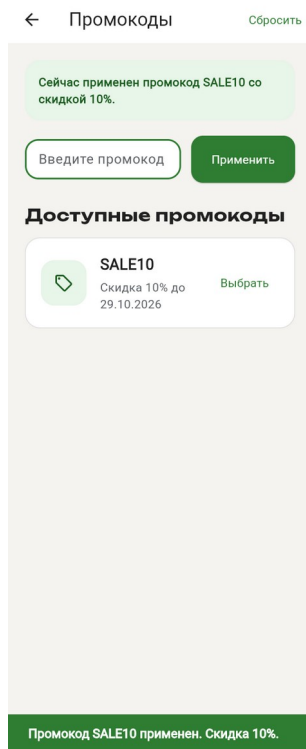


Рисунок 24 – Применение промокода

После возврата в окно оформления итоговая сумма пересчитывается с учетом скидки (рисунок 25). Для завершения покупки нажимается кнопка «Разместить заказ».

Моя корзина

Молоко 3.2%

Оформление заказа

Адрес доставки Москва, Тверская 1

Время доставки Как можно скорее (...)

Способ оплаты

☒ Онлайн

Карта **** * 1111

☐ Наличными

Промокод SALE10 (10%) [Выбрать](#)

Сумма товаров	₽180.00
Скидка	-₽18.00
Доставка	₽149.00
Итого	₽311.00

Разместить заказ

Рисунок 25 – Оформление заказа с промокодом

После успешной транзакции остатки товаров списываются, корзина очищается, а покупатель видит подтверждение с номером заказа (рисунок 26).

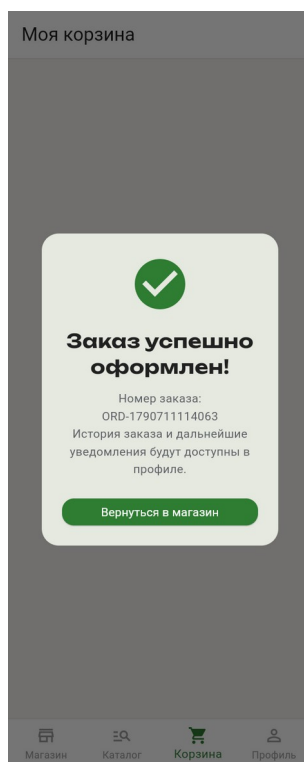


Рисунок 26 – Успешное оформление заказа

Историю покупок покупатель просматривает в разделе «Профиль» → «Заказы». Для каждого заказа отображаются номер, дата и шаги выполнения «Создан – У курьера – Доставлен» (рисунок 27).

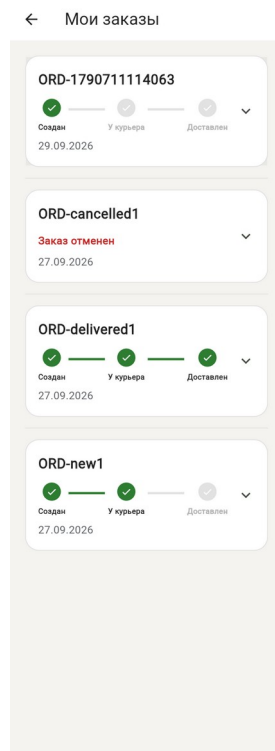


Рисунок 27 – Экран «Мои заказы»

При нажатии на заказ карточка раскрывается: указываются адрес, время доставки, оплата, курьер и его телефон, итоговая сумма и состав заказа. Завершенный или отмененный заказ можно скрыть из списка кнопкой «Скрыть из списка» (рисунок 28).

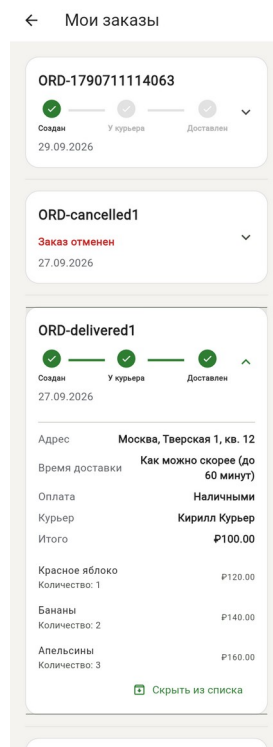


Рисунок 28 – Детали заказа

Пока заказ находится у курьера, отображается кнопка «Подтвердить, что всё хорошо» (рисунок 29), после нажатия на которую выводится сообщение о подтверждении получения (рисунок 30).

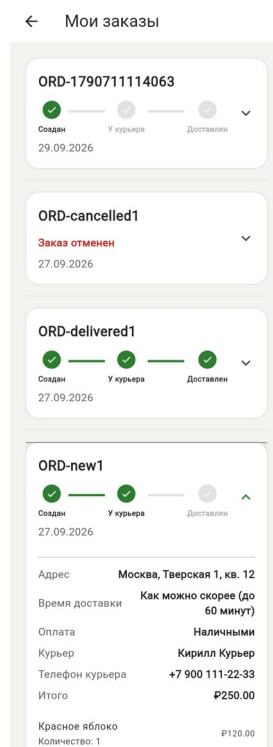


Рисунок 29 – Подтверждение получения заказа

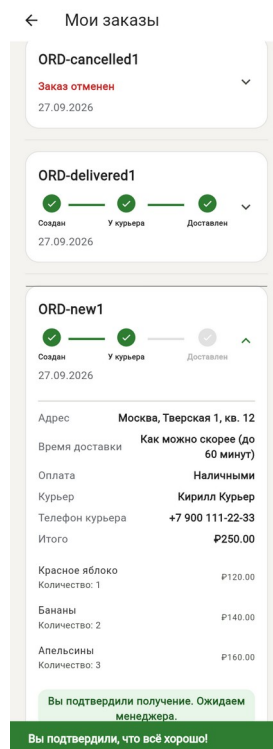


Рисунок 30 – Заказ с подтвержденным получением

О смене статуса заказа покупатель узнает из раздела «Профиль» → «Уведомления». Здесь же переключателями настраиваются Push-, SMS- и Email-уведомления, а кнопка «Очистить все» удаляет историю (рисунок 31).

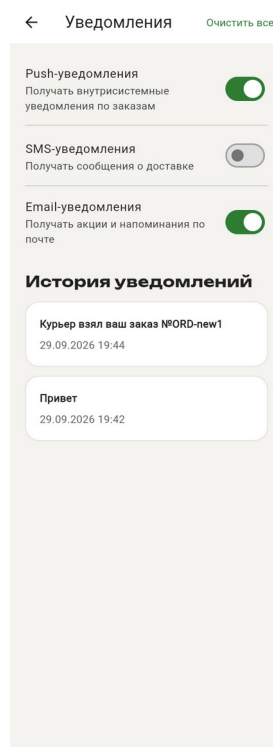


Рисунок 31 – Экран «Уведомления»

На экране «Профиль» отображаются аватар, имя, email и роль. Нажатие на аватар позволяет выбрать новое изображение (JPEG или PNG, размер уменьшается автоматически). Меню содержит разделы «Заказы», «Мои данные», «Адрес доставки», «Способы оплаты», «Промокоды», «Сменить пароль», переключатель языка, «Уведомления», «Помощь» и «О приложении» (рисунок 32).

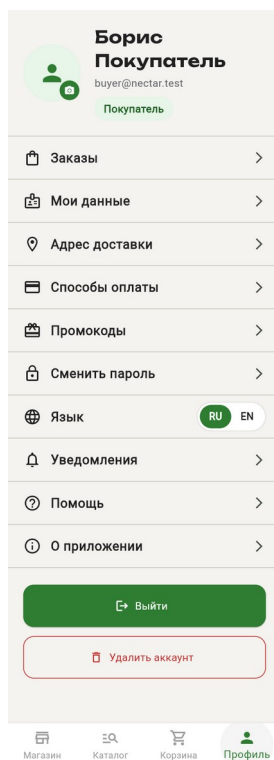


Рисунок 32 – Профиль пользователя

В разделе «Мои данные» можно изменить имя и телефон; email изменить нельзя (рисунок 33).

← Мои данные

Полное имя
Борис Покупатель

Номер телефона
+7 900 000-00-00

Email (нельзя изменить)
buyer@nectar.test

Сохранить

Рисунок 33 – Экран «Мои данные»

В разделе «Адрес доставки» хранятся адреса, которые затем выбираются при оформлении заказа. Кнопка «Добавить» открывает окно ввода адреса (до 300 символов), значок корзины удаляет адрес (рисунок 34).

← Адрес доставки

Москва, Тверская 1

Добавить адрес

Москва, Арбат 10, кв. 5

23/300

Отмена Сохранить

+ Добавить

Рисунок 34 – Добавление адреса доставки

В разделе «Способы оплаты» добавляются банковские карты: вводится 16 цифр, а в профиле сохраняется только маска вида «**** *
**** 4444» (рисунок 35).

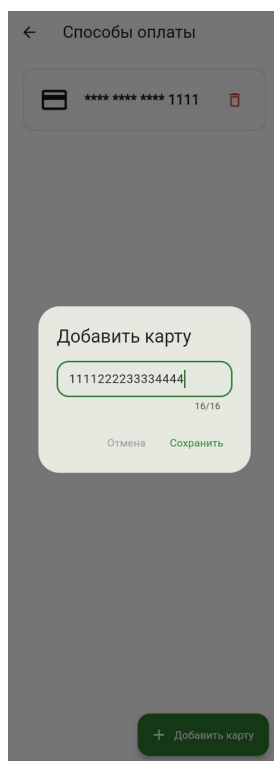


Рисунок 35 – Добавление банковской карты

Раздел «Промокоды» позволяет ввести промокод и просмотреть доступные скидки со сроком действия (рисунок 36).

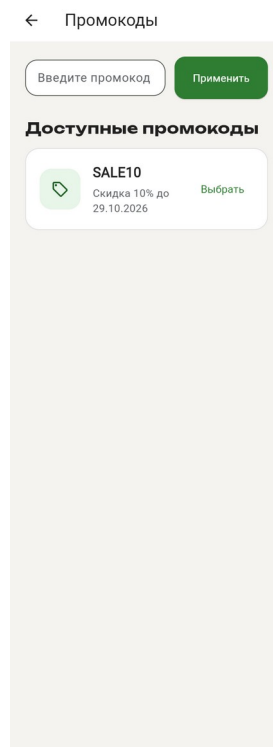


Рисунок 36 – Экран «Промокоды»

Для смены пароля выбирается пункт «Сменить пароль», вводятся текущий пароль, новый пароль и его подтверждение (рисунок 37).

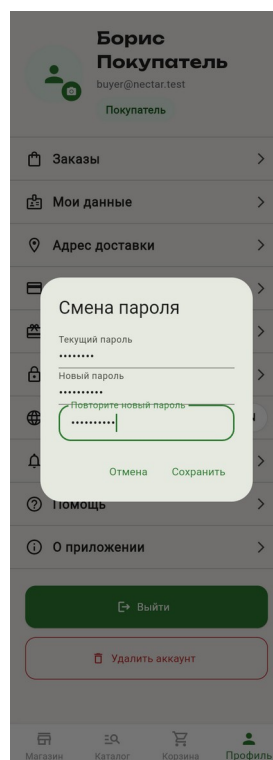


Рисунок 37 – Смена пароля

Переключатель RU/EN меняет язык всего приложения; выбор запоминается на устройстве (рисунок 38).

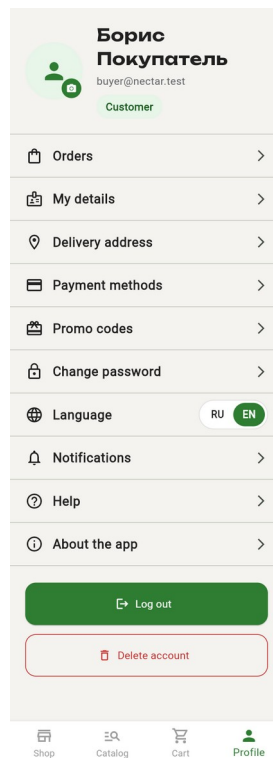


Рисунок 38 – Профиль при выбранном английском языке

Раздел «Помощь» содержит ответы на часто задаваемые вопросы (рисунок 39), раздел «О приложении» – версию и ссылки на условия использования и политику конфиденциальности.

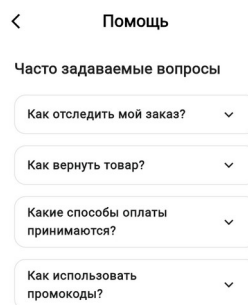


Рисунок 39 – Экран «Помощь»

Кнопка «Выйти» завершает сеанс. Кнопка «Удалить аккаунт» после подтверждения отключает учетную запись, если у пользователя нет незавершенных заказов (рисунок 40).

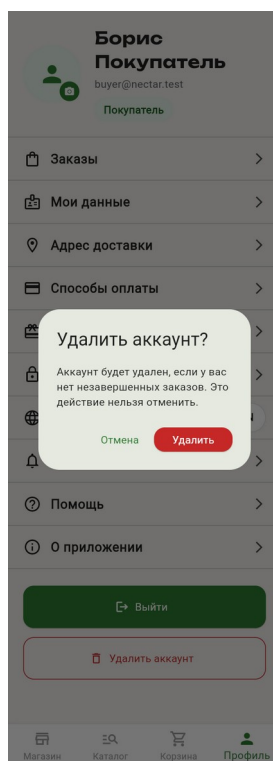


Рисунок 40 – Подтверждение удаления аккаунта

2.2.4. Выполнение программы «Курьер»

Пользователь с ролью «Курьер» после входа попадает на экран «Доставка» с двумя вкладками. На вкладке «Свободные» отображаются заказы без курьера в статусе «Новый» или «В сборке»: адрес, время, состав, способ оплаты и сумма. Для заказа с оплатой наличными выделена сумма к получению. Кнопка «Взять заказ» назначает заказ на курьера; если заказ уже взял другой курьер, выводится сообщение об ошибке (рисунок 41).

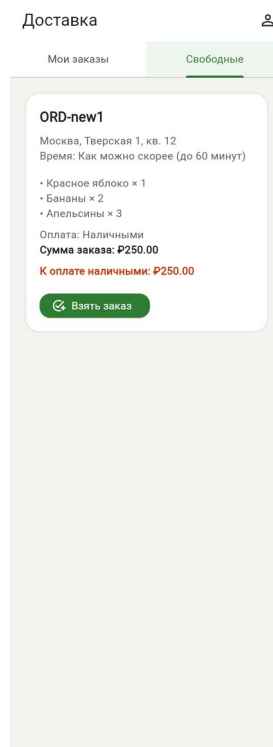


Рисунок 41 – Вкладка «Свободные»

На вкладке «Мои заказы» в блоке «Активные заказы» отображаются взятые заказы с данными клиента. Телефон клиента копируется нажатием (рисунок 42). Статус нового заказа – «Передан курьеру».

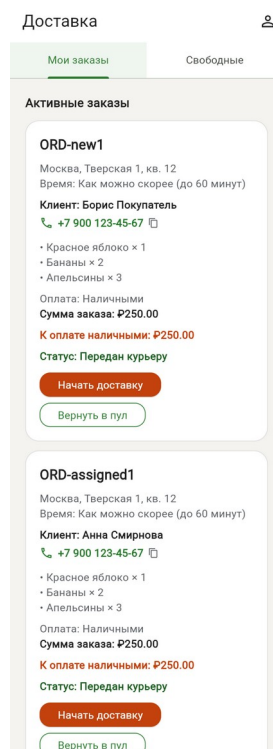


Рисунок 42 – Вкладка «Мои заказы»

Пока заказ не в пути, его можно вернуть в общий пул кнопкой «Вернуть в пул» после подтверждения (рисунок 43).

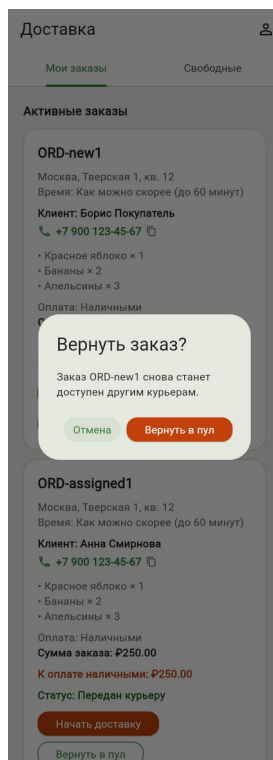


Рисунок 43 – Возврат заказа в пул

Кнопка «Начать доставку» переводит заказ в статус «В пути», покупатель получает уведомление. Вместо неё отображается кнопка «Подтвердить доставку» (рисунок 44).

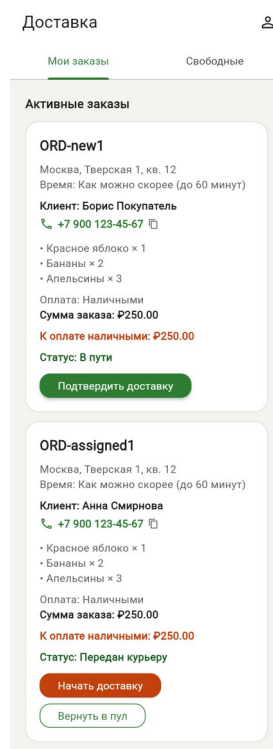


Рисунок 44 – Заказ в пути

При нажатии «Подтвердить доставку» выводится запрос на подтверждение (рисунок 45). После подтверждения заказ получает статус «Доставлен» и переходит в блок «История доставок»; отменить это действие нельзя.

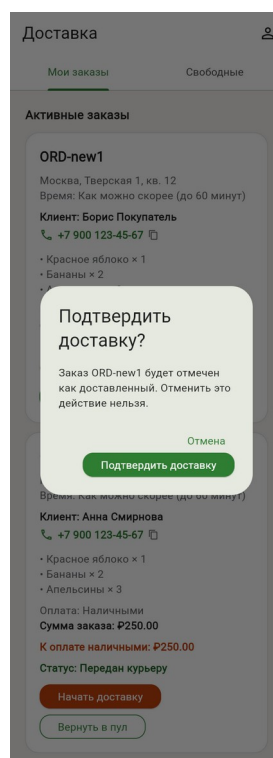


Рисунок 45 – Подтверждение доставки

2.2.5. Выполнение программы «Администратор и менеджер»

Для управления магазином используется веб-панель. Вход выполняется по email и паролю на странице авторизации; вход разрешен только ролям «Администратор» и «Менеджер» (рисунок 46).

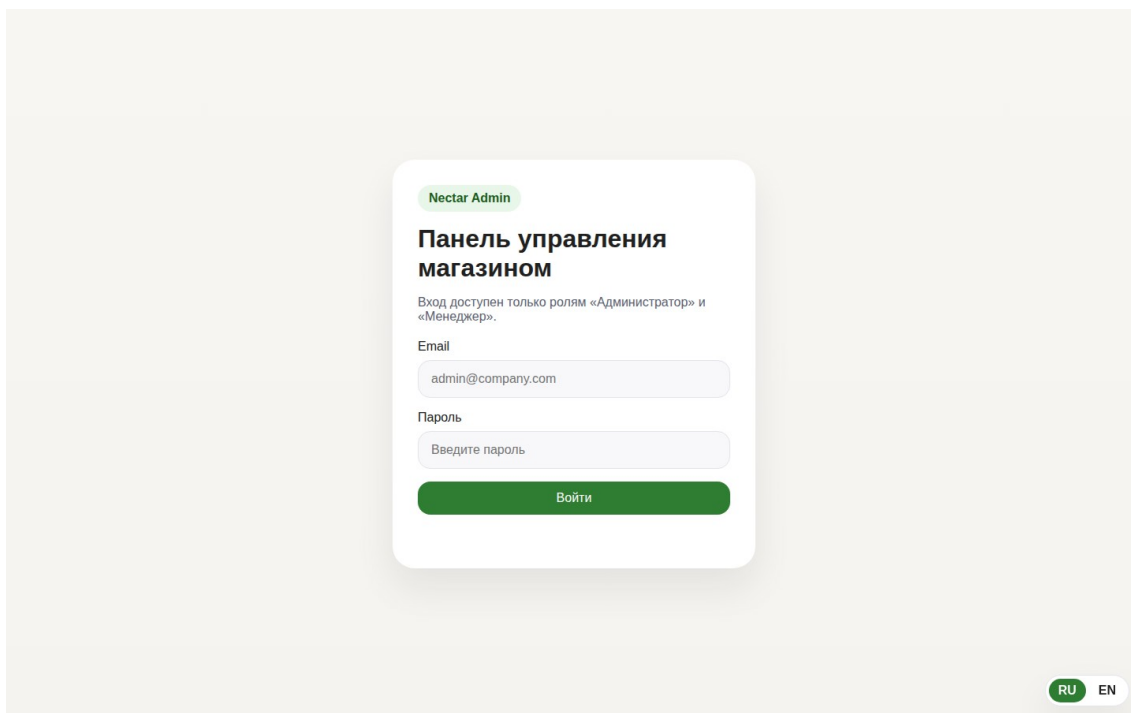


Рисунок 46 – Вход в веб-панель

На странице «Дашборд» отображаются количество покупателей, активных заказов, товаров с низким остатком, общая выручка, диаграммы «Продажи по месяцам» и «ТОП-5 товаров». Верхнее меню содержит разделы панели, кнопки «Сменить пароль», «Обновить» и «Выйти» (рисунок 47).

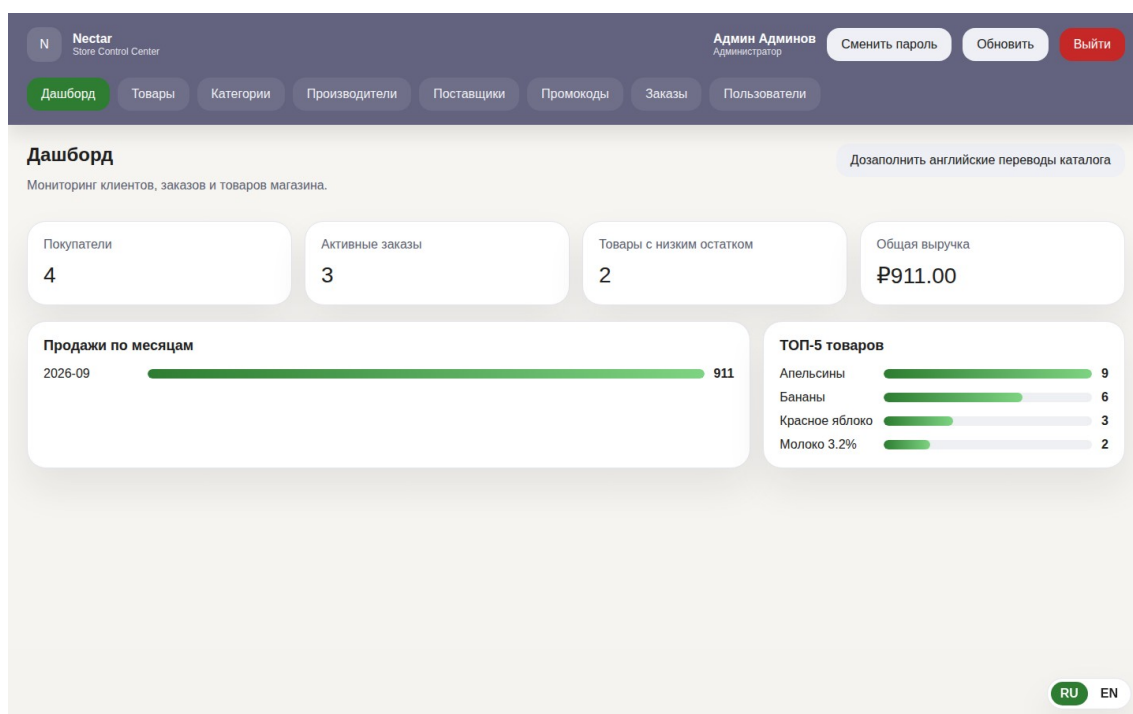


Рисунок 47 – Дашборд

Раздел «Товары» содержит каталог с поиском по названию, ценой, остатком и статусом (рисунок 48). Кнопка «Добавить товар» открывает форму создания, «Редактировать» – форму изменения. Кнопка «Удалить» после подтверждения скрывает товар из каталога приложения, а кнопка «Восстановить» возвращает его.

Товары
Создание, редактирование и мягкое удаление товаров.

Добавить товар

Поиск по названию товара

Название	Категория	Цена	Остаток	Статус	Действия
Красное яблоко	Фрукты	₹120.00	60	Активен	Редактировать Удалить
Авокадо	Фрукты	₹150.00	40	Активен	Редактировать Удалить
Бекон нарезка	Мясо	₹210.00	40	Активен	Редактировать Удалить
Бублик с маком	Хлеб и выпечка	₹95.00	30	Активен	Редактировать Удалить
Багет французский	Хлеб и выпечка	₹65.00	40	Активен	Редактировать Удалить
Бананы	Фрукты	₹140.00	80	Активен	Редактировать Удалить
Хлеб пшеничный	Хлеб и выпечка	₹55.00	70	Активен	Редактировать Удалить
Брокколи	Овощи	₹180.00	30	Активен	Редактировать Удалить

Рисунок 48 – Управление каталогом товаров

В форме товара указываются название, цена, количество на складе, единица, категория, производитель, поставщик, ссылка на изображение, страна, срок годности, пищевая ценность, состав и описание на русском и английском языках (рисунок 49). Неверные значения блокируют сохранение и сопровождаются сообщением.

The screenshot displays the 'Add Item' form within the Nectar Store Control Center. The form is titled 'Добавить товар' and includes a note: 'Все изменения сохраняются прямо в Cloud Firestore.' The form fields are as follows:

- Название** (Name): A text input field, currently highlighted with a green border.
- Цена** (Price): A text input field.
- Количество** (Quantity): A text input field with the value '0'.
- Единица** (Unit): A text input field with the value '1 шт'.
- Категория** (Category): A dropdown menu with the value 'Бакалея'.
- Производитель** (Manufacturer): A dropdown menu with the value 'Bake House'.
- Поставщик** (Supplier): A dropdown menu with the value 'Не выбран'.
- Ссылка на изображение** (Image Link): A text input field.
- Страна происхождения** (Country of Origin): A text input field.
- Срок годности до** (Expiry Date): A text input field with the value 'mm/dd/yyyy' and a calendar icon.
- Калорийность, ккал / 100 г** (Caloricity): A text input field with the value '0'.
- Белки, г** (Protein): A text input field with the value '0'.
- Жиры, г** (Fat): A text input field with the value '0'.
- Углеводы, г** (Carbohydrates): A text input field with the value '0'.
- Состав** (Composition): A text input field.
- Описание** (Description): A text input field.

The background shows a list of items with 'Удалить' (Delete) buttons and a language toggle at the bottom right.

Рисунок 49 – Форма добавления товара

Раздел «Категории» служит для ведения категорий товаров (рисунок 50). Название категории не должно повторяться; категорию с привязанными товарами удалить нельзя.

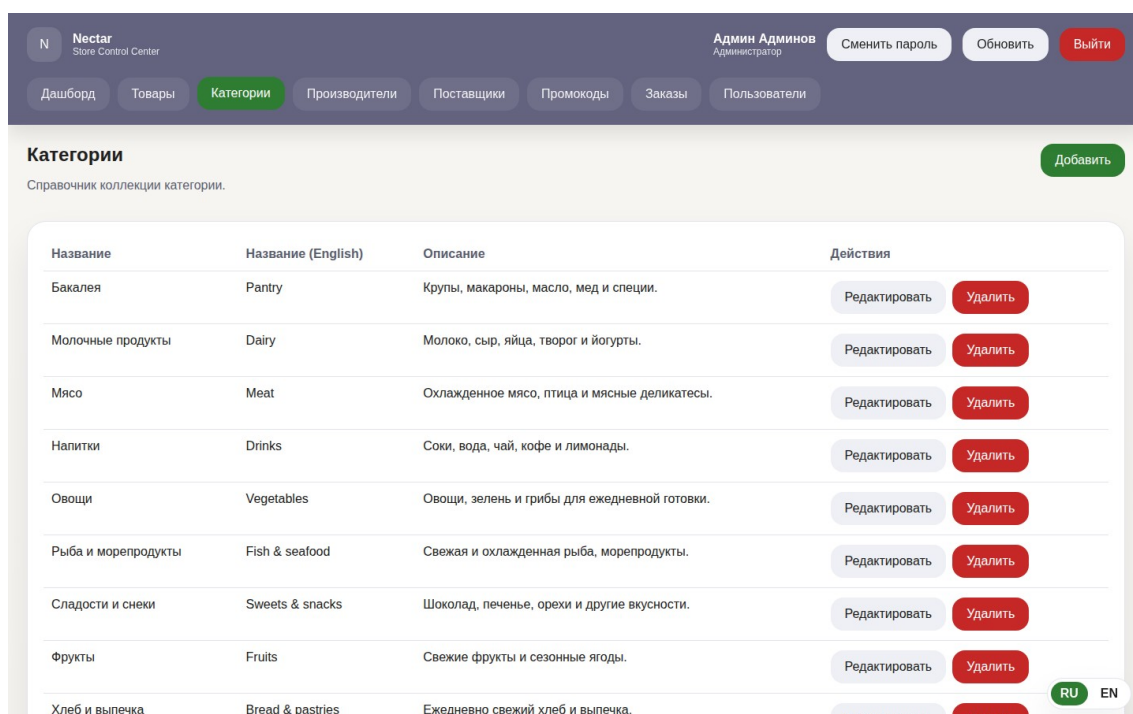


Рисунок 50 – Справочник категорий

Кнопка «Добавить» открывает форму с названием, английским названием и описанием (рисунок 51).

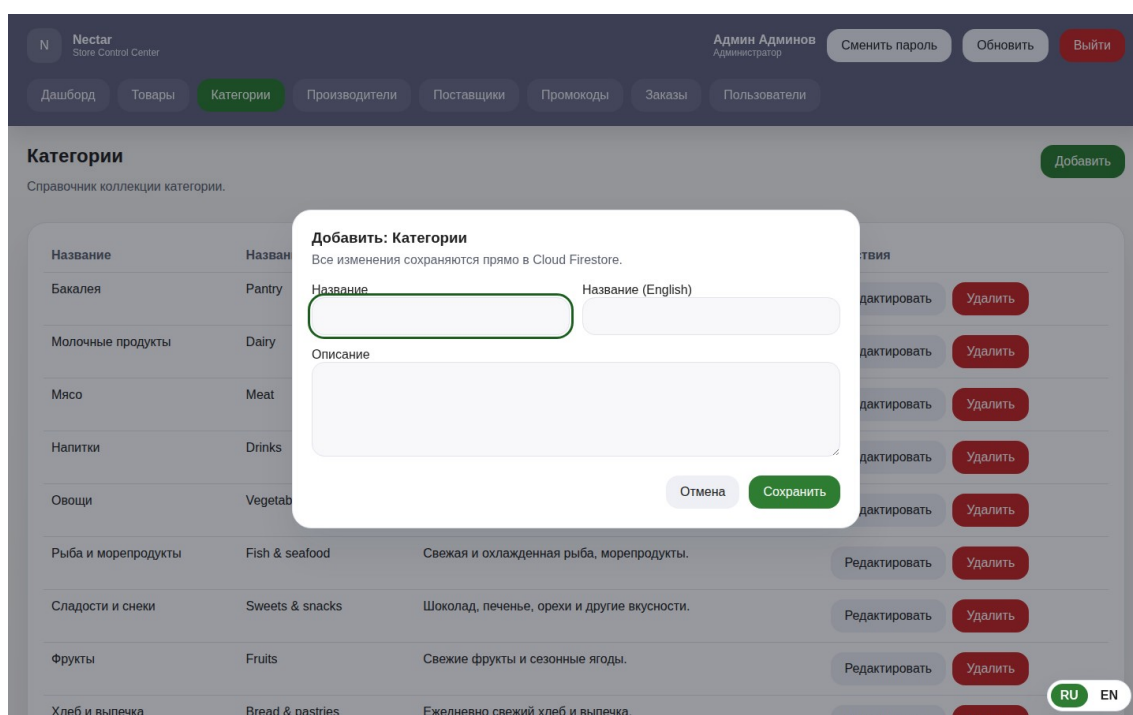


Рисунок 51 – Добавление категории

Раздел «Производители» содержит справочник производителей с названием, страной и телефоном (рисунок 52). Он определяет фильтр по брендам в мобильном приложении.

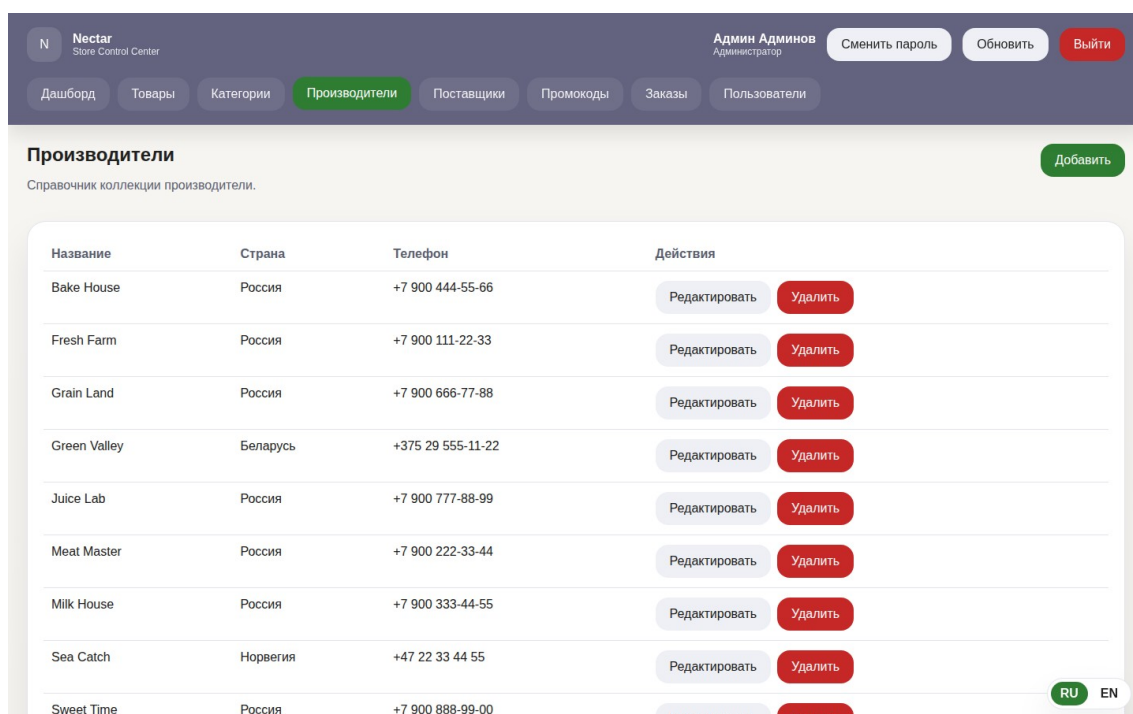


Рисунок 52 – Справочник производителей

Раздел «Поставщики» содержит название, ИНН, адрес, телефон и email. Администратор редактирует записи, менеджер только просматривает (рисунок 53).

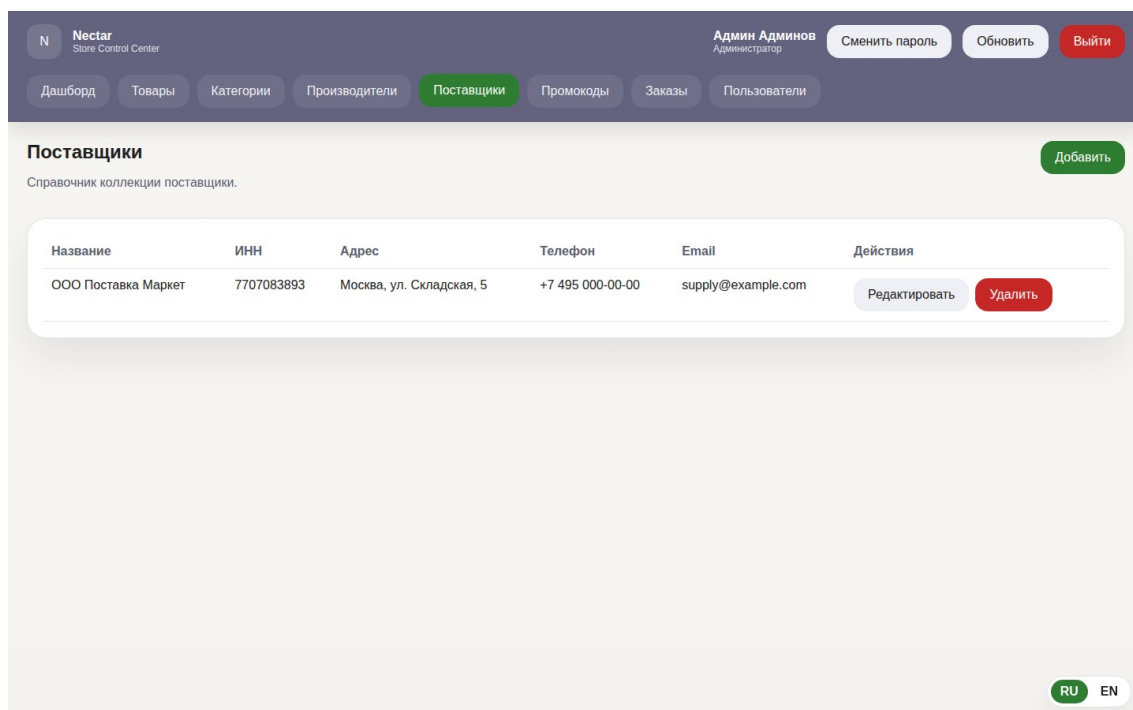


Рисунок 53 – Справочник поставщиков

Раздел «Промокоды» предназначен для маркетинговых акций: отображаются код, скидка и срок действия (рисунок 54).

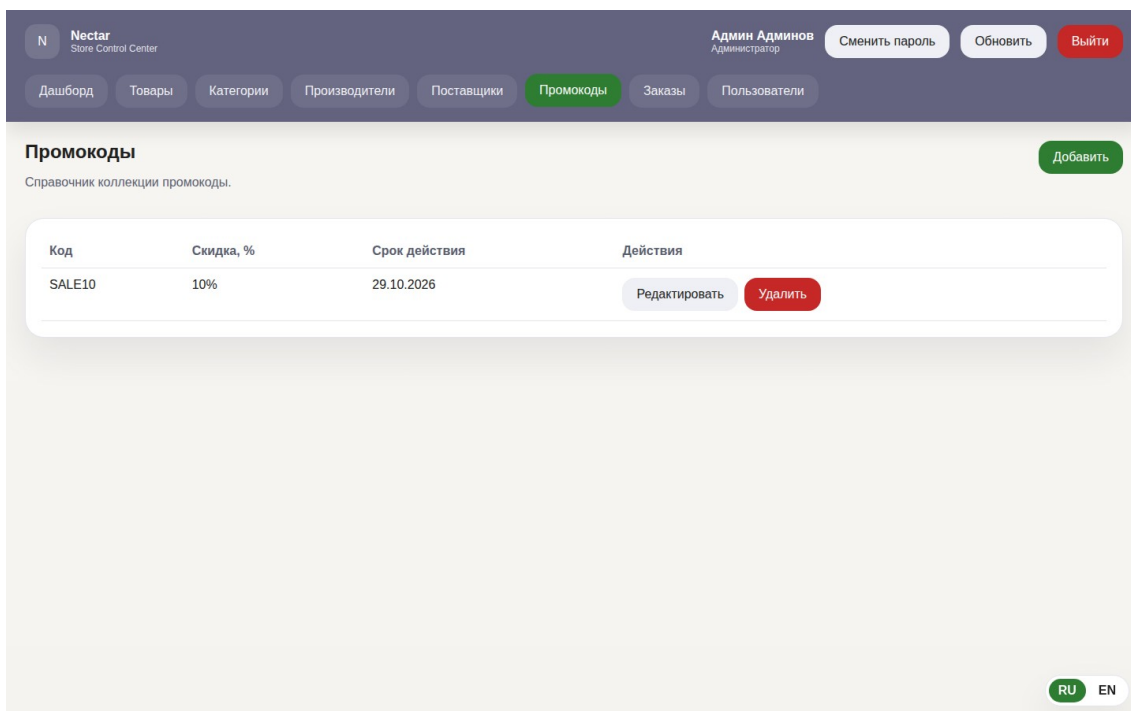


Рисунок 54 – Список промокодов

При создании промокода указываются код (латиница и цифры), скидка от 1 до 99 % и срок действия (рисунок 55).

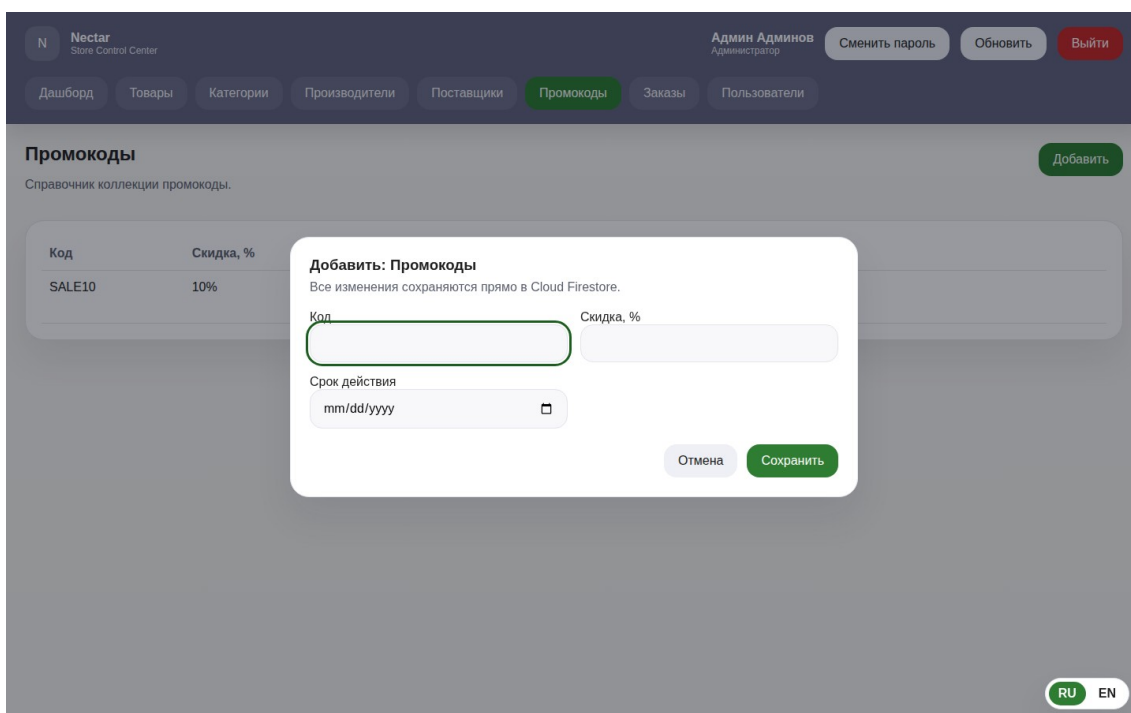


Рисунок 55 – Добавление промокода

В разделе «Заказы» отображаются все заказы с номером, клиентом, суммой, статусом и курьером. Для смены статуса или назначения курьера необходимо выбрать значения в списках и нажать «Сохранить»;

покупатель и курьер получают уведомления. Для статусов «Передан курьеру», «В пути» и «Доставлен» курьер обязателен. При отмене заказа товары возвращаются на склад, завершённые заказы изменить нельзя (рисунок 56).

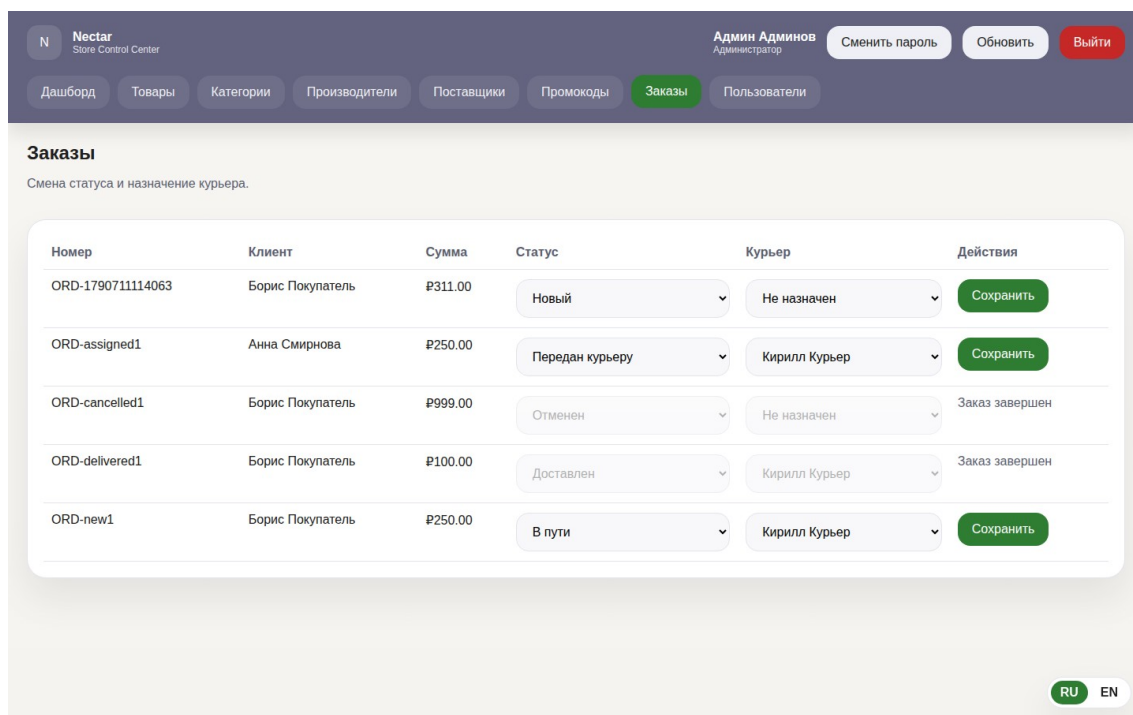


Рисунок 56 – Управление заказами

Раздел «Пользователи» показывает имя, логин, email, телефон и роль. Администратор может изменить роль и нажать «Отключить» (после подтверждения пользователь не сможет войти), а также восстановить отключенных, включив флажок «Показывать отключенных». Менеджеру раздел доступен только для просмотра (рисунок 57).

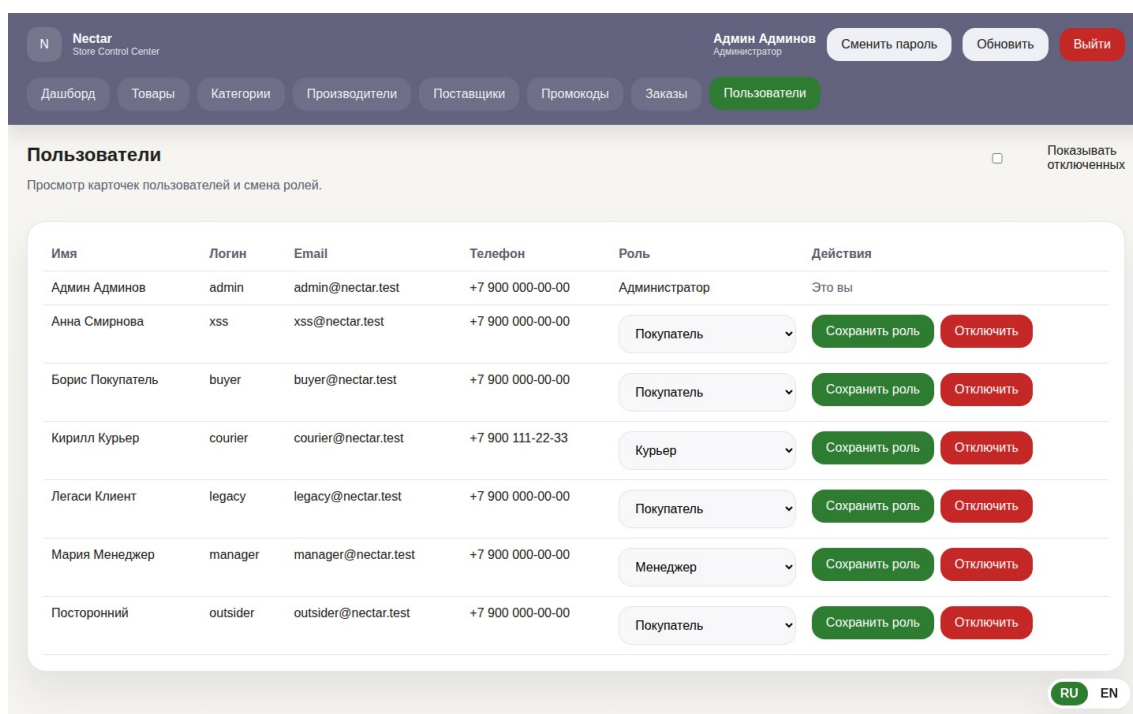


Рисунок 57 – Управление пользователями

Кнопка «Сменить пароль» открывает окно ввода текущего и нового пароля (рисунок 58).

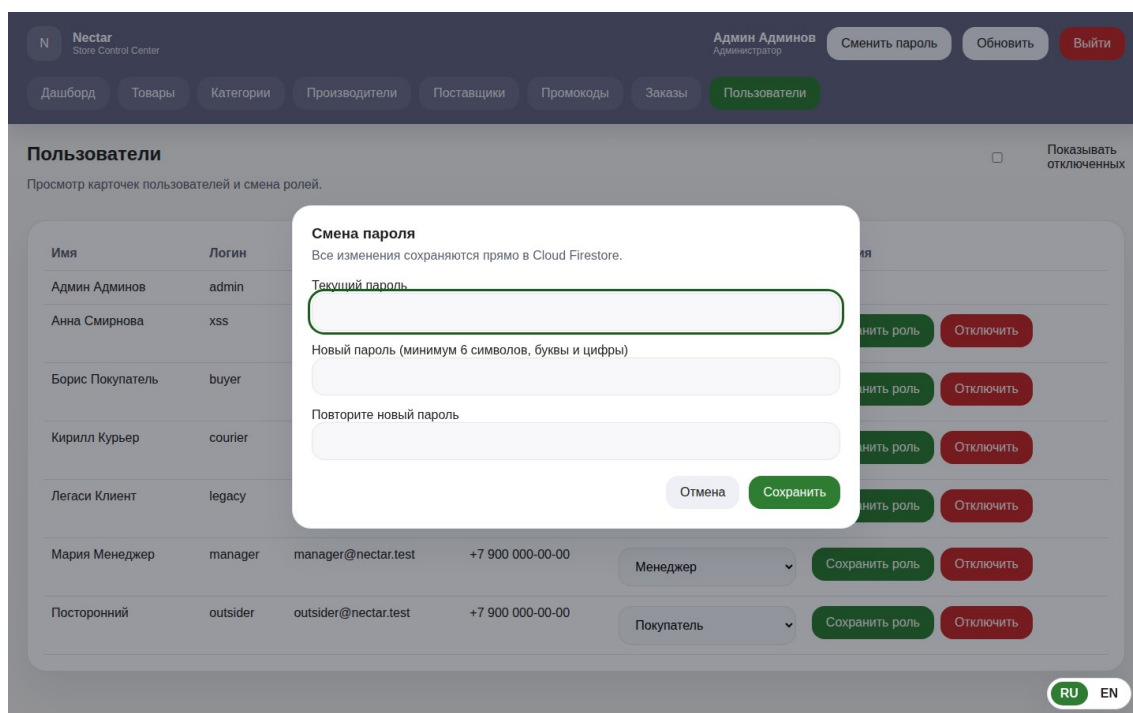


Рисунок 58 – Смена пароля

3. СООБЩЕНИЯ ОПЕРАТОРУ

№	Текст сообщения	Содержание сообщения	Действия оператора (пользователя)
1	2	3	4
1	Введите логин или email	Ошибка проверки формы входа: поле «Логин или email» пустое	Заполнить поле и повторить вход
2	Логин или email указан некорректно	Ошибка проверки формы входа: неверный формат email или логина	Ввести email в формате user@mail.ru или логин из латинских букв, цифр и символов ._-
3	Введите пароль	Ошибка проверки формы: поле пароля пустое	Заполнить поле пароля
4	Пользователь с таким логином не найден.	Ошибка авторизации: указанный логин не зарегистрирован	Проверить логин или войти по email, либо перейти к регистрации
5	Неверный email или пароль.	Ошибка авторизации: аккаунт не найден или пароль не подходит	Проверить раскладку клавиатуры и правильность ввода, при необходимости восстановить пароль
6	Этот аккаунт был удален.	Аккаунт отключен администратором или удален пользователем	Обратиться к администратору магазина
7	Этот email уже используется.	Ошибка регистрации: адрес электронной почты уже занят	Войти с этим email или использовать другой адрес
8	Пользователь с таким логином уже существует.	Ошибка регистрации: логин уже занят	Придумать другой логин
9	Длина логина от 3 до 50 символов	Логин не соответствует требованиям	Ввести логин длиной от 3 до 50 символов
10	Только латиница, цифры и символы ._-	Логин содержит недопустимые символы	Использовать только латинские буквы, цифры и символы ._-
11	Только буквы, пробелы и дефисы	Имя содержит недопустимые символы	Ввести имя без цифр и специальных символов
12	Неверный формат email	Ошибка проверки адреса электронной почты	Ввести email в правильном формате (например, user@mail.ru)
13	Неверный формат номера телефона	Ошибка проверки номера телефона	Ввести от 10 до 15 цифр, допустимы +, пробелы, скобки и дефис
14	Минимум 6 символов	Пароль короче допустимой длины	Ввести пароль длиной не менее 6 символов
15	Пароль должен содержать минимум одну цифру	Пароль не соответствует требованиям безопасности	Добавить в пароль хотя бы одну цифру
16	Пароли не совпадают	Ошибка подтверждения	Ввести одинаковый

		пароля при регистрации или смене пароля	пароль в оба поля
17	Неверный текущий пароль.	Ошибка смены пароля: текущий пароль указан неверно	Ввести правильный текущий пароль
18	Письмо отправлено!	Инструкция по сбросу пароля отправлена на указанную почту	Нажать «Понятно», проверить почту (в том числе папку «Спам») и создать новый пароль по ссылке
19	Больше нет в наличии	Количество товара в корзине достигло остатка на складе	Оставить текущее количество или выбрать другой товар
20	Неверный или просроченный промокод.	Промокод не найден, неактивен или истек срок действия	Проверить правильность ввода или оформить заказ без промокода
21	Добавьте адрес в профиле	Для оформления заказа нет сохраненного адреса доставки	Открыть «Профиль» → «Адрес доставки» и добавить адрес
22	Для онлайн-оплаты выберите сохраненную карту.	При способе оплаты «Онлайн» не выбрана карта	Добавить карту в профиле и выбрать её либо выбрать оплату «Наличными»
23	Недостаточно товара «...» на складе.	Товар закончился во время оформления заказа	Уменьшить количество или удалить товар из корзины
24	Заказ успешно оформлен!	Заказ создан, остатки товаров списаны, корзина очищена	Нажать «Вернуться в магазин»; статус заказа отслеживать в разделе «Заказы»
25	Данные успешно сохранены	Имя и телефон профиля сохранены	Закрыть экран
26	Пароль успешно изменен.	Новый пароль установлен	Использовать новый пароль при следующем входе
27	Номер карты должен содержать 16 цифр	Ошибка проверки номера банковской карты	Ввести ровно 16 цифр
28	Разрешены только изображения JPEG и PNG.	Выбран файл неподдерживаемого формата для аватара	Выбрать изображение в формате JPEG или PNG
29	Нельзя удалить пользователя с незавершенными заказами.	Удаление аккаунта запрещено, пока есть незавершенные заказы	Дождаться завершения заказов и повторить
30	Этот заказ уже взял другой курьер.	Курьер попытался взять заказ, который уже назначен	Выбрать другой заказ в списке «Свободные»
31	Заказ нельзя вернуть: он уже в пути или назначен не вам.	Возврат заказа в пул невозможен	Довести доставку до конца

32	Не удалось загрузить данные. Проверьте соединение.	Проблемы с сетью или недоступность сервера Firebase	Проверить подключение к интернету и повторить попытку
33	Роль «Администратор» не может работать в мобильном приложении.	Администратор или менеджер вошел в мобильное приложение	Нажать «Выйти из аккаунта» и использовать веб-панель
34	Недостаточно прав для входа в веб-панель.	В веб-панель вошел пользователь без роли «Администратор» или «Менеджер»	Использовать мобильное приложение
35	Не удалось выполнить вход. Проверьте email и пароль.	Ошибка входа в веб-панель	Проверить email и пароль и повторить
36	Для этого статуса назначьте курьера.	Статус «Передан курьеру», «В пути» или «Доставлен» выбран без курьера	Выбрать курьера в списке и сохранить заказ
37	Завершенный или отмененный заказ изменить нельзя.	Попытка изменить завершенный заказ	Действий не требуется
38	Цена должна быть больше нуля.	Ошибка проверки формы товара	Ввести цену больше 0 (не более 1 000 000)
39	Нельзя удалить категорию, к которой привязаны продукты.	Удаление категории (производителя) запрещено, пока к ней привязаны товары	Переназначить или скрыть товары и повторить
40	Нельзя удалить промокод: он используется в незавершенных заказах.	Промокод применен в активных заказах	Дождаться завершения заказов
41	Срок действия промокода уже истек.	Указана дата в прошлом	Указать сегодняшнюю или будущую дату
42	Нельзя снять роль с последнего администратора.	Попытка оставить систему без администратора	Сначала назначить другого администратора
43	Товар скрыт из каталога. / Товар снова доступен в каталоге.	Изменена видимость товара	Действий не требуется

ПРИЛОЖЕНИЕ Г. ТЕКСТ ПРОГРАММЫ АННОТАЦИЯ

В данном программном документе приведён текст программы для мобильного приложения и веб-панели управления «Интернет-магазин продуктов «Нектар».

В разделе «Текст программы» указано наименование программы, область применения программы, модули программы в виде таблицы с указанием описания, размера и количества строк, а также исходный код программы.

СОДЕРЖАНИЕ

1.	ТЕКСТ ПРОГРАММЫ.....	3
1.1	Наименование программы.....	3
1.2	Область применения программы.....	3
1.3	Модули.....	3
1.4	Код программы.....	6

1. ТЕКСТ ПРОГРАММЫ

1.1. Наименование программы

Наименование – «Интернет-магазин продуктов «Нектар».

1.2. Область применения программы

Программный комплекс предназначен для автоматизации процессов розничной торговли продуктами питания с доставкой. Комплекс включает мобильное приложение для покупателей и курьеров, административную веб-панель для сотрудников магазина и правила безопасности облачной базы данных; отдельной серверной части нет.

1.3. Модули

Таблица 1 – Модули мобильного приложения, веб-панели и правил безопасности

№	Название модуля	Описание модуля	Размер модуля	Кол-во строк
1	2	3	4	5
Запуск и корневые файлы приложения (lib/)				
1	main.dart	Точка входа: инициализация Firebase, провайдеров состояния и корневого виджета, выбор стартового экрана	5 КБ	154
2	main_screen.dart	Основной экран покупателя с нижней панелью навигации	2 КБ	70
3	session_app.dart	Оболочка сессии: тема приложения, язык, пересоздание MaterialApp при смене пользователя	5 КБ	123
Модели данных (lib/models/)				
4	category_model.dart	Модель категории	1 КБ	23
5	order_model.dart	Модель заказа	2 КБ	43
6	product_model.dart	Модель товара: разбор документа Firestore, тексты на языке интерфейса	5 КБ	143
7	user_model.dart	Модель профиля пользователя	2 КБ	45
Провайдеры состояния (lib/providers/)				
8	cart_provider.dart	Корзина: позиции, количество, промокод, расчет скидки, доставки и итога	5 КБ	161
9	favorite_provider.dart	Избранное: подписка на профиль и переключение товара	2 КБ	75
10	locale_provider.dart	Язык интерфейса с	2 КБ	48

№	Название модуля	Описание модуля	Размер модуля	Кол-во строк
1	2	3	4	5
		сохранением выбора		
Сервисы (lib/services/)				
11	auth_service.dart	Регистрация, вход по логину/email и через Google, сброс и смена пароля, выход	7 КБ	206
12	database_service.dart	Работа с Cloud Firestore: потоки данных, транзакция заказа, промокоды, справочники, уведомления, курьерские операции	57 КБ	1522
Вспомогательные модули (lib/utills/, lib/l10n/)				
13	app_palette.dart	Цветовая палитра приложения	2 КБ	24
14	auth_target.dart	Назначение входа (приложение / панель)	1 КБ	4
15	courier_orders.dart	Правила переходов и группировка заказов курьера	2 КБ	39
16	order_status.dart	Подписи статусов заказа	1 КБ	22
17	parsers.dart	Преобразование значений и форматирование дат	2 КБ	35
18	pricing.dart	Расчет доставки и итога, слоты доставки	2 КБ	53
19	product_filters.dart	Фильтрация и сортировка товаров	2 КБ	62
20	role_utills.dart	Роли и права доступа	2 КБ	69
21	validators.dart	Проверка полей форм	6 КБ	126
22	l10n.dart	Доступ к строкам локализации	2 КБ	34
Экраны запуска и авторизации (lib/screens/)				
23	splash_screen.dart	Экран загрузки	2 КБ	68
24	onboarding_screen.dart	Экран приветствия	7 КБ	167
25	auth_wrapper.dart	Определение роли и выбор экрана	6 КБ	167
26	login_screen.dart	Экран входа	12 КБ	308
27	register_screen.dart	Экран регистрации	11 КБ	273
28	forgot_password_screen.dart	Экран восстановления пароля	6 КБ	137
Экраны покупателя (lib/screens/)				
29	shop_screen.dart	Экран «Магазин»	11 КБ	306
30	category_screen.dart	Список товаров категории	4 КБ	93
31	product_detail_screen.dart	Карточка товара	14 КБ	345
32	explore_screen.dart	Экран «Каталог»: категории, поиск, фильтры	9 КБ	229
33	favorites_screen.dart	Экран «Избранное»	4 КБ	106
34	cart_screen.dart	Экран корзины	9 КБ	243
35	checkout_bottom_sheet.dart	Окно оформления заказа	15 КБ	403
36	account_screen.dart	Профиль пользователя	19 КБ	493

№	Название модуля	Описание модуля	Размер модуля	Кол-во строк
1	2	3	4	5
37	orders_screen.dart	История заказов	13 КБ	291
38	my_details_screen.dart	Редактирование имени и телефона	6 КБ	159
39	delivery_address_screen.dart	Адреса доставки	5 КБ	119
40	payment_methods_screen.dart	Банковские карты	5 КБ	121
41	promo_code_screen.dart	Промокоды	9 КБ	239
42	notifications_screen.dart	Уведомления	8 КБ	172
43	change_password_dialog.dart	Смена пароля	5 КБ	125
44	help_screen.dart	Помощь	2 КБ	54
45	about_screen.dart	О приложении	3 КБ	59
Экраны курьера и веб-заглушки (lib/screens/)				
46	courier_home_screen.dart	Экран курьера: свободные и свои заказы, взять, вернуть, начать и подтвердить доставку	16 КБ	386
47	public_web_landing.dart	Публичная страница Flutter Web	10 КБ	213
48	web_auth_wrapper.dart	Проверка роли в Flutter Web	2 КБ	54
49	web_role_blocked_screen.dart	Сообщение о запрете доступа	3 КБ	83
50	admin_home_screen.dart	Заклушка административной части Flutter Web	15 КБ	367
Общие виджеты (lib/widgets/)				
51	app_snackbar.dart	Всплывающие сообщения	1 КБ	32
52	centered_message.dart	Сообщение по центру экрана	1 КБ	32
53	input_dialog.dart	Диалог ввода строки	4 КБ	131
54	language_switcher.dart	Переключатель языка	3 КБ	63
55	product_card.dart	Карточка товара	5 КБ	128
56	product_filters_sheet.dart	Окно фильтров	6 КБ	159
57	product_grid.dart	Сетка товаров	1 КБ	30
58	product_image.dart	Изображение товара	1 КБ	32
Веб-панель управления (web/)				
59	index.html	Разметка веб-панели: форма входа и разделы	4 КБ	91
60	admin.js	Логика веб-панели: вход, разделы, CRUD, заказы, пользователи	55 КБ	1427
61	validation.js	Проверка форм и расчеты веб-панели	15 КБ	289
Правила безопасности и конфигурация Firebase				
62	firestore.rules	Правила безопасности Cloud Firestore	12 КБ	263
63	firebase.json	Конфигурация Firebase: правила, эмуляторы, хостинг	3 КБ	86

1.4. Код программы

lib/main.dart

```
import 'dart:async';

import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'app/session_app.dart';
import 'firebase_options.dart';
import 'providers/cart_provider.dart';
import 'providers/favorite_provider.dart';
import 'providers/locale_provider.dart';
import 'screens/auth/auth_wrapper.dart';
import 'screens/auth/login_screen.dart';
import 'screens/onboarding_screen.dart';
import 'screens/splash_screen.dart';
import 'screens/web/public_web_landing.dart';
import 'screens/web/web_auth_wrapper.dart';
import 'services/auth_service.dart';
import 'utils/app_palette.dart';

/// Адрес эмуляторов Firebase для локальной отладки (только в debug-сборке):
/// `flutter run --dart-define=FIREBASE_EMULATOR=localhost` (Android-эмулятор: 10.0.2.2).
/// Эмуляторы запускаются с `--project нестар-41848` (локально, в боевую базу не ходят).
/// Пусто — боевой проект.
const String _emulatorHost = String.fromEnvironment('FIREBASE_EMULATOR');

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  final useEmulator = kDebugMode && _emulatorHost.isNotEmpty;
```

```

if (useEmulator) {
  await FirebaseAuth.instance.useAuthEmulator(_emulatorHost, 9099);
  FirebaseFirestore.instance.useFirestoreEmulator(_emulatorHost, 8080);
}

runApp(
  MultiProvider(
    providers: [
      ChangeNotifierProvider(create: (_) => CartProvider()),
      ChangeNotifierProvider(create: (_) => FavoriteProvider()),
      ChangeNotifierProvider(create: (_) => LocaleProvider()..load()),
    ],
    child: const MyApp(),
  ),
);
}

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  final AuthService _authService = AuthService();
  late final Stream<User?> _authStream = _authService.authStateChanges;
  StreamSubscription<User?>? _sessionSubscription;
  String? _sessionUserId;

  bool _splashFinished = kIsWeb;
  bool _onboardingFinished = false;

  @override
  void initState() {
    super.initState();
    _sessionUserId = _authService.currentUser?.uid;
    context.read<FavoriteProvider>().bindUser(_sessionUserId);
    _sessionSubscription = _authService.authStateChanges.listen(_onAuthChanged);
  }
}

```

```

@override
void dispose() {
  _sessionSubscription?.cancel();
  super.dispose();
}

void _onAuthChanged(User? user) {
  if (!mounted || user?.uid == _sessionUserId) return;

  _sessionUserId = user?.uid;
  context.read<CartProvider>().clear();
  context.read<FavoriteProvider>().bindUser(_sessionUserId);
}

Widget _homeFor(User? user, {required bool authResolved}) {
  if (!authResolved) {
    return const _LoadingScreen();
  }

  if (!_splashFinished) {
    return SplashScreen(onFinished: () => setState(() => _splashFinished = true));
  }

  if (kIsWeb) {
    return user == null ? const PublicWebLandingScreen() : WebRoleGate(user: user);
  }

  if (user != null) {
    return AuthenticatedShell(user: user);
  }

  if (!_onboardingFinished) {
    return OnboardingScreen(onFinished: () => setState(() => _onboardingFinished = true));
  }

  return const LoginScreen();
}

@override
Widget build(BuildContext context) {
  return ValueListenableBuilder<bool>{

```

```

valueListenable: AuthService.gateHold,
builder: (context, isGateHeld, _) {
  return StreamBuilder<User?>(
    stream: _authStream,
    initialData: _authService.currentUser,
    builder: (context, snapshot) {
      // On the web the persisted session is restored asynchronously, so
      // the first frame must wait for the first auth event.
      final authResolved = !kIsWeb ||
        snapshot.connectionState != ConnectionState.waiting ||
        snapshot.data != null;
      final user = isGateHeld ? null : snapshot.data;

      return SessionApp(
        locale: context.watch<LocaleProvider>().locale,
        sessionId: authResolved ? user?.uid : null,
        home: _homeFor(user, authResolved: authResolved),
      );
    },
  );
},
);
}
}

```

```

class _LoadingScreen extends StatelessWidget {
  const _LoadingScreen();

  @override
  Widget build(BuildContext context) {
    return const Scaffold(
      body: Center(child: CircularProgressIndicator(color: AppPalette.primary)),
    );
  }
}

```

lib/main_screen.dart

```

import 'package:flutter/material.dart';

import 'screens/account/account_screen.dart';
import 'screens/cart/cart_screen.dart';
import 'screens/explore/explore_screen.dart';

```

```

import 'screens/shop/shop_screen.dart';
import 'l10n/l10n.dart';
import 'utils/app_palette.dart';

class MainScreen extends StatefulWidget {
  const MainScreen({super.key});

  @override
  State<MainScreen> createState() => _MainScreenState();
}

class _MainScreenState extends State<MainScreen> {
  int _currentIndex = 0;

  static final List<Widget> _pages = [
    ShopScreen(),
    ExploreScreen(),
    CartScreen(),
    AccountScreen(),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: IndexedStack(
        index: _currentIndex,
        children: _pages,
      ),
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _currentIndex,
        onTap: (index) {
          setState() {
            _currentIndex = index;
          };
        },
        selectedItemColor: AppPalette.primary,
        unselectedItemColor: Colors.grey,
        showUnselectedLabels: true,
        type: BottomNavigationBarType.fixed,
        items: [
          BottomNavigationBarItem(

```



```

        icon: Icon(Icons.store_outlined),
        activeIcon: Icon(Icons.store),
        label: context.l10n.navShop,
      ),
      BottomNavigationBarItem(
        icon: Icon(Icons.manage_search_outlined),
        activeIcon: Icon(Icons.manage_search),
        label: context.l10n.navCatalog,
      ),
      BottomNavigationBarItem(
        icon: Icon(Icons.shopping_cart_outlined),
        activeIcon: Icon(Icons.shopping_cart),
        label: context.l10n.navCart,
      ),
      BottomNavigationBarItem(
        icon: Icon(Icons.person_outline),
        activeIcon: Icon(Icons.person),
        label: context.l10n.navProfile,
      ),
    ],
  ),
);
}
}

```

lib/app/session_app.dart

```

import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';

import '../l10n/l10n.dart';
import '../utils/app_palette.dart';

final ThemeData nectarTheme = _buildTheme();

ThemeData _buildTheme() {
  final colorScheme = ColorScheme.fromSeed(seedColor: AppPalette.primary).copyWith(
    primary: AppPalette.primary,
    onPrimary: Colors.white,
    error: AppPalette.danger,
    onError: Colors.white,
    surface: Colors.white,
    onSurface: AppPalette.textPrimary,
  );
}

```

);

final buttonShape = RoundedRectangleBorder(borderRadius: BorderRadius.circular(14));

```
return ThemeData(  
  useMaterial3: true,  
  colorScheme: colorScheme,  
  scaffoldBackgroundColor: Colors.white,  
  appBarTheme: const AppBarTheme(  
    backgroundColor: Colors.white,  
    foregroundColor: AppPalette.textPrimary,  
    surfaceTintColor: Colors.transparent,  
    elevation: 0,  
  ),  
  elevatedButtonTheme: ElevatedButtonThemeData(  
    style: ElevatedButton.styleFrom(  
      backgroundColor: AppPalette.primary,  
      foregroundColor: Colors.white,  
      disabledBackgroundColor: AppPalette.disabledBackground,  
      disabledForegroundColor: AppPalette.disabledForeground,  
      shape: buttonShape,  
    ),  
  ),  
  filledButtonTheme: FilledButtonThemeData(  
    style: FilledButton.styleFrom(  
      backgroundColor: AppPalette.primary,  
      foregroundColor: Colors.white,  
      disabledBackgroundColor: AppPalette.disabledBackground,  
      disabledForegroundColor: AppPalette.disabledForeground,  
      shape: buttonShape,  
    ),  
  ),  
  textButtonTheme: TextButtonThemeData(  
    style: TextButton.styleFrom(foregroundColor: AppPalette.primary),  
  ),  
  outlinedButtonTheme: OutlinedButtonThemeData(  
    style: OutlinedButton.styleFrom(  
      foregroundColor: AppPalette.primary,  
      side: const BorderSide(color: AppPalette.primary),  
      shape: buttonShape,  
    ),  
  ),  
);
```

```

    ),
    inputDecorationTheme: InputDecorationTheme(
      hintStyle: const TextStyle(color: AppPalette.textSecondary),
      focusedBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(16),
        borderSide: const BorderSide(color: AppPalette.primary, width: 2),
      ),
    ),
    snackBarTheme: const SnackBarThemeData(
      contentTextStyle: TextStyle(color: Colors.white, fontSize: 15, fontWeight: FontWeight.w600),
    ),
    switchTheme: SwitchThemeData(
      thumbColor: WidgetStateProperty.resolveWith(
        (states) => states.contains(WidgetState.selected) ? Colors.white : Colors.grey.shade600,
      ),
      trackColor: WidgetStateProperty.resolveWith(
        (states) => states.contains(WidgetState.selected) ? AppPalette.primary : Colors.grey.shade300,
      ),
      // Включенный переключатель: зеленая дорожка и белый «бегунок» без обводки.
      trackOutlineColor: WidgetStateProperty.resolveWith(
        (states) => states.contains(WidgetState.selected) ? Colors.transparent : Colors.grey.shade500,
      ),
    ),
  );
}

```

```

/// The [MaterialApp] for one auth session.

```

```

///

```

```

/// It is keyed by [sessionId], so a sign-in or sign-out rebuilds the whole

```

```

/// Navigator/Overlay tree in a single frame. Swapping only the page inside a

```

```

/// live Navigator leaves routes, dialogs and bottom sheets alive while the

```

```

/// InheritedElements they depend on are deactivated, which fails Flutter's

```

```

/// `_dependents.isEmpty` assertion.

```

```

class SessionApp extends StatelessWidget {

```

```

  const SessionApp({

```

```

    super.key,

```

```

    required this.sessionId,

```

```

    required this.home,

```

```

    this.theme,

```

```

    this.locale = AppLocale.russian,

```

```

  });

```

```

final String? sessionId;
final Widget home;
final ThemeData? theme;
final Locale locale;

@override
Widget build(BuildContext context) {
  return MaterialApp(
    key: ValueKey<String>(sessionId ?? 'guest'),
    debugShowCheckedModeBanner: false,
    onGenerateTitle: (context) => context.l10n.appTitle,
    theme: theme ?? nectarTheme,
    locale: locale,
    supportedLocales: AppLocalizations.supportedLocales,
    localizationsDelegates: const [
      AppLocalizations.delegate,
      GlobalMaterialLocalizations.delegate,
      GlobalWidgetsLocalizations.delegate,
      GlobalCupertinoLocalizations.delegate,
    ],
    home: home,
  );
}
}

```

lib/models/category_model.dart

```

class CategoryModel {
  final String id;
  final String name;

  CategoryModel({
    required this.id,
    required this.name,
  });

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'name': name.trim(),
    };
  }
}

```

```

factory CategoryModel.fromJson(Map<String, dynamic> json) {
  return CategoryModel(
    id: json['id'] ?? "",
    name: json['name'] ?? "",
  );
}
}

```

lib/models/order_model.dart

```

class OrderModel {
  final String id;
  final String userId;
  final List<Map<String, dynamic>> items;
  final double totalAmount;
  final String status;
  final String address;
  final DateTime createdAt;

```

```

  OrderModel({
    required this.id,
    required this.userId,
    required this.items,
    required this.totalAmount,
    required this.status,
    required this.address,
    required this.createdAt,
  });

```

```

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'userId': userId,
      'items': items,
      'totalAmount': totalAmount,
      'status': status,
      'address': address.trim(),
      'createdAt': createdAt.toIso8601String(),
    };
  }
}

```

```

factory OrderModel.fromJson(Map<String, dynamic> json) {

```

```

return OrderModel(
  id: json['id'] ?? "",
  userId: json['userId'] ?? "",
  items: List<Map<String, dynamic>>.from(json['items'] ?? []),
  totalAmount: (json['totalAmount'] ?? 0).toDouble(),
  status: json['status'] ?? 'В работе',
  address: json['address'] ?? "",
  createdAt: DateTime.parse(json['createdAt'] ?? DateTime.now().toIso8601String()),
);
}
}

```

lib/models/product_model.dart

```

import '../data/catalog_translations.dart';
import '../utils/parsers.dart';

```

```

class Product {
  const Product({
    required this.id,
    required this.name,
    required this.price,
    required this.stock,
    required this.qty,
    required this.description,
    required this.imageUrl,
    required this.categoryName,
    this.nameEn = "",
    this.descriptionEn = "",
    this.compositionEn = "",
    this.categoryNameEn = "",
    this.brand = "",
    this.country = "",
    this.composition = "",
    this.calories = 0,
    this.proteins = 0,
    this.fats = 0,
    this.carbs = 0,
    this.expiryDate,
    this.popularity = 0,
  });
}

```

```

final String id;

```

```

final String name;
final double price;
final int stock;
final String qty;
final String description;
final String imageUrl;
final String categoryName;
final String nameEn;
final String descriptionEn;
final String compositionEn;
final String categoryNameEn;
final String brand;
final String country;
final String composition;
final double calories;
final double proteins;
final double fats;
final double carbs;
final DateTime? expiryDate;
final int popularity;

bool get hasNutrition => calories > 0 || proteins > 0 || fats > 0 || carbs > 0;

bool get isOutOfStock => stock <= 0;

String nameFor({required bool english}) => _pick(english, name, nameEn);

String descriptionFor({required bool english}) => _pick(english, description, descriptionEn);

String compositionFor({required bool english}) => _pick(english, composition, compositionEn);

String categoryFor({required bool english}) => _pick(english, categoryName, categoryNameEn);

String qtyFor({required bool english}) => localizeQty(qty, english: english);

String countryFor({required bool english}) {
  return english ? (countryTranslations[country] ?? country) : country;
}

static String _pick(bool english, String russian, String englishValue) {
  return english && englishValue.trim().isEmpty ? englishValue : russian;
}

```

```
}
```

```
factory Product.fromMap(Map<String, dynamic> map) {  
    final categoryName = toStringValue(map['categoryName']).isEmpty  
        ? toStringValue(map['categoryName'])  
        : toStringValue(map['category']);  
    final description = toStringValue(map['description']).isEmpty  
        ? toStringValue(map['description'])  
        : toStringValue(map['details']);  
  
    final id = toStringValue(map['id']);  
    // Для товаров стартового каталога английские тексты берутся из приложения,  
    // если в базе их еще нет; значения из базы имеют приоритет.  
    final builtIn = productTranslations[id];  
    final builtInCategory = categoryTranslations[toStringValue(map['categoryId'])];  
    String english(String field, String? fallback) {  
        final stored = toStringValue(map[field]);  
        return stored.isEmpty ? stored : (fallback ?? "");  
    }  
  
    return Product(  
        id: id,  
        name: toStringValue(map['name']),  
        price: toDoubleValue(map['price']),  
        stock: toIntValue(map['stockQuantity'] ?? map['stock']),  
        qty: toStringValue(map['qty']),  
        description: description,  
        imageUrl: toStringValue(map['imageUrl']),  
        categoryName: categoryName,  
        nameEn: english('nameEn', builtIn?.name),  
        descriptionEn: english('descriptionEn', builtIn?.description),  
        compositionEn: english('compositionEn', builtIn?.composition),  
        categoryNameEn: english('categoryNameEn', builtInCategory?.name),  
        brand: toStringValue(map['manufacturerName']),  
        country: toStringValue(map['country']),  
        composition: toStringValue(map['composition']),  
        calories: toDoubleValue(map['calories']),  
        proteins: toDoubleValue(map['proteins']),  
        fats: toDoubleValue(map['fats']),  
        carbs: toDoubleValue(map['carbs']),  
        expiryDate: map['expiryDate'] == null ? null : toDateTimeValue(map['expiryDate']),  
    );  
}
```



```

        popularity: toIntValue(map['popularity']),
    );
}

static final RegExp _unitPattern = RegExp(
    r'(?<=\d)\s*(kg|ml|pcs|кг|мл|шт|г|л|л)(?![\p{L}])',
    caseSensitive: false,
    unicode: true,
);

static const Map<String, (String, String)> _units = {
    'kg': ('кг', 'kg'),
    'кг': ('кг', 'kg'),
    'g': ('г', 'g'),
    'г': ('г', 'g'),
    'ml': ('мл', 'ml'),
    'мл': ('мл', 'ml'),
    'l': ('л', 'l'),
    'л': ('л', 'l'),
    'pcs': ('шт', 'pcs'),
    'шт': ('шт', 'pcs'),
};

/// Приводит единицы измерения («1 kg», «500 г») к языку интерфейса.
static String localizeQty(String value, {required bool english}) {
    return value.replaceAllMapped(_unitPattern, (match) {
        final unit = _units[match.group(1)!.toLowerCase()];
        if (unit == null) return match.group(0)!;
        return ' ${english ? unit.$2 : unit.$1}';
    });
}
}

```

lib/models/user_model.dart

```

class UserModel {
    final String id;
    final String login;
    final String name;
    final String email;
    final String phoneNumber;
    final String role;
    final String? photoUrl;
}

```

```
UserModel({
  required this.id,
  required this.login,
  required this.name,
  required this.email,
  required this.phoneNumber,
  required this.role,
  this.photoUrl,
});
```

```
Map<String, dynamic> toJson() {
  return {
    'uid': id,
    'login': login.trim(),
    'loginNormalized': login.trim().toLowerCase(),
    'displayName': name.trim(),
    'email': email.trim().toLowerCase(),
    'emailNormalized': email.trim().toLowerCase(),
    'phoneNumber': phoneNumber.trim(),
    'role': role,
    'photoUrl': photoUrl,
  };
}
```

```
factory UserModel.fromJson(Map<String, dynamic> json) {
  return UserModel(
    id: json['uid'] ?? json['id'] ?? "",
    login: json['login'] ?? "",
    name: json['displayName'] ?? json['name'] ?? "",
    email: json['email'] ?? "",
    phoneNumber: json['phoneNumber'] ?? "",
    role: json['role'] ?? 'buyer',
    photoUrl: json['photoUrl'],
  );
}
```

lib/providers/cart_provider.dart

```
import 'dart:math' as math;
```

```
import 'package:flutter/foundation.dart';
```

```

import '../models/product_model.dart';
import '../utils/pricing.dart';

class CartItem {
  const CartItem({
    required this.id,
    required this.name,
    required this.quantity,
    required this.price,
    required this.imageUrl,
    required this.qty,
    this.stockQuantity,
    this.nameEn = "",
  });

  final String id;
  final String name;
  final String nameEn;
  final int quantity;
  final double price;
  final String imageUrl;
  final String qty;
  final int? stockQuantity;

  double get lineTotal => price * quantity;

  String nameFor({required bool english}) => english && nameEn.isNotEmpty ? nameEn : name;

  String qtyFor({required bool english}) => Product.localizeQty(qty, english: english);

  CartItem copyWith({int? quantity, int? stockQuantity}) {
    return CartItem(
      id: id,
      name: name,
      quantity: quantity ?? this.quantity,
      price: price,
      imageUrl: imageUrl,
      qty: qty,
      stockQuantity: stockQuantity ?? this.stockQuantity,
      nameEn: nameEn,
    );
  }
}

```

```

    );
}
}

class CartProvider with ChangeNotifier {
    static const int maxQuantityPerItem = 99;

    final Map<String, CartItem> _items = {};
    String? _appliedPromoCode;
    double _discountPercent = 0;

    Map<String, CartItem> get items => Map.unmodifiable(_items);

    String? get appliedPromoCode => _appliedPromoCode;

    double get discountPercent => _discountPercent;

    int get itemCount => _items.length;

    int get totalQuantity => _items.values.fold(0, (sum, item) => sum + item.quantity);

    bool get isEmpty => _items.isEmpty;

    double get subtotalAmount => _items.values.fold(0.0, (sum, item) => sum + item.lineTotal);

    double get discountAmount => subtotalAmount * (_discountPercent / 100);

    double get amountAfterDiscount => math.max(0, subtotalAmount - discountAmount);

    double get deliveryAmount => calculateDeliveryFee(amountAfterDiscount);

    double get totalAmount => amountAfterDiscount + deliveryAmount;

    bool get hasPromoCode => _appliedPromoCode != null && _appliedPromoCode!.isNotEmpty;

    void applyPromoCode(String promoCode, double percent) {
        _appliedPromoCode = promoCode;
        _discountPercent = percent;
        notifyListeners();
    }
}

```

```

void clearPromoCode() {
    _appliedPromoCode = null;
    _discountPercent = 0;
    notifyListeners();
}

/// Adds one unit of [product]. Returns false when the stock limit is reached.
bool addProduct(Product product) {
    final existingItem = _items[product.id];
    final maxAllowed = math.min(product.stock, maxQuantityPerItem);
    final nextQuantity = (existingItem?.quantity ?? 0) + 1;

    if (product.isOutOfStock || nextQuantity > maxAllowed) {
        return false;
    }

    _items[product.id] = existingItem?.copyWith(
        quantity: nextQuantity,
        stockQuantity: product.stock,
    ) ??
    CartItem(
        id: product.id,
        name: product.name,
        quantity: nextQuantity,
        price: product.price,
        imageUrl: product.imageUrl,
        qty: product.qty,
        stockQuantity: product.stock,
        nameEn: product.nameEn,
    );
    notifyListeners();
    return true;
}

/// Adds one more unit of an item that is already in the cart.
bool increment(String productId) {
    final item = _items[productId];
    if (item == null) return false;

    final maxAllowed = math.min(item.stockQuantity ?? maxQuantityPerItem, maxQuantityPerItem);
    if (item.quantity + 1 > maxAllowed) {

```

```

        return false;
    }

    _items[productId] = item.copyWith(quantity: item.quantity + 1);
    notifyListeners();
    return true;
}

void decrement(String productId) {
    final item = _items[productId];
    if (item == null) return;

    if (item.quantity > 1) {
        _items[productId] = item.copyWith(quantity: item.quantity - 1);
    } else {
        _items.remove(productId);
    }
    notifyListeners();
}

void removeItem(String productId) {
    if (_items.remove(productId) != null) {
        notifyListeners();
    }
}

void clear() {
    if (_items.isEmpty && _appliedPromoCode == null) return;

    _items.clear();
    _appliedPromoCode = null;
    _discountPercent = 0;
    notifyListeners();
}
}

```

lib/providers/favorite_provider.dart

```

import 'dart:async';

import 'package:flutter/foundation.dart';

import '../services/database_service.dart';

```

```

/// Keeps the ids of the signed-in user's favorite products, synced with the
/// `favorites` array of the user document. Product data itself is always read
/// from the live catalog so price and stock never go stale.
class FavoriteProvider with ChangeNotifier {
  FavoriteProvider({DatabaseService? database}) : _databaseOverride = database;

  final DatabaseService? _databaseOverride;
  late final DatabaseService _database = _databaseOverride ?? DatabaseService();

  Set<String> _ids = <String>{};
  StreamSubscription<Set<String>>? _subscription;
  String? _userId;

  Set<String> get ids => Set.unmodifiable(_ids);

  int get count => _ids.length;

  bool isFavorite(String id) => _ids.contains(id);

  void bindUser(String? userId) {
    if (userId == _userId) return;

    _userId = userId;
    _subscription?.cancel();
    _subscription = null;

    if (_ids.isNotEmpty) {
      _ids = <String>{};
      notifyListeners();
    }

    if (userId == null) return;

    _subscription = _database.watchFavoriteIds(userId).listen(
      (ids) {
        _ids = ids;
        notifyListeners();
      },
      onError: (Object error) => debugPrint('Favorites sync failed: $error'),
    );
  }
}

```

```
}
```

```
Future<void> toggle(String productId) async {  
  final userId = _userId;  
  if (userId == null) return;  
  
  final shouldAdd = !_ids.contains(productId);  
  _ids = shouldAdd ? {..._ids, productId} : {..._ids}..remove(productId);  
  notifyListeners();  
  
  try {  
    await _database.setFavorite(  
      userId: userId,  
      productId: productId,  
      isFavorite: shouldAdd,  
    );  
  } catch (error) {  
    debugPrint('Could not update favorites: $error');  
    _ids = shouldAdd ? {..._ids}..remove(productId) : {..._ids, productId};  
    notifyListeners();  
  }  
}
```

```
@override  
void dispose() {  
  _subscription?.cancel();  
  super.dispose();  
}
```

lib/providers/locale_provider.dart

```
import 'dart:ui' as ui;  
  
import 'package:flutter/material.dart';  
import 'package:shared_preferences/shared_preferences.dart';  
  
import '../l10n/l10n.dart';  
  
/// Хранит выбранный язык приложения (русский по умолчанию, английский вторым).  
class LocaleProvider with ChangeNotifier {  
  static const String _storageKey = 'app_language';
```



```

Locale _locale = AppLocale.russian;

Locale get locale => _locale;

bool get isEnglish => _locale.languageCode == 'en';

/// Загружает сохраненный язык. Если он не выбран, берется язык устройства
/// (только русский или английский), иначе русский.
Future<void> load() async {
  String? stored;
  try {
    stored = (await SharedPreferences.getInstance()).getString(_storageKey);
  } catch (_) {
    stored = null;
  }

  final code = stored ?? ui.PlatformDispatcher.instance.locale.languageCode;
  _apply(code == 'en' ? AppLocale.english : AppLocale.russian);
}

Future<void> setLocale(Locale locale) async {
  if (locale.languageCode == _locale.languageCode) return;

  _apply(locale);
  try {
    await (await SharedPreferences.getInstance()).setString(_storageKey, locale.languageCode);
  } catch (_) {
    // Язык применится, но не запомнится до следующего запуска.
  }
}

void _apply(Locale locale) {
  _locale = locale;
  AppLocale.current = locale;
  notifyListeners();
}
}

```

lib/services/auth_service.dart

```

import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/foundation.dart';
import 'package:google_sign_in/google_sign_in.dart';

```

```

import 'database_service.dart';
import '../models/user_model.dart';
import '../l10n/l10n.dart';
import '../utils/role_utils.dart';

class AuthService {
  factory AuthService() => _instance;

  AuthService._();

  static final AuthService _instance = AuthService._();

  /// While true the app keeps showing the signed-out UI even though Firebase
  /// already has a user. Login/registration flip it so the signed-in shell is
  /// only built after the profile has been created and validated.
  static final ValueNotifier<bool> gateHold = ValueNotifier<bool>(false);

  final FirebaseAuth _auth = FirebaseAuth.instance;
  final GoogleSignIn _googleSignIn = GoogleSignIn();
  final DatabaseService _dbService = DatabaseService();

  User? get currentUser => _auth.currentUser;

  Stream<User?> get authStateChanges => _auth.authStateChanges();

  String getErrorMessage(Object error) {
    if (error is DatabaseOperationException) {
      return error.message;
    }
  }

  final l10n = AppLocale.strings;

  if (error is FirebaseAuthException) {
    switch (error.code) {
      case 'invalid-email':
        return l10n.authInvalidEmail;
      case 'user-disabled':
        return l10n.authUserDisabled;
      case 'user-not-found':
      case 'wrong-password':

```

```

        case 'invalid-credential':
            return l10n.authWrongCredentials;
        case 'email-already-in-use':
            return l10n.authEmailInUse;
        case 'weak-password':
            return l10n.authWeakPassword;
        case 'network-request-failed':
            return l10n.authNetworkError;
        case 'too-many-requests':
            return l10n.authTooManyRequests;
    }
}

return l10n.genericError;
}

Future<UserCredential> registerBuyer({
  required String login,
  required String name,
  required String email,
  required String phoneNumber,
  required String password,
}) async {
  final normalizedEmail = email.trim().toLowerCase();
  final normalizedName = name.trim();
  final normalizedLogin = login.trim();

  gateHold.value = true;
  try {
    await _dbService.assertLoginAvailable(normalizedLogin);

    final userCredential = await _auth.createUserWithEmailAndPassword(
      email: normalizedEmail,
      password: password,
    );

    try {
      await userCredential.user?.updateDisplayName(normalizedName);

      await _dbService.createUser(
        UserModel(

```

```

        id: userCredential.user!.uid,
        login: normalizedLogin,
        name: normalizedName,
        email: normalizedEmail,
        phoneNumber: phoneNumber.trim(),
        role: AppRoles.buyer,
    ),
);
return userCredential;
} catch (_) {
    await userCredential.user?.delete();
    rethrow;
}
} finally {
    gateHold.value = false;
}
}

```

```

Future<UserCredential> login(String loginOrEmail, String password) async {
    final resolvedEmail = await _dbService.resolveEmailForIdentifier(loginOrEmail);
    return _signInGuarded(
        () => _auth.signInWithEmailAndPassword(
            email: resolvedEmail,
            password: password,
        ),
    );
}

```

```

Future<UserCredential?> signInWithGoogle() async {
    final googleUser = await _googleSignIn.signIn();
    if (googleUser == null) return null;

    final googleAuth = await googleUser.authentication;
    final credential = GoogleAuthProvider.credential(
        accessToken: googleAuth.accessToken,
        idToken: googleAuth.idToken,
    );

    return _signInGuarded(() => _auth.signInWithCredential(credential));
}

```

```

Future<UserCredential> _signInGuarded(Future<UserCredential> Function() signIn) async {
  gateHold.value = true;
  try {
    final userCredential = await signIn();
    final user = userCredential.user;
    if (user != null) {
      await _rejectIfDeleted(user);
    }
    return userCredential;
  } finally {
    gateHold.value = false;
  }
}

```

```

Future<void> _rejectIfDeleted(User user) async {
  // Сначала читаем профиль: удаленному пользователю запись в профиль запрещена
  // правилами, а сбой чтения не должен пропускать его в приложение.
  Map<String, dynamic>? profile;
  try {
    profile = await _dbService.getUserProfile(user.uid);
  } catch (error) {
    debugPrint('Profile check after sign-in failed: $error');
  }

  if (profile?['isDeleted'] == true) {
    await logout();
    throw DatabaseOperationException(AppLocale.strings.authAccountDeleted);
  }

  try {
    await _dbService.ensureUserProfileFromAuth(user);
  } catch (error) {
    debugPrint('Profile sync after sign-in failed: $error');
  }
}

```

```

Future<void> sendPasswordReset(String loginOrEmail) async {
  _auth.setLanguageCode(AppLocale.current.languageCode);
  final resolvedEmail = await _dbService.resolveEmailForIdentifier(loginOrEmail);
  await _auth.sendPasswordResetEmail(email: resolvedEmail);
}

```

```

bool get canChangePassword {
  return _auth.currentUser?.providerData.any((info) => info.providerId == 'password') ?? false;
}

/// Перед установкой нового пароля запрашивает текущий (повторная аутентификация).
Future<void> changePassword({
  required String currentPassword,
  required String newPassword,
}) async {
  final user = _auth.currentUser;
  final email = user?.email;
  if (user == null || email == null) {
    throw DatabaseOperationException(AppLocale.strings.authNotSignedIn);
  }

  try {
    await user.reauthenticateWithCredential(
      EmailAuthProvider.credential(email: email, password: currentPassword),
    );
  } on FirebaseAuthException catch (error) {
    if (error.code == 'wrong-password' || error.code == 'invalid-credential') {
      throw DatabaseOperationException(AppLocale.strings.authWrongCurrentPassword);
    }
    rethrow;
  }

  await user.updatePassword(newPassword);
}

Future<void> logout() async {
  try {
    await _googleSignIn.signOut();
  } catch (error) {
    debugPrint('Google sign-out failed: $error');
  }
  await _auth.signOut();
}

```

lib/services/database_service.dart

```
import 'dart:convert';
```

```

import 'dart:typed_data';

import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';

import '../data/catalog_translations.dart';
import '../data/product_catalog.dart';
import '../l10n/l10n.dart';
import '../models/product_model.dart';
import '../models/user_model.dart';
import '../utils/parsers.dart';
import '../utils/pricing.dart';
import '../utils/role_utils.dart';

class DatabaseOperationException implements Exception {
  final String message;

  const DatabaseOperationException(this.message);

  @override
  String toString() => message;
}

class DatabaseService {
  DatabaseService({
    FirebaseFirestore? firestore,
    String? Function()? currentUserId,
  }) : _database = firestore ?? FirebaseFirestore.instance,
       _currentUserId = currentUserId ?? () => FirebaseAuth.instance.currentUser?.uid;

  final FirebaseFirestore _database;
  final String? Function() _currentUserId;

  CollectionReference<Map<String, dynamic>> get _usersCollection => _database.collection('users');
  CollectionReference<Map<String, dynamic>> get _productsCollection => _database.collection('products');
  CollectionReference<Map<String, dynamic>> get _categoriesCollection =>
    _database.collection('categories');
  CollectionReference<Map<String, dynamic>> get _manufacturersCollection =>
    _database.collection('manufacturers');
  CollectionReference<Map<String, dynamic>> get _suppliersCollection => _database.collection('suppliers');
  CollectionReference<Map<String, dynamic>> get _rolesCollection => _database.collection('roles');

```

```

CollectionReference<Map<String, dynamic>> get _promoCodesCollection =>
_database.collection('promoCodes');
CollectionReference<Map<String, dynamic>> get _orderStatusesCollection =>
_database.collection('orderStatuses');
CollectionReference<Map<String, dynamic>> get _ordersCollection => _database.collection('orders');
CollectionReference<Map<String, dynamic>> get _notificationsCollection =>
_database.collection('notifications');

/// Публичный индекс «логин -> email»: по нему гость находит почту при входе
/// по логину, не имея доступа к самим профилям. Документ создается один раз,
/// что гарантирует уникальность логина.
CollectionReference<Map<String, dynamic>> get _loginIndexCollection =>
_database.collection('loginIndex');

Future<void> createUser(UserModel user) async {
  final normalizedLogin = user.login.trim().toLowerCase();
  final normalizedEmail = user.email.trim().toLowerCase();
  final indexReference = _loginIndexCollection.doc(normalizedLogin);
  final userReference = _usersCollection.doc(user.id);

  await _database.runTransaction((transaction) async {
    final indexDocument = await transaction.get(indexReference);
    if (indexDocument.exists && indexDocument.data()?['uid'] != user.id) {
      throw DatabaseOperationException(AppLocale.strings.errLoginTaken);
    }

    if (!indexDocument.exists) {
      transaction.set(indexReference, {'uid': user.id, 'email': normalizedEmail});
    }

    transaction.set(userReference, {
      ...user.toJson(),
      'addresses': <String>[],
      'cards': <String>[],
      'notificationSettings': {
        'push': true,
        'sms': false,
        'email': true,
      },
      'isDeleted': false,
      'createdAt': FieldValue.serverTimestamp(),
    });
  });
}

```



```

        'updatedAt': FieldValue.serverTimestamp(),
    }, SetOptions(merge: true));
});
}

Future<void> ensureUserProfileFromAuth(
    User firebaseUser, {
    String defaultRole = AppRoles.buyer,
}) async {
    final userReference = _usersCollection.doc(firebaseUser.uid);
    final email = (firebaseUser.email ?? "").trim().toLowerCase();

    await _database.runTransaction((transaction) async {
        final userDocument = await transaction.get(userReference);
        final currentData = userDocument.data() ?? <String, dynamic>{};

        var loginValue = (currentData['login'] ??
            _buildDefaultLogin(email, firebaseUser.displayName ?? ""))
            .toString();

        var indexReference = _loginIndexCollection.doc(loginValue.trim().toLowerCase());
        var indexDocument = await transaction.get(indexReference);
        var shouldCreateIndex = !indexDocument.exists;
        final loginTakenByAnother = indexDocument.exists && indexDocument.data()?['uid'] !=
firebaseUser.uid;

        if (loginTakenByAnother && currentData['login'] == null) {
            // Сгенерированный логин занят другим пользователем: подбираем свободный.
            loginValue = '$loginValue${firebaseUser.uid.substring(0, 4).toLowerCase()}';
            indexReference = _loginIndexCollection.doc(loginValue.toLowerCase());
            indexDocument = await transaction.get(indexReference);
            shouldCreateIndex = !indexDocument.exists;
        }

        final defaults = <String, dynamic>{
            'uid': firebaseUser.uid,
            'login': loginValue,
            'loginNormalized': loginValue.trim().toLowerCase(),
            'email': email,
            'emailNormalized': email,
            'displayName': firebaseUser.displayName ?? 'Пользователь',
            'phoneNumber': "",

```

```

        'role': defaultRole,
        'addresses': <String>[],
        'cards': <String>[],
        'notificationSettings': {
            'push': true,
            'sms': false,
            'email': true,
        },
        'isDeleted': false,
        'createdAt': FieldValue.serverTimestamp(),
    };

    final updates = <String, dynamic>{
        for (final entry in defaults.entries)
            if (currentData[entry.key] == null) entry.key: entry.value,
    };

    final authPhotoUrl = firebaseUser.photoURL;
    if (authPhotoUrl != null && authPhotoUrl != currentData['photoUrl']) {
        updates['photoUrl'] = authPhotoUrl;
    }

    if (shouldCreateIndex && email.isNotEmpty) {
        transaction.set(indexReference, {'uid': firebaseUser.uid, 'email': email});
    }

    if (updates.isEmpty) {
        return;
    }

    updates['updatedAt'] = FieldValue.serverTimestamp();
    transaction.set(userReference, updates, SetOptions(merge: true));
});
}

Future<String> resolveEmailForIdentifier(String loginOrEmail) async {
    final normalizedIdentifier = loginOrEmail.trim();

    if (normalizedIdentifier.contains('@')) {
        return normalizedIdentifier.toLowerCase();
    }
}

```

```

final indexDocument = await _loginIndexCollection.doc(normalizedIdentifier.toLowerCase()).get();
final email = _stringValue(indexDocument.data()?['email']);
if (email.isEmpty) {
  throw DatabaseOperationException(AppLocale.strings.errLoginNotFound);
}
return email;
}

```

```

Future<Map<String, dynamic>?> getUserProfile(String userId) async {
  final userDocument = await _usersCollection.doc(userId).get();
  if (!userDocument.exists) {
    return null;
  }
  return _documentToMap(userDocument);
}

```

```

Stream<Map<String, dynamic>?> watchUserProfile(String userId) {
  return _usersCollection.doc(userId).snapshots().map((userDocument) {
    if (!userDocument.exists) {
      return null;
    }
    return _documentToMap(userDocument);
  });
}

```

```

Stream<List<Map<String, dynamic>>> watchProducts({
  bool includeDeleted = false,
}) {
  return _productsCollection.snapshots().map((snapshot) {
    final products = snapshot.docs
      .map(_documentToMap)
      .where((product) => includeDeleted || product['isDeleted'] != true)
      .toList();
    products.sort((leftProduct, rightProduct) {
      return _stringValue(leftProduct['name']).compareTo(_stringValue(rightProduct['name']));
    });
    return products;
  });
}

```

```

Stream<List<Product>>> watchCatalog() {
    return watchProducts().map((products) => products.map(Product.fromMap).toList());
}

Stream<List<Map<String, dynamic>>>> watchCategories() {
    return _watchOrderedCollection(_categoriesCollection, 'name');
}

Stream<List<Map<String, dynamic>>>> watchManufacturers() {
    return _watchOrderedCollection(_manufacturersCollection, 'name');
}

Stream<List<Map<String, dynamic>>>> watchSuppliers() {
    return _watchOrderedCollection(_suppliersCollection, 'name');
}

Stream<List<Map<String, dynamic>>>> watchRoles() {
    return _watchOrderedCollection(_rolesCollection, 'name');
}

Stream<List<Map<String, dynamic>>>> watchPromoCodes() {
    return _promoCodesCollection.snapshots().map((snapshot) {
        final promoCodes = snapshot.docs.map(_documentToMap).toList();
        promoCodes.sort((leftPromoCode, rightPromoCode) {
            return _stringValue(leftPromoCode['code']).compareTo(_stringValue(rightPromoCode['code']));
        });
        return promoCodes;
    });
}

Stream<List<Map<String, dynamic>>>> watchOrderStatuses() {
    return _watchOrderedCollection(_orderStatusesCollection, 'name');
}

Stream<List<Map<String, dynamic>>>> watchUsers() {
    return _usersCollection.snapshots().map((snapshot) {
        final users = snapshot.docs
            .map(_documentToMap)
            .where((user) => user['isDeleted'] != true)
            .toList();
        users.sort((leftUser, rightUser) {

```

```

        return _stringValue(leftUser['displayName']).compareTo(_stringValue(rightUser['displayName']));
    });
    return users;
});
}

```

```

Stream<List<Map<String, dynamic>>> watchOrders() {
    return _ordersCollection.snapshots().map((snapshot) {
        final orders = snapshot.docs.map(_documentToMap).toList();
        orders.sort((leftOrder, rightOrder) {
            return _dateTimeValue(rightOrder['createdAt']).compareTo(_dateTimeValue(leftOrder['createdAt']));
        });
        return orders;
    });
}

```

```

Stream<List<Map<String, dynamic>>> watchOrdersForUser(String userId) {
    return _watchOrdersWhere('userId', userId);
}

```

```

Stream<List<Map<String, dynamic>>> watchAssignedOrders(String courierId) {
    return _watchOrdersWhere('courierId', courierId);
}

```

```

Stream<List<Map<String, dynamic>>> _watchOrdersWhere(String field, String value) {
    return _ordersCollection.where(field, isEqualTo: value).snapshots().map((snapshot) {
        final orders = snapshot.docs.map(_documentToMap).toList();
        orders.sort((leftOrder, rightOrder) {
            return _dateTimeValue(rightOrder['createdAt']).compareTo(_dateTimeValue(leftOrder['createdAt']));
        });
        return orders;
    });
}

```

/// Свободные заказы, которые курьер может взять: без курьера и еще не в работе.

/// Старые заказы идут первыми.

```

Stream<List<Map<String, dynamic>>> watchAvailableOrders() {
    return _ordersCollection
        .where('courierId', isNull: true)
        .where('statusId', whereIn: const ['new', 'processing'])
        .snapshots()
}

```

```

        .map((snapshot) {
            final orders = snapshot.docs.map(_documentToMap).toList();
            orders.sort((leftOrder, rightOrder) {
                return _dateTimeValue(leftOrder['createdAt']).compareTo(_dateTimeValue(rightOrder['createdAt']));
            });
            return orders;
        });
    }
}

```

```

Stream<List<Map<String, dynamic>>> watchNotifications(String userId) {
    return _notificationsCollection.where('userId', isEqualTo: userId).snapshots().map((snapshot) {
        final notifications = snapshot.docs.map(_documentToMap).toList();
        notifications.sort((leftNotification, rightNotification) {
            return
                _dateTimeValue(rightNotification['createdAt']).compareTo(_dateTimeValue(leftNotification['createdAt']));
        });
        return notifications;
    });
}

```

```

Stream<Set<String>> watchFavoriteIds(String userId) {
    return _usersCollection.doc(userId).snapshots().map((userDocument) {
        final favorites = userDocument.data()?['favorites'];
        if (favorites is! List) {
            return <String>{};
        }
        return favorites.map((id) => id.toString()).toSet();
    });
}

```

```

Future<void> setFavorite({
    required String userId,
    required String productId,
    required bool isFavorite,
}) async {
    await _usersCollection.doc(userId).set({
        'favorites': isFavorite
            ? FieldValue.arrayUnion([productId])
            : FieldValue.arrayRemove([productId]),
        'updatedAt': FieldValue.serverTimestamp(),
    }, SetOptions(merge: true));
}

```

```
}
```

```
Future<void> updateUserProfile({  
    required String userId,  
    required String displayName,  
    required String phoneNumber,  
}) async {  
    await _usersCollection.doc(userId).set({  
        'displayName': displayName.trim(),  
        'phoneNumber': phoneNumber.trim(),  
        'updatedAt': FieldValue.serverTimestamp(),  
    }, SetOptions(merge: true));  
}
```

```
/// Максимальный размер аватара после сжатия. Картинка хранится прямо в  
/// документе профиля (Firebase Storage платный), поэтому она маленькая.  
static const int maxAvatarBytes = 150 * 1024;
```

```
/// Сохраняет аватар в профиле как base64 (JPEG/PNG уже уменьшены на устройстве).
```

```
Future<void> saveUserAvatar({  
    required String userId,  
    required Uint8List imageBytes,  
}) async {  
    if (imageBytes.isEmpty || imageBytes.length > maxAvatarBytes) {  
        throw DatabaseOperationException(AppLocale.strings.avatarTooBig);  
    }  
    await _usersCollection.doc(userId).set({  
        'photoData': base64Encode(imageBytes),  
        'updatedAt': FieldValue.serverTimestamp(),  
    }, SetOptions(merge: true));  
}
```

```
Future<void> saveNotificationSettings({  
    required String userId,  
    required bool pushEnabled,  
    required bool smsEnabled,  
    required bool emailEnabled,  
}) async {  
    await _usersCollection.doc(userId).set({  
        'notificationSettings': {  
            'push': pushEnabled,
```

```

        'sms': smsEnabled,
        'email': emailEnabled,
    },
    'updatedAt': FieldValue.serverTimestamp(),
}, SetOptions(merge: true));
}

```

```

Future<void> addDeliveryAddress({
    required String userId,
    required String address,
}) async {
    final normalizedAddress = address.trim();
    if (normalizedAddress.isEmpty) {
        throw DatabaseOperationException(AppLocale.strings.errAddressEmpty);
    }
    if (normalizedAddress.length > 300) {
        throw DatabaseOperationException(AppLocale.strings.errAddressTooLong);
    }

```

```

    await _usersCollection.doc(userId).set({
        'addresses': FieldValue.arrayUnion([normalizedAddress]),
        'updatedAt': FieldValue.serverTimestamp(),
    }, SetOptions(merge: true));
}

```

```

Future<void> removeDeliveryAddress({
    required String userId,
    required String address,
}) async {
    await _usersCollection.doc(userId).set({
        'addresses': FieldValue.arrayRemove([address]),
        'updatedAt': FieldValue.serverTimestamp(),
    }, SetOptions(merge: true));
}

```

```

Future<void> addPaymentCard({
    required String userId,
    required String cardNumber,
}) async {
    final normalizedCardNumber = cardNumber.replaceAll(RegExp(r'\D'), "");
    if (normalizedCardNumber.length != 16) {

```



```

        throw DatabaseOperationException(AppLocale.strings.errCardLength);
    }

    final maskedCardNumber = '**** *' + normalizedCardNumber.substring(12) + '*';

    await _usersCollection.doc(userId).set({
        'cards': FieldValue.arrayUnion([maskedCardNumber]),
        'updatedAt': FieldValue.serverTimestamp(),
    }, SetOptions(merge: true));
}

Future<void> removePaymentCard({
    required String userId,
    required String maskedCardNumber,
}) async {
    await _usersCollection.doc(userId).set({
        'cards': FieldValue.arrayRemove([maskedCardNumber]),
        'updatedAt': FieldValue.serverTimestamp(),
    }, SetOptions(merge: true));
}

Future<Map<String, dynamic>> validatePromoCode(String promoCode) async {
    final normalizedPromoCode = promoCode.trim().toUpperCase();
    final invalidMessage = AppLocale.strings.errPromoInvalid;

    final promoCodes = await _promoCodesCollection
        .where('codeNormalized', isEqualTo: normalizedPromoCode)
        .get();
    final currentDate = DateTime.now();

    for (final promoCodeDocument in promoCodes.docs) {
        final promoCodeData = promoCodeDocument.data();
        if (promoCodeData['isActive'] == false) {
            continue;
        }

        if (_dateTimeValue(promoCodeData['expiresAt']).isBefore(currentDate)) {
            continue;
        }

        return _documentToMap(promoCodeDocument);
    }
}

```

```

    }

    throw DatabaseOperationException(invalidMessage);
}

Future<Map<String, dynamic>> placeOrder({
    required String userId,
    required List<Map<String, dynamic>> orderItems,
    required String deliveryAddress,
    required String paymentMethod,
    String? paymentCard,
    String? deliverySlot,
    Map<String, dynamic>? promoCode,
}) async {
    if (orderItems.isEmpty) {
        throw DatabaseOperationException(AppLocale.strings.errCartEmpty);
    }

    if (deliveryAddress.trim().isEmpty) {
        throw DatabaseOperationException(AppLocale.strings.errDeliveryAddressRequired);
    }

    if (paymentMethod.trim().isEmpty) {
        throw DatabaseOperationException(AppLocale.strings.errPaymentMethodRequired);
    }

    // Одинаковые товары объединяются: остаток проверяется по суммарному количеству.
    final mergedItems = <String, Map<String, dynamic>>{};
    for (final orderItem in orderItems) {
        final productId = _stringValue(orderItem['id']);
        final productQuantity = _intValue(orderItem['quantity']);
        if (productQuantity <= 0) {
            throw DatabaseOperationException(AppLocale.strings.errQuantityPositive);
        }
        final existingItem = mergedItems[productId];
        mergedItems[productId] = {
            ...(existingItem ?? orderItem),
            'quantity': (existingItem == null ? 0 : _intValue(existingItem['quantity'])) + productQuantity,
        };
    }
    final normalizedOrderItems = mergedItems.values.toList();
}

```

```

final userDocument = await _usersCollection.doc(userId).get();
final userData = userDocument.data();
if (!userDocument.exists || userData == null) {
    throw DatabaseOperationException(AppLocale.strings.errUserNotFound);
}

final orderStatusDocument = await _orderStatusesCollection.doc('new').get();
final orderStatusData = orderStatusDocument.data() ??
{
    'code': 'new',
    'name': 'Новый',
};

final orderDocumentReference = _ordersCollection.doc();

final createdOrder = await _database.runTransaction<Map<String, dynamic>>((transaction) async {
    // Firestore требует, чтобы все чтения шли до записей.
    final productDocs = <String, DocumentSnapshot<Map<String, dynamic>>>{};
    for (final orderItem in normalizedOrderItems) {
        final productId = _stringValue(orderItem['id']);
        productDocs[productId] = await transaction.get(_productsCollection.doc(productId));
    }

    double subtotalAmount = 0;
    final normalizedItems = <Map<String, dynamic>>[];
    final stockUpdates = <DocumentReference<Map<String, dynamic>>, int>{};

    for (final orderItem in normalizedOrderItems) {
        final productId = _stringValue(orderItem['id']);
        final productQuantity = _intValue(orderItem['quantity']);
        if (productQuantity <= 0) {
            throw DatabaseOperationException(AppLocale.strings.errQuantityPositive);
        }

        final productDocument = productDocs[productId]!;
        final productData = productDocument.data();

        if (productData == null || productData['isDeleted'] == true) {
            throw
DatabaseOperationException(AppLocale.strings.errProductUnavailable(orderItem['name'].toString()));

```

```

    }

    final productStockQuantity = _intValue(productData['stockQuantity'] ?? productData['stock'] ?? 0);
    if (productStockQuantity < productQuantity) {
        throw
DatabaseOperationException(AppLocale.strings.errNotEnoughStock(_stringValue(productData['name'])));
    }

    final productPrice = _doubleValue(productData['price']);
    subtotalAmount += productPrice * productQuantity;

    normalizedItems.add({
        'id': productId,
        'name': productData['name'],
        'nameEn': _stringValue(productData['nameEn']).isEmpty
            ? productData['nameEn']
            : (productTranslations[productId]?.name ?? ''),
        'price': productPrice,
        'quantity': productQuantity,
        'imageUrl': productData['imageUrl'],
        'categoryName': productData['categoryName'],
    });

    stockUpdates[productDocument.reference] = productStockQuantity - productQuantity;
}

// Записи выполняются только после проверки всех позиций.
stockUpdates.forEach((reference, remainingStock) {
    transaction.update(reference, {
        'stockQuantity': remainingStock,
        'updatedAt': FieldValue.serverTimestamp(),
    });
});

final discountPercent = _intValue(promoCode?['discountPercent']);
final discountAmount = subtotalAmount * discountPercent / 100;
final deliveryAmount = calculateDeliveryFee(subtotalAmount - discountAmount);
final totalAmount = subtotalAmount - discountAmount + deliveryAmount;

final orderData = {
    'number': 'ORD-${DateTime.now().millisecondsSinceEpoch}',

```

```

'userId': userId,
'customerName': userData['displayName'] ?? 'Покупатель',
'customerEmail': userData['email'] ?? '',
'customerPhone': userData['phoneNumber'] ?? '',
'customerRole': userData['role'] ?? AppRoles.buyer,
'address': deliveryAddress.trim(),
'paymentMethod': paymentMethod.trim(),
'paymentCard': paymentCard,
'deliverySlot': deliverySlot,
'promoCodeId': promoCode?['id'],
'promoCode': promoCode?['code'],
'discountPercent': discountPercent,
'discountAmount': discountAmount,
'subtotalAmount': subtotalAmount,
'deliveryAmount': deliveryAmount,
'totalAmount': totalAmount,
'statusId': orderStatusDocument.id,
'statusName': orderStatusData['name'] ?? 'Новый',
'courierId': null,
'courierName': null,
'items': normalizedItems,
'createdAt': FieldValue.serverTimestamp(),
'updatedAt': FieldValue.serverTimestamp(),
};

transaction.set(orderDocumentReference, orderData);

if (promoCode != null) {
  transaction.update(_promoCodesCollection.doc(promoCode['id'].toString()), {
    'usageCount': FieldValue.increment(1),
    'updatedAt': FieldValue.serverTimestamp(),
  });
}

return {
  'id': orderDocumentReference.id,
  ...orderData,
};
});

return createdOrder;

```

```

}

Future<void> updateOrder({
    required String orderId,
    required String statusId,
    String? courierId,
}) async {
    final statusSnapshot = await _orderStatusesCollection.doc(statusId).get();
    final fallbackStatusSnapshot = await _orderStatusesCollection.doc('processing').get();
    final statusData = statusSnapshot.data() ?? fallbackStatusSnapshot.data() ?? {'name': 'В обработке'};

    final orderDocument = await _ordersCollection.doc(orderId).get();
    final orderData = orderDocument.data();
    if (orderData == null) {
        throw DatabaseOperationException(AppLocale.strings.errOrderNotFound);
    }

    String? courierName;
    String? courierPhone;
    if (courierId != null && courierId.isNotEmpty) {
        final courierDocument = await _usersCollection.doc(courierId).get();
        courierName = courierDocument.data()?['displayName']?.toString();
        courierPhone = courierDocument.data()?['phoneNumber']?.toString();
    }

    await _ordersCollection.doc(orderId).set({
        'statusId': statusId,
        'statusName': statusData['name'] ?? 'В обработке',
        'courierId': courierId,
        'courierName': courierName,
        'courierPhone': courierPhone,
        'updatedAt': FieldValue.serverTimestamp(),
    }, SetOptions(merge: true));

    if (statusId == 'delivered' && orderData['statusId'] != 'delivered') {
        final orderNumber = orderData['number'] ?? orderId.padRight(6, '0').substring(0, 6).toUpperCase();
        await createNotification(
            userId: _stringValue(orderData['userId']),
            text: 'Заказ №$orderNumber доставлен',
            textEn: 'Order #$orderNumber has been delivered',
        );
    }
}

```

```

    }
}

/// Курьер берет свободный заказ. Транзакция гарантирует, что два курьера
/// не возьмут один заказ: второй получит [DatabaseOperationException].
Future<void> claimOrder({required String orderId, required String courierId}) async {
    final courierData = (await _usersCollection.doc(courierId).get()).data();
    final orderReference = _ordersCollection.doc(orderId);

    await _database.runTransaction((transaction) async {
        final orderDocument = await transaction.get(orderReference);
        final orderData = orderDocument.data();
        if (orderData == null) {
            throw DatabaseOperationException(AppLocale.strings.errOrderNotFound);
        }

        final isFree = orderData['courierId'] == null && const {'new',
'processing'}.contains(orderData['statusId']);
        if (!isFree) {
            throw DatabaseOperationException(AppLocale.strings.errOrderAlreadyTaken);
        }

        transaction.update(orderReference, {
            'statusId': 'assigned',
            'statusName': 'Передан курьеру',
            'courierId': courierId,
            'courierName': courierData?['displayName']?.toString() ?? '',
            'courierPhone': courierData?['phoneNumber']?.toString() ?? '',
            'updatedAt': FieldValue.serverTimestamp(),
        });

        final customerId = _stringValue(orderData['userId']);
        if (customerId.isNotEmpty) {
            final orderNumber = orderData['number'] ?? orderId;
            transaction.set(_notificationsCollection.doc(), {
                'userId': customerId,
                'text': 'Курьер взял ваш заказ №$orderNumber',
                'textEn': 'A courier has taken your order #$orderNumber',
                'createdAt': FieldValue.serverTimestamp(),
                'updatedAt': FieldValue.serverTimestamp(),
            });
        }
    });
}

```

```

    }
  });
}

/// Курьер возвращает свой заказ в пул, пока не выехал.
Future<void> releaseOrder({required String orderId, required String courierId}) async {
  final orderReference = _ordersCollection.doc(orderId);

  await _database.runTransaction((transaction) async {
    final orderData = (await transaction.get(orderReference)).data();
    if (orderData == null) {
      throw DatabaseOperationException(AppLocale.strings.errOrderNotFound);
    }
    if (orderData['courierId'] != courierId || orderData['statusId'] != 'assigned') {
      throw DatabaseOperationException(AppLocale.strings.errOrderCannotRelease);
    }

    transaction.update(orderReference, {
      'statusId': 'processing',
      'statusName': 'В сборке',
      'courierId': null,
      'courierName': null,
      'courierPhone': null,
      'updatedAt': FieldValue.serverTimestamp(),
    });
  });
}

Future<void> createNotification({
  required String userId,
  required String text,
  String textEn = "",
}) async {
  await _notificationsCollection.add({
    'userId': userId,
    'text': text,
    'textEn': textEn,
    'createdAt': FieldValue.serverTimestamp(),
    'updatedAt': FieldValue.serverTimestamp(),
  });
}

```



```

Future<void> clearNotifications(String userId) async {
  final notificationsSnapshot = await _notificationsCollection.where('userId', isEqualTo: userId).get();

  const batchLimit = 400;
  for (var start = 0; start < notificationsSnapshot.docs.length; start += batchLimit) {
    final batch = _database.batch();
    final end = start + batchLimit < notificationsSnapshot.docs.length
      ? start + batchLimit
      : notificationsSnapshot.docs.length;
    for (final notificationDocument in notificationsSnapshot.docs.sublist(start, end)) {
      batch.delete(notificationDocument.reference);
    }
    await batch.commit();
  }
}

```

/// Заказ физически не удаляется: для финансовой отчетности он только
 /// скрывается из списка покупателя.

```

Future<void> hideOrderForUser(String orderId) async {
  await _ordersCollection.doc(orderId).set({
    'hiddenByUser': true,
    'updatedAt': FieldValue.serverTimestamp(),
  }, SetOptions(merge: true));
}

```

```

Future<void> confirmOrderReceived(String orderId) async {
  await _ordersCollection.doc(orderId).set({
    'userConfirmed': true,
    'updatedAt': FieldValue.serverTimestamp(),
  }, SetOptions(merge: true));
}

```

```

Future<void> deleteUser(String userId) async {
  await assertUserCanBeDeleted(userId);
  await _setUserDeleted(userId, true);
}

```

```

Future<void> restoreUser(String userId) async {
  await _setUserDeleted(userId, false);
}

```

```

/// Кто пометил профиль удаленным, хранится в `deletedBy`: по нему правила
/// Firestore отличают самоудаление (его владелец может откатить) от блокировки
/// администратором (снять ее может только администратор).
Future<void> _setUserDeleted(String userId, bool isDeleted) async {
  await _usersCollection.doc(userId).set({
    'isDeleted': isDeleted,
    if (isDeleted) 'deletedBy': _currentUserId() ?? userId,
    if (!isDeleted) 'deletedBy': FieldValue.delete(),
    'deletedAt': isDeleted ? FieldValue.serverTimestamp() : FieldValue.delete(),
    'updatedAt': FieldValue.serverTimestamp(),
  }, SetOptions(merge: true));
}

/// Освобождает логин пользователя перед удалением его учетной записи и
/// возвращает данные записи, чтобы при неудаче ее можно было вернуть.
Future<Map<String, dynamic>?> releaseLoginIndex(String userId) async {
  final profile = await getUserProfile(userId);
  final login = _stringValue(profile?['loginNormalized']).isNotEmpty
    ? _stringValue(profile?['loginNormalized'])
    : _stringValue(profile?['login']).toLowerCase();
  if (login.isEmpty) {
    return null;
  }

  final indexReference = _loginIndexCollection.doc(login);
  final indexDocument = await indexReference.get();
  final indexData = indexDocument.data();
  if (indexData == null || indexData['uid'] != userId) {
    return null;
  }

  await indexReference.delete();
  return {'login': login, ...indexData};
}

Future<void> restoreLoginIndex(Map<String, dynamic>? released) async {
  if (released == null) return;
  await _loginIndexCollection.doc(released['login'].toString()).set({
    'uid': released['uid'],
    'email': released['email'],
  });
}

```

```
});
}
```

```
Future<void> assertUserCanBeDeleted(String userId) async {
  final ordersSnapshot = await _ordersCollection.where('userId', isEqualTo: userId).get();
  for (final orderDocument in ordersSnapshot.docs) {
    final orderData = orderDocument.data();
    final statusId = _stringValue(orderData['statusId']);
    final statusName = _stringValue(orderData['statusName']).toLowerCase();
    final isFinished = statusId == 'delivered' || statusId == 'cancelled' || _isFinishedOrderStatus(statusName);
    if (!isFinished) {
      throw DatabaseOperationException(AppLocale.strings.errUserHasActiveOrders);
    }
  }
}
```

```
Future<void> saveCategory({
  String? categoryId,
  required String name,
  required String description,
}) async {
  final normalizedName = name.trim();
  if (normalizedName.isEmpty) {
    throw DatabaseOperationException(AppLocale.strings.errCategoryNameRequired);
  }
  if (normalizedName.length > 100) {
    throw DatabaseOperationException(AppLocale.strings.errCategoryNameTooLong);
  }
  if (description.trim().length > 500) {
    throw DatabaseOperationException(AppLocale.strings.errCategoryDescriptionTooLong);
  }
}
```

```
await _ensureUniqueValue(
  collection: _categoriesCollection,
  fieldName: 'nameNormalized',
  normalizedValue: normalizedName.toLowerCase(),
  currentDocumentId: categoryId,
  errorMessage: AppLocale.strings.errCategoryExists,
);
```

```
final categoryReference = categoryId == null || categoryId.isEmpty
```

```

        ? _categoriesCollection.doc()
        : _categoriesCollection.doc(categoryId);

await categoryReference.set({
    'name': normalizedName,
    'nameNormalized': normalizedName.toLowerCase(),
    'description': description.trim(),
    'updatedAt': FieldValue.serverTimestamp(),
    'createdAt': FieldValue.serverTimestamp(),
}, SetOptions(merge: true));
}

Future<void> deleteCategory(String categoryId) async {
    final categoryDocument = await _categoriesCollection.doc(categoryId).get();
    final categoryData = categoryDocument.data();
    if (categoryData == null) {
        throw DatabaseOperationException(AppLocale.strings.errCategoryNotFound);
    }

    final productsSnapshot = await _productsCollection.get();
    for (final productDocument in productsSnapshot.docs) {
        final productData = productDocument.data();
        final categoryLinkedById = productData['categoryId'] == categoryId;
        final categoryLinkedByName = _stringValue(productData['categoryName']) ==
        _stringValue(categoryData['name']) ||
        _stringValue(productData['category']) == _stringValue(categoryData['name']);

        if (categoryLinkedById || categoryLinkedByName) {
            throw DatabaseOperationException(AppLocale.strings.errCategoryHasProducts);
        }
    }

    await _categoriesCollection.doc(categoryId).delete();
}

Future<void> saveManufacturer({
    String? manufacturerId,
    required String name,
    required String country,
    required String contactPhone,
}) async {

```

```

final normalizedName = name.trim();
if (normalizedName.isEmpty) {
    throw DatabaseOperationException(AppLocale.strings.errManufacturerNameRequired);
}
if (normalizedName.length > 100) {
    throw DatabaseOperationException(AppLocale.strings.errManufacturerNameTooLong);
}

await _ensureUniqueValue(
    collection: _manufacturersCollection,
    fieldName: 'nameNormalized',
    normalizedValue: normalizedName.toLowerCase(),
    currentDocumentId: manufacturerId,
    errorMessage: AppLocale.strings.errManufacturerExists,
);

final manufacturerReference = manufacturerId == null || manufacturerId.isEmpty
    ? _manufacturersCollection.doc()
    : _manufacturersCollection.doc(manufacturerId);

await manufacturerReference.set({
    'name': normalizedName,
    'nameNormalized': normalizedName.toLowerCase(),
    'country': country.trim(),
    'contactPhone': contactPhone.trim(),
    'updatedAt': FieldValue.serverTimestamp(),
    'createdAt': FieldValue.serverTimestamp(),
}, SetOptions(merge: true));
}

Future<void> deleteManufacturer(String manufacturerId) async {
    final manufacturerDocument = await _manufacturersCollection.doc(manufacturerId).get();
    final manufacturerData = manufacturerDocument.data();
    if (manufacturerData == null) {
        throw DatabaseOperationException(AppLocale.strings.errManufacturerNotFound);
    }

    final productsSnapshot = await _productsCollection.get();
    for (final productDocument in productsSnapshot.docs) {
        final productData = productDocument.data();
        final isLinked = productData['manufacturerId'] == manufacturerId ||

```

```

        _stringValue(productData['manufacturerName']) == _stringValue(manufacturerData['name']));
    if (isLinked) {
        throw DatabaseOperationException(AppLocale.strings.errManufacturerHasProducts);
    }
}

await _manufacturersCollection.doc(manufacturerId).delete();
}

Future<void> saveSupplier({
    String? supplierId,
    required String name,
    required String taxId,
    required String address,
    required String contactPhone,
    required String email,
}) async {
    final normalizedName = name.trim();
    if (normalizedName.isEmpty) {
        throw DatabaseOperationException(AppLocale.strings.errSupplierNameRequired);
    }

    await _ensureUniqueValue(
        collection: _suppliersCollection,
        fieldName: 'nameNormalized',
        normalizedValue: normalizedName.toLowerCase(),
        currentDocumentId: supplierId,
        errorMessage: AppLocale.strings.errSupplierExists,
    );

    final supplierReference = supplierId == null || supplierId.isEmpty
        ? _suppliersCollection.doc()
        : _suppliersCollection.doc(supplierId);

    await supplierReference.set({
        'name': normalizedName,
        'nameNormalized': normalizedName.toLowerCase(),
        'taxId': taxId.trim(),
        'address': address.trim(),
        'contactPhone': contactPhone.trim(),
        'email': email.trim().toLowerCase(),
    });
}

```

```

        'updatedAt': FieldValue.serverTimestamp(),
        'createdAt': FieldValue.serverTimestamp(),
    }, SetOptions(merge: true));
}

```

```

Future<void> deleteSupplier(String supplierId) async {
    final supplierDocument = await _suppliersCollection.doc(supplierId).get();
    final supplierData = supplierDocument.data();
    if (supplierData == null) {
        throw DatabaseOperationException(AppLocale.strings.errSupplierNotFound);
    }

    final productsSnapshot = await _productsCollection.get();
    for (final productDocument in productsSnapshot.docs) {
        final productData = productDocument.data();
        final isLinked = productData['supplierId'] == supplierId ||
            _stringValue(productData['supplierName']) == _stringValue(supplierData['name']);
        if (isLinked) {
            throw DatabaseOperationException(AppLocale.strings.errSupplierHasSupplies);
        }
    }

    await _suppliersCollection.doc(supplierId).delete();
}

```

```

Future<void> saveRole({
    String? roleId,
    required String name,
}) async {
    final normalizedName = name.trim();
    if (normalizedName.isEmpty) {
        throw DatabaseOperationException(AppLocale.strings.errRoleNameRequired);
    }
    if (normalizedName.length > 50) {
        throw DatabaseOperationException(AppLocale.strings.errRoleNameTooLong);
    }

    final normalizedRoleCode = normalizedName.toLowerCase().replaceAll(RegExp(r'^a-zA-я0-9]+'), '_');

    await _ensureUniqueValue(
        collection: _rolesCollection,

```

```

        fieldName: 'nameNormalized',
        normalizedValue: normalizedName.toLowerCase(),
        currentDocumentId: roleId,
        errorMessage: AppLocale.strings.errRoleExists,
    );

    final roleReference = roleId == null || roleId.isEmpty
        ? _rolesCollection.doc(normalizedRoleCode)
        : _rolesCollection.doc(roleId);

    await roleReference.set({
        'name': normalizedName,
        'nameNormalized': normalizedName.toLowerCase(),
        'code': roleReference.id,
        'updatedAt': FieldValue.serverTimestamp(),
        'createdAt': FieldValue.serverTimestamp(),
    }, SetOptions(merge: true));
}

Future<void> deleteRole(String roleId) async {
    final usersSnapshot = await _usersCollection.get();
    for (final userDocument in usersSnapshot.docs) {
        final userData = userDocument.data();
        if (userData['role'] == roleId) {
            throw DatabaseOperationException(AppLocale.strings.errRoleHasUsers);
        }
    }

    await _rolesCollection.doc(roleId).delete();
}

Future<void> saveOrderStatus({
    String? statusId,
    required String name,
}) async {
    final normalizedName = name.trim();
    if (normalizedName.isEmpty) {
        throw DatabaseOperationException(AppLocale.strings.errStatusNameRequired);
    }
    if (normalizedName.length > 50) {
        throw DatabaseOperationException(AppLocale.strings.errStatusNameTooLong);
    }
}

```



```

}

await _ensureUniqueValue(
    collection: _orderStatusesCollection,
    fieldName: 'nameNormalized',
    normalizedValue: normalizedName.toLowerCase(),
    currentDocumentId: statusId,
    errorMessage: AppLocale.strings.errStatusExists,
);

final normalizedCode = normalizedName.toLowerCase().replaceAll(RegExp(r'^a-zA-я0-9'+'), '_');
final statusReference = statusId == null || statusId.isEmpty
    ? _orderStatusesCollection.doc(normalizedCode)
    : _orderStatusesCollection.doc(statusId);

await statusReference.set({
    'name': normalizedName,
    'nameNormalized': normalizedName.toLowerCase(),
    'code': statusReference.id,
    'updatedAt': FieldValue.serverTimestamp(),
    'createdAt': FieldValue.serverTimestamp(),
}, SetOptions(merge: true));
}

Future<void> deleteOrderStatus(String statusId) async {
    final ordersSnapshot = await _ordersCollection.get();
    for (final orderDocument in ordersSnapshot.docs) {
        final orderData = orderDocument.data();
        if (orderData['statusId'] == statusId) {
            throw DatabaseOperationException(AppLocale.strings.errStatusInUse);
        }
    }

    await _orderStatusesCollection.doc(statusId).delete();
}

Future<void> savePromoCode({
    String? promoCodeId,
    required String code,
    required int discountPercent,
    required DateTime expiresAt,

```

```

}) async {
    final normalizedCode = code.trim().toUpperCase();
    if (normalizedCode.isEmpty) {
        throw DatabaseOperationException(AppLocale.strings.errPromoCodeRequired);
    }
    if (!RegExp(r'^[A-Z0-9]+$').hasMatch(normalizedCode)) {
        throw DatabaseOperationException(AppLocale.strings.errPromoCodeChars);
    }
    if (discountPercent < 1 || discountPercent > 99) {
        throw DatabaseOperationException(AppLocale.strings.errPromoDiscountRange);
    }
    if (expiresAt.isBefore(DateTime.now().subtract(const Duration(days: 1)))) {
        throw DatabaseOperationException(AppLocale.strings.errPromoExpiryPast);
    }

    await _ensureUniqueValue(
        collection: _promoCodesCollection,
        fieldName: 'codeNormalized',
        normalizedValue: normalizedCode,
        currentDocumentId: promoCodeId,
        errorMessage: AppLocale.strings.errPromoCodeExists,
    );

    final promoCodeReference = promoCodeId == null || promoCodeId.isEmpty
        ? _promoCodesCollection.doc()
        : _promoCodesCollection.doc(promoCodeId);

    await promoCodeReference.set({
        'code': normalizedCode,
        'codeNormalized': normalizedCode,
        'discountPercent': discountPercent,
        'expiresAt': Timestamp.fromDate(expiresAt),
        'isActive': true,
        'updatedAt': FieldValue.serverTimestamp(),
        'createdAt': FieldValue.serverTimestamp(),
    }, SetOptions(merge: true));
}

Future<void> deletePromoCode(String promoCodeId) async {
    final ordersSnapshot = await _ordersCollection.get();
    for (final orderDocument in ordersSnapshot.docs) {

```

```

        final orderData = orderDocument.data();
        final isPromoLinked = orderData['promoCodeId'] == promoCodeId;
        final statusName = _stringValue(orderData['statusName']).toLowerCase();
        if (isPromoLinked && !_isFinishedOrderStatus(statusName)) {
            throw DatabaseOperationException(AppLocale.strings.errPromoInActiveOrders);
        }
    }

    await _promoCodesCollection.doc(promoCodeId).delete();
}

Future<void> saveProduct({
    String? productId,
    required String name,
    required String description,
    required double price,
    required int stockQuantity,
    required String categoryId,
    required String manufacturerId,
    String? supplierId,
    required String quantityLabel,
    String imageUrl = "",
    DateTime? expiryDate,
}) async {
    if (name.trim().isEmpty) {
        throw DatabaseOperationException(AppLocale.strings.errProductNameRequired);
    }
    if (name.trim().length > 255) {
        throw DatabaseOperationException(AppLocale.strings.errProductNameTooLong);
    }
    if (description.trim().length > 2000) {
        throw DatabaseOperationException(AppLocale.strings.errProductDescriptionTooLong);
    }
    if (price <= 0) {
        throw DatabaseOperationException(AppLocale.strings.errProductPricePositive);
    }
    if (stockQuantity < 0 || stockQuantity > 9999) {
        throw DatabaseOperationException(AppLocale.strings.errProductStockRange);
    }

    final categoryDocument = await _categoriesCollection.doc(categoryId).get();

```

```

final manufacturerDocument = await _manufacturersCollection.doc(manufacturerId).get();
final supplierDocument = supplierId == null || supplierId.isEmpty
    ? null
    : await _suppliersCollection.doc(supplierId).get();

if (!categoryDocument.exists || !manufacturerDocument.exists) {
    throw DatabaseOperationException(AppLocale.strings.errProductNeedsCategory);
}

final productReference = productId == null || productId.isEmpty
    ? _productsCollection.doc()
    : _productsCollection.doc(productId);

await productReference.set({
    'name': name.trim(),
    'description': description.trim(),
    'price': price,
    'stockQuantity': stockQuantity,
    'qty': quantityLabel.trim(),
    'categoryId': categoryId,
    'categoryName': categoryDocument.data()?['name'],
    'manufacturerId': manufacturerId,
    'manufacturerName': manufacturerDocument.data()?['name'],
    'supplierId': supplierId,
    'supplierName': supplierDocument?.data()?['name'],
    'imageUrl': imageUrl.trim(),
    'expiryDate': expiryDate == null ? null : Timestamp.fromDate(expiryDate),
    'minStock': 5,
    'isDeleted': false,
    'updatedAt': FieldValue.serverTimestamp(),
    'createdAt': FieldValue.serverTimestamp(),
}, SetOptions(merge: true));
}

Future<void> deleteProduct(String productId) async {
    final ordersSnapshot = await _ordersCollection.get();
    for (final orderDocument in ordersSnapshot.docs) {
        final orderData = orderDocument.data();
        final orderItems = List<Map<String, dynamic>>.from(orderData['items'] ?? []);
        final statusName = _stringValue(orderData['statusName']).toLowerCase();
        final containsProduct = orderItems.any((orderItem) => orderItem['id'] == productId);
    }
}

```

```

    if (containsProduct && !_isFinishedOrderStatus(statusName)) {
        await _productsCollection.doc(productId).set({
            'isDeleted': true,
            'updatedAt': FieldValue.serverTimestamp(),
            'deletedAt': FieldValue.serverTimestamp(),
        }, SetOptions(merge: true));
        return;
    }
}

await _productsCollection.doc(productId).delete();
}

Future<Map<String, dynamic>> getDashboardData() async {
    final productsSnapshot = await _productsCollection.get();
    final usersSnapshot = await _usersCollection.get();
    final ordersSnapshot = await _ordersCollection.get();

    final products = productsSnapshot.docs.map(_documentToMap).where((product) => product['isDeleted'] !
= true).toList();
    final users = usersSnapshot.docs.map(_documentToMap).where((user) => user['isDeleted'] !=
true).toList();
    final orders = ordersSnapshot.docs.map(_documentToMap).toList();
    final now = DateTime.now();

    final expiringProducts = products.where((product) {
        final expiryDate = product['expiryDate'];
        if (expiryDate == null) {
            return false;
        }
        final parsedDate = _dateTimeValue(expiryDate);
        return parsedDate.isAfter(now.subtract(const Duration(days: 1))) &&
            parsedDate.isBefore(now.add(const Duration(days: 7)));
    }).toList();

    expiringProducts.sort((leftProduct, rightProduct) {
        return
        _dateTimeValue(leftProduct['expiryDate']).compareTo(_dateTimeValue(rightProduct['expiryDate']));
    });

    final buyersCount = users.where((user) => user['role'] == AppRoles.buyer).length;

```

```

final monthlySales = <String, double>{};
final productPopularity = <String, int>{};

for (final order in orders) {
    final orderDate = _dateTimeValue(order['createdAt']);
    final monthKey = '${orderDate.year}-${orderDate.month.toString().padLeft(2, '0')}';
    monthlySales[monthKey] = (monthlySales[monthKey] ?? 0) + _doubleValue(order['totalAmount']);

    final orderItems = List<Map<String, dynamic>>.from(order['items'] ?? []);
    for (final orderItem in orderItems) {
        final productName = _stringValue(orderItem['name']);
        productPopularity[productName] = (productPopularity[productName] ?? 0) +
        _intValue(orderItem['quantity']);
    }
}

final topProducts = productPopularity.entries.toList()
    ..sort((leftEntry, rightEntry) => rightEntry.value.compareTo(leftEntry.value));

return {
    'expiringProducts': expiringProducts.take(5).toList(),
    'buyersCount': buyersCount,
    'monthlySales': monthlySales,
    'topProducts': topProducts.take(5).map((entry) {
        return {
            'name': entry.key,
            'count': entry.value,
        };
    }).toList(),
};
}

Future<List<Map<String, dynamic>>> getCategoriesList() async {
    final categoriesSnapshot = await _categoriesCollection.get();
    final categories = categoriesSnapshot.docs.map(_documentToMap).toList();
    categories.sort((leftCategory, rightCategory) {
        return _stringValue(leftCategory['name']).compareTo(_stringValue(rightCategory['name']));
    });
    return categories;
}

```

```

Future<List<Map<String, dynamic>>> getManufacturersList() async {
  final manufacturersSnapshot = await _manufacturersCollection.get();
  final manufacturers = manufacturersSnapshot.docs.map(_documentToMap).toList();
  manufacturers.sort((leftManufacturer, rightManufacturer) {
    return _stringValue(leftManufacturer['name']).compareTo(_stringValue(rightManufacturer['name']));
  });
  return manufacturers;
}

```

```

Future<List<Map<String, dynamic>>> getSuppliersList() async {
  final suppliersSnapshot = await _suppliersCollection.get();
  final suppliers = suppliersSnapshot.docs.map(_documentToMap).toList();
  suppliers.sort((leftSupplier, rightSupplier) {
    return _stringValue(leftSupplier['name']).compareTo(_stringValue(rightSupplier['name']));
  });
  return suppliers;
}

```

```

Future<List<Map<String, dynamic>>> getRolesList() async {
  final rolesSnapshot = await _rolesCollection.get();
  final roles = rolesSnapshot.docs.map(_documentToMap).toList();
  roles.sort((leftRole, rightRole) {
    return _stringValue(leftRole['name']).compareTo(_stringValue(rightRole['name']));
  });
  return roles;
}

```

```

Future<List<Map<String, dynamic>>> getOrderStatusesList() async {
  final statusesSnapshot = await _orderStatusesCollection.get();
  final statuses = statusesSnapshot.docs.map(_documentToMap).toList();
  statuses.sort((leftStatus, rightStatus) {
    return _stringValue(leftStatus['name']).compareTo(_stringValue(rightStatus['name']));
  });
  return statuses;
}

```

```

Future<List<Map<String, dynamic>>> getUsersList() async {
  final usersSnapshot = await _usersCollection.get();
  final users = usersSnapshot.docs.map(_documentToMap).where((user) => user['isDeleted'] !=
true).toList();

```

```

users.sort((leftUser, rightUser) {
    return _stringValue(leftUser['displayName']).compareTo(_stringValue(rightUser['displayName']));
});
return users;
}

```

```

Future<void> _seedCategories() async {
    final batch = _database.batch();
    for (final category in catalogCategories) {
        final translation = categoryTranslations[category.id];
        batch.set(_categoriesCollection.doc(category.id), {
            'name': category.name,
            'nameEn': translation?.name ?? "",
            'nameNormalized': category.name.toLowerCase(),
            'description': category.description,
            'descriptionEn': translation?.description ?? "",
            'imageUrl': category.imageUrl,
            'updatedAt': FieldValue.serverTimestamp(),
            'createdAt': FieldValue.serverTimestamp(),
        }, SetOptions(merge: true));
    }
    await batch.commit();
}

```

```

Future<void> _seedManufacturers() async {
    final batch = _database.batch();
    for (final manufacturer in catalogManufacturers) {
        batch.set(_manufacturersCollection.doc(manufacturer.id), {
            'name': manufacturer.name,
            'nameNormalized': manufacturer.name.toLowerCase(),
            'country': manufacturer.country,
            'contactPhone': manufacturer.contactPhone,
            'updatedAt': FieldValue.serverTimestamp(),
            'createdAt': FieldValue.serverTimestamp(),
        }, SetOptions(merge: true));
    }
    await batch.commit();
}

```

```

Map<String, dynamic> _catalogProductData(CatalogProduct product) {
    final category = catalogCategories.firstWhere((category) => category.id == product.categoryId);
}

```



```

final manufacturer = catalogManufacturers.firstWhere(
  (manufacturer) => manufacturer.id == product.manufacturerId,
);
final translation = productTranslations[product.id];
final categoryTranslation = categoryTranslations[category.id];
final supplierIndex = product.id.codeUnits.fold<int>(0, (total, unit) => total + unit) % 2;

return {
  'name': product.name,
  'nameEn': translation?.name ?? "",
  'nameNormalized': product.name.toLowerCase(),
  'description': product.description,
  'descriptionEn': translation?.description ?? "",
  'composition': product.composition,
  'compositionEn': translation?.composition ?? "",
  'country': product.country,
  'price': product.price,
  'unit': product.unit,
  'qty': product.qty,
  'stockQuantity': product.stock,
  'minStock': 5,
  'imageUrl': product.imageUrl,
  'categoryId': category.id,
  'categoryName': category.name,
  'categoryNameEn': categoryTranslation?.name ?? "",
  'manufacturerId': manufacturer.id,
  'manufacturerName': manufacturer.name,
  'supplierId': supplierIndex == 0 ? 'supply-one' : 'supply-two',
  'supplierName': supplierIndex == 0 ? 'ООО Поставка Маркет' : 'ИП Экспресс Поставка',
  'calories': product.calories,
  'proteins': product.proteins,
  'fats': product.fats,
  'carbs': product.carbs,
  'shelfLifeDays': product.shelfLifeDays,
  'expiryDate': Timestamp.fromDate(DateTime.now().add(Duration(days: product.shelfLifeDays))),
  'popularity': product.popularity,
  'isDeleted': false,
};
}

```

/// Полностью пересоздает каталог: удаляет все товары и загружает стартовый набор.

```

Future<void> reseedCatalog() async {
    await _seedCategories();
    await _seedManufacturers();

    final existingSnapshot = await _productsCollection.get();
    var batch = _database.batch();
    var operations = 0;

    for (final document in existingSnapshot.docs) {
        batch.delete(document.reference);
        if (++operations >= 400) {
            await batch.commit();
            batch = _database.batch();
            operations = 0;
        }
    }

    for (final product in catalogProducts) {
        batch.set(_productsCollection.doc(product.id), {
            ..._catalogProductData(product),
            'createdAt': FieldValue.serverTimestamp(),
            'updatedAt': FieldValue.serverTimestamp(),
        });
        if (++operations >= 400) {
            await batch.commit();
            batch = _database.batch();
            operations = 0;
        }
    }

    await batch.commit();
}

Stream<List<Map<String, dynamic>>>> _watchOrderedCollection(
    CollectionReference<Map<String, dynamic>>> collection,
    String orderField,
) {
    return collection.snapshots().map((snapshot) {
        final records = snapshot.docs.map(_documentToMap).toList();
        records.sort((leftRecord, rightRecord) {
            return _stringValue(leftRecord[orderField]).compareTo(_stringValue(rightRecord[orderField]));
        });
    });
}

```

```

});
return records;
});
}

```

```

Map<String, dynamic> _documentToMap(DocumentSnapshot<Map<String, dynamic>>
documentSnapshot) {
    final documentData = documentSnapshot.data() ?? <String, dynamic>{};
    return {
        'id': documentSnapshot.id,
        ...documentData,
    };
}

```

```

Future<void> assertLoginAvailable(String login) async {
    final indexDocument = await _loginIndexCollection.doc(login.trim().toLowerCase()).get();
    if (indexDocument.exists) {
        throw DatabaseOperationException(AppLocale.strings.errLoginTaken);
    }
}

```

```

Future<void> _ensureUniqueValue({
    required CollectionReference<Map<String, dynamic>> collection,
    required String fieldName,
    required String normalizedValue,
    String? currentDocumentId,
    required String errorMessage,
}) async {
    final snapshot = await collection.where(fieldName, isEqualTo: normalizedValue).get();
    for (final document in snapshot.docs) {
        if (document.id == currentDocumentId || document.data()['isDeleted'] == true) {
            continue;
        }
        throw DatabaseOperationException(errorMessage);
    }
}

```

```

String _buildDefaultLogin(String email, String displayName) {
    final sourceValue = email.isNotEmpty ? email.split('@').first : displayName;
    final normalizedSource = sourceValue
        .trim()

```

```

        .toLowerCase()
        .replaceAll(RegExp(r'^a-z0-9'], " ");
    if (normalizedSource.length >= 3) {
        return normalizedSource;
    }
    return 'user${DateTime.now().millisecondsSinceEpoch.toString().substring(8)}';
}

bool _isFinishedOrderStatus(String statusName) {
    return statusName.contains('доставлен') ||
        statusName.contains('отмен') ||
        statusName.contains('cancel');
}

String _stringValue(dynamic value) => toStringValue(value);

int _intValue(dynamic value) => toIntValue(value);

double _doubleValue(dynamic value) => toDoubleValue(value);

DateTime _dateTimeValue(dynamic value) => toDateTimeValue(value);
}

```

lib/utils/app_palette.dart

```

import 'package:flutter/material.dart';

/// Палитра бренда. Все сочетания «текст на фоне» подобраны по контрасту WCAG AA
/// (не ниже 4.5:1): белый текст допустим только на [primary], [primaryDark],
/// [danger] и [warning], а на светлых поверхностях используется [textPrimary]
/// или [textSecondary].
class AppPalette {
    static const Color background = Color(0xFFFF5F3EF);
    static const Color primary = Color(0xFF2E7D32);
    static const Color primaryDark = Color(0xFF1B5E20);
    static const Color textPrimary = Color(0xFF212121);
    static const Color textSecondary = Color(0xFF5F6368);
    static const Color surface = Colors.white;
    static const Color border = Color(0xFFE0E0E0);
    static const Color navigation = Color(0xFF636380);
    static const Color warning = Color(0xFFC2410C);
    static const Color danger = Color(0xFFC62828);
    static const Color addAction = Color(0xFF94FF66);
}

```

```

static const Color editAction = Color(0xFFFFC466);

static const Color lightPrimary = Color(0xFFE8F5E9);
static const Color disabledBackground = Color(0xFFE0E0E0);
static const Color disabledForeground = Color(0xFF616161);
}

lib/utils/auth_target.dart

enum AuthFlowTarget {
  mobile,
  adminPanel,
}

lib/utils/courier_orders.dart

import 'parsers.dart';

const Set<String> _startableStatuses = {'new', 'processing', 'assigned'};
const Set<String> _finishedStatuses = {'delivered', 'cancelled'};

/// Курьер может начать доставку, пока заказ не в пути и не завершен.
bool canStartDelivery(String statusId) => _startableStatuses.contains(statusId);

/// Завершить доставку можно только у заказа «в пути».
bool canCompleteDelivery(String statusId) => statusId == 'delivering';

bool isFinishedOrder(String statusId) => _finishedStatuses.contains(statusId);

class CourierOrderGroups {
  const CourierOrderGroups({required this.active, required this.history});

  final List<Map<String, dynamic>> active;
  final List<Map<String, dynamic>> history;
}

/// Делит заказы курьера на активные и историю.
/// Активные: сначала «в пути», затем по времени создания (старые раньше).
/// История: новые сверху.
CourierOrderGroups groupCourierOrders(List<Map<String, dynamic>> orders) {
  String statusOf(Map<String, dynamic> order) => order['statusId']?.toString() ?? 'new';
  DateTime createdAt(Map<String, dynamic> order) => toDateTimeValue(order['createdAt']);

  final active = orders.where((order) => !isFinishedOrder(statusOf(order))).toList()

```

```

    ..sort((left, right) {
      final leftOnTheWay = canCompleteDelivery(statusOf(left));
      final rightOnTheWay = canCompleteDelivery(statusOf(right));
      if (leftOnTheWay != rightOnTheWay) return leftOnTheWay ? -1 : 1;
      return createdAt(left).compareTo(createdAt(right));
    });
    final history = orders.where((order) => isFinishedOrder(statusOf(order))).toList()
    ..sort((left, right) => createdAt(right).compareTo(createdAt(left)));

    return CourierOrderGroups(active: active, history: history);
  }

```

lib/utils/order_status.dart

```

import '../l10n/l10n.dart';

/// Подпись статуса заказа на языке интерфейса. Если статус неизвестен
/// (создан вручную), показывается сохраненное название.
String orderStatusLabel(String? statusId, String? storedName, AppLocalizations l10n) {
  switch (statusId) {
    case 'new':
      return l10n.statusNew;
    case 'processing':
      return l10n.statusProcessing;
    case 'assigned':
      return l10n.statusAssigned;
    case 'delivering':
      return l10n.statusDelivering;
    case 'delivered':
      return l10n.statusDelivered;
    case 'cancelled':
      return l10n.statusCancelled;
    default:
      return storedName ?? l10n.statusNew;
  }
}

```

lib/utils/parsers.dart

```

import 'package:cloud_firestore/cloud_firestore.dart';

int toIntValue(dynamic value) {
  if (value is int) return value;
  if (value is num) return value.round();
}

```

```

return int.tryParse(value?.toString().trim() ?? "") ?? 0;
}

```

```

double toDoubleValue(dynamic value) {
  if (value is num) return value.toDouble();
  return double.tryParse(value?.toString().trim().replaceAll(',', '.') ?? "") ?? 0;
}

```

```

String toStringValue(dynamic value) => value?.toString().trim() ?? "";

```

```

DateTime toDateTimeValue(dynamic value) {
  if (value is Timestamp) return value.toDate();
  if (value is DateTime) return value;
  if (value is String && value.isNotEmpty) {
    return DateTime.tryParse(value) ?? DateTime.fromMillisecondsSinceEpoch(0);
  }
  return DateTime.fromMillisecondsSinceEpoch(0);
}

```

```

String formatDate(DateTime date) {
  return '${date.day.toString().padLeft(2, '0')}. '
    '${date.month.toString().padLeft(2, '0')}. '
    '${date.year}';
}

```

```

String formatDateTime(DateTime date) {
  return '${formatDate(date)} '
    '${date.hour.toString().padLeft(2, '0')}:'
    '${date.minute.toString().padLeft(2, '0')}';
}

```

lib/utils/pricing.dart

```

import 'dart:math' as math;

```

```

import '../l10n/l10n.dart';

```

```

const double freeDeliveryThreshold = 1500;
const double standardDeliveryFee = 149;

```

```

/// Стоимость доставки зависит от суммы заказа после скидки.

```

```

double calculateDeliveryFee(double amountAfterDiscount) {
  if (amountAfterDiscount <= 0 || amountAfterDiscount >= freeDeliveryThreshold) {

```

```

        return 0;
    }
    return standardDeliveryFee;
}

double calculateOrderTotal({
    required double subtotal,
    required double discountPercent,
}) {
    final discount = subtotal * discountPercent / 100;
    final afterDiscount = math.max(0.0, subtotal - discount);
    return afterDiscount + calculateDeliveryFee(afterDiscount);
}

/// В заказе способ оплаты хранится русским словом (для персонала), пользователю показывается
/// перевод.
String paymentMethodLabel(String? stored, AppLocalizations l10n) {
    switch (stored) {
        case 'Онлайн':
            return l10n.paymentOnline;
        case 'Наличными':
            return l10n.paymentCash;
        default:
            return stored ?? l10n.notSpecifiedFem;
    }
}

String formatMoney(num value) => '₽${value.toStringAsFixed(2)}';

/// Ключ первого слота: в заказ пишется русская подпись (для персонала),
/// а пользователю показывается перевод через [deliverySlotLabel].
const String asapSlot = 'Как можно скорее (до 60 минут)';

String deliverySlotLabel(String slot, AppLocalizations l10n) => slot == asapSlot ? l10n.slotAsap : slot;

const List<String> deliverySlots = [
    asapSlot,
    '09:00 – 11:00',
    '11:00 – 13:00',
    '13:00 – 15:00',
    '15:00 – 17:00',

```



```
'17:00 – 19:00',  
'19:00 – 21:00',  
];
```

lib/utils/product_filters.dart

```
import '../l10n/l10n.dart';  
import '../models/product_model.dart';
```

```
enum ProductSort {  
  none,  
  freshFirst,  
  priceDescending,  
  priceAscending;
```

```
String label(AppLocalizations l10n) {  
  switch (this) {  
    case ProductSort.none:  
      return l10n.sortDefault;  
    case ProductSort.freshFirst:  
      return l10n.sortFreshFirst;  
    case ProductSort.priceDescending:  
      return l10n.sortPriceDesc;  
    case ProductSort.priceAscending:  
      return l10n.sortPriceAsc;  
  }  
}  
}
```

```
class ProductFilters {  
  const ProductFilters({  
    this.minPrice,  
    this.maxPrice,  
    this.brands = const {},  
    this.sort = ProductSort.none,  
  });
```

```
  final double? minPrice;  
  final double? maxPrice;  
  final Set<String> brands;  
  final ProductSort sort;
```

```
  bool get isActive => minPrice != null || maxPrice != null || brands.isNotEmpty || sort != ProductSort.none;
```

```

List<Product> apply(List<Product> products) {
  final filtered = products.where((product) {
    if (minPrice != null && product.price < minPrice!) return false;
    if (maxPrice != null && product.price > maxPrice!) return false;
    if (brands.isNotEmpty && !brands.contains(product.brand)) return false;
    return true;
  }).toList();

  switch (sort) {
    case ProductSort.none:
      break;
    case ProductSort.priceAscending:
      filtered.sort((left, right) => left.price.compareTo(right.price));
    case ProductSort.priceDescending:
      filtered.sort((left, right) => right.price.compareTo(left.price));
    case ProductSort.freshFirst:
      final unknownDate = DateTime.fromMillisecondsSinceEpoch(0);
      filtered.sort((left, right) {
        return (right.expiryDate ?? unknownDate).compareTo(left.expiryDate ?? unknownDate);
      });
  }
  return filtered;
}

```

lib/utils/role_utils.dart

```

import '../l10n/l10n.dart';

class AppRoles {
  static const String guest = 'guest';
  static const String admin = 'admin';
  static const String manager = 'manager';
  static const String courier = 'courier';
  static const String buyer = 'buyer';

  /// Синоним роли покупателя из технического задания.
  static const String customer = 'customer';
}

String roleLabel(String roleCode) {
  final l10n = AppLocale.strings;

```

```

switch (roleCode) {
    case AppRoles.admin:
        return l10n.roleAdmin;
    case AppRoles.manager:
        return l10n.roleManager;
    case AppRoles.courier:
        return l10n.roleCourier;
    case AppRoles.buyer:
    case AppRoles.customer:
        return l10n.roleBuyer;
    default:
        return l10n.roleGuest;
}
}

bool canAccessAdminPanel(String roleCode) {
    return roleCode == AppRoles.admin || roleCode == AppRoles.manager;
}

bool canAccessBuyerApp(String roleCode) {
    return roleCode == AppRoles.buyer || roleCode == AppRoles.customer;
}

bool canAccessCourierApp(String roleCode) {
    return roleCode == AppRoles.courier;
}

bool isAdminRole(String roleCode) {
    return roleCode == AppRoles.admin;
}

bool isManagerRole(String roleCode) {
    return roleCode == AppRoles.manager;
}

bool canManageProducts(String roleCode) {
    return isAdminRole(roleCode) || isManagerRole(roleCode);
}

bool canDeleteProducts(String roleCode) {
    return isAdminRole(roleCode);
}

```

```

}

bool canManagePromoCodes(String roleCode) {
  return isAdminRole(roleCode) || isManagerRole(roleCode);
}

bool canDeletePromoCodes(String roleCode) {
  return isAdminRole(roleCode) || isManagerRole(roleCode);
}

bool canManageOrders(String roleCode) {
  return isAdminRole(roleCode) || isManagerRole(roleCode);
}

```

lib/utils/validators.dart

```

import '../l10n/l10n.dart';

class Validators {
  /// Firebase Authentication не принимает пароли короче 6 символов.
  static const int minPasswordLength = 6;

  static AppLocalizations get l10n => AppLocale.strings;

  static String? validateLogin(String? value) {
    if (value == null || value.trim().isEmpty) return l10n.valLoginRequired;
    final trimmed = value.trim();
    if (trimmed.length < 3 || trimmed.length > 50) return l10n.valLoginLength;
    if (!RegExp(r'^[a-zA-Z0-9._-]+$').hasMatch(trimmed)) return l10n.valLoginChars;
    return null;
  }

  static String? validateName(String? value) {
    if (value == null || value.trim().isEmpty) return l10n.valFieldRequired;
    final trimmed = value.trim();
    if (trimmed.length < 2 || trimmed.length > 50) return l10n.valNameLength;
    if (!RegExp(r'^[a-zA-Za-яA-ЯёЁ\s-]+$').hasMatch(trimmed)) return l10n.valNameChars;
    return null;
  }

  static String? validateEmail(String? value) {
    if (value == null || value.trim().isEmpty) return l10n.valEmailRequired;
    final trimmed = value.trim().toLowerCase();

```

```

        if (trimmed.length > 100) return l10n.valEmailLength;
        if (!RegExp(r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]+$').hasMatch(trimmed)) return
l10n.valEmailFormat;
        return null;
    }

```

```

static String? validatePhone(String? value) {
    if (value == null || value.trim().isEmpty) return l10n.valPhoneRequired;
    if (!RegExp(r'^\+?[\d\s\(-)]+$').hasMatch(value.trim())) return l10n.valPhoneFormat;
    final digitsOnly = value.replaceAll(RegExp(r'\D'), "");
    if (digitsOnly.length < 10 || digitsOnly.length > 15) return l10n.valPhoneFormat;
    return null;
}

```

/// Поле входа принимает либо email, либо логин.

```

static String? validateLoginIdentifier(String? value) {
    if (value == null || value.trim().isEmpty) return l10n.loginIdentifierRequired;
    final trimmed = value.trim();
    final error = trimmed.contains('@') ? validateEmail(trimmed) : validateLogin(trimmed);
    return error == null ? null : l10n.loginIdentifierInvalid;
}

```

```

static String? validatePassword(String? value) {
    if (value == null || value.trim().isEmpty) return l10n.valPasswordRequired;
    if (value.contains(' ')) return l10n.valPasswordNoSpaces;
    if (value.length < minPasswordLength) return l10n.valPasswordMinLength;
    if (!RegExp(r'\d').hasMatch(value)) return l10n.valPasswordDigit;
    if (value.length > 128) return l10n.valPasswordMaxLength;
    return null;
}

```

/// При входе проверяется только заполненность: пароль, созданный раньше или

/// другим способом, не должен блокироваться правилами регистрации.

```

static String? validatePasswordForLogin(String? value) {
    if (value == null || value.isEmpty) return l10n.valPasswordRequired;
    return null;
}

```

```

static String? validatePasswordConfirmation(String? value, String originalPassword) {
    if (value == null || value.isEmpty) return l10n.valPasswordConfirmRequired;
    if (value != originalPassword) return l10n.valPasswordMismatch;
}

```

```
    return null;
}
```

```
static String? validateRequiredText(
    String? value, {
    required String fieldName,
    int? maxLength,
}) {
    if (value == null || value.trim().isEmpty) return l10n.valNamedFieldRequired(fieldName);
    if (maxLength != null && value.trim().length > maxLength) {
        return l10n.valNamedFieldMaxLength(fieldName, maxLength);
    }
    return null;
}
```

```
static String? validatePositivePrice(String? value) {
    if (value == null || value.trim().isEmpty) return l10n.valPriceRequired;
    final parsedValue = double.tryParse(value.replaceAll(',', '.'));
    if (parsedValue == null || parsedValue <= 0) return l10n.valPricePositive;
    return null;
}
```

```
static String? validateIntegerRange(
    String? value, {
    required String fieldName,
    required int minValue,
    required int maxValue,
}) {
    if (value == null || value.trim().isEmpty) return l10n.valNamedValueRequired(fieldName);
    final parsedValue = int.tryParse(value);
    if (parsedValue == null || parsedValue < minValue || parsedValue > maxValue) {
        return l10n.valNamedRange(fieldName, minValue, maxValue);
    }
    return null;
}
```

```
static String? validatePromoCode(String? value) {
    if (value == null || value.trim().isEmpty) return l10n.valPromoRequired;
    final normalizedValue = value.trim().toUpperCase();
    if (!RegExp(r'^[A-Z0-9]+$').hasMatch(normalizedValue)) return l10n.valPromoChars;
    return null;
}
```

```

}

static String? validateTaxId(String? value) {
  if (value == null || value.trim().isEmpty) return l10n.valTaxIdRequired;
  if (!RegExp(r'^\d{10}(\d{2})?$').hasMatch(value.trim())) {
    return l10n.valTaxIdFormat;
  }
  return null;
}

static String? validateCardNumber(String? value) {
  if (value == null || value.trim().isEmpty) return l10n.valCardRequired;
  if (!RegExp(r'^[\d\s-]+$').hasMatch(value.trim())) return l10n.valCardLength;
  final digitsOnly = value.replaceAll(RegExp(r'\D'), "");
  if (digitsOnly.length != 16) return l10n.valCardLength;
  return null;
}
}

```

lib/l10n/l10n.dart

```

import 'package:flutter/material.dart';

import 'app_localizations.dart';

export 'app_localizations.dart';

extension L10nContext on BuildContext {
  AppLocalizations get l10n => AppLocalizations.of(this);

  bool get isEnglish => Localizations.localeOf(this).languageCode == 'en';
}

/// Текущий язык для кода без BuildContext (сервисы, валидаторы).
/// Обновляется провайдером [LocaleProvider].
class AppLocale {
  const AppLocale._();

  static const Locale russian = Locale('ru');
  static const Locale english = Locale('en');

  static Locale current = russian;
}

```

```

static bool get isEnglish => current.languageCode == 'en';

static AppLocalizations get strings => lookupAppLocalizations(current);
}

/// Выбирает значение для языка: английский вариант, если он задан и язык английский.
String pickLocalized({required bool english, required String russian, String? englishValue}) {
  if (english && englishValue != null && englishValue.trim().isEmpty) {
    return englishValue;
  }
  return russian;
}

```

lib/screens/splash_screen.dart

```

import 'dart:async';

import 'package:flutter/material.dart';
import '../l10n/l10n.dart';
import '../utils/app_palette.dart';

class SplashScreen extends StatefulWidget {
  const SplashScreen({super.key, required this.onFinished});

  final VoidCallback onFinished;

  @override
  State<SplashScreen> createState() => _SplashScreenState();
}

class _SplashScreenState extends State<SplashScreen> {
  Timer? _timer;

  @override
  void initState() {
    super.initState();
    _timer = Timer(const Duration(seconds: 3), () {
      if (mounted) widget.onFinished();
    });
  }

  @override
  void dispose() {

```



```

    _timer?.cancel();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppPalette.primary,
      body: Center(
        child: Row(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Image.network(
              'https://cdn-icons-png.flaticon.com/512/1147/1147805.png',
              color: Colors.white,
              width: 55,
              errorBuilder: (context, error, stackTrace) =>
                const Icon(Icons.eco, color: Colors.white, size: 55),
            ),
            const SizedBox(width: 15),
            Column(
              mainAxisAlignment: MainAxisAlignment.min,
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  context.l10n.splashBrand,
                  style: TextStyle(fontSize: 45, fontWeight: FontWeight.bold, color: Colors.white, height: 1.0),
                ),
                Text(
                  context.l10n.splashTagline,
                  style: TextStyle(fontSize: 14, color: Colors.white, letterSpacing: 3, fontWeight:
FontWeight.w500),
                ),
              ],
            ),
          ],
        ),
      );
  }
}

```

lib/screens/onboarding_screen.dart

```
import 'package:flutter/material.dart';

import '../l10n/l10n.dart';
import '../utils/app_palette.dart';
import '../widgets/language_switcher.dart';

class OnboardingScreen extends StatefulWidget {
  const OnboardingScreen({super.key, required this.onFinished});

  final VoidCallback onFinished;

  @override
  State<OnboardingScreen> createState() => _OnboardingScreenState();
}

class _OnboardingScreenState extends State<OnboardingScreen> {
  final PageController _controller = PageController();
  int _page = 0;

  static const int _pageCount = 3;

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  void _next() {
    if (_page == _pageCount - 1) {
      widget.onFinished();
      return;
    }
    _controller.nextPage(duration: const Duration(milliseconds: 300), curve: Curves.easeOut);
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final slides = [
      (Icons.eco, l10n.onboarding1Title, l10n.onboarding1Body),
    ];
```

```

(Icons.delivery_dining, l10n.onboarding2Title, l10n.onboarding2Body),
(Icons.local_offer, l10n.onboarding3Title, l10n.onboarding3Body),
];
final isLast = _page == _pageCount - 1;

return Scaffold(
  backgroundColor: const Color(0xFF181B19),
  body: Stack(
    fit: StackFit.expand,
    children: [
      Image.network(
        'https://images.unsplash.com/photo-1542838132-92c53300491e?
q=80&w=1974&auto=format&fit=crop',
        fit: BoxFit.cover,
        errorBuilder: (context, error, stackTrace) => const SizedBox.shrink(),
      ),
      // Тёмный градиент гарантирует читаемость белого текста на любом фото.
      DecoratedBox(
        decoration: BoxDecoration(
          gradient: LinearGradient(
            colors: [Colors.black.withValues(alpha: 0.35), Colors.black.withValues(alpha: 0.9)],
            begin: Alignment.topCenter,
            end: Alignment.bottomCenter,
          ),
        ),
      ),
      SafeArea(
        child: Column(
          children: [
            Padding(
              padding: const EdgeInsets.fromLTRB(20, 12, 20, 0),
              child: Row(
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: [
                  const LanguageSwitcher(onDark: true),
                  if (!isLast)
                    TextButton(
                      style: TextButton.styleFrom(backgroundColor: Colors.white),
                      onPressed: widget.onFinished,
                      child: Text(l10n.onboardingSkip, style: const TextStyle(fontSize: 16)),
                    ),
                ],
              ),
            ),
          ],
        ),
      ),
    ],
  ),
);

```

```

    ],
  ),
),
Expanded(
  child: PageView.builder(
    controller: _controller,
    itemCount: _pageCount,
    onPageChanged: (index) => setState(() => _page = index),
    itemBuilder: (context, index) {
      final (icon, title, body) = slides[index];
      return Padding(
        padding: const EdgeInsets.symmetric(horizontal: 30),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.end,
          children: [
            Container(
              padding: const EdgeInsets.all(22),
              decoration: const BoxDecoration(color: AppPalette.primary, shape: BoxShape.circle),
              child: Icon(icon, color: Colors.white, size: 48),
            ),
            const SizedBox(height: 28),
            Text(
              title,
              textAlign: TextAlign.center,
              style: const TextStyle(
                color: Colors.white,
                fontSize: 34,
                fontWeight: FontWeight.bold,
                height: 1.15,
                fontFamily: 'Unbounded',
              ),
            ),
            const SizedBox(height: 14),
            Text(
              body,
              textAlign: TextAlign.center,
              style: const TextStyle(color: Colors.white, fontSize: 16, height: 1.5),
            ),
          ],
        ),
      );
    },
  );

```

```
},  
),  
),  
Row(  
  mainAxisAlignment: MainAxisAlignment.center,  
  children: [  
    for (var i = 0; i < _pageCount; i++)  
      AnimatedContainer(  
        duration: const Duration(milliseconds: 200),  
        margin: const EdgeInsets.symmetric(horizontal: 4, vertical: 24),  
        width: i == _page ? 26 : 8,  
        height: 8,  
        decoration: BoxDecoration(  
          color: i == _page ? Colors.white : Colors.white54,  
          borderRadius: BorderRadius.circular(8),  
        ),  
      ),  
  ],  
),  
Padding(  
  padding: const EdgeInsets.fromLTRB(30, 0, 30, 32),  
  child: SizedBox(  
    width: double.infinity,  
    height: 62,  
    child: ElevatedButton(  
      style: ElevatedButton.styleFrom(  
        backgroundColor: AppPalette.primary,  
        foregroundColor: Colors.white,  
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(19)),  
      ),  
      onPressed: _next,  
      child: Text(  
        isLast ? l10n.onboardingStart : l10n.onboardingNext,  
        style: const TextStyle(fontSize: 18, fontWeight: FontWeight.w600),  
      ),  
    ),  
  ),  
),  
),  
],  
),  
),
```

```

    ],
  ),
);
}
}

```

lib/screens/auth/auth_wrapper.dart

```

import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';

import '../main_screen.dart';
import '../services/auth_service.dart';
import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/role_utils.dart';
import '../courier/courier_home_screen.dart';
import '../l10n/l10n.dart';

/// Home of a signed-in session: resolves the user's role and shows the matching
/// app. Sign-out is handled by the app root, which replaces the whole session.
class AuthenticatedShell extends StatefulWidget {
  const AuthenticatedShell({super.key, required this.user});

  final User user;

  @override
  State<AuthenticatedShell> createState() => _AuthenticatedShellState();
}

class _AuthenticatedShellState extends State<AuthenticatedShell> {
  late Future<String> _roleFuture = _resolveRole();

  Future<String> _resolveRole() async {
    final databaseService = DatabaseService();
    var userProfile = await databaseService.getUserProfile(widget.user.uid);

    if (userProfile?['isDeleted'] == true) {
      await AuthService().logout();
      return AppRoles.guest;
    }

    if (userProfile == null) {

```

```

        await databaseService.ensureUserProfileFromAuth(widget.user, defaultRole: AppRoles.buyer);
        userProfile = await databaseService.getUserProfile(widget.user.uid);
    }

    return userProfile?['role']?.toString() ?? AppRoles.buyer;
}

void _retry() {
    setState(() => _roleFuture = _resolveRole());
}

Widget _homeForRole(String roleCode) {
    if (canAccessBuyerApp(roleCode)) {
        return const MainScreen();
    }
    if (canAccessCourierApp(roleCode)) {
        return const CourierHomeScreen();
    }
    return MobileRoleBlockedScreen(roleName: roleLabel(roleCode));
}

@override
Widget build(BuildContext context) {
    return FutureBuilder<String>(
        future: _roleFuture,
        builder: (context, snapshot) {
            if (snapshot.hasError) {
                return _ProfileLoadError(onRetry: _retry);
            }

            if (!snapshot.hasData) {
                return const Scaffold(
                    body: Center(child: CircularProgressIndicator(color: AppPalette.primary)),
                );
            }

            return _homeForRole(snapshot.data!);
        },
    );
}

```

```

class _ProfileLoadError extends StatelessWidget {
  const _ProfileLoadError({required this.onRetry});

  final VoidCallback onRetry;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppPalette.background,
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              const Icon(Icons.cloud_off, size: 56, color: AppPalette.textSecondary),
              const SizedBox(height: 16),
              Text(
                context.l10n.profileLoadFailed,
                textAlign: TextAlign.center,
                style: TextStyle(fontSize: 16, color: AppPalette.textSecondary),
              ),
              const SizedBox(height: 20),
              ElevatedButton(
                style: ElevatedButton.styleFrom(backgroundColor: AppPalette.primary, foregroundColor:
Colors.white),
                onPressed: onRetry,
                child: Text(context.l10n.retry),
              ),
              TextButton(
                onPressed: () => AuthService().logout(),
                child: Text(context.l10n.logoutFromAccount),
              ),
            ],
          ),
        ),
      );
  }
}

```



```

class MobileRoleBlockedScreen extends StatelessWidget {
  const MobileRoleBlockedScreen({super.key, required this.roleName});

  final String roleName;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppPalette.background,
      body: Center(
        child: ConstrainedBox(
          constraints: const BoxConstraints(maxWidth: 460),
          child: Card(
            margin: const EdgeInsets.all(20),
            child: Padding(
              padding: const EdgeInsets.all(24),
              child: Column(
                mainAxisAlignment: MainAxisAlignment.min,
                children: [
                  const Icon(Icons.lock_outline_rounded, color: AppPalette.warning, size: 56),
                  const SizedBox(height: 16),
                  Text(
                    context.l10n.roleBlockedTitle(roleName),
                    textAlign: TextAlign.center,
                    style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
                  ),
                  const SizedBox(height: 10),
                  Text(
                    context.l10n.roleBlockedBody,
                    textAlign: TextAlign.center,
                    style: TextStyle(color: AppPalette.textSecondary),
                  ),
                  const SizedBox(height: 20),
                  SizedBox(
                    width: double.infinity,
                    child: ElevatedButton(
                      style: ElevatedButton.styleFrom(backgroundColor: AppPalette.primary, foregroundColor:
Colors.white),
                      onPressed: () => AuthService().logout(),
                      child: Text(context.l10n.logoutFromAccount),
                    ),
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
}

```

```

        ),
      ),
    ],
  ),
),
),
),
),
),
);
}
}

```

lib/screens/auth/login_screen.dart

```

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';

import '../services/auth_service.dart';
import '../utils/app_palette.dart';
import '../utils/auth_target.dart';
import '../utils/validators.dart';
import '../widgets/app_snackbar.dart';
import '../widgets/language_switcher.dart';
import 'register_screen.dart';
import 'forgot_password_screen.dart';
import '../l10n/l10n.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({
    super.key,
    this.authFlowTarget = AuthFlowTarget.mobile,
  });

  final AuthFlowTarget authFlowTarget;

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
  final TextEditingController _identifierController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();

```

```

final AuthService _authService = AuthService();

bool _isLoading = false;
bool _obscureText = true;

bool get _isAdminFlow => widget.authFlowTarget == AuthFlowTarget.adminPanel;

@override
void dispose() {
  _identifierController.dispose();
  _passwordController.dispose();
  super.dispose();
}

// Successful sign-in needs no navigation: the app root replaces this whole
// session once the auth state changes, disposing this screen with it.
Future<void> _submit() async {
  if (!_formKey.currentState!.validate()) return;

  setState(() => _isLoading = true);
  try {
    await _authService.login(
      _identifierController.text,
      _passwordController.text,
    );
  } catch (error) {
    _handleAuthError(error);
  }
}

Future<void> _loginWithGoogle() async {
  setState(() => _isLoading = true);
  try {
    final userCredential = await _authService.signInWithGoogle();
    if (userCredential == null && mounted) {
      setState(() => _isLoading = false);
    }
  } catch (error) {
    _handleAuthError(error);
  }
}

```

```

void _handleAuthError(Object error) {
  if (!mounted) return;
  _showMessage(_authService.getErrorMessage(error), isError: true);
  setState(() => _isLoading = false);
}

void _showMessage(String message, {bool isError = false}) {
  showAppSnackBar(context, message, isError: isError);
}

@override
Widget build(BuildContext context) {
  final isCompact = MediaQuery.of(context).size.width < 720;

  return Scaffold(
    backgroundColor: AppPalette.background,
    appBar: _isAdminFlow
      ? AppBar(
        automaticallyImplyLeading: Navigator.of(context).canPop(),
        title: Text(context.l10n.loginPanelAppBar),
      )
      : null,
    body: SafeArea(
      child: Center(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: ConstrainedBox(
            constraints: BoxConstraints(maxWidth: _isAdminFlow ? 520 : 480),
            child: Container(
              padding: const EdgeInsets.all(28),
              decoration: BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.circular(28),
                boxShadow: [
                  BoxShadow(
                    color: Colors.black.withValues(alpha: 0.05),
                    blurRadius: 24,
                    offset: const Offset(0, 14),
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    ),
  );
}

```

```

),
child: Form(
  key: _formKey,
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const Align(
        alignment: Alignment.centerRight,
        child: LanguageSwitcher(),
      ),
      const SizedBox(height: 8),
      Center(
        child: Container(
          width: 72,
          height: 72,
          decoration: BoxDecoration(
            color: AppPalette.lightPrimary,
            borderRadius: BorderRadius.circular(22),
          ),
          child: const Icon(
            Icons.local_grocery_store,
            color: AppPalette.primary,
            size: 36,
          ),
        ),
      ),
      const SizedBox(height: 28),
      Text(
        _isAdminFlow ? context.l10n.loginPanelTitle : context.l10n.loginTitle,
        style: const TextStyle(
          fontSize: 30,
          fontWeight: FontWeight.bold,
          color: AppPalette.textPrimary,
          fontFamily: 'Unbounded',
        ),
      ),
      const SizedBox(height: 12),
      Text(
        _isAdminFlow
          ? context.l10n.loginPanelSubtitle
          : context.l10n.loginSubtitle,

```

```

style: const TextStyle(
  fontSize: 15,
  height: 1.5,
  color: AppPalette.textSecondary,
),
),
const SizedBox(height: 28),
Text(context.l10n.loginIdentifierLabel),
const SizedBox(height: 8),
TextFormField(
  controller: _identifierController,
  textInputAction: TextInputAction.next,
  validator: Validators.validateLoginIdentifier,
  decoration: _inputDecoration(
    hintText: context.l10n.loginIdentifierHint,
    icon: Icons.person_outline,
  ),
),
const SizedBox(height: 18),
Text(context.l10n.labelPassword),
const SizedBox(height: 8),
TextFormField(
  controller: _passwordController,
  obscureText: _obscureText,
  validator: Validators.validatePasswordForLogin,
  onFieldSubmitted: (_) => _submit(),
  decoration: _inputDecoration(
    hintText: context.l10n.passwordHint,
    icon: Icons.lock_outline,
    suffixIcon: IconButton(
      onPressed: () => setState(() => _obscureText = !_obscureText),
      icon: Icon(
        _obscureText ? Icons.visibility_off : Icons.visibility,
      ),
    ),
  ),
),
const SizedBox(height: 12),
Align(
  alignment: Alignment.centerRight,
  child: TextButton(

```

```

onPressed: () {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (_) => const ForgotPasswordScreen()),
  );
},
child: Text(context.l10n.forgotPasswordLink),
),
const SizedBox(height: 12),
SizedBox(
  width: double.infinity,
  height: 58,
  child: _isLoading
    ? const Center(
      child: CircularProgressIndicator(
        color: AppPalette.primary,
      ),
    )
    : ElevatedButton(
      style: ElevatedButton.styleFrom(
        backgroundColor: AppPalette.primary,
        foregroundColor: Colors.white,
      ),
      onPressed: _submit,
      child: Text(_isAdminFlow ? context.l10n.signInToPanel : context.l10n.signIn),
    ),
),
if (!_isAdminFlow) ...[
  const SizedBox(height: 16),
  SizedBox(
    width: double.infinity,
    height: 56,
    child: OutlinedButton.icon(
      onPressed: _isLoading ? null : _loginWithGoogle,
      icon: const Icon(Icons.g_mobiledata, size: 34, color: Colors.red),
      label: Text(context.l10n.continueWithGoogle),
    ),
  ),
  const SizedBox(height: 18),
  Wrap(

```

```

alignment: WrapAlignment.center,
crossAxisAlignment: WrapCrossAlignment.center,
children: [
  Text(context.l10n.noAccount),
  TextButton(
    onPressed: () {
      Navigator.push(
        context,
        MaterialPageRoute(builder: (_) => const RegisterScreen()),
      );
    },
    child: Text(context.l10n.registerAction),
  ),
],
),
],
if (_isAdminFlow && !isCompact && kIsWeb) ...[
  const SizedBox(height: 16),
  const Divider(),
  const SizedBox(height: 8),
  Text(
    context.l10n.panelInfoNote,
    style: TextStyle(
      fontSize: 13,
      height: 1.5,
      color: AppPalette.textSecondary,
    ),
  ),
],
],
),
),
),
),
),
),
);
}

```

```

InputDecoration _inputDecoration({

```



```

required String hintText,
required IconData icon,
Widget? suffixIcon,
}) {
  return InputDecoration(
    hintText: hintText,
    prefixIcon: Icon(icon),
    suffixIcon: suffixIcon,
    filled: true,
    fillColor: AppPalette.background,
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(18),
      borderSide: const BorderSide(color: AppPalette.border),
    ),
    enabledBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(18),
      borderSide: const BorderSide(color: AppPalette.border),
    ),
    focusedBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(18),
      borderSide: const BorderSide(color: AppPalette.primary),
    ),
    errorBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(18),
      borderSide: const BorderSide(color: AppPalette.danger),
    ),
    focusedErrorBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(18),
      borderSide: const BorderSide(color: AppPalette.danger),
    ),
  );
}
}

```

lib/screens/auth/register_screen.dart

```

import 'package:flutter/material.dart';

import '../services/auth_service.dart';
import '../utils/app_palette.dart';
import '../utils/validators.dart';
import '../widgets/app_snackbar.dart';
import '../l10n/l10n.dart';

```

```

class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
  final TextEditingController _loginController = TextEditingController();
  final TextEditingController _nameController = TextEditingController();
  final TextEditingController _phoneController = TextEditingController();
  final TextEditingController _emailController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();
  final TextEditingController _confirmPasswordController = TextEditingController();
  final AuthService _authService = AuthService();

  bool _isLoading = false;
  bool _obscurePassword = true;
  bool _obscureConfirmPassword = true;

  @override
  void dispose() {
    _loginController.dispose();
    _nameController.dispose();
    _phoneController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    _confirmPasswordController.dispose();
    super.dispose();
  }

  // Successful registration needs no navigation: the app root replaces this
  // whole session once the new account is signed in.
  Future<void> _register() async {
    if (!_formKey.currentState!.validate()) {
      return;
    }

    setState(() => _isLoading = true);
  }

```

```

try {
  await _authService.registerBuyer(
    login: _loginController.text,
    name: _nameController.text,
    email: _emailController.text,
    phoneNumber: _phoneController.text,
    password: _passwordController.text,
  );
} catch (error) {
  if (!mounted) return;
  showAppSnackBar(context, _authService.getErrorMessage(error), isError: true);
  setState(() => _isLoading = false);
}
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: AppPalette.background,
    appBar: AppBar(
      title: Text(context.l10n.registerTitle),
    ),
    body: SafeArea(
      child: Center(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: ConstrainedBox(
            constraints: const BoxConstraints(maxWidth: 520),
            child: Container(
              padding: const EdgeInsets.all(28),
              decoration: BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.circular(28),
                boxShadow: [
                  BoxShadow(
                    color: Colors.black.withValues(alpha: 0.05),
                    blurRadius: 24,
                    offset: const Offset(0, 14),
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    ),
  );
}

```

```

child: Form(
  key: _formKey,
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(
        context.l10n.registerHeading,
        style: TextStyle(
          fontSize: 28,
          fontWeight: FontWeight.bold,
          color: AppPalette.textPrimary,
          fontFamily: 'Unbounded',
        ),
      ),
      const SizedBox(height: 12),
      Text(
        context.l10n.registerRoleNote,
        style: TextStyle(color: AppPalette.textSecondary, height: 1.5),
      ),
      const SizedBox(height: 24),
      _buildLabel(context.l10n.labelLogin),
      TextFormField(
        controller: _loginController,
        validator: Validators.validateLogin,
        decoration: _inputDecoration(
          hintText: context.l10n.registerLoginHint,
          icon: Icons.alternate_email,
        ),
      ),
      const SizedBox(height: 16),
      _buildLabel(context.l10n.labelName),
      TextFormField(
        controller: _nameController,
        validator: Validators.validateName,
        decoration: _inputDecoration(
          hintText: context.l10n.registerNameHint,
          icon: Icons.badge_outlined,
        ),
      ),
      const SizedBox(height: 16),
      _buildLabel(context.l10n.labelPhone),
    ],
  ),
),

```

```

TextFormField(
  controller: _phoneController,
  validator: Validators.validatePhone,
  keyboardType: TextInputType.phone,
  decoration: _inputDecoration(
    hintText: '+7 900 123-45-67',
    icon: Icons.phone_outlined,
  ),
),
const SizedBox(height: 16),
_buildLabel(context.l10n.labelEmail),
TextFormField(
  controller: _emailController,
  validator: Validators.validateEmail,
  keyboardType: TextInputType.emailAddress,
  decoration: _inputDecoration(
    hintText: 'example@mail.com',
    icon: Icons.email_outlined,
  ),
),
const SizedBox(height: 16),
_buildLabel(context.l10n.labelPassword),
TextFormField(
  controller: _passwordController,
  validator: Validators.validatePassword,
  obscureText: _obscurePassword,
  decoration: _inputDecoration(
    hintText: context.l10n.registerPasswordHint,
    icon: Icons.lock_outline,
    suffixIcon: IconButton(
      onPressed: () => setState(() => _obscurePassword = !_obscurePassword),
      icon: Icon(
        _obscurePassword ? Icons.visibility_off : Icons.visibility,
      ),
    ),
  ),
),
const SizedBox(height: 16),
_buildLabel(context.l10n.labelPasswordConfirm),
TextFormField(
  controller: _confirmPasswordController,

```

```

validator: (value) => Validators.validatePasswordConfirmation(
  value,
  _passwordController.text,
),
obscureText: _obscureConfirmPassword,
decoration: _inputDecoration(
  hintText: context.l10n.registerPasswordRepeatHint,
  icon: Icons.lock_reset_outlined,
  suffixIcon: IconButton(
    onPressed: () => setState(
      () => _obscureConfirmPassword = !_obscureConfirmPassword,
    ),
    icon: Icon(
      _obscureConfirmPassword ? Icons.visibility_off : Icons.visibility,
    ),
  ),
),
),
const SizedBox(height: 24),
SizedBox(
  width: double.infinity,
  height: 58,
  child: _isLoading
    ? const Center(
        child: CircularProgressIndicator(color: AppPalette.primary),
      )
    : ElevatedButton(
        style: ElevatedButton.styleFrom(
          backgroundColor: AppPalette.primary,
          foregroundColor: Colors.white,
        ),
        onPressed: _register,
        child: Text(context.l10n.registerAction),
      ),
),
const SizedBox(height: 12),
Align(
  alignment: Alignment.center,
  child: TextButton(
    onPressed: () => Navigator.pop(context),
    child: Text(context.l10n.haveAccountSignIn),
  ),
),

```

```

        ),
      ),
    ],
  ),
),
),
),
),
),
),
);
}

```

```

Widget _buildLabel(String text) {
  return Padding(
    padding: const EdgeInsets.only(bottom: 8),
    child: Text(
      text,
      style: const TextStyle(
        fontSize: 15,
        fontWeight: FontWeight.w600,
        color: AppPalette.textPrimary,
      ),
    ),
  );
}

```

```

InputDecoration _inputDecoration({
  required String hintText,
  required IconData icon,
  Widget? suffixIcon,
}) {
  return InputDecoration(
    hintText: hintText,
    prefixIcon: Icon(icon),
    suffixIcon: suffixIcon,
    filled: true,
    fillColor: AppPalette.background,
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(18),
      borderSide: const BorderSide(color: AppPalette.border),
    ),
  );
}

```

```

    ),
    enabledBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(18),
      borderSide: const BorderSide(color: AppPalette.border),
    ),
    focusedBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(18),
      borderSide: const BorderSide(color: AppPalette.primary),
    ),
    errorBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(18),
      borderSide: const BorderSide(color: AppPalette.danger),
    ),
    focusedErrorBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(18),
      borderSide: const BorderSide(color: AppPalette.danger),
    ),
  );
}
}

```

lib/screens/auth/forgot_password_screen.dart

```

import 'package:flutter/material.dart';
import '../services/auth_service.dart';
import '../utils/app_palette.dart';
import '../utils/validators.dart';
import '../l10n/l10n.dart';

class ForgotPasswordScreen extends StatefulWidget {
  const ForgotPasswordScreen({super.key});

  @override
  State<ForgotPasswordScreen> createState() => _ForgotPasswordScreenState();
}

class _ForgotPasswordScreenState extends State<ForgotPasswordScreen> {
  final _emailController = TextEditingController();
  final _authService = AuthService();
  bool _isLoading = false;

  @override
  void dispose() {

```



```

_emailController.dispose();
super.dispose();
}

Future<void> _resetPassword() async {
  final email = _emailController.text.trim();
  final error = Validators.validateEmail(email);

  if (error != null) {
    ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(error), backgroundColor:
AppPalette.danger));
    return;
  }

  setState(() => _isLoading = true);
  try {
    await _authService.sendPasswordReset(email);
    if (!mounted) return;

    await showDialog<void>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
        title: const Icon(Icons.mark_email_read, color: AppPalette.primary, size: 60),
        content: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Text(context.l10n.resetSentTitle, style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
textAlign: TextAlign.center),
            const SizedBox(height: 10),
            Text(context.l10n.resetSentBody(email), textAlign: TextAlign.center, style: const TextStyle(color:
AppPalette.textSecondary)),
          ],
        ),
        actions: [
          SizedBox(
            width: double.infinity,
            child: ElevatedButton(
              style: ElevatedButton.styleFrom(backgroundColor: AppPalette.primary, foregroundColor:
Colors.white),
              onPressed: () => Navigator.pop(dialogContext),

```

```

        child: Text(context.l10n.gotIt),
      ),
    ),
  ],
),
);
if (mounted) Navigator.pop(context);
} catch (e) {
  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(SnackBar(content:
Text(_authService.getErrorMessage(e)), backgroundColor: AppPalette.danger));
  }
} finally {
  if (mounted) setState(() => _isLoading = false);
}
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: AppPalette.background,
    appBar: AppBar(
      title: Text(context.l10n.resetPasswordTitle),
      backgroundColor: AppPalette.background,
      elevation: 0,
    ),
    body: SafeArea(
      child: Center(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: ConstrainedBox(
            constraints: const BoxConstraints(maxWidth: 480),
            child: Container(
              padding: const EdgeInsets.all(28),
              decoration: BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.circular(28),
                boxShadow: [BoxShadow(color: Colors.black.withValues(alpha: 0.05), blurRadius: 24, offset:
const Offset(0, 14))],
              ),
              child: Column(

```

```
crossAxisAlignment: CrossAxisAlignment.start,
children: [
  Text(context.l10n.forgotPasswordLink, style: TextStyle(fontSize: 28, fontWeight:
FontWeight.bold, fontFamily: 'Unbounded')),
  const SizedBox(height: 12),
  Text(context.l10n.resetPasswordInfo, style: TextStyle(color: AppPalette.textSecondary, height:
1.5)),
  const SizedBox(height: 24),
  Text(context.l10n.resetEmailLabel, style: TextStyle(fontWeight: FontWeight.bold)),
  const SizedBox(height: 8),
  TextField(
    controller: _emailController,
    keyboardType: TextInputType.emailAddress,
    decoration: InputDecoration(
      hintText: 'example@mail.com',
      prefixIcon: const Icon(Icons.email_outlined),
      filled: true,
      fillColor: AppPalette.background,
      border: OutlineInputBorder(borderRadius: BorderRadius.circular(18), borderSide:
BorderSide.none),
    ),
  ),
  const SizedBox(height: 24),
  SizedBox(
    width: double.infinity,
    height: 58,
    child: _isLoading
      ? const Center(child: CircularProgressIndicator(color: AppPalette.primary))
      : ElevatedButton(
        style: ElevatedButton.styleFrom(background-color: AppPalette.primary, foregroundColor:
Colors.white, shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(18))),
        onPressed: _resetPassword,
        child: Text(context.l10n.resetSendButton, style: TextStyle(fontSize: 16, font-weight:
FontWeight.bold)),
      ),
  ],
),
),
),
),
),
```

```

    ),
  ),
);
}
}

```

lib/screens/shop/shop_screen.dart

```

import 'package:flutter/material.dart';

import '../models/product_model.dart';
import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../widgets/centered_message.dart';
import '../widgets/product_card.dart';
import '../widgets/product_grid.dart';
import '../favorites/favorites_screen.dart';
import 'category_screen.dart';
import '../l10n/l10n.dart';

class ShopScreen extends StatefulWidget {
  const ShopScreen({super.key});

  @override
  State<ShopScreen> createState() => _ShopScreenState();
}

class _ShopScreenState extends State<ShopScreen> {
  static const List<String> _preferredCategoryOrder = [
    'Фрукты',
    'Овощи',
    'Молочные продукты',
    'Мясо',
    'Напитки',
    'Бакалея',
  ];

  static const Map<String, String> _sectionTitles = {
    'Фрукты': 'Свежие фрукты',
  };

  final TextEditingController _searchController = TextEditingController();
  late final Stream<List<Product>> _catalog = DatabaseService().watchCatalog();

```

```
String _searchQuery = "";
```

```
@override
```

```
void dispose() {  
  _searchController.dispose();  
  super.dispose();  
}
```

```
Map<String, List<Product>> _groupByCategory(List<Product> products) {  
  final grouped = <String, List<Product>>{};  
  for (final product in products) {  
    final category = product.categoryName.isEmpty ? context.l10n.categoryOther : product.categoryName;  
    grouped.putIfAbsent(category, () => []).add(product);  
  }  
}
```

```
final orderedKeys = grouped.keys.toList()  
  ..sort((left, right) {  
    final leftIndex = _preferredCategoryOrder.indexOf(left);  
    final rightIndex = _preferredCategoryOrder.indexOf(right);  
    if (leftIndex != -1 && rightIndex != -1) return leftIndex.compareTo(rightIndex);  
    if (leftIndex != -1) return -1;  
    if (rightIndex != -1) return 1;  
    return left.compareTo(right);  
  });
```

```
return {for (final key in orderedKeys) key: grouped[key]!};  
}
```

```
@override
```

```
Widget build(BuildContext context) {  
  return Scaffold(  
    backgroundColor: Colors.white,  
    appBar: AppBar(  
      backgroundColor: Colors.white,  
      elevation: 0,  
      centerTitle: true,  
      title: Text(  
        context.l10n.navShop,  
        style: TextStyle(color: AppPalette.textPrimary, fontSize: 20, fontWeight: FontWeight.bold),  
      ),  
      actions: [  
        
```

```

IconButton(
  icon: const Icon(Icons.favorite_border, color: Colors.black, size: 28),
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => const FavoritesScreen()),
    );
  },
),
],
),
body: Column(
  children: [
    const SizedBox(height: 10),
    Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Icon(Icons.eco, color: AppPalette.primary, size: 32),
        SizedBox(width: 8),
        Text(context.l10n.splashBrand, style: TextStyle(fontSize: 26, fontWeight: FontWeight.bold, color:
AppPalette.textPrimary)),
      ],
    ),
    const SizedBox(height: 10),
    Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Icon(Icons.location_on, color: AppPalette.textSecondary, size: 20),
        SizedBox(width: 5),
        Text(context.l10n.shopLocation, style: TextStyle(fontSize: 15, color: AppPalette.textSecondary,
fontWeight: FontWeight.w600)),
      ],
    ),
    const SizedBox(height: 15),
    Padding(
      padding: const EdgeInsets.symmetric(horizontal: 20),
      child: Container(
        decoration: BoxDecoration(
          color: const Color(0xFFFF2F3F2),
          borderRadius: BorderRadius.circular(15),
        ),
      ),
    ),
  ],
),
)

```

```

child: TextField(
  controller: _searchController,
  onChanged: (value) => setState(() => _searchQuery = value.trim()),
  decoration: InputDecoration(
    hintText: context.l10n.searchHint,
    hintStyle: const TextStyle(color: AppPalette.textSecondary, fontSize: 14),
    prefixIcon: const Icon(Icons.search, color: AppPalette.textPrimary),
    suffixIcon: _searchQuery.isEmpty
      ? null
      : IconButton(
        icon: const Icon(Icons.clear, color: AppPalette.textSecondary),
        onPressed: () {
          _searchController.clear();
          setState(() => _searchQuery = "");
          FocusScope.of(context).unfocus();
        },
      ),
    border: InputBorder.none,
    contentPadding: const EdgeInsets.symmetric(vertical: 15),
  ),
),
),
),
),
const SizedBox(height: 20),
Expanded(
  child: StreamBuilder<List<Product>>(
    stream: _catalog,
    builder: (context, snapshot) {
      if (snapshot.hasError) {
        return CenteredMessage(context.l10n.productsLoadFailedConnection, icon: Icons.cloud_off);
      }
      if (!snapshot.hasData) {
        return const Center(child: CircularProgressIndicator(color: AppPalette.primary));
      }

      final products = snapshot.data!;
      if (products.isEmpty) {
        return CenteredMessage(context.l10n.noProductsInDatabase);
      }

      if (_searchQuery.isNotEmpty) {

```

```

        final query = _searchQuery.toLowerCase();
        final results = products.where((product) =>
product.name.toLowerCase().contains(query)).toList();
        if (results.isEmpty) {
            return CenteredMessage(context.l10n.nothingFound);
        }
        return ProductGrid(products: results);
    }

    return _buildCatalog(products);
  },
),
),
],
),
);
}

```

```

Widget _buildCatalog(List<Product> products) {
    final sections = _groupByCategory(products).entries.toList();
    final popularProducts = (products.where((product) => product.popularity > 0 && !
product.isOutOfStock)).toList()
        ..sort((left, right) => right.popularity.compareTo(left.popularity)))
        .take(10)
        .toList();

    return ListView(
        padding: const EdgeInsets.only(bottom: 20),
        children: [
            Padding(
                padding: const EdgeInsets.symmetric(horizontal: 20),
                child: Container(
                    height: 115,
                    padding: const EdgeInsets.all(20),
                    decoration: BoxDecoration(
                        borderRadius: BorderRadius.circular(15),
                        gradient: const LinearGradient(
                            colors: [AppPalette.primaryDark, AppPalette.primary],
                            begin: Alignment.topLeft,
                            end: Alignment.bottomRight,
                        ),
                    ),

```



```

    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text(
          context.l10n.promoBannerTitle,
          style: const TextStyle(color: Colors.white, fontSize: 22, fontWeight: FontWeight.bold),
        ),
        const SizedBox(height: 4),
        Text(
          context.l10n.promoBannerCode,
          style: const TextStyle(color: AppPalette.addAction, fontSize: 16, fontWeight: FontWeight.w600),
        ),
      ],
    ),
  ),
  const SizedBox(height: 10),
  if (popularProducts.isNotEmpty)
    _ProductCarousel(title: context.l10n.popular, products: popularProducts),
  for (final section in sections)
    _CategorySection(
      title: context.isEnglish
        ? section.value.first.categoryFor(english: true)
        : (_sectionTitles[section.key] ?? section.key),
      categoryName: section.key,
      products: section.value,
    ),
  ],
);
}
}

```

```

class _CategorySection extends StatelessWidget {
  const _CategorySection({
    required this.title,
    required this.categoryName,
    required this.products,
  });
}

```

```

final String title;
final String categoryName;
final List<Product> products;

@override
Widget build(BuildContext context) {
  return _ProductCarousel(
    title: title,
    products: products,
    onSeeAll: () {
      FocusScope.of(context).unfocus();
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (_) => CategoryScreen(title: title, categoryName: [categoryName]),
        ),
      );
    },
  );
}

class _ProductCarousel extends StatelessWidget {
  const _ProductCarousel({required this.title, required this.products, this.onSeeAll});

  final String title;
  final List<Product> products;
  final VoidCallback? onSeeAll;

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Padding(
          padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 15),
          child: GestureDetector(
            behavior: HitTestBehavior.opaque,
            onTap: onSeeAll,
            child: Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [

```

```

        Flexible(
          child: Text(
            title,
            overflow: TextOverflow.ellipsis,
            style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold, color:
AppPalette.textPrimary),
          ),
        ),
        if (onSeeAll != null)
          Text(context.l10n.seeAll, style: TextStyle(fontSize: 16, fontWeight: FontWeight.w600, color:
AppPalette.primary)),
      ],
    ),
  ),
  SizedBox(
    height: 260,
    child: ListView.builder(
      scrollDirection: Axis.horizontal,
      padding: const EdgeInsets.symmetric(horizontal: 15),
      itemCount: products.length,
      itemBuilder: (context, index) => Padding(
        padding: const EdgeInsets.symmetric(horizontal: 5),
        child: ProductCard(product: products[index], width: 173),
      ),
    ),
  ),
],
);
}
}

```

lib/screens/shop/category_screen.dart

```

import 'package:flutter/material.dart';

import '../models/product_model.dart';
import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/product_filters.dart';
import '../widgets/centered_message.dart';
import '../widgets/product_filters_sheet.dart';
import '../widgets/product_grid.dart';

```

```

import '../110n/110n.dart';

class CategoryScreen extends StatefulWidget {
  const CategoryScreen({
    super.key,
    required this.title,
    required this.categoryNames,
  });

  final String title;
  final List<String> categoryNames;

  @override
  State<CategoryScreen> createState() => _CategoryScreenState();
}

class _CategoryScreenState extends State<CategoryScreen> {
  late final Stream<List<Product>> _catalog = DatabaseService().watchCatalog();
  ProductFilters _filters = const ProductFilters();
  List<Product> _categoryProducts = const [];

  Future<void> _openFilters() async {
    final result = await showProductFiltersSheet(context, products: _categoryProducts, current: _filters);
    if (result != null && mounted) setState(() => _filters = result);
  }

  bool _matches(Product product) {
    final category = product.categoryName.toLowerCase();
    return widget.categoryNames.any((name) => name.toLowerCase() == category);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.white,
      appBar: AppBar(
        backgroundColor: Colors.white,
        elevation: 0,
        centerTitle: true,
        leading: IconButton(
          icon: const Icon(Icons.arrow_back_ios, color: Colors.black),

```

```

        onPressed: () => Navigator.pop(context),
      ),
      title: Text(
        widget.title,
        style: const TextStyle(color: AppPalette.textPrimary, fontSize: 20, fontWeight: FontWeight.bold),
      ),
      actions: [
        IconButton(
          tooltip: context.l10n.filtersTitle,
          onPressed: _openFilters,
          icon: Icon(
            _filters.isActive ? Icons.filter_alt : Icons.filter_alt_outlined,
            color: _filters.isActive ? AppPalette.primary : Colors.black,
          ),
        ),
      ],
    ),
    body: StreamBuilder<List<Product>>(
      stream: _catalog,
      builder: (context, snapshot) {
        if (snapshot.hasError) {
          return CenteredMessage(context.l10n.productsLoadFailedLater, icon: Icons.cloud_off);
        }
        if (!snapshot.hasData) {
          return const Center(child: CircularProgressIndicator(color: AppPalette.primary));
        }

        _categoryProducts = snapshot.data!.where(_matches).toList();
        if (_categoryProducts.isEmpty) {
          return CenteredMessage(context.l10n.categoryEmpty);
        }

        final products = _filters.apply(_categoryProducts);
        if (products.isEmpty) {
          return CenteredMessage(context.l10n.noProductsForFilters);
        }

        return ProductGrid(products: products, padding: const EdgeInsets.symmetric(horizontal: 20, vertical:
10));
      },
    ),
  ),

```

```

    );
  }
}

```

lib/screens/shop/product_detail_screen.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:share_plus/share_plus.dart';

import '../models/product_model.dart';
import '../providers/cart_provider.dart';
import '../providers/favorite_provider.dart';
import '../utils/app_palette.dart';
import '../utils/parsers.dart';
import '../widgets/app_snackbar.dart';
import '../widgets/product_image.dart';
import '../l10n/l10n.dart';

class ProductDetailScreen extends StatefulWidget {
  const ProductDetailScreen({super.key, required this.product});

  final Product product;

  @override
  State<ProductDetailScreen> createState() => _ProductDetailScreenState();
}

class _ProductDetailScreenState extends State<ProductDetailScreen> {
  int _quantity = 1;

  Product get product => widget.product;

  int get _maxQuantity => product.stock.clamp(1, CartProvider.maxQuantityPerItem);

  void _share() {
    final shareText = context.l10n.shareMessage(
      product.nameFor(english: context.isEnglish),
      product.price.toStringAsFixed(0),
      'https://nectar.app/product/${product.id}',
    );
    SharePlus.instance.share(ShareParams(text: shareText));
  }
}

```

```

void _addToCart() {
  final cart = context.read<CartProvider>();
  var addedCount = 0;
  for (var i = 0; i < _quantity; i++) {
    if (cart.addProduct(product)) addedCount++;
  }

  showAppSnackBar(
    context,
    addedCount == 0
      ? context.l10n.notEnoughStock
      : addedCount < _quantity
        ? context.l10n.addedPartial(addedCount)
        : context.l10n.addedQuantityToCart(product.nameFor(english: context.isEnglish), addedCount),
    icon: addedCount > 0 ? Icons.shopping_bag : Icons.info_outline,
    isError: addedCount == 0,
    floating: true,
  );
}

@override
Widget build(BuildContext context) {
  final isOutOfStock = product.isOutOfStock;

  return Scaffold(
    backgroundColor: Colors.white,
    appBar: AppBar(
      backgroundColor: const Color(0xFFFF2F3F2),
      elevation: 0,
      leading: IconButton(
        icon: const Icon(Icons.arrow_back_ios, color: Colors.black),
        onPressed: () => Navigator.pop(context),
      ),
      actions: [
        IconButton(
          icon: const Icon(Icons.share_outlined, color: Colors.black),
          onPressed: _share,
        ),
      ],
    ),
  );
}

```

```

body: SafeArea(
  child: Column(
    children: [
      Container(
        width: double.infinity,
        height: 250,
        decoration: const BoxDecoration(
          color: Color(0xFFF2F3F2),
          borderRadius: BorderRadius.only(
            bottomLeft: Radius.circular(25),
            bottomRight: Radius.circular(25),
          ),
        ),
        child: Padding(
          padding: const EdgeInsets.all(20),
          child: InteractiveViewer(minScale: 1, maxScale: 4, child: ProductImage(url: product.imageUrl,
iconSize: 100)),
        ),
      ),
      Expanded(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(20),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Row(
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: [
                  Expanded(
                    child: Text(
                      product.nameFor(english: context.isEnglish),
                      style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold, color:
AppPalette.textPrimary),
                    ),
                  ),
                  Consumer<FavoriteProvider>(
                    builder: (context, favorites, child) {
                      final isFavorite = favorites.isFavorite(product.id);
                      return IconButton(
                        icon: Icon(
                          isFavorite ? Icons.favorite : Icons.favorite_border,

```



```

        color: isFavorite ? AppPalette.primary : AppPalette.textSecondary,
        size: 28,
      ),
      onPressed: () => favorites.toggle(product.id),
    );
  },
),
],
),
const SizedBox(height: 10),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    Text(
      product.qtyFor(english: context.isEnglish),
      style: const TextStyle(fontSize: 16, color: AppPalette.textSecondary, fontWeight:
FontWeight.w600),
    ),
    Container(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
      decoration: BoxDecoration(
        color: isOutOfStock ? const Color(0xFFFDECEA) : AppPalette.lightPrimary,
        borderRadius: BorderRadius.circular(8),
      ),
      child: Text(
        isOutOfStock ? context.l10n.outOfStock : context.l10n.stockLeft(product.stock),
        style: TextStyle(
          color: isOutOfStock ? AppPalette.danger : AppPalette.primaryDark,
          fontWeight: FontWeight.bold,
        ),
      ),
    ),
  ],
),
const SizedBox(height: 30),
Text(
  '${product.price.toStringAsFixed(0)} ₪',
  style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold, color:
AppPalette.textPrimary),
),
const SizedBox(height: 30),

```

```

const Divider(thickness: 1, color: Color(0xFFE2E2E2)),
Theme(
  data: Theme.of(context).copyWith(dividerColor: Colors.transparent),
  child: ExpansionTile(
    initiallyExpanded: true,
    iconColor: AppPalette.textPrimary,
    collapsedIconColor: AppPalette.textPrimary,
    tilePadding: EdgeInsets.zero,
    title: Text(
      context.l10n.productDescriptionTitle,
      style: TextStyle(fontSize: 16, fontWeight: FontWeight.w600, color: AppPalette.textPrimary),
    ),
    children: [
      Align(
        alignment: Alignment.centerLeft,
        child: Text(
          product.descriptionFor(english: context.isEnglish).isEmpty
            ? context.l10n.noDescription
            : product.descriptionFor(english: context.isEnglish),
          style: const TextStyle(fontSize: 14, color: AppPalette.textSecondary, height: 1.5),
        ),
      ),
    ],
  ),
const Divider(thickness: 1, color: Color(0xFFE2E2E2)),
_ProductFacts(product: product),
],
),
),
),
Padding(
  padding: const EdgeInsets.fromLTRB(20, 8, 20, 20),
  child: Row(
    children: [
      if (!isOutOfStock) ...[
        _QuantitySelector(
          quantity: _quantity,
          onDecrement: _quantity > 1 ? () => setState(() => _quantity--) : null,
          onIncrement: _quantity < _maxQuantity ? () => setState(() => _quantity++) : null,
        ),

```

```

        const SizedBox(width: 16),
      ],
      Expanded(
        child: SizedBox(
          height: 60,
          child: ElevatedButton(
            style: ElevatedButton.styleFrom(
              backgroundColor: AppPalette.primary,
              disabledBackgroundColor: Colors.grey,
              shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(19)),
            ),
            onPressed: isOutOfStock ? null : _addToCart,
            child: Text(
              isOutOfStock ? context.l10n.soldOut : context.l10n.addToCartWithPrice((product.price *
                _quantity).toStringAsFixed(0)),
              style: const TextStyle(fontSize: 17, fontWeight: FontWeight.w600, color: Colors.white),
            ),
          ),
        ),
      ),
    ],
  ),
),
],
),
),
);
}
}

```

```

class _QuantitySelector extends StatelessWidget {
  const _QuantitySelector({required this.quantity, this.onDecrement, this.onIncrement});

  final int quantity;
  final VoidCallback? onDecrement;
  final VoidCallback? onIncrement;

  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(

```

```

        border: Border.all(color: const Color(0xFFE2E2E2)),
        borderRadius: BorderRadius.circular(16),
    ),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        IconButton(onPressed: onDecrement, icon: const Icon(Icons.remove)),
        SizedBox(
          width: 28,
          child: Text('$quantity', textAlign: TextAlign.center, style: const TextStyle(fontSize: 18, fontWeight:
FontWeight.bold)),
        ),
        IconButton(onPressed: onIncrement, icon: const Icon(Icons.add, color: AppPalette.primary)),
      ],
    ),
  );
}
}

```

```

class _ProductFacts extends StatelessWidget {
  const _ProductFacts({required this.product});

  final Product product;

  @override
  Widget build(BuildContext context) {
    final rows = <MapEntry<String, String>>[
      if (product.brand.isNotEmpty) MapEntry(context.l10n.factManufacturer, product.brand),
      if (product.country.isNotEmpty) MapEntry(context.l10n.factCountry, product.countryFor(english:
context.isEnglish)),
      if (product.expiryDate != null) MapEntry(context.l10n.factBestBefore,
formatDate(product.expiryDate!)),
      if (product.compositionFor(english: context.isEnglish).isNotEmpty)
MapEntry(context.l10n.factComposition, product.compositionFor(english: context.isEnglish)),
    ];

    if (rows.isEmpty && !product.hasNutrition) {
      return const SizedBox.shrink();
    }

    return Column(

```

```

crossAxisAlignment: CrossAxisAlignment.start,
children: [
  const SizedBox(height: 16),
  Text(
    context.l10n.productFactsTitle,
    style: TextStyle(fontSize: 16, fontWeight: FontWeight.w600, color: AppPalette.textPrimary),
  ),
  const SizedBox(height: 8),
  for (final row in rows)
    Padding(
      padding: const EdgeInsets.symmetric(vertical: 4),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Expanded(flex: 4, child: Text(row.key, style: const TextStyle(color: AppPalette.textSecondary))),
          Expanded(flex: 6, child: Text(row.value, textAlign: TextAlign.right, style: const
TextStyle(fontWeight: FontWeight.w600))),
        ],
      ),
    ),
  if (product.hasNutrition) ...[
    const SizedBox(height: 16),
    Text(
      context.l10n.nutritionTitle,
      style: TextStyle(fontSize: 16, fontWeight: FontWeight.w600, color: AppPalette.textPrimary),
    ),
    const SizedBox(height: 8),
    Row(
      children: [
        _NutritionCell(label: context.l10n.nutritionCalories, value: product.calories),
        _NutritionCell(label: context.l10n.nutritionProteins, value: product.proteins),
        _NutritionCell(label: context.l10n.nutritionFats, value: product.fats),
        _NutritionCell(label: context.l10n.nutritionCarbs, value: product.carbs),
      ],
    ),
  ],
  const SizedBox(height: 8),
],
);
}
}

```

```

class _NutritionCell extends StatelessWidget {
  const _NutritionCell({required this.label, required this.value});

  final String label;
  final double value;

  @override
  Widget build(BuildContext context) {
    return Expanded(
      child: Container(
        margin: const EdgeInsets.symmetric(horizontal: 3),
        padding: const EdgeInsets.symmetric(vertical: 10),
        decoration: BoxDecoration(
          color: AppPalette.lightPrimary,
          borderRadius: BorderRadius.circular(12),
        ),
      child: Column(
        children: [
          Text(
            value == value.roundToDouble() ? value.toStringAsFixed(0) : value.toStringAsFixed(1),
            style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold, color: AppPalette.primaryDark),
          ),
          const SizedBox(height: 2),
          Text(label, style: const TextStyle(fontSize: 11, color: AppPalette.textSecondary)),
        ],
      ),
    );
  }
}

```

lib/screens/explore/explore_screen.dart

```

import 'package:flutter/material.dart';

import '../data/catalog_translations.dart';
import '../models/product_model.dart';
import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/parsers.dart';
import '../utils/product_filters.dart';
import '../widgets/centered_message.dart';

```

```

import '../widgets/product_filters_sheet.dart';
import '../widgets/product_image.dart';
import '../widgets/product_grid.dart';
import '../shop/category_screen.dart';
import '../l10n/l10n.dart';

class _TileColors {
  const _TileColors(this.background, this.border);

  final Color background;
  final Color border;
}

class ExploreScreen extends StatefulWidget {
  const ExploreScreen({super.key});

  @override
  State<ExploreScreen> createState() => _ExploreScreenState();
}

class _ExploreScreenState extends State<ExploreScreen> {
  static const List<_TileColors> _palette = [
    _TileColors(Color(0x1A53B175), AppPalette.primary),
    _TileColors(Color(0x1AF8A44C), Color(0xFFFF8A44C)),
    _TileColors(Color(0x1AF7A593), Color(0xFFFF7A593)),
    _TileColors(Color(0x1AD3B0E0), Color(0xFFD3B0E0)),
    _TileColors(Color(0x1AFDE598), Color(0xFFFFDE598)),
    _TileColors(Color(0x1AB7DFF5), Color(0xFFB7DFF5)),
  ];

  final TextEditingController _searchController = TextEditingController();
  final DatabaseService _database = DatabaseService();
  late final Stream<List<Product>> _catalog = _database.watchCatalog();
  late final Stream<List<Map<String, dynamic>>> _categories = _database.watchCategories();
  String _searchQuery = "";
  ProductFilters _filters = const ProductFilters();
  List<Product> _searchMatches = const [];

  Future<void> _openFilters() async {
    final result = await showProductFiltersSheet(context, products: _searchMatches, current: _filters);
    if (result != null && mounted) setState(() => _filters = result);
  }
}

```

```
}
```

```
@override
```

```
void dispose() {  
  _searchController.dispose();  
  super.dispose();  
}
```

```
@override
```

```
Widget build(BuildContext context) {  
  return Scaffold(  
    backgroundColor: Colors.white,  
    appBar: AppBar(  
      backgroundColor: Colors.white,  
      elevation: 0,  
      centerTitle: true,  
      title: Text(  
        context.l10n.navCatalog,  
        style: TextStyle(color: AppPalette.textPrimary, fontSize: 20, fontWeight: FontWeight.bold),  
      ),  
    ),  
    body: Column(  
      children: [  
        Padding(  
          padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 10),  
          child: Container(  
            decoration: BoxDecoration(  
              color: const Color(0xFFFF2F3F2),  
              borderRadius: BorderRadius.circular(15),  
            ),  
            child: TextField(  
              controller: _searchController,  
              onChanged: (value) => setState(() => _searchQuery = value.trim()),  
              decoration: InputDecoration(  
                hintText: context.l10n.searchHint,  
                hintStyle: const TextStyle(color: AppPalette.textSecondary, fontSize: 14),  
                prefixIcon: const Icon(Icons.search, color: AppPalette.textPrimary),  
                suffixIcon: _searchQuery.isEmpty  
                  ? null  
                  : Row(  
                    mainAxisAlignment: MainAxisAlignment.min,
```



```

children: [
  IconButton(
    tooltip: context.l10n.filtersTitle,
    icon: Icon(
      _filters.isActive ? Icons.filter_alt : Icons.filter_alt_outlined,
      color: _filters.isActive ? AppPalette.primary : AppPalette.textSecondary,
    ),
    onPressed: _openFilters,
  ),
  IconButton(
    icon: const Icon(Icons.clear, color: AppPalette.textSecondary),
    onPressed: () {
      _searchController.clear();
      setState(() {
        _searchQuery = "";
        _filters = const ProductFilters();
      });
      FocusScope.of(context).unfocus();
    },
  ),
],
),
border: InputBorder.none,
contentPadding: const EdgeInsets.symmetric(vertical: 15),
),
),
),
Expanded(
  child: _searchQuery.isEmpty ? _buildTiles() : _buildSearchResults(),
),
],
),
);
}

```

```

Widget _buildTiles() {
  return StreamBuilder<List<Map<String, dynamic>>>>(
    stream: _categories,
    builder: (context, snapshot) {
      if (snapshot.hasError) {

```

```

        return CenteredMessage(context.l10n.categoriesLoadFailed, icon: Icons.cloud_off);
    }
    if (!snapshot.hasData) {
        return const Center(child: CircularProgressIndicator(color: AppPalette.primary));
    }

    final categories = snapshot.data!;
    if (categories.isEmpty) {
        return CenteredMessage(context.l10n.categoriesEmpty);
    }

    return GridView.builder(
        padding: const EdgeInsets.all(20),
        gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
            crossAxisCount: 2,
            crossAxisSpacing: 15,
            mainAxisSpacing: 15,
            childAspectRatio: 0.85,
        ),
        itemCount: categories.length,
        itemBuilder: (context, index) {
            final category = categories[index];
            final name = toStringValue(category['name']);
            final displayName = pickLocalized(
                english: context.isEnglish,
                russian: name,
                englishValue: category['nameEn']?.toString() ?? categoryTranslations[category['id']]?.name,
            );
            final imageUrl = toStringValue(category['imageUrl']);
            final colors = _palette[index % _palette.length];

            return GestureDetector(
                onTap: () {
                    FocusScope.of(context).unfocus();
                    Navigator.push(
                        context,
                        MaterialPageRoute(
                            builder: (_) => CategoryScreen(title: displayName, categoryNames: [name]),
                        ),
                    );
                },
            ),
        ),
    );
}

```

```

        child: Container(
          padding: const EdgeInsets.all(12),
          decoration: BoxDecoration(
            color: colors.background,
            border: Border.all(color: colors.border),
            borderRadius: BorderRadius.circular(18),
          ),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              SizedBox(height: 70, child: ProductImage(url: imageUrl, iconSize: 56)),
              const SizedBox(height: 16),
              Text(
                displayName,
                textAlign: TextAlign.center,
                style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold, color:
AppPalette.textPrimary),
              ),
            ],
          ),
        ),
      ),
    );
  },
);
}

```

```

Widget _buildSearchResults() {
  return StreamBuilder<List<Product>>(
    stream: _catalog,
    builder: (context, snapshot) {
      if (snapshot.hasError) {
        return CenteredMessage(context.l10n.searchFailed, icon: Icons.cloud_off);
      }
      if (!snapshot.hasData) {
        return const Center(child: CircularProgressIndicator(color: AppPalette.primary));
      }

      final query = _searchQuery.toLowerCase();
      _searchMatches = snapshot.data!.where((product) {

```

```

        return product.name.toLowerCase().contains(query) ||
product.categoryName.toLowerCase().contains(query);
    }).toList();

    if (_searchMatches.isEmpty) {
        return CenteredMessage(context.l10n.nothingFound);
    }

    final results = _filters.apply(_searchMatches);
    if (results.isEmpty) {
        return CenteredMessage(context.l10n.noProductsForFilters);
    }
    return ProductGrid(products: results);
},
);
}
}

```

lib/screens/favorites/favorites_screen.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/product_model.dart';
import '../providers/cart_provider.dart';
import '../providers/favorite_provider.dart';
import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../widgets/app_snackbar.dart';
import '../widgets/centered_message.dart';
import '../widgets/product_grid.dart';
import '../l10n/l10n.dart';

class FavoritesScreen extends StatefulWidget {
  const FavoritesScreen({super.key});

  @override
  State<FavoritesScreen> createState() => _FavoritesScreenState();
}

class _FavoritesScreenState extends State<FavoritesScreen> {
  late final Stream<List<Product>> _catalog = DatabaseService().watchCatalog();

```

```

void _addAllToCart(List<Product> products) {
  final cart = context.read<CartProvider>();
  var addedCount = 0;
  for (final product in products) {
    if (cart.addProduct(product)) addedCount++;
  }

  showAppSnackBar(
    context,
    addedCount == 0 ? context.l10n.favoritesNothingToAdd : context.l10n.favoritesAdded(addedCount),
    isError: addedCount == 0,
    floating: true,
  );
}

@override
Widget build(BuildContext context) {
  final favoriteIds = context.watch<FavoriteProvider>().ids;

  return Scaffold(
    backgroundColor: Colors.white,
    appBar: AppBar(
      backgroundColor: Colors.white,
      elevation: 0,
      centerTitle: true,
      leading: IconButton(
        icon: const Icon(Icons.arrow_back_ios, color: Colors.black),
        onPressed: () => Navigator.pop(context),
      ),
      title: Text(
        context.l10n.favoritesTitle,
        style: TextStyle(color: AppPalette.textPrimary, fontSize: 20, fontWeight: FontWeight.bold),
      ),
    ),
    body: StreamBuilder<List<Product>>(
      stream: _catalog,
      builder: (context, snapshot) {
        if (snapshot.hasError) {
          return CenteredMessage(context.l10n.favoritesLoadFailed, icon: Icons.cloud_off);
        }
        if (!snapshot.hasData) {

```

```

        return const Center(child: CircularProgressIndicator(color: AppPalette.primary));
    }

    final favorites = snapshot.data!.where((product) => favoriteIds.contains(product.id)).toList();
    if (favorites.isEmpty) {
        return CenteredMessage(context.l10n.favoritesEmpty);
    }

    return Stack(
      children: [
        ProductGrid(
          products: favorites,
          padding: const EdgeInsets.only(left: 20, right: 20, top: 10, bottom: 100),
        ),
        Align(
          alignment: Alignment.bottomCenter,
          child: Padding(
            padding: const EdgeInsets.all(20),
            child: SizedBox(
              width: double.infinity,
              height: 65,
              child: ElevatedButton(
                style: ElevatedButton.styleFrom(
                  backgroundColor: AppPalette.primary,
                  shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(19)),
                ),
                onPressed: () => _addAllToCart(favorites),
                child: Text(
                  context.l10n.addAllToCart,
                  style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600, color: Colors.white),
                ),
              ),
            ),
          ),
        ),
      ],
    );
  },
);
}

```

```
}
```

lib/screens/cart/cart_screen.dart

```
import 'package:flutter/material.dart';  
import 'package:provider/provider.dart';
```

```
import '../providers/cart_provider.dart';  
import '../utils/app_palette.dart';  
import '../utils/pricing.dart';  
import '../widgets/app_snackbar.dart';  
import '../widgets/centered_message.dart';  
import '../widgets/product_image.dart';  
import 'checkout_bottom_sheet.dart';  
import '../l10n/l10n.dart';
```

```
class CartScreen extends StatelessWidget {  
  const CartScreen({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {  
    final cart = context.watch<CartProvider>();  
    final cartItems = cart.items.values.toList();
```

```
    return Scaffold(  
      backgroundColor: AppPalette.background,  
      appBar: AppBar(title: Text(context.l10n.cartTitle)),  
      body: Column(  
        children: [  
          const Divider(thickness: 1, height: 1, color: Color(0xFFE2E2E2)),  
          Expanded(  
            child: cartItems.isEmpty  
              ? CenteredMessage(context.l10n.cartEmpty, icon: Icons.shopping_cart_outlined)  
              : ListView.separated(  
                itemCount: cartItems.length,  
                separatorBuilder: (_, __) =>  
                  const Divider(thickness: 1, color: Color(0xFFE2E2E2), indent: 20, endIndent: 20),  
                itemBuilder: (context, index) => _CartRow(item: cartItems[index]),  
              ),  
          ),  
          if (cartItems.isNotEmpty) _CheckoutPanel(cart: cart),  
        ],  
      ),
```

```

    );
  }
}

class _CartRow extends StatelessWidget {
  const _CartRow({required this.item});

  final CartItem item;

  @override
  Widget build(BuildContext context) {
    final cart = context.read<CartProvider>();

    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 10),
      child: Row(
        children: [
          SizedBox(
            width: 60,
            height: 60,
            child: ProductImage(url: item.imageUrl, iconSize: 40),
          ),
          const SizedBox(width: 20),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Row(
                  mainAxisAlignment: MainAxisAlignment.spaceBetween,
                  children: [
                    Expanded(
                      child: Text(
                        item.nameFor(english: context.isEnglish),
                        style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold, color:
AppPalette.textPrimary),
                      ),
                    ),
                    IconButton(
                      icon: const Icon(Icons.close, color: Color(0xFFB3B3B3)),
                      onPressed: () => cart.removeItem(item.id),
                    ),
                  ],
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

```



```

        ],
      ),
      Text(item.qtyFor(english: context.isEnglish), style: const TextStyle(fontSize: 14, color:
AppPalette.textSecondary)),
      const SizedBox(height: 10),
      Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          Row(
            children: [
              _QuantityButton(icon: Icons.remove, onTap: () => cart.decrement(item.id)),
              const SizedBox(width: 15),
              Text('${item.quantity}', style: const TextStyle(fontSize: 16, fontWeight: FontWeight.w600)),
              const SizedBox(width: 15),
              _QuantityButton(
                icon: Icons.add,
                color: AppPalette.primary,
                onTap: () {
                  if (!cart.increment(item.id)) {
                    showAppSnackBar(context, context.l10n.notEnoughStock, isError: true);
                  }
                },
              ),
            ],
          ),
          Text(
            '₹${item.lineTotal.toStringAsFixed(2)}',
            style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold, color:
AppPalette.textPrimary),
          ),
        ],
      ),
    ],
  );
}
}

```

```

class _QuantityButton extends StatelessWidget {
  const _QuantityButton({
    required this.icon,
    required this.onTap,
    this.color = const Color(0xFFB3B3B3),
  });

  final IconData icon;
  final VoidCallback onTap;
  final Color color;

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: onTap,
      child: Container(
        padding: const EdgeInsets.all(5),
        decoration: BoxDecoration(
          border: Border.all(color: const Color(0xFFE2E2E2)),
          borderRadius: BorderRadius.circular(12),
        ),
      child: Icon(icon, color: color, size: 20),
    ),
  );
}

```

```

class _CheckoutPanel extends StatelessWidget {
  const _CheckoutPanel({required this.cart});

  final CartProvider cart;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(20),
      child: Column(
        children: [
          if (cart.hasPromoCode)
            Container(
              width: double.infinity,

```

```

margin: const EdgeInsets.only(bottom: 12),
padding: const EdgeInsets.all(14),
decoration: BoxDecoration(
  color: AppPalette.lightPrimary,
  borderRadius: BorderRadius.circular(16),
),
child: Text(
  context.l10n.cartPromoApplied(cart.appliedPromoCode ?? ",
cart.discountPercent.toStringAsFixed(0), formatMoney(cart.discountAmount)),
  style: const TextStyle(color: AppPalette.primaryDark, fontWeight: FontWeight.w600),
),
),
_SummaryLine(context.l10n.summarySubtotal, formatMoney(cart.subtotalAmount)),
if (cart.hasPromoCode) _SummaryLine(context.l10n.summaryDiscount, '-$
{formatMoney(cart.discountAmount)}'),
_SummaryLine(
  context.l10n.summaryDelivery,
  cart.deliveryAmount == 0
    ? context.l10n.free
    : context.l10n.deliveryFeeWithThreshold(formatMoney(cart.deliveryAmount),
freeDeliveryThreshold.toStringAsFixed(0)),
),
const SizedBox(height: 12),
SizedBox(
  width: double.infinity,
  height: 65,
  child: ElevatedButton(
    style: ElevatedButton.styleFrom(
      backgroundColor: AppPalette.primary,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(19)),
    ),
    onPressed: () {
      showModalBottomSheet<void>(
        context: context,
        isScrollControlled: true,
        backgroundColor: Colors.transparent,
        builder: (_) => const CheckoutBottomSheet(),
      );
    },
  ),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,

```

```

children: [
  Text(
    context.l10n.checkout,
    style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600, color: Colors.white),
  ),
  Container(
    padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
    decoration: BoxDecoration(
      color: AppPalette.primaryDark,
      borderRadius: BorderRadius.circular(6),
    ),
    child: Text(
      '₹${cart.totalAmount.toStringAsFixed(2)}',
      style: const TextStyle(fontSize: 14, fontWeight: FontWeight.w600, color: Colors.white),
    ),
  ),
],
),
),
),
),
],
),
);
}
}

```

```

class _SummaryLine extends StatelessWidget {
  const _SummaryLine(this.label, this.value);

  final String label;
  final String value;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 2),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          Text(label, style: const TextStyle(color: AppPalette.textSecondary)),

```

```

        Flexible(child: Text(value, textAlign: TextAlign.right, style: const TextStyle(fontWeight:
FontWeight.w600))),
      ],
    ),
  );
}
}

```

lib/screens/cart/checkout_bottom_sheet.dart

```

import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/cart_provider.dart';
import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/pricing.dart';
import '../widgets/app_snackbar.dart';
import '../account/promo_code_screen.dart';
import '../l10n/l10n.dart';

class CheckoutBottomSheet extends StatefulWidget {
  const CheckoutBottomSheet({super.key});

  @override
  State<CheckoutBottomSheet> createState() => _CheckoutBottomSheetState();
}

class _CheckoutBottomSheetState extends State<CheckoutBottomSheet> {
  static const String _paymentOnline = 'Онлайн';
  static const String _paymentCash = 'Наличными';

  final DatabaseService _databaseService = DatabaseService();
  final User? _currentUser = FirebaseAuth.instance.currentUser;
  late final Stream<Map<String, dynamic>?>? _profileStream =
    _currentUser == null ? null : _databaseService.watchUserProfile(_currentUser.uid);

  String? _selectedAddress;
  String _selectedSlot = deliverySlots.first;
  String _selectedPaymentMethod = _paymentOnline;
  String? _selectedCard;
  bool _isProcessing = false;

```

```

Future<void> _placeOrder(CartProvider cartProvider) async {
  final currentUser = _currentUser;
  if (currentUser == null) {
    _showMessage(context.l10n.checkoutSignInRequired, isError: true);
    return;
  }

  final address = _selectedAddress?.trim() ?? "";
  if (address.isEmpty) {
    _showMessage(context.l10n.checkoutSelectAddress, isError: true);
    return;
  }

  final card = _selectedCard;
  if (_selectedPaymentMethod == _paymentOnline && (card == null || card.isEmpty)) {
    _showMessage(context.l10n.checkoutSelectCard, isError: true);
    return;
  }

  setState(() => _isProcessing = true);
  try {
    Map<String, dynamic>? promoCodeData;
    final appliedPromoCode = cartProvider.appliedPromoCode;
    if (cartProvider.hasPromoCode && appliedPromoCode != null) {
      promoCodeData = await _databaseService.validatePromoCode(appliedPromoCode);
    }

    final orderData = await _databaseService.placeOrder(
      userId: currentUser.uid,
      orderItems: cartProvider.items.values.map((cartItem) {
        return {
          'id': cartItem.id,
          'name': cartItem.name,
          'price': cartItem.price,
          'quantity': cartItem.quantity,
          'imageUrl': cartItem.imageUrl,
        };
      }).toList(),
      deliveryAddress: address,
      paymentMethod: _selectedPaymentMethod,
    );
  } catch (e) {
    _showMessage(context.l10n.checkoutError, isError: true);
  }
}

```

```

        paymentCard: _selectedPaymentMethod == _paymentOnline ? card : null,
        deliverySlot: _selectedSlot,
        promoCode: promoCodeData,
    );

    cartProvider.clear();

    if (!mounted) {
        return;
    }

    final navigator = Navigator.of(context);
    final orderNumber = orderData['number']?.toString() ?? orderData['id'].toString();

    // Закрываем шторку, а диалог показываем в контексте навигатора: контекст
    // этого State после pop использовать нельзя.
    navigator.pop();
    await showDialog<void>(
        context: navigator.context,
        barrierDismissible: false,
        builder: (dialogContext) => _OrderSuccessDialog(orderNumber: orderNumber),
    );
  } on DatabaseOperationException catch (error) {
    _showMessage(error.message, isError: true);
  } catch (error) {
    debugPrint('Order placement error: $error');
    _showMessage(AppLocale.strings.checkoutFailed, isError: true);
  } finally {
    if (mounted) {
      setState(() => _isProcessing = false);
    }
  }
}

void _showMessage(String message, {bool isError = false}) {
  if (!mounted) return;
  showAppSnackBar(context, message, isError: isError);
}

@override
Widget build(BuildContext context) {

```

```

final cartProvider = context.watch<CartProvider>();

return Container(
  padding: const EdgeInsets.all(24),
  decoration: const BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.vertical(top: Radius.circular(30)),
  ),
  child: SafeArea(
    top: false,
    child: Column(
      mainAxisAlignment: MainAxisAlignment.min,
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Expanded(
              child: Text(
                context.l10n.checkoutTitle,
                style: const TextStyle(
                  fontSize: 22,
                  fontWeight: FontWeight.bold,
                  fontFamily: 'Unbounded',
                ),
              ),
            ),
            IconButton(
              onPressed: () => Navigator.pop(context),
              icon: const Icon(Icons.close),
            ),
          ],
        ),
        const SizedBox(height: 12),
        StreamBuilder<Map<String, dynamic>?>(
          stream: _profileStream,
          builder: (context, snapshot) {
            final addresses = List<String>.from(snapshot.data?['addresses'] ?? const []);
            final cards = List<String>.from(snapshot.data?['cards'] ?? const []);

            if (_selectedAddress == null && addresses.isNotEmpty) {

```



```

        _selectedAddress = addresses.first;
    }
    if (_selectedCard == null && cards.isNotEmpty) {
        _selectedCard = cards.first;
    }
    if (_selectedAddress != null && !addresses.contains(_selectedAddress)) {
        _selectedAddress = addresses.isEmpty ? null : addresses.first;
    }
    if (_selectedCard != null && !cards.contains(_selectedCard)) {
        _selectedCard = cards.isEmpty ? null : cards.first;
    }

    return Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
            const Divider(),
            _buildDropDownRow(
                title: context.l10n.menuDeliveryAddress,
                items: addresses,
                selectedValue: _selectedAddress,
                emptyMessage: context.l10n.addAddressInProfile,
                onChanged: (value) => setState(() => _selectedAddress = value),
            ),
            const Divider(),
            _buildDropDownRow(
                title: context.l10n.deliveryTime,
                items: deliverySlots,
                selectedValue: _selectedSlot,
                emptyMessage: "",
                labelOf: (slot) => deliverySlotLabel(slot, context.l10n),
                onChanged: (value) {
                    if (value != null) setState(() => _selectedSlot = value);
                },
            ),
            const Divider(),
            Text(
                context.l10n.paymentMethodTitle,
                style: TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w700,
                    color: AppPalette.textPrimary,

```

```

    ),
  ),
  RadioGroup<String>(
    groupValue: _selectedPaymentMethod,
    onChanged: (value) {
      if (value != null) setState() => _selectedPaymentMethod = value;
    },
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        RadioListTile<String>(
          contentPadding: EdgeInsets.zero,
          value: _paymentOnline,
          title: Text(context.l10n.paymentOnline),
        ),
        if (_selectedPaymentMethod == _paymentOnline)
          _buildDropDownRow(
            title: context.l10n.cardLabel,
            items: cards,
            selectedValue: _selectedCard,
            emptyMessage: context.l10n.addCardInProfile,
            onChanged: (value) => setState() => _selectedCard = value,
          ),
        RadioListTile<String>(
          contentPadding: EdgeInsets.zero,
          value: _paymentCash,
          title: Text(context.l10n.paymentCash),
        ),
      ],
    ),
  ),
],
);
},
),
const Divider(),
ListTile(
  contentPadding: EdgeInsets.zero,
  title: Text(context.l10n.promoCodeLabel),
  subtitle: Text(
    cartProvider.hasPromoCode

```

```

        ? '${cartProvider.appliedPromoCode} (${cartProvider.discountPercent.toStringAsFixed(0)}%)'
        : context.l10n.promoNotSelected,
    ),
    trailing: TextButton(
      onPressed: () {
        Navigator.push(
          context,
          MaterialPageRoute(builder: (_) => const PromoCodeScreen()),
        );
      },
      child: Text(context.l10n.choose),
    ),
  ),
  const Divider(),
  _buildSummaryRow(context.l10n.summaryItemsTotal, 'P$
{cartProvider.subtotalAmount.toStringAsFixed(2)}'),
  _buildSummaryRow(context.l10n.summaryDiscount, '-P$
{cartProvider.discountAmount.toStringAsFixed(2)}'),
  _buildSummaryRow(
    context.l10n.summaryDelivery,
    cartProvider.deliveryAmount == 0 ? context.l10n.free : 'P$
{cartProvider.deliveryAmount.toStringAsFixed(2)}',
  ),
  _buildSummaryRow(
    context.l10n.total,
    'P$${cartProvider.totalAmount.toStringAsFixed(2)}',
    emphasize: true,
  ),
  const SizedBox(height: 18),
  SizedBox(
    width: double.infinity,
    height: 58,
    child: _isProcessing
      ? const Center(child: CircularProgressIndicator(color: AppPalette.primary))
      : ElevatedButton(
          style: ElevatedButton.styleFrom(
            backgroundColor: AppPalette.primary,
            foregroundColor: Colors.white,
          ),
          onPressed: () => _placeOrder(cartProvider),
          child: Text(context.l10n.placeOrder),
        ),
  ),

```

```

        ),
      ),
    ],
  ),
),
);
}

```

```

Widget _buildDropDownRow({
  required String title,
  required List<String> items,
  required String? selectedValue,
  required String emptyMessage,
  required ValueChanged<String?> onChanged,
  String Function(String)? labelOf,
}) {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 10),
    child: Row(
      children: [
        Expanded(
          flex: 4,
          child: Text(
            title,
            style: const TextStyle(
              fontWeight: FontWeight.w700,
              color: AppPalette.textPrimary,
            ),
          ),
        ),
        Expanded(
          flex: 6,
          child: items.isEmpty
            ? Text(
                emptyMessage,
                textAlign: TextAlign.right,
                style: const TextStyle(color: AppPalette.danger),
              )
            : DropdownButtonHideUnderline(
                child: DropdownButton<String>(
                  isExpanded: true,

```

```

        value: selectedValue,
        items: items.map((item) {
          return DropdownMenuItem<String>(
            value: item,
            child: Text(
              labelOf?.call(item) ?? item,
              overflow: TextOverflow.ellipsis,
              textAlign: TextAlign.right,
            ),
          );
        }).toList(),
        onChanged: onChanged,
      ),
    ),
  ],
),
);
}

```

```

Widget _buildSummaryRow(String label, String value, {bool emphasize = false}) {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 6),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text(
          label,
          style: const TextStyle(color: AppPalette.textSecondary),
        ),
        Text(
          value,
          style: TextStyle(
            fontSize: emphasize ? 18 : 16,
            fontWeight: emphasize ? FontWeight.bold : FontWeight.w600,
            color: AppPalette.textPrimary,
          ),
        ),
      ],
    ),
  );
}

```

```

}
}

```

```

class _OrderSuccessDialog extends StatelessWidget {
  const _OrderSuccessDialog({required this.orderNumber});

  final String orderNumber;

  @override
  Widget build(BuildContext context) {
    return AlertDialog(
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
      title: const Icon(Icons.check_circle, color: AppPalette.primary, size: 72),
      content: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Text(
            context.l10n.orderSuccessTitle,
            textAlign: TextAlign.center,
            style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold, fontFamily: 'Unbounded'),
          ),
          const SizedBox(height: 12),
          Text(
            context.l10n.orderSuccessBody(orderNumber),
            textAlign: TextAlign.center,
            style: const TextStyle(color: AppPalette.textSecondary, height: 1.5),
          ),
        ],
      ),
      actions: [
        SizedBox(
          width: double.infinity,
          child: ElevatedButton(
            style: ElevatedButton.styleFrom(
              backgroundColor: AppPalette.primary,
              foregroundColor: Colors.white,
            ),
            onPressed: () => Navigator.of(context).popUntil((route) => route.isFirst),
            child: Text(context.l10n.backToShop),
          ),
        ),
      ],
    );
  }
}

```

```

    ],
  );
}
}

```

lib/screens/account/account_screen.dart

```

import 'dart:convert';
import 'dart:typed_data';

import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';

import '../services/auth_service.dart';
import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/role_utils.dart';
import '../widgets/app_snackbar.dart';
import '../widgets/language_switcher.dart';
import 'about_screen.dart';
import 'change_password_dialog.dart';
import 'delivery_address_screen.dart';
import 'help_screen.dart';
import 'my_details_screen.dart';
import 'notifications_screen.dart';
import 'orders_screen.dart';
import 'payment_methods_screen.dart';
import 'promo_code_screen.dart';
import '../l10n/l10n.dart';

class AccountScreen extends StatefulWidget {
  const AccountScreen({super.key});

  @override
  State<AccountScreen> createState() => _AccountScreenState();
}

class _AccountScreenState extends State<AccountScreen> {
  final DatabaseService _databaseService = DatabaseService();
  final AuthService _authService = AuthService();
  final ImagePicker _imagePicker = ImagePicker();

```

```

bool _isUploadingAvatar = false;
bool _isDeletingAccount = false;

User? get _currentUser => FirebaseAuth.instance.currentUser;

Future<void> _pickAndUploadImage() async {
  final currentUser = _currentUser;
  if (currentUser == null) {
    return;
  }

  // Аватар хранится в Firestore, поэтому сжимаем его до маленького квадрата.
  final pickedFile = await _imagePicker.pickImage(
    source: ImageSource.gallery,
    maxWidth: 256,
    maxHeight: 256,
    imageQuality: 75,
  );

  if (pickedFile == null) {
    return;
  }

  final fileName = pickedFile.name.toLowerCase();
  final isSupportedFormat =
    fileName.endsWith('.jpg') ||
    fileName.endsWith('.jpeg') ||
    fileName.endsWith('.png');
  if (!isSupportedFormat) {
    _showMessage(AppLocale.strings.avatarFormatError, isError: true);
    return;
  }

  final Uint8List fileBytes = await pickedFile.readAsBytes();
  if (!mounted) {
    return;
  }
  if (fileBytes.length > DatabaseService.maxAvatarBytes) {
    _showMessage(context.l10n.avatarTooBig, isError: true);
    return;
  }
}

```



```

setState(() => _isUploadingAvatar = true);
try {
  await _databaseService.saveUserAvatar(
    userId: currentUser.uid,
    imageBytes: fileBytes,
  );

  if (!mounted) {
    return;
  }

  _showMessage(context.l10n.avatarUpdated);
} catch (error) {
  _showMessage(context.l10n.avatarUploadFailed, isError: true);
} finally {
  if (mounted) {
    setState(() => _isUploadingAvatar = false);
  }
}
}

/// Аватар из профиля (base64), иначе фото Google-аккаунта.
ImageProvider? _avatarImage(Map<String, dynamic> profileData, User currentUser) {
  final photoData = profileData['photoData']?.toString() ?? "";
  if (photoData.isNotEmpty) {
    try {
      return MemoryImage(base64Decode(photoData));
    } on FormatException {
      // Поврежденные данные: показываем запасной вариант.
    }
  }
  final photoUrl = profileData['photoUrl']?.toString().isEmpty == true
    ? profileData['photoUrl'].toString()
    : currentUser.photoURL;
  return photoUrl == null ? null : NetworkImage(photoUrl);
}

Future<void> _logout() async {
  await _authService.logout();
}

```

```

Future<void> _deleteCurrentAccount() async {
  final currentUser = _currentUser;
  if (currentUser == null) {
    return;
  }

  final shouldDelete =
    await showDialog<bool>(
      context: context,
      builder: (dialogContext) {
        return AlertDialog(
          title: Text(context.l10n.deleteAccountTitle),
          content: Text(
            context.l10n.deleteAccountBody,
          ),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(dialogContext, false),
              child: Text(context.l10n.cancel),
            ),
            ElevatedButton(
              style: ElevatedButton.styleFrom(
                backgroundColor: AppPalette.danger,
                foregroundColor: Colors.white,
              ),
              onPressed: () => Navigator.pop(dialogContext, true),
              child: Text(context.l10n.delete),
            ),
          ],
        );
      },
    ) ??
    false;

  if (!shouldDelete || !mounted) {
    return;
  }

  setState(() => _isDeletingAccount = true);
  var profileMarkedDeleted = false;

```

```

Map<String, dynamic>? releasedLogin;
try {
    // Профиль помечаем удаленным и освобождаем логин, пока пользователь еще
    // авторизован, и возвращаем все обратно, если Firebase не даст удалить
    // учетную запись.
    await _databaseService.deleteUser(currentUser.uid);
    profileMarkedDeleted = true;
    releasedLogin = await _databaseService.releaseLoginIndex(currentUser.uid);
    await currentUser.delete();
} on FirebaseAuthException catch (error) {
    await _restoreProfileIfNeeded(currentUser.uid, profileMarkedDeleted, releasedLogin);
    _showMessage(
        error.code == 'requires-recent-login'
        ? AppLocale.strings.deleteAccountRecentLogin
        : AppLocale.strings.deleteAccountFailed,
        isError: true,
    );
} on DatabaseOperationException catch (error) {
    _showMessage(error.message, isError: true);
} catch (_) {
    await _restoreProfileIfNeeded(currentUser.uid, profileMarkedDeleted, releasedLogin);
    _showMessage(AppLocale.strings.deleteAccountFailed, isError: true);
} finally {
    if (mounted) {
        setState(() => _isDeletingAccount = false);
    }
}
}

Future<void> _restoreProfileIfNeeded(
    String userId,
    bool profileMarkedDeleted,
    Map<String, dynamic>? releasedLogin,
) async {
    if (!profileMarkedDeleted) return;
    try {
        await _databaseService.restoreLoginIndex(releasedLogin);
        await _databaseService.restoreUser(userId);
    } catch (_) {
        // Профиль останется помеченным удаленным; при следующем входе пользователь будет
        разлогинен.
    }
}

```

```

    }
  }

void _showMessage(String message, {bool isError = false}) {
  if (!mounted) return;
  showAppSnackBar(context, message, isError: isError);
}

void _navigateTo(Widget screen) {
  Navigator.push(context, MaterialPageRoute(builder: (_) => screen));
}

@override
Widget build(BuildContext context) {
  final currentUser = _currentUser;
  if (currentUser == null) {
    return Scaffold(
      body: Center(child: Text(context.l10n.authNotSignedIn)),
    );
  }

  return StreamBuilder<Map<String, dynamic>?>(
    stream: _databaseService.watchUserProfile(currentUser.uid),
    builder: (context, snapshot) {
      final profileData = snapshot.data ?? <String, dynamic>{};
      final displayName =
        profileData['displayName']?.toString().isEmpty == true
        ? profileData['displayName'].toString()
        : (currentUser.displayName ?? context.l10n.userDefaultName);
      final email = profileData['email']?.toString().isEmpty == true
        ? profileData['email'].toString()
        : (currentUser.email ?? context.l10n.noEmail);
      final roleCode = profileData['role']?.toString() ?? AppRoles.buyer;
      final avatarImage = _avatarImage(profileData, currentUser);

      return Scaffold(
        backgroundColor: AppPalette.background,
        body: SafeArea(
          child: SingleChildScrollView(
            padding: const EdgeInsets.only(bottom: 32),
            child: Column(

```

```

children: [
  Padding(
    padding: const EdgeInsets.all(20),
    child: Row(
      children: [
        GestureDetector(
          onTap: _isUploadingAvatar
            ? null
            : _pickAndUploadImage,
          child: Stack(
            children: [
              Container(
                width: 76,
                height: 76,
                decoration: BoxDecoration(
                  color: AppPalette.lightPrimary,
                  shape: BoxShape.circle,
                  image: avatarImage == null
                    ? null
                    : DecorationImage(
                        image: avatarImage,
                        fit: BoxFit.cover,
                      ),
                ),
              ),
              child: avatarImage == null && !_isUploadingAvatar
                ? const Icon(
                    Icons.person,
                    size: 36,
                    color: AppPalette.primary,
                  )
                : null,
            ),
            if (_isUploadingAvatar)
              const Positioned.fill(
                child: CircularProgressIndicator(
                  color: AppPalette.primary,
                  strokeWidth: 3,
                ),
              ),
            Positioned(
              bottom: 2,

```

```

right: 2,
child: Container(
  padding: const EdgeInsets.all(6),
  decoration: const BoxDecoration(
    color: AppPalette.primary,
    shape: BoxShape.circle,
  ),
  child: const Icon(
    Icons.camera_alt,
    color: Colors.white,
    size: 14,
  ),
),
),
],
),
),
const SizedBox(width: 18),
Expanded(
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(
        displayName,
        style: const TextStyle(
          fontSize: 22,
          fontWeight: FontWeight.bold,
          color: AppPalette.textPrimary,
          fontFamily: 'Unbounded',
        ),
      ),
      const SizedBox(height: 4),
      Text(
        email,
        style: const TextStyle(
          color: AppPalette.textSecondary,
        ),
      ),
      const SizedBox(height: 8),
      Container(
        padding: const EdgeInsets.symmetric(

```

```

        horizontal: 12,
        vertical: 6,
      ),
      decoration: BoxDecoration(
        color: AppPalette.lightPrimary,
        borderRadius: BorderRadius.circular(999),
      ),
      child: Text(
        roleLabel(roleCode),
        style: const TextStyle(
          color: AppPalette.primaryDark,
          fontWeight: FontWeight.w600,
        ),
      ),
    ),
  ],
),
),
],
),
),
const Divider(height: 1),
if (canAccessBuyerApp(roleCode)) ...[
  _buildListTile(
    icon: Icons.shopping_bag_outlined,
    title: context.l10n.menuOrders,
    onTap: () => _navigateTo(const OrdersScreen()),
  ),
  _buildListTile(
    icon: Icons.badge_outlined,
    title: context.l10n.menuMyDetails,
    onTap: () => _navigateTo(const MyDetailsScreen()),
  ),
  _buildListTile(
    icon: Icons.location_on_outlined,
    title: context.l10n.menuDeliveryAddress,
    onTap: () => _navigateTo(const DeliveryAddressScreen()),
  ),
  _buildListTile(
    icon: Icons.payment_outlined,
    title: context.l10n.menuPaymentMethods,

```

```

        onTap: () => _navigateTo(const PaymentMethodsScreen()),
      ),
      _buildListTile(
        icon: Icons.card_giftcard_outlined,
        title: context.l10n.menuPromoCodes,
        onTap: () => _navigateTo(const PromoCodeScreen()),
      ),
    ],
    if (_authService.canChangePassword)
      _buildListTile(
        icon: Icons.lock_outline,
        title: context.l10n.menuChangePassword,
        onTap: () => showChangePasswordDialog(context),
      ),
    Column(
      children: [
        ListTile(
          contentPadding: const EdgeInsets.symmetric(horizontal: 20, vertical: 4),
          leading: const Icon(Icons.language, color: AppPalette.textPrimary),
          title: Text(
            context.l10n.language,
            style: const TextStyle(
              fontSize: 17,
              fontWeight: FontWeight.w600,
              color: AppPalette.textPrimary,
            ),
          ),
          trailing: const LanguageSwitcher(),
        ),
        const Divider(height: 1),
      ],
    ),
    _buildListTile(
      icon: Icons.notifications_none_outlined,
      title: context.l10n.menuNotifications,
      onTap: () => _navigateTo(const NotificationsScreen()),
    ),
    _buildListTile(
      icon: Icons.help_outline,
      title: context.l10n.menuHelp,
      onTap: () => _navigateTo(const HelpScreen()),
    ),
  ],
);

```



```

),
_buildListTile(
  icon: Icons.info_outline,
  title: context.l10n.menuAbout,
  onTap: () => _navigateTo(const AboutScreen()),
),
const SizedBox(height: 22),
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 20),
  child: Column(
    children: [
      SizedBox(
        width: double.infinity,
        height: 56,
        child: ElevatedButton.icon(
          style: ElevatedButton.styleFrom(
            backgroundColor: AppPalette.primary,
            foregroundColor: Colors.white,
          ),
          onPressed: _logout,
          icon: const Icon(Icons.logout),
          label: Text(context.l10n.logout),
        ),
      ),
      const SizedBox(height: 12),
      SizedBox(
        width: double.infinity,
        height: 56,
        child: OutlinedButton.icon(
          style: OutlinedButton.styleFrom(
            foregroundColor: AppPalette.danger,
            side: const BorderSide(color: AppPalette.danger),
          ),
          onPressed: _isDeletingAccount
            ? null
            : _deleteCurrentAccount,
          icon: _isDeletingAccount
            ? const SizedBox(
                width: 18,
                height: 18,
                child: CircularProgressIndicator(

```

```

        strokeWidth: 2,
      ),
    ),
    : const Icon(Icons.delete_outline),
    label: Text(context.l10n.deleteAccount),
  ),
),
],
),
),
],
),
),
),
);
},
);
}

```

```

Widget _buildListTile({
  required IconData icon,
  required String title,
  required VoidCallback onTap,
}) {
  return Column(
    children: [
      ListTile(
        contentPadding: const EdgeInsets.symmetric(
          horizontal: 20,
          vertical: 4,
        ),
        leading: Icon(icon, color: AppPalette.textPrimary),
        title: Text(
          title,
          style: const TextStyle(
            fontSize: 17,
            fontWeight: FontWeight.w600,
            color: AppPalette.textPrimary,
          ),
        ),
        trailing: const Icon(Icons.arrow_forward_ios, size: 18),
      ),
    ],
  );
}

```

```

        onTap: onTap,
      ),
      const Divider(height: 1),
    ],
  );
}
}

```

lib/screens/account/orders_screen.dart

```

import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/parsers.dart';
import '../utils/pricing.dart';
import '../widgets/app_snackbar.dart';
import '../l10n/l10n.dart';

class OrdersScreen extends StatelessWidget {
  const OrdersScreen({super.key});

  String _itemName(Map<String, dynamic> item, BuildContext context) {
    return pickLocalized(
      english: context.isEnglish,
      russian: item['name']?.toString() ?? context.l10n.itemDefaultName,
      englishValue: item['nameEn']?.toString(),
    );
  }

  int _getStep(String statusId) {
    switch (statusId) {
      case 'new':
      case 'processing':
        return 0;
      case 'assigned':
      case 'delivering':
        return 1;
      case 'delivered':
        return 2;
      default:
        return 0;
    }
  }

```

```

    }
}

@override
Widget build(BuildContext context) {
  final user = FirebaseAuth.instance.currentUser;
  final databaseService = DatabaseService();

  return Scaffold(
    backgroundColor: AppPalette.background,
    appBar: AppBar(title: Text(context.l10n.myOrders)),
    body: user == null
      ? Center(child: Text(context.l10n.ordersSignInRequired))
      : StreamBuilder<List<Map<String, dynamic>>>(
        stream: databaseService.watchOrdersForUser(user.uid),
        builder: (context, snapshot) {
          if (snapshot.hasError) {
            return Center(child: Text(context.l10n.ordersLoadFailed));
          }
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator(color: AppPalette.primary));
          }

          final orders = snapshot.data!.where((order) => order['hiddenByUser'] != true).toList();
          if (orders.isEmpty) {
            return Center(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  Icon(Icons.shopping_bag_outlined, size: 80, color: AppPalette.textSecondary.withValues(alpha:
0.5)),
                  const SizedBox(height: 20),
                  Text(context.l10n.ordersEmptyTitle, style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold,
color: Color(0xFF181B19))),
                  const SizedBox(height: 10),
                  Text(context.l10n.ordersEmptyBody, style: TextStyle(color: AppPalette.textSecondary)),
                  const SizedBox(height: 30),
                  ElevatedButton(
                    style: ElevatedButton.styleFrom(
                      backgroundColor: AppPalette.primary,
                      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(15)),

```

```

        padding: const EdgeInsets.symmetric(horizontal: 40, vertical: 15),
      ),
      onPressed: () => Navigator.popUntil(context, (route) => route.isFirst),
      child: Text(context.l10n.startShopping, style: TextStyle(color: Colors.white, fontSize: 16)),
    ),
  ],
),
);
}

return ListView.separated(
  padding: const EdgeInsets.all(20),
  itemCount: orders.length,
  separatorBuilder: (context, index) => const Divider(color: Color(0xFFE2E2E2), thickness: 1, height:
30),
  itemBuilder: (context, index) {
    final orderData = orders[index];
    final totalAmount = (orderData['totalAmount'] as num?)?.toDouble() ?? 0;
    final statusId = orderData['statusId']?.toString() ?? 'new';
    final isCancelled = statusId == 'cancelled';
    final currentStep = _getStep(statusId);
    final orderItems = List<Map<String, dynamic>>.from(orderData['items'] ?? []);
    final createdAt = orderData['createdAt'];
    final createdAtDate = createdAt == null ? null : toDateTimeValue(createdAt);
    final userConfirmed = orderData['userConfirmed'] == true;

    return Container(
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(18),
        border: Border.all(color: AppPalette.border),
      ),
      child: ExpansionTile(
        tilePadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
        childrenPadding: const EdgeInsets.only(bottom: 16, left: 16, right: 16),
        title: Text(
          orderData['number']?.toString() ??
            context.l10n.orderNumberFallback((orderData['id']?.toString() ?? '000000').padRight(6,
'0')).substring(0, 6).toUpperCase(),
          style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
        ),
      ),
    ),
  ),
);

```

```

        subtitle: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const SizedBox(height: 12),
            if (isCancelled)
              Text(context.l10n.orderCancelled, style: TextStyle(color: AppPalette.danger, fontWeight:
FontWeight.bold))
            else
              Row(
                children: [
                  _buildStepItem(context.l10n.stepCreated, currentStep >= 0),
                  _buildLine(currentStep >= 1),
                  _buildStepItem(context.l10n.stepWithCourier, currentStep >= 1),
                  _buildLine(currentStep >= 2),
                  _buildStepItem(context.l10n.stepDelivered, currentStep >= 2),
                ],
              ),
            const SizedBox(height: 8),
            if (createdAtDate != null)
              Text(
                formatDate(createdAtDate),
                style: const TextStyle(color: AppPalette.textSecondary),
              ),
          ],
        ),
        children: [
          const Divider(),
          _OrderInfoRow(label: context.l10n.labelAddress, value: orderData['address']?.toString() ??
context.l10n.notSpecified),
          if ((orderData['deliverySlot']?.toString() ?? "").isEmpty)
            _OrderInfoRow(label: context.l10n.deliveryTime, value: orderData['deliverySlot'].toString()),
          _OrderInfoRow(label: context.l10n.orderInfoPayment, value:
paymentMethodLabel(orderData['paymentMethod']?.toString(), context.l10n)),
          if ((orderData['courierName']?.toString() ?? "").isEmpty)
            _OrderInfoRow(label: context.l10n.orderInfoCourier, value:
orderData['courierName'].toString()),
          if ((orderData['courierPhone']?.toString() ?? "").isEmpty)
            _OrderInfoRow(label: context.l10n.orderInfoCourierPhone, value:
orderData['courierPhone'].toString()),
          if (toDoubleValue(orderData['deliveryAmount']) > 0)

```

```

        _OrderInfoRow(label: context.l10n.summaryDelivery, value:
formatMoney(toDoubleValue(orderData['deliveryAmount']))),
        if (toDoubleValue(orderData['discountAmount']) > 0)
            _OrderInfoRow(label: context.l10n.summaryDiscount, value: '-$
{formatMoney(toDoubleValue(orderData['discountAmount'])))}),
        _OrderInfoRow(label: context.l10n.total, value: '₹${totalAmount.toStringAsFixed(2)}'),
        const SizedBox(height: 10),
        ...orderItems.map((orderItem) {
            return ListTile(
                dense: true,
                contentPadding: EdgeInsets.zero,
                title: Text(_itemName(orderItem, context)),
                subtitle: Text(context.l10n.quantityLabel('${orderItem['quantity']} ?? 0)'),
                trailing: Text('₹${((orderItem['price'] as num?)?.toDouble() ?? 0).toStringAsFixed(2)}'),
            );
        }),
        if (isCancelled || currentStep >= 2)
            Align(
                alignment: Alignment.centerRight,
                child: TextButton.icon(
                    onPressed: () async {
                        try {
                            await databaseService.hideOrderForUser(orderData['id'].toString());
                        } catch (_) {
                            if (context.mounted) {
                                showAppSnackBar(context, context.l10n.hideOrderFailed, isError: true);
                            }
                        }
                    },
                    icon: const Icon(Icons.archive_outlined),
                    label: Text(context.l10n.hideOrder),
                ),
            ),
        if (!isCancelled && currentStep >= 1 && currentStep < 2)
            Padding(
                padding: const EdgeInsets.only(top: 16),
                child: SizedBox(
                    width: double.infinity,
                    height: 50,
                    child: userConfirmed
                        ? Container(

```

```
alignment: Alignment.center,
decoration: BoxDecoration(
  color: AppPalette.lightPrimary,
  borderRadius: BorderRadius.circular(12),
),
child: Text(
  context.l10n.orderConfirmedWaiting,
  style: TextStyle(color: AppPalette.primaryDark, fontWeight: FontWeight.bold),
  textAlign: TextAlign.center,
),
)
: ElevatedButton(
style: ElevatedButton.styleFrom(
  backgroundColor: AppPalette.primary,
  foregroundColor: Colors.white,
  shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
),
onPressed: () async {
  try {
    await databaseService.confirmOrderReceived(orderData['id'].toString());
    if (context.mounted) {
      showAppSnackBar(context, context.l10n.orderConfirmedToast);
    }
  } catch (_) {
    if (context.mounted) {
      showAppSnackBar(context, context.l10n.orderConfirmFailed, isError: true);
    }
  }
},
child: Text(context.l10n.orderConfirmButton),
),
),
],
),
);
},
);
},
);
```



```

}

Widget _buildStepItem(String title, bool isActive) {
  return Column(
    children: [
      CircleAvatar(
        radius: 12,
        backgroundColor: isActive ? AppPalette.primary : const Color(0xFFE2E2E2),
        child: const Icon(Icons.check, size: 14, color: Colors.white),
      ),
      const SizedBox(height: 6),
      Text(
        title,
        style: TextStyle(
          fontSize: 10,
          fontWeight: FontWeight.bold,
          color: isActive ? AppPalette.textPrimary : const Color(0xFFB3B3B3),
        ),
      ),
    ],
  );
}

Widget _buildLine(bool isActive) {
  return Expanded(
    child: Container(
      margin: const EdgeInsets.only(bottom: 16, left: 4, right: 4),
      height: 3,
      color: isActive ? AppPalette.primary : const Color(0xFFE2E2E2),
    ),
  );
}

class _OrderInfoRow extends StatelessWidget {
  const _OrderInfoRow({
    required this.label,
    required this.value,
  });
}

```

```

final String label;
final String value;

@override
Widget build(BuildContext context) {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 4),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text(label, style: const TextStyle(color: AppPalette.textSecondary)),
        const SizedBox(width: 16),
        Flexible(
          child: Text(
            value,
            textAlign: TextAlign.right,
            style: const TextStyle(fontWeight: FontWeight.w600),
          ),
        ),
      ],
    ),
  );
}
}

```

lib/screens/account/my_details_screen.dart

```

import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/validators.dart';
import '../l10n/l10n.dart';

class MyDetailsScreen extends StatefulWidget {
  const MyDetailsScreen({super.key});

  @override
  State<MyDetailsScreen> createState() => _MyDetailsScreenState();
}

class _MyDetailsScreenState extends State<MyDetailsScreen> {

```

```

final user = FirebaseAuth.instance.currentUser;
final DatabaseService _databaseService = DatabaseService();
final GlobalKey<FormState> _formKey = GlobalKey<FormState>();

final TextEditingController _nameController = TextEditingController();
final TextEditingController _emailController = TextEditingController();
final TextEditingController _phoneController = TextEditingController();

bool _isLoading = false;

@override
void initState() {
  super.initState();
  _nameController.text = user?.displayName ?? "";
  _emailController.text = user?.email ?? "";
  _loadUserData();
}

@override
void dispose() {
  _nameController.dispose();
  _emailController.dispose();
  _phoneController.dispose();
  super.dispose();
}

Future<void> _loadUserData() async {
  if (user != null) {
    try {
      final userProfile = await _databaseService.getUserProfile(user!.uid);
      if (userProfile != null && mounted) {
        setState() {
          _nameController.text = userProfile['displayName']?.toString() ?? user?.displayName ?? "";
          _phoneController.text = userProfile['phoneNumber']?.toString() ?? "";
        };
      }
    } catch (_) {
      // Поля остаются заполненными данными из аккаунта Firebase.
    }
  }
}

```

```

Future<void> _updateProfile() async {
  if (user == null) return;
  if (!_formKey.currentState!.validate()) return;

  setState(() => _isLoading = true);
  try {
    await user!.updateDisplayName(_nameController.text.trim());

    await _databaseService.updateUserProfile(
      userId: user!.uid,
      displayName: _nameController.text,
      phoneNumber: _phoneController.text,
    );

    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(context.l10n.detailsSaved), backgroundColor: AppPalette.primary),
      );
      Navigator.pop(context);
    }
  } catch (e) {
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(context.l10n.detailsSaveFailed), backgroundColor: AppPalette.danger),
      );
    }
  } finally {
    if (mounted) setState(() => _isLoading = false);
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.white,
    appBar: AppBar(
      title: Text(context.l10n.menuMyDetails),
    ),
    body: Form(
      key: _formKey,

```

```

child: Padding(
  padding: const EdgeInsets.all(20.0),
  child: Column(
    children: [
      TextFormField(
        controller: _nameController,
        validator: Validators.validateName,
        decoration: InputDecoration(
          labelText: context.l10n.fullName,
          labelStyle: const TextStyle(color: AppPalette.textSecondary),
          border: OutlineInputBorder(borderRadius: BorderRadius.circular(16)),
          focusedBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(16), borderSide: const
BorderSide(color: AppPalette.primary)),
        ),
      ),
      const SizedBox(height: 20),
      TextFormField(
        controller: _phoneController,
        validator: Validators.validatePhone,
        keyboardType: TextInputType.phone,
        decoration: InputDecoration(
          labelText: context.l10n.phoneNumber,
          labelStyle: const TextStyle(color: AppPalette.textSecondary),
          border: OutlineInputBorder(borderRadius: BorderRadius.circular(16)),
          focusedBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(16), borderSide: const
BorderSide(color: AppPalette.primary)),
        ),
      ),
      const SizedBox(height: 20),
      TextField(
        controller: _emailController,
        enabled: false,
        decoration: InputDecoration(
          labelText: context.l10n.emailReadOnly,
          labelStyle: const TextStyle(color: AppPalette.textSecondary),
          border: OutlineInputBorder(borderRadius: BorderRadius.circular(16)),
          filled: true,
          fillColor: const Color(0xFF2F3F2),
        ),
      ),
      const Spacer(),

```

```

    SizedBox(
      width: double.infinity,
      height: 56,
      child: _isLoading
        ? const Center(child: CircularProgressIndicator(color: AppPalette.primary))
        : ElevatedButton(
            style: ElevatedButton.styleFrom(
              backgroundColor: AppPalette.primary,
              foregroundColor: Colors.white,
              shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(19)),
            ),
            onPressed: _updateProfile,
            child: Text(context.l10n.save, style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),
          ),
    ),
  ],
),
);
}
}

```

lib/screens/account/delivery_address_screen.dart

```

import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/validators.dart';
import '../widgets/app_snackbar.dart';
import '../widgets/input_dialog.dart';
import '../l10n/l10n.dart';

class DeliveryAddressScreen extends StatefulWidget {
  const DeliveryAddressScreen({super.key});

  @override
  State<DeliveryAddressScreen> createState() => _DeliveryAddressScreenState();
}

class _DeliveryAddressScreenState extends State<DeliveryAddressScreen> {

```

```

final user = FirebaseAuth.instance.currentUser;
final DatabaseService _databaseService = DatabaseService();

void _showAddAddressDialog() {
  final userId = user?.uid;
  if (userId == null) return;

  showInputDialog(
    context,
    title: context.l10n.addAddressTitle,
    hint: context.l10n.addAddressHint,
    maxLines: 2,
    maxLength: 300,
    validator: (value) => Validators.validateRequiredText(value, fieldName: context.l10n.labelAddress,
    maxLength: 300),
    onSubmit: (value) => _databaseService.addDeliveryAddress(userId: userId, address: value),
  );
}

Future<void> _removeAddress(String address) async {
  final userId = user?.uid;
  if (userId == null) return;

  try {
    await _databaseService.removeDeliveryAddress(userId: userId, address: address);
  } catch (_) {
    if (mounted) showAppSnackBar(context, context.l10n.addressRemoveFailed, isError: true);
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.white,
    appBar: AppBar(
      title: Text(context.l10n.menuDeliveryAddress),
    ),
    body: user == null
      ? Center(child: Text(context.l10n.addressesSignIn))
      : StreamBuilder<Map<String, dynamic>?>(
        stream: _databaseService.watchUserProfile(user!.uid),

```

```

builder: (context, snapshot) {
  if (snapshot.connectionState == ConnectionState.waiting) {
    return const Center(child: CircularProgressIndicator(color: AppPalette.primary));
  }

  final addresses = List<String>.from(snapshot.data?['addresses'] ?? []);

  if (addresses.isEmpty) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(Icons.location_on_outlined, size: 80, color: AppPalette.textSecondary.withValues(alpha:
0.5)),
          const SizedBox(height: 20),
          Text(context.l10n.addressesEmptyTitle, style: TextStyle(fontSize: 20, fontWeight:
FontWeight.bold, color: Color(0xFF181B19))),
          const SizedBox(height: 10),
          Text(context.l10n.addressesEmptyBody, style: TextStyle(color: AppPalette.textSecondary),
textAlign: TextAlign.center),
        ],
      ),
    );
  }

  return ListView.separated(
    padding: const EdgeInsets.all(20),
    itemCount: addresses.length,
    separatorBuilder: (_, __) => const SizedBox(height: 15),
    itemBuilder: (context, index) {
      final address = addresses[index];
      return Container(
        padding: const EdgeInsets.all(20),
        decoration: BoxDecoration(
          border: Border.all(color: const Color(0xFFE2E2E2)),
          borderRadius: BorderRadius.circular(15),
        ),
        child: Row(
          children: [
            const Icon(Icons.location_on, color: AppPalette.primary),
            const SizedBox(width: 15),

```



```

        Expanded(
          child: Text(address, style: const TextStyle(fontSize: 16)),
        ),
        IconButton(
          icon: const Icon(Icons.delete_outline, color: Colors.red),
          onPressed: () => _removeAddress(address),
        )
      ],
    ),
  );
},
);
},
),
floatingActionButton: FloatingActionButton.extended(
  backgroundColor: AppPalette.primary,
  onPressed: _showAddAddressDialog,
  icon: const Icon(Icons.add, color: Colors.white),
  label: Text(context.l10n.add, style: TextStyle(color: Colors.white)),
),
);
}
}

```

lib/screens/account/payment_methods_screen.dart

```

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:firebase_auth/firebase_auth.dart';

import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/validators.dart';
import '../widgets/app_snackbar.dart';
import '../widgets/input_dialog.dart';
import '../l10n/l10n.dart';

class PaymentMethodsScreen extends StatefulWidget {
  const PaymentMethodsScreen({super.key});

  @override
  State<PaymentMethodsScreen> createState() => _PaymentMethodsScreenState();
}

```

```

class _PaymentMethodsScreenState extends State<PaymentMethodsScreen> {
  final user = FirebaseAuth.instance.currentUser;
  final DatabaseService _databaseService = DatabaseService();

  void _showAddCardDialog() {
    final userId = user?.uid;
    if (userId == null) return;

    showInputDialog(
      context,
      title: context.l10n.addCardTitle,
      hint: context.l10n.addCardHint,
      keyboardType: TextInputType.number,
      maxLength: 16,
      inputFormatters: [FilteringTextInputFormatter.digitsOnly],
      validator: Validators.validateCardNumber,
      onSubmit: (value) => _databaseService.addPaymentCard(userId: userId, cardNumber: value),
    );
  }

  Future<void> _removeCard(String card) async {
    final userId = user?.uid;
    if (userId == null) return;

    try {
      await _databaseService.removePaymentCard(userId: userId, maskedCardNumber: card);
    } catch (_) {
      if (mounted) showAppSnackBar(context, context.l10n.cardRemoveFailed, isError: true);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.white,
      appBar: AppBar(
        title: Text(context.l10n.menuPaymentMethods),
      ),
      body: user == null
        ? Center(child: Text(context.l10n.cardsSignIn))
  }

```

```

: StreamBuilder<Map<String, dynamic>?>(
stream: _databaseService.watchUserProfile(user!.uid),
builder: (context, snapshot) {
  if (snapshot.connectionState == ConnectionState.waiting) {
    return const Center(child: CircularProgressIndicator(color: AppPalette.primary));
  }

  final cards = List<String>.from(snapshot.data?['cards'] ?? []);

  if (cards.isEmpty) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(Icons.payment_outlined, size: 80, color: AppPalette.textSecondary.withValues(alpha: 0.5)),
          const SizedBox(height: 20),
          Text(context.l10n.cardsEmptyTitle, style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold,
color: Color(0xFF181B19))),
          const SizedBox(height: 10),
          Text(context.l10n.cardsEmptyBody, style: TextStyle(color: AppPalette.textSecondary)),
        ],
      ),
    );
  }

  return ListView.separated(
    padding: const EdgeInsets.all(20),
    itemCount: cards.length,
    separatorBuilder: (_, __) => const SizedBox(height: 15),
    itemBuilder: (context, index) {
      final card = cards[index];
      return Container(
        padding: const EdgeInsets.all(20),
        decoration: BoxDecoration(
          border: Border.all(color: const Color(0xFFE2E2E2)),
          borderRadius: BorderRadius.circular(15),
        ),
        child: Row(
          children: [
            const Icon(Icons.credit_card, size: 40, color: Color(0xFF181B19)),
            const SizedBox(width: 20),

```

```

        Expanded(
          child: Text(card, style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),
        ),
        IconButton(
          icon: const Icon(Icons.delete_outline, color: Colors.red),
          onPressed: () => _removeCard(card),
        )
      ],
    ),
  );
},
);
},
),
floatingActionButton: FloatingActionButton.extended(
  backgroundColor: AppPalette.primary,
  onPressed: _showAddCardDialog,
  icon: const Icon(Icons.add, color: Colors.white),
  label: Text(context.l10n.addCardTitle, style: TextStyle(color: Colors.white)),
),
);
}
}

```

lib/screens/account/promo_code_screen.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/cart_provider.dart';
import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/parsers.dart';
import '../utils/validators.dart';
import '../widgets/app_snackbar.dart';
import '../l10n/l10n.dart';

class PromoCodeScreen extends StatefulWidget {
  const PromoCodeScreen({super.key});

  @override
  State<PromoCodeScreen> createState() => _PromoCodeScreenState();
}

```

```

class _PromoCodeScreenState extends State<PromoCodeScreen> {
  final TextEditingController _promoController = TextEditingController();
  final DatabaseService _databaseService = DatabaseService();

  bool _isApplying = false;

  @override
  void dispose() {
    _promoController.dispose();
    super.dispose();
  }

  Future<void> _applyPromo(String promoCode) async {
    final formError = Validators.validatePromoCode(promoCode);
    if (formError != null) {
      _showMessage(formError, isError: true);
      return;
    }

    setState(() => _isApplying = true);
    try {
      final promoData = await _databaseService.validatePromoCode(promoCode);
      if (!mounted) {
        return;
      }

      context.read<CartProvider>().applyPromoCode(
        promoData['code'].toString(),
        toDoubleValue(promoData['discountPercent']),
      );
      _promoController.clear();
      _showMessage(
        context.l10n.promoApplied('${promoData['code']}', '${promoData['discountPercent']}'),
      );
    } on DatabaseOperationException catch (error) {
      _showMessage(error.message, isError: true);
    } catch (_) {
      _showMessage(context.l10n.promoCheckFailed, isError: true);
    } finally {
      if (mounted) {

```

```

        setState(() => _isApplying = false);
    }
}

void _showMessage(String message, {bool isError = false}) {
    if (!mounted) return;
    showAppSnackBar(context, message, isError: isError);
}

@override
Widget build(BuildContext context) {
    final cartProvider = context.watch<CartProvider>();

    return Scaffold(
        backgroundColor: AppPalette.background,
        appBar: AppBar(
            title: Text(context.l10n.menuPromoCodes),
            actions: [
                if (cartProvider.hasPromoCode)
                    TextButton(
                        onPressed: () {
                            cartProvider.clearPromoCode();
                            _showMessage(context.l10n.promoRemoved);
                        },
                        child: Text(context.l10n.reset),
                    ),
            ],
        ),
        body: Padding(
            padding: const EdgeInsets.all(20),
            child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                    if (cartProvider.hasPromoCode)
                        Container(
                            width: double.infinity,
                            padding: const EdgeInsets.all(16),
                            margin: const EdgeInsets.only(bottom: 18),
                            decoration: BoxDecoration(
                                color: AppPalette.lightPrimary,

```

```

        borderRadius: BorderRadius.circular(18),
      ),
      child: Text(
        context.l10n.promoCurrentlyApplied(cartProvider.appliedPromoCode ?? ",
cartProvider.discountPercent.toStringAsFixed(0)),
        style: const TextStyle(
          color: AppPalette.primaryDark,
          fontWeight: FontWeight.w600,
        ),
      ),
    ),
  Row(
    children: [
      Expanded(
        child: TextField(
          controller: _promoController,
          textCapitalization: TextCapitalization.characters,
          decoration: InputDecoration(
            hintText: context.l10n.valPromoRequired,
            filled: true,
            fillColor: Colors.white,
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(16),
              borderSide: const BorderSide(color: AppPalette.border),
            ),
          ),
        ),
      ),
      const SizedBox(width: 12),
      SizedBox(
        height: 56,
        child: _isApplying
          ? const Center(child: CircularProgressIndicator(color: AppPalette.primary))
          : ElevatedButton(
              style: ElevatedButton.styleFrom(
                backgroundColor: AppPalette.primary,
                foregroundColor: Colors.white,
              ),
              onPressed: () => _applyPromo(_promoController.text),
              child: Text(context.l10n.apply),
            ),
      ),
    ],
  ),

```

```

    ),
  ],
),
const SizedBox(height: 24),
Text(
  context.l10n.promoAvailableTitle,
  style: TextStyle(
    fontSize: 20,
    fontWeight: FontWeight.bold,
    color: AppPalette.textPrimary,
    fontFamily: 'Unbounded',
  ),
),
const SizedBox(height: 12),
Expanded(
  child: StreamBuilder<List<Map<String, dynamic>>>>(
    stream: _databaseService.watchPromoCodes(),
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        return const Center(
          child: CircularProgressIndicator(color: AppPalette.primary),
        );
      }

      final currentDate = DateTime.now();
      final activePromoCodes = (snapshot.data ?? []).where((promoCode) {
        final expirationDate = toDateTimeValue(promoCode['expiresAt']);
        return promoCode['isActive'] != false && expirationDate.isAfter(currentDate);
      }).toList();

      if (activePromoCodes.isEmpty) {
        return Center(
          child: Text(context.l10n.promoNoneActive),
        );
      }

      return ListView.separated(
        itemCount: activePromoCodes.length,
        separatorBuilder: (_, __) => const SizedBox(height: 12),
        itemBuilder: (context, index) {
          final promoCode = activePromoCodes[index];

```



```

final expirationDate = toDateTimeValue(promoCode['expiresAt']);
return Container(
  padding: const EdgeInsets.all(18),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(18),
    border: Border.all(color: AppPalette.border),
  ),
  child: Row(
    children: [
      Container(
        width: 54,
        height: 54,
        decoration: BoxDecoration(
          color: AppPalette.lightPrimary,
          borderRadius: BorderRadius.circular(16),
        ),
        child: const Icon(Icons.local_offer_outlined, color: AppPalette.primaryDark),
      ),
      const SizedBox(width: 16),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              promoCode['code'].toString(),
              style: const TextStyle(
                fontSize: 18,
                fontWeight: FontWeight.bold,
                color: AppPalette.textPrimary,
              ),
            ),
            const SizedBox(height: 4),
            Text(
              context.l10n.promoDiscountUntil('${promoCode['discountPercent']}',
formatDate(expirationDate)),
              style: const TextStyle(color: AppPalette.textSecondary),
            ),
          ],
        ),
      ),
    ],
  ),
);

```

```

        TextButton(
          onPressed: () => _applyPromo(promoCode['code'].toString()),
          child: Text(context.l10n.choose),
        ),
      ],
    ),
  );
},
);
},
),
),
],
),
),
);
}

}

```

lib/screens/account/notifications_screen.dart

```

import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';

import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/parsers.dart';
import '../widgets/app_snackbar.dart';
import '../l10n/l10n.dart';

class NotificationsScreen extends StatelessWidget {
  const NotificationsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final currentUser = FirebaseAuth.instance.currentUser;
    final databaseService = DatabaseService();

    if (currentUser == null) {
      return Scaffold(
        body: Center(child: Text(context.l10n.notificationsSignIn)),
      );
    }
  }
}

```

```

}

return StreamBuilder<Map<String, dynamic>?>(
  stream: databaseService.watchUserProfile(currentUser.uid),
  builder: (context, profileSnapshot) {
    final notificationSettings =
      profileSnapshot.data?['notificationSettings'] as Map<String, dynamic>? ??
      {
        'push': true,
        'sms': false,
        'email': true,
      };

    return Scaffold(
      backgroundColor: AppPalette.background,
      appBar: AppBar(
        title: Text(context.l10n.menuNotifications),
        actions: [
          TextButton(
            onPressed: () async {
              try {
                await databaseService.clearNotifications(currentUser.uid);
                if (context.mounted) showAppSnackBar(context, context.l10n.notificationsCleared);
              } catch (_) {
                if (context.mounted) {
                  showAppSnackBar(context, context.l10n.notificationsClearFailed, isError: true);
                }
              }
            },
            child: Text(context.l10n.clearAll),
          ),
        ],
      ),
      body: StreamBuilder<List<Map<String, dynamic>>>(
        stream: databaseService.watchNotifications(currentUser.uid),
        builder: (context, notificationsSnapshot) {
          final notifications = notificationsSnapshot.data ?? [];
          return ListView(
            padding: const EdgeInsets.all(20),
            children: [
              SwitchListTile(

```

```

        contentPadding: EdgeInsets.zero,
        title: Text(context.l10n.notifPushTitle),
        subtitle: Text(context.l10n.notifPushSubtitle),
        value: notificationSettings['push'] == true,
        onChanged: (value) {
          databaseService.saveNotificationSettings(
            userId: currentUser.uid,
            pushEnabled: value,
            smsEnabled: notificationSettings['sms'] == true,
            emailEnabled: notificationSettings['email'] == true,
          );
        },
      ),
      const Divider(),
      SwitchListTile(
        contentPadding: EdgeInsets.zero,
        title: Text(context.l10n.notifSmsTitle),
        subtitle: Text(context.l10n.notifSmsSubtitle),
        value: notificationSettings['sms'] == true,
        onChanged: (value) {
          databaseService.saveNotificationSettings(
            userId: currentUser.uid,
            pushEnabled: notificationSettings['push'] == true,
            smsEnabled: value,
            emailEnabled: notificationSettings['email'] == true,
          );
        },
      ),
      const Divider(),
      SwitchListTile(
        contentPadding: EdgeInsets.zero,
        title: Text(context.l10n.notifEmailTitle),
        subtitle: Text(context.l10n.notifEmailSubtitle),
        value: notificationSettings['email'] == true,
        onChanged: (value) {
          databaseService.saveNotificationSettings(
            userId: currentUser.uid,
            pushEnabled: notificationSettings['push'] == true,
            smsEnabled: notificationSettings['sms'] == true,
            emailEnabled: value,
          );
        },
      ),
    ],
  ),
);

```

```

    },
  ),
  const SizedBox(height: 24),
  Text(
    context.l10n.notifHistoryTitle,
    style: TextStyle(
      fontSize: 20,
      fontWeight: FontWeight.bold,
      color: AppPalette.textPrimary,
      fontFamily: 'Unbounded',
    ),
  ),
  const SizedBox(height: 12),
  if (notifications.isEmpty)
    Container(
      padding: const EdgeInsets.all(18),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(18),
      ),
      child: Text(
        context.l10n.notifEmpty,
        style: TextStyle(color: AppPalette.textSecondary),
      ),
    )
  else
    ...notifications.map((notification) {
      final notificationDate = toDateTimeValue(notification['createdAt']);
      return Container(
        margin: const EdgeInsets.only(bottom: 12),
        padding: const EdgeInsets.all(18),
        decoration: BoxDecoration(
          color: Colors.white,
          borderRadius: BorderRadius.circular(18),
          border: Border.all(color: AppPalette.border),
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              pickLocalized(

```



```

);

if (changed == true && context.mounted) {
  showAppSnackBar(context, context.l10n.passwordChanged);
}
}

class _ChangePasswordDialog extends StatefulWidget {
  const _ChangePasswordDialog();

  @override
  State<_ChangePasswordDialog> createState() => _ChangePasswordDialogState();
}

class _ChangePasswordDialogState extends State<_ChangePasswordDialog> {
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
  final TextEditingController _currentController = TextEditingController();
  final TextEditingController _newController = TextEditingController();
  final TextEditingController _confirmController = TextEditingController();
  final AuthService _authService = AuthService();

  bool _isSaving = false;
  String? _errorText;

  @override
  void dispose() {
    _currentController.dispose();
    _newController.dispose();
    _confirmController.dispose();
    super.dispose();
  }

  Future<void> _submit() async {
    if (!_formKey.currentState!.validate()) return;

    setState() {
      _isSaving = true;
      _errorText = null;
    });

    try {

```

```

    await _authService.changePassword(
      currentPassword: _currentController.text,
      newPassword: _newController.text,
    );
    if (mounted) Navigator.pop(context, true);
  } catch (error) {
    if (!mounted) return;
    setState(() {
      _isSaving = false;
      _errorText = _authService.getErrorMessage(error);
    });
  }
}

@override
Widget build(BuildContext context) {
  return AlertDialog(
    title: Text(context.l10n.changePasswordTitle),
    content: Form(
      key: _formKey,
      child: SingleChildScrollView(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            TextFormField(
              controller: _currentController,
              obscureText: true,
              enabled: !_isSaving,
              validator: (value) => (value == null || value.isEmpty) ? context.l10n.currentPasswordRequired :
null,
              decoration: InputDecoration(labelText: context.l10n.currentPasswordLabel),
            ),
            TextFormField(
              controller: _newController,
              obscureText: true,
              enabled: !_isSaving,
              validator: (value) {
                final error = Validators.validatePassword(value);
                if (error != null) return error;
                if (value == _currentController.text) return context.l10n.newPasswordSame;
                return null;
              },
            ),
          ],
        ),
      ),
    ),
  );
}

```



```

    },
    decoration: InputDecoration(labelText: context.l10n.newPasswordLabel),
  ),
  TextFormField(
    controller: _confirmController,
    obscureText: true,
    enabled: !_isSaving,
    validator: (value) => Validators.validatePasswordConfirmation(value, _newController.text),
    decoration: InputDecoration(labelText: context.l10n.newPasswordRepeatLabel),
  ),
  if (_errorText != null)
    Padding(
      padding: const EdgeInsets.only(top: 12),
      child: Text(_errorText!, style: const TextStyle(color: AppPalette.danger)),
    ),
],
),
),
),
actions: [
  TextButton(
    onPressed: _isSaving ? null : () => Navigator.pop(context, false),
    child: Text(context.l10n.cancel),
  ),
  TextButton(
    onPressed: _isSaving ? null : _submit,
    child: Text(context.l10n.save, style: TextStyle(color: AppPalette.primary)),
  ),
],
);
}
}

```

lib/screens/account/help_screen.dart

```

import 'package:flutter/material.dart';
import '../l10n/l10n.dart';
import '../utils/app_palette.dart';

class HelpScreen extends StatelessWidget {
  const HelpScreen({super.key});

  @override

```

```

Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.white,
    appBar: AppBar(
      backgroundColor: Colors.white,
      elevation: 0,
      centerTitle: true,
      leading: IconButton(
        icon: const Icon(Icons.arrow_back_ios, color: Colors.black),
        onPressed: () => Navigator.pop(context),
      ),
      title: Text(context.l10n.menuHelp, style: TextStyle(color: Colors.black, fontWeight: FontWeight.bold)),
    ),
    body: ListView(
      padding: const EdgeInsets.all(20),
      children: [
        Text(context.l10n.faqTitle, style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),
        const SizedBox(height: 20),
        _buildFaq(context.l10n.faqTrackQ, context.l10n.faqTrackA),
        _buildFaq(context.l10n.faqReturnQ, context.l10n.faqReturnA),
        _buildFaq(context.l10n.faqPaymentQ, context.l10n.faqPaymentA),
        _buildFaq(context.l10n.faqPromoQ, context.l10n.faqPromoA),
      ],
    ),
  );
}

```

```

Widget _buildFaq(String question, String answer) {
  return Container(
    margin: const EdgeInsets.only(bottom: 15),
    decoration: BoxDecoration(
      border: Border.all(color: const Color(0xFFE2E2E2)),
      borderRadius: BorderRadius.circular(15),
    ),
    child: ExpansionTile(
      title: Text(question, style: const TextStyle(fontWeight: FontWeight.w600, color: Color(0xFF181B19))),
      iconColor: AppPalette.primary,
      collapsedIconColor: const Color(0xFF181B19),
      childrenPadding: const EdgeInsets.only(left: 15, right: 15, bottom: 15),
      children: [
        Text(answer, style: const TextStyle(color: AppPalette.textSecondary, height: 1.5)),
      ],
    ),
  );
}

```

```

    ],
  ),
);
}
}

```

lib/screens/account/about_screen.dart

```

import 'package:flutter/material.dart';
import '../l10n/l10n.dart';
import '../utils/app_palette.dart';

class AboutScreen extends StatelessWidget {
  const AboutScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.white,
      appBar: AppBar(
        backgroundColor: Colors.white,
        elevation: 0,
        centerTitle: true,
        leading: IconButton(
          icon: const Icon(Icons.arrow_back_ios, color: Colors.black),
          onPressed: () => Navigator.pop(context),
        ),
        title: Text(context.l10n.menuAbout, style: TextStyle(color: Colors.black, fontWeight:
FontWeight.bold)),
      ),
      body: Column(
        children: [
          const SizedBox(height: 40),
          Center(
            child: Column(
              children: [
                Icon(Icons.eco, color: AppPalette.primary, size: 80),
                SizedBox(height: 10),
                Text(context.l10n.splashBrand, style: TextStyle(fontSize: 34, fontWeight: FontWeight.bold, color:
Color(0xFF181B19))),
                SizedBox(height: 5),
                Text(context.l10n.aboutVersion, style: TextStyle(color: AppPalette.textSecondary)),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

```

```

    ),
  ),
  const SizedBox(height: 40),
  const Divider(thickness: 1, color: Color(0xFFE2E2E2)),
  _buildLinkTile(context.l10n.aboutTerms),
  _buildLinkTile(context.l10n.aboutPrivacy),
  _buildLinkTile(context.l10n.aboutLicenses),
],
),
);
}

Widget _buildLinkTile(String title) {
  return Column(
    children: [
      ListTile(
        contentPadding: const EdgeInsets.symmetric(horizontal: 20),
        title: Text(title, style: const TextStyle(fontSize: 18, fontWeight: FontWeight.w600, color:
Color(0xFF181B19))),
        trailing: const Icon(Icons.arrow_forward_ios, color: Color(0xFF181B19), size: 16),
        onTap: () {},
      ),
      const Divider(thickness: 1, color: Color(0xFFE2E2E2)),
    ],
  );
}
}

```

lib/screens/courier/courier_home_screen.dart

```

import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/courier_orders.dart';
import '../utils/order_status.dart';
import '../utils/parsers.dart';
import '../utils/pricing.dart';
import '../widgets/app_snackbar.dart';
import '../account/account_screen.dart';
import '../l10n/l10n.dart';

```

```

class CourierHomeScreen extends StatefulWidget {
  const CourierHomeScreen({super.key});

  @override
  State<CourierHomeScreen> createState() => _CourierHomeScreenState();
}

class _CourierHomeScreenState extends State<CourierHomeScreen> {
  final DatabaseService _databaseService = DatabaseService();
  final Set<String> _busyOrderIds = {};
  late final Stream<List<Map<String, dynamic>>>? _assignedOrders = FirebaseAuth.instance.currentUser
  == null
    ? null
    : _databaseService.watchAssignedOrders(FirebaseAuth.instance.currentUser!.uid);
  late final Stream<List<Map<String, dynamic>>> _availableOrders =
    _databaseService.watchAvailableOrders();

  Future<void> _changeStatus(String orderId, String statusId, String courierId) async {
    setState(() => _busyOrderIds.add(orderId));
    try {
      await _databaseService.updateOrder(orderId: orderId, statusId: statusId, courierId: courierId);
    } catch (_) {
      if (mounted) showAppSnackBar(context, context.l10n.courierUpdateFailed, isError: true);
    } finally {
      if (mounted) setState(() => _busyOrderIds.remove(orderId));
    }
  }

  /// Завершение доставки необратимо (заказ закрывается), поэтому просим подтверждение.
  Future<void> _confirmComplete(Map<String, dynamic> order, String courierId) async {
    final l10n = context.l10n;
    final number = order['number']?.toString() ?? order['id'].toString();
    final confirmed = await showDialog<bool>({
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: Text(l10n.courierConfirmTitle),
        content: Text(l10n.courierConfirmBody(number)),
        actions: [
          TextButton(onPressed: () => Navigator.pop(dialogContext, false), child: Text(l10n.cancel)),
          ElevatedButton(

```

```

        style: ElevatedButton.styleFrom(backgroundColor: AppPalette.primary, foregroundColor:
Colors.white),
        onPressed: () => Navigator.pop(dialogContext, true),
        child: Text(l10n.courierComplete),
      ),
    ],
  ),
);
if (confirmed == true) {
  await _changeStatus(order['id'].toString(), 'delivered', courierId);
}
}

```

```

Future<void> _claim(String orderId, String courierId) async {
  setState(() => _busyOrderIds.add(orderId));
  try {
    await _databaseService.claimOrder(orderId: orderId, courierId: courierId);
    if (mounted) showAppSnackBar(context, context.l10n.courierClaimed);
  } on DatabaseOperationException catch (error) {
    if (mounted) showAppSnackBar(context, error.message, isError: true);
  } catch (_) {
    if (mounted) showAppSnackBar(context, context.l10n.courierUpdateFailed, isError: true);
  } finally {
    if (mounted) setState(() => _busyOrderIds.remove(orderId));
  }
}

```

```

Future<void> _confirmRelease(Map<String, dynamic> order, String courierId) async {
  final l10n = context.l10n;
  final orderId = order['id'].toString();
  final number = order['number']?.toString() ?? orderId;
  final confirmed = await showDialog<bool>(<
    context: context,
    builder: (dialogContext) => AlertDialog(
      title: Text(l10n.courierReleaseTitle),
      content: Text(l10n.courierReleaseBody(number)),
      actions: [
        TextButton(onPressed: () => Navigator.pop(dialogContext, false), child: Text(l10n.cancel)),
        ElevatedButton(
          style: ElevatedButton.styleFrom(backgroundColor: AppPalette.warning, foregroundColor:
Colors.white),

```

```

        onPressed: () => Navigator.pop(dialogContext, true),
        child: Text(l10n.courierRelease),
      ),
    ],
  ),
);
if (confirmed != true || !mounted) return;

setState(() => _busyOrderIds.add(orderId));
try {
  await _databaseService.releaseOrder(orderId: orderId, courierId: courierId);
  if (mounted) showAppSnackBar(context, l10n.courierReleased);
} on DatabaseOperationException catch (error) {
  if (mounted) showAppSnackBar(context, error.message, isError: true);
} catch (_) {
  if (mounted) showAppSnackBar(context, l10n.courierUpdateFailed, isError: true);
} finally {
  if (mounted) setState(() => _busyOrderIds.remove(orderId));
}
}

Future<void> _copyPhone(String phone) async {
  await Clipboard.setData(ClipboardData(text: phone));
  if (mounted) showAppSnackBar(context, context.l10n.courierPhoneCopied);
}

@override
Widget build(BuildContext context) {
  final currentUser = FirebaseAuth.instance.currentUser;
  if (currentUser == null) {
    return Scaffold(body: Center(child: Text(context.l10n.courierNotSignedIn)));
  }

  return DefaultTabController(
    length: 2,
    child: Scaffold(
      backgroundColor: AppPalette.background,
      appBar: AppBar(
        title: Text(context.l10n.courierTitle),
        actions: [
          IconButton(

```

```

        onPressed: () {
          Navigator.push(context, MaterialPageRoute(builder: (_) => const AccountScreen()));
        },
        icon: const Icon(Icons.person_outline),
      ),
    ],
    bottom: TabBar(
      labelColor: AppPalette.primary,
      indicatorColor: AppPalette.primary,
      tabs: [
        Tab(text: context.l10n.courierTabMine),
        Tab(text: context.l10n.courierTabAvailable),
      ],
    ),
  ),
  body: TabBarView(children: [_buildMineTab(currentUser.uid), _buildAvailableTab(currentUser.uid)]),
),
);
}

```

```

Widget _buildMineTab(String courierId) {
  return StreamBuilder<List<Map<String, dynamic>>>>(
    stream: _assignedOrders,
    builder: (context, snapshot) {
      if (snapshot.hasError) {
        return Center(child: Text(context.l10n.ordersLoadFailed));
      }
      if (!snapshot.hasData) {
        return const Center(child: CircularProgressIndicator(color: AppPalette.primary));
      }

      final assignedOrders = snapshot.data!;
      if (assignedOrders.isEmpty) {
        return Center(
          child: Padding(
            padding: const EdgeInsets.all(24),
            child: Text(context.l10n.courierNoOrders, textAlign: TextAlign.center),
          ),
        );
      }
    }
  );
}

```



```

final groups = groupCourierOrders(assignedOrders);
return ListView(
  padding: const EdgeInsets.all(20),
  children: [
    _SectionTitle(context.l10n.courierActiveTitle),
    if (groups.active.isEmpty)
      Padding(
        padding: const EdgeInsets.only(bottom: 12),
        child: Text(context.l10n.courierNoActive, style: const TextStyle(color:
AppPalette.textSecondary)),
      ),
    for (final order in groups.active) ...[_buildOrderCard(order, courierId), const SizedBox(height: 12)],
    if (groups.history.isNotEmpty) ...[
      const SizedBox(height: 8),
      _SectionTitle(context.l10n.courierHistoryTitle),
      for (final order in groups.history) ...[_buildOrderCard(order, courierId), const SizedBox(height: 12)],
    ],
  ],
);
},
);
}

```

```

Widget _buildAvailableTab(String courierId) {
  return StreamBuilder<List<Map<String, dynamic>>>>(
    stream: _availableOrders,
    builder: (context, snapshot) {
      if (snapshot.hasError) {
        return Center(child: Text(context.l10n.ordersLoadFailed));
      }
      if (!snapshot.hasData) {
        return const Center(child: CircularProgressIndicator(color: AppPalette.primary));
      }
    }
  );
}

```

```

final orders = snapshot.data!;
if (orders.isEmpty) {
  return Center(
    child: Padding(
      padding: const EdgeInsets.all(24),
      child: Text(context.l10n.courierNoAvailable, textAlign: TextAlign.center),
    ),
  );
}

```

```

    );
}

return ListView(
  padding: const EdgeInsets.all(20),
  children: [
    for (final order in orders) ...[
      _buildOrderCard(order, courierId, isPool: true),
      const SizedBox(height: 12),
    ],
  ],
);
},
);
}

Widget _buildOrderCard(Map<String, dynamic> order, String courierId, {bool isPool = false}) {
  final l10n = context.l10n;
  final statusId = order['statusId']?.toString() ?? 'new';
  final statusName = orderStatusLabel(statusId, order['statusName']?.toString(), l10n);
  final orderId = order['id'].toString();
  final isBusy = _busyOrderIds.contains(orderId);
  final phone = order['customerPhone']?.toString() ?? "";
  final items = List<Map<String, dynamic>>.from(order['items'] ?? const []);
  final totalAmount = toDoubleValue(order['totalAmount']);
  final isCash = (order['paymentMethod']?.toString() ?? "") == 'Наличными';
  final isFinished = isFinishedOrder(statusId);

  return Opacity(
    opacity: isFinished ? 0.75 : 1,
    child: Container(
      padding: const EdgeInsets.all(18),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(18),
        border: Border.all(color: AppPalette.border),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(

```

```

order['number']?.toString() ?? l10n.courierOrderDefault,
style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold, color: AppPalette.textPrimary),
),
const SizedBox(height: 8),
Text(
  order['address']?.toString() ?? l10n.courierAddressMissing,
  style: const TextStyle(color: AppPalette.textSecondary),
),
if ((order['deliverySlot']?.toString() ?? "").isNotEmpty)
  Text(
    l10n.courierSlot(deliverySlotLabel(order['deliverySlot'].toString(), l10n)),
    style: const TextStyle(color: AppPalette.textSecondary),
  ),
const SizedBox(height: 8),
if (!isPool)
  Text(
    l10n.courierCustomer(order['customerName']?.toString() ?? '—', ),
    style: const TextStyle(fontWeight: FontWeight.w600),
  ),
if (phone.isNotEmpty && !isPool)
  InkWell(
    onTap: () => _copyPhone(phone),
    child: Padding(
      padding: const EdgeInsets.symmetric(vertical: 4),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          const Icon(Icons.phone_outlined, size: 18, color: AppPalette.primary),
          const SizedBox(width: 6),
          Text(
            phone,
            style: const TextStyle(color: AppPalette.primary, fontWeight: FontWeight.w600),
          ),
          const SizedBox(width: 6),
          const Icon(Icons.copy, size: 14, color: AppPalette.textSecondary),
        ],
      ),
    ),
  ),
const SizedBox(height: 8),
for (final item in items)

```

```

Text(
  '• ${pickLocalized(english: context.isEnglish, russian: item['name']?.toString() ?? "", englishValue:
item['nameEn']?.toString())} × ${item['quantity']}',
  style: const TextStyle(color: AppPalette.textSecondary),
),
const SizedBox(height: 8),
Text(
  l10n.courierPayment(paymentMethodLabel(order['paymentMethod']?.toString(), l10n)),
  style: const TextStyle(color: AppPalette.textSecondary),
),
Text(l10n.courierTotal(formatMoney(totalAmount)), style: const TextStyle(fontWeight:
FontWeight.w600)),
if (isCash && !isFinished) ...[
  const SizedBox(height: 8),
  Text(
    l10n.courierCashDue(formatMoney(totalAmount)),
    style: const TextStyle(fontWeight: FontWeight.bold, color: AppPalette.warning),
  ),
],
const SizedBox(height: 8),
if (!isPool)
  Text(
    l10n.courierStatus(statusName),
    style: TextStyle(
      color: statusId == 'cancelled' ? AppPalette.danger : AppPalette.primaryDark,
      fontWeight: FontWeight.w600,
    ),
  ),
if (statusId == 'cancelled')
  Text(l10n.courierCancelledNote, style: const TextStyle(color: AppPalette.textSecondary)),
if (isPool) ...[
  const SizedBox(height: 12),
  ElevatedButton.icon(
    style: ElevatedButton.styleFrom(backgroundColor: AppPalette.primary, foregroundColor:
Colors.white),
    onPressed: isBusy ? null : () => _claim(orderId, courierId),
    icon: const Icon(Icons.add_task),
    label: Text(l10n.courierClaim),
  ),
] else if (!isFinished) ...[
  const SizedBox(height: 12),

```

```

Wrap(
  spacing: 12,
  runSpacing: 8,
  children: [
    if (canStartDelivery(statusId))
      ElevatedButton(
        style: ElevatedButton.styleFrom(
          backgroundColor: AppPalette.warning,
          foregroundColor: Colors.white,
        ),
        onPressed: isBusy ? null : () => _changeStatus(orderId, 'delivering', courierId),
        child: Text(l10n.courierStart),
      ),
    if (canCompleteDelivery(statusId))
      ElevatedButton(
        style: ElevatedButton.styleFrom(
          backgroundColor: AppPalette.primary,
          foregroundColor: Colors.white,
        ),
        onPressed: isBusy ? null : () => _confirmComplete(order, courierId),
        child: Text(l10n.courierComplete),
      ),
    if (statusId == 'assigned')
      OutlinedButton(
        onPressed: isBusy ? null : () => _confirmRelease(order, courierId),
        child: Text(l10n.courierRelease),
      ),
  ],
),
],
),
);
}
}

```

```

class _SectionTitle extends StatelessWidget {
  const _SectionTitle(this.text);

```

```

  final String text;

```

```

@override
Widget build(BuildContext context) {
  return Padding(
    padding: const EdgeInsets.only(bottom: 12),
    child: Text(
      text,
      style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold, color: AppPalette.textPrimary),
    ),
  );
}

```

lib/screens/web/public_web_landing.dart

```

import 'package:flutter/material.dart';
import '../utils/app_palette.dart';
import '../utils/auth_target.dart';
import '../auth/login_screen.dart';
import '../l10n/l10n.dart';

class PublicWebLandingScreen extends StatelessWidget {
  const PublicWebLandingScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.white,
      body: Column(
        children: [
          // Шапка сайта (Header)
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 40, vertical: 20),
            decoration: const BoxDecoration(
              border: Border(bottom: BorderSide(color: AppPalette.border, width: 1)),
            ),
            child: Row(
              children: [
                const Icon(Icons.eco, color: AppPalette.primary, size: 36),
                const SizedBox(width: 12),
                const Text(
                  'Nectar',
                  style: TextStyle(

```

```

        color: AppPalette.textPrimary,
        fontSize: 28,
        fontWeight: FontWeight.bold,
        fontFamily: 'Unbounded',
    ),
),
const Spacer(),
TextButton(
    onPressed: () {},
    child: Text(context.l10n.landingForCustomers, style: TextStyle(color: AppPalette.textPrimary,
fontSize: 16)),
),
const SizedBox(width: 20),
TextButton(
    onPressed: () {},
    child: Text(context.l10n.landingAbout, style: TextStyle(color: AppPalette.textPrimary, fontSize:
16)),
),
const SizedBox(width: 30),
// Вход для сотрудников
OutlinedButton.icon(
    style: OutlinedButton.styleFrom(
        foregroundColor: AppPalette.primary,
        side: const BorderSide(color: AppPalette.primary),
        padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
    ),
    onPressed: () {
        Navigator.push(
            context,
            MaterialPageRoute(
                builder: (_) => const LoginScreen(
                    authFlowTarget: AuthFlowTarget.adminPanel,
                ),
            ),
        );
    },
    icon: const Icon(Icons.admin_panel_settings),
    label: Text(context.l10n.landingStaffLogin, style: TextStyle(fontWeight: FontWeight.bold)),
),
],

```

```

    ),
  ),

  // Основной контент (Hero Section)
  Expanded(
    child: Container(
      color: AppPalette.background,
      child: Center(
        child: ConstrainedBox(
          constraints: const BoxConstraints(maxWidth: 1200),
          child: Row(
            children: [
              // Текстовая часть
              Expanded(
                child: Padding(
                  padding: const EdgeInsets.all(40.0),
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Container(
                        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
                        decoration: BoxDecoration(
                          color: AppPalette.lightPrimary,
                          borderRadius: BorderRadius.circular(20),
                        ),
                      ),
                      child: Text(
                        context.l10n.landingBadge,
                        style: TextStyle(color: AppPalette.primaryDark, fontWeight: FontWeight.bold),
                      ),
                    ],
                  ),
                ),
                child: const SizedBox(height: 24),
                child: Text(
                  context.l10n.landingHeadline,
                  style: TextStyle(
                    fontSize: 56,
                    fontWeight: FontWeight.bold,
                    color: AppPalette.textPrimary,
                    height: 1.1,
                    fontFamily: 'Unbounded',
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    ),
  ),

```



```

    ),
    const SizedBox(height: 24),
    Text(
      context.l10n.landingBody,
      style: TextStyle(
        fontSize: 18,
        color: AppPalette.textSecondary,
        height: 1.5,
      ),
    ),
    const SizedBox(height: 40),
    Text(
      context.l10n.landingDownload,
      style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
    ),
    const SizedBox(height: 16),
    Row(
      children: [
        _buildStoreButton(context, Icons.apple, 'App Store', Colors.black),
        const SizedBox(width: 16),
        _buildStoreButton(context, Icons.android, 'Google Play', AppPalette.primaryDark),
      ],
    ),
  ],
),
),
// Визуальная часть (Картинка телефона)
Expanded(
  child: Stack(
    alignment: Alignment.center,
    children: [
      Container(
        width: 400,
        height: 400,
        decoration: const BoxDecoration(
          color: AppPalette.lightPrimary,
          shape: BoxShape.circle,
        ),
      ),
      const Icon(Icons.phone_iphone, size: 300, color: AppPalette.primary),
    ],
  ),
)

```



```

padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 14),
decoration: BoxDecoration(
  color: color,
  borderRadius: BorderRadius.circular(16),
),
child: Row(
  mainAxisAlignment: MainAxisAlignment.min,
  children: [
    Icon(icon, color: Colors.white, size: 28),
    const SizedBox(width: 12),
    Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Text(context.l10n.landingDownloadIn, style: TextStyle(color: Colors.white70, fontSize: 10)),
        Text(text, style: const TextStyle(color: Colors.white, fontWeight: FontWeight.bold, fontSize: 16)),
      ],
    ),
  ],
),
);
}
}

```

lib/screens/web/web_auth_wrapper.dart

```

import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';

import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../utils/role_utils.dart';
import 'admin_home_screen.dart';
import 'web_role_blocked_screen.dart';

/// Web home of a signed-in session: only admins and managers get the panel.
class WebRoleGate extends StatefulWidget {
  const WebRoleGate({super.key, required this.user});

  final User user;

  @override
  State<WebRoleGate> createState() => _WebRoleGateState();

```

```

}

class _WebRoleGateState extends State<WebRoleGate> {
  late final Future<String> _roleFuture = _resolveRole();

  Future<String> _resolveRole() async {
    final databaseService = DatabaseService();
    await databaseService.ensureUserProfileFromAuth(widget.user, defaultRole: AppRoles.buyer);
    final userProfile = await databaseService.getUserProfile(widget.user.uid);
    return userProfile?['role']?.toString() ?? AppRoles.guest;
  }

  @override
  Widget build(BuildContext context) {
    return FutureBuilder<String>(
      future: _roleFuture,
      builder: (context, snapshot) {
        if (snapshot.hasError) {
          return WebRoleBlockedScreen(roleName: roleLabel(AppRoles.guest));
        }

        if (!snapshot.hasData) {
          return const Scaffold(
            body: Center(child: CircularProgressIndicator(color: AppPalette.primary)),
          );
        }

        final roleCode = snapshot.data!;
        if (canAccessAdminPanel(roleCode)) {
          return const AdminHomeScreen();
        }

        return WebRoleBlockedScreen(roleName: roleLabel(roleCode));
      },
    );
  }
}

```

lib/screens/web/web_role_blocked_screen.dart

```

import 'package:flutter/material.dart';
import '../services/auth_service.dart';
import '../utils/app_palette.dart';

```

```

import '../l10n/l10n.dart';

class WebRoleBlockedScreen extends StatelessWidget {
  const WebRoleBlockedScreen({super.key, required this.roleName});

  final String roleName;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppPalette.background,
      body: Center(
        child: ConstrainedBox(
          constraints: const BoxConstraints(maxWidth: 500),
          child: Container(
            padding: const EdgeInsets.all(40),
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(28),
              boxShadow: [
                BoxShadow(
                  color: Colors.black.withValues(alpha: 0.05),
                  blurRadius: 24,
                  offset: const Offset(0, 10),
                ),
              ],
            ),
            child: Column(
              mainAxisAlignment: MainAxisAlignment.min,
              children: [
                Container(
                  padding: const EdgeInsets.all(20),
                  decoration: BoxDecoration(
                    color: AppPalette.lightPrimary,
                    shape: BoxShape.circle,
                  ),
                  child: const Icon(Icons.shopping_cart_checkout, size: 60, color: AppPalette.primary),
                ),
                const SizedBox(height: 24),
                Text(
                  context.l10n.webBlockedTitle,

```

```

        textAlign: TextAlign.center,
        style: TextStyle(
          fontSize: 24,
          fontWeight: FontWeight.bold,
          color: AppPalette.textPrimary,
        ),
      ),
      const SizedBox(height: 16),
      Text(
        context.l10n.webBlockedBody(roleName),
        textAlign: TextAlign.center,
        style: const TextStyle(
          fontSize: 16,
          color: AppPalette.textSecondary,
          height: 1.5,
        ),
      ),
      const SizedBox(height: 32),
      SizedBox(
        width: double.infinity,
        height: 56,
        child: ElevatedButton(
          style: ElevatedButton.styleFrom(
            backgroundColor: AppPalette.primary,
            foregroundColor: Colors.white,
            shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
          ),
          onPressed: () => AuthService().logout(),
          child: Text(context.l10n.logoutFromAccount, style: TextStyle(fontSize: 16, fontWeight:
FontWeight.bold)),
        ),
      ),
    ],
  ),
);
}
}

```

lib/screens/web/admin_home_screen.dart

```

import 'package:flutter/material.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import '../services/auth_service.dart';
import '../utils/pricing.dart';
import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../l10n/l10n.dart';

class AdminHomeScreen extends StatefulWidget {
  const AdminHomeScreen({super.key});

  @override
  State<AdminHomeScreen> createState() => _AdminHomeScreenState();
}

class _AdminHomeScreenState extends State<AdminHomeScreen> {
  int _selectedIndex = 0;
  final DatabaseService _dbService = DatabaseService();

  Future<void> _confirmAndReseed() async {
    final confirmed = await showDialog<bool>(<
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: Text(context.l10n.reseedTitle),
        content: Text(context.l10n.reseedBody),
        actions: [
          TextButton(onPressed: () => Navigator.pop(dialogContext, false), child: Text(context.l10n.cancel)),
          ElevatedButton(
            style: ElevatedButton.styleFrom(backgroundColor: AppPalette.danger, foregroundColor:
Colors.white),
            onPressed: () => Navigator.pop(dialogContext, true),
            child: Text(context.l10n.reseedConfirm),
          ),
        ],
      ),
    );
    if (confirmed != true || !mounted) return;

    final messenger = ScaffoldMessenger.of(context);
    final l10n = context.l10n;
    messenger.showSnackBar(SnackBar(content: Text(l10n.reseedLoading)));
  }
}

```

```

try {
  await _dbService.reseedCatalog();
  messenger.showSnackBar(SnackBar(content: Text(l10n.reseedDone)));
} catch (_) {
  messenger.showSnackBar(SnackBar(
    content: Text(l10n.reseedFailed),
    backgroundColor: AppPalette.danger,
  ));
}
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: AppPalette.background,
    body: Row(
      children: [
        NavigationRail(
          backgroundColor: Colors.white,
          selectedIndex: _selectedIndex,
          onDestinationSelected: (int index) {
            setState(() => _selectedIndex = index);
          },
          leading: const Padding(
            padding: EdgeInsets.symmetric(vertical: 20),
            child: Icon(Icons.eco, color: AppPalette.primary, size: 40),
          ),
          trailing: Expanded(
            child: Align(
              alignment: Alignment.bottomCenter,
              child: Padding(
                padding: const EdgeInsets.only(bottom: 20),
                child: IconButton(
                  icon: const Icon(Icons.logout, color: AppPalette.danger),
                  onPressed: () => AuthService().logout(),
                  tooltip: context.l10n.logout,
                ),
              ),
            ),
          selectedIconTheme: const IconThemeData(color: AppPalette.primary),

```



```

        selectedLabelTextStyle: TextStyle(color: AppPalette.primary, fontWeight: FontWeight.bold),
        unselectedIconTheme: IconThemeData(color: AppPalette.textSecondary),
        unselectedLabelTextStyle: TextStyle(color: AppPalette.textSecondary),
        labelType: NavigationRailLabelType.all,
        destinations: [
          NavigationRailDestination(icon: Icon(Icons.dashboard_outlined), selectedIcon:
Icon(Icons.dashboard), label: Text(context.l10n.adminHome)),
          NavigationRailDestination(icon: Icon(Icons.shopping_bag_outlined), selectedIcon:
Icon(Icons.shopping_bag), label: Text(context.l10n.menuOrders)),
        ],
      ),
      const VerticalDivider(thickness: 1, width: 1, color: AppPalette.border),
      Expanded(child: _buildBody()),
    ],
  ),
);
}

Widget _buildBody() {
  if (_selectedIndex == 0) return _buildDashboardTab();
  if (_selectedIndex == 1) return _buildOrdersTab();
  return Center(child: Text(context.l10n.sectionInDevelopment));
}

Widget _buildDashboardTab() {
  return Padding(
    padding: const EdgeInsets.all(40.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(context.l10n.adminDashboardTitle, style: TextStyle(fontSize: 32, fontWeight: FontWeight.bold,
color: AppPalette.textPrimary)),
        const SizedBox(height: 30),
        Row(
          children: [
            _buildStatCard(context.l10n.adminNewOrders, context.l10n.adminActive, Icons.shopping_basket,
AppPalette.primary),
            const SizedBox(width: 20),
            _buildStatCard(context.l10n.adminProducts, context.l10n.adminInCatalog, Icons.inventory,
AppPalette.warning),
          ],
        ),
      ],
    ),
  );
}

```

```

    ),
    const SizedBox(height: 40),
    Container(
      padding: const EdgeInsets.all(30),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(20),
        border: Border.all(color: AppPalette.border),
      ),
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(context.l10n.adminQuickActions, style: TextStyle(fontSize: 20, fontWeight:
FontWeight.bold)),
        const SizedBox(height: 20),
        ElevatedButton.icon(
          style: ElevatedButton.styleFrom(
            backgroundColor: AppPalette.primary,
            foregroundColor: Colors.white,
            padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
          ),
          onPressed: _confirmAndReseed,
          icon: const Icon(Icons.refresh),
          label: Text(context.l10n.adminReseedButton),
        ),
        const SizedBox(height: 10),
        Text(context.l10n.adminReseedHint, style: TextStyle(color: AppPalette.textSecondary)),
      ],
    ),
  ),
],
),
],
);
}

```

```

Widget _buildOrdersTab() {
  return Padding(
    padding: const EdgeInsets.all(40.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [

```

```

        Text(context.l10n.adminOrdersTitle, style: TextStyle(fontSize: 32, fontWeight: FontWeight.bold,
color: AppPalette.textPrimary)),
        const SizedBox(height: 20),
        Expanded(
            child: StreamBuilder<List<Map<String, dynamic>>>>(
                stream: _dbService.watchOrders(),
                builder: (context, snapshot) {
                    if (snapshot.connectionState == ConnectionState.waiting) return const Center(child:
CircularProgressIndicator());
                    final orders = snapshot.data ?? [];
                    if (orders.isEmpty) return Center(child: Text(context.l10n.adminNoOrders, style:
TextStyle(fontSize: 18, color: AppPalette.textSecondary)));

                    return ListView.builder(
                        itemCount: orders.length,
                        itemBuilder: (context, index) {
                            final order = orders[index];
                            return _OrderAdminCard(order: order, dbService: _dbService);
                        },
                    );
                },
            ),
        ),
    ],
),
);
}

```

```

Widget _buildStatCard(String title, String subtitle, IconData icon, Color color) {
    return Expanded(
        child: Container(
            padding: const EdgeInsets.all(24),
            decoration: BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.circular(20),
                border: Border.all(color: AppPalette.border),
                boxShadow: [BoxShadow(color: Colors.black.withValues(alpha: 0.02), blurRadius: 10, offset: const
Offset(0, 4))],
            ),
        child: Row(
            children: [

```

```

        Container(
          padding: const EdgeInsets.all(16),
          decoration: BoxDecoration(color: color.withValues(alpha: 0.1), shape: BoxShape.circle),
          child: Icon(icon, color: color, size: 32),
        ),
        const SizedBox(width: 20),
        Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(title, style: const TextStyle(color: AppPalette.textSecondary, fontSize: 16)),
            const SizedBox(height: 8),
            Text(subtitle, style: const TextStyle(color: AppPalette.textPrimary, fontSize: 18, fontWeight:
FontWeight.bold)),
          ],
        ),
      ],
    );
  }
}

```

```

class _OrderAdminCard extends StatefulWidget {
  final Map<String, dynamic> order;
  final DatabaseService dbService;

  const _OrderAdminCard({required this.order, required this.dbService});

  @override
  State<_OrderAdminCard> createState() => _OrderAdminCardState();
}

```

```

class _OrderAdminCardState extends State<_OrderAdminCard> {
  List<Map<String, dynamic>> _couriers = [];
  String? _selectedCourierId;
  String? _selectedStatusId;

  @override
  void initState() {
    super.initState();
    _selectedCourierId = widget.order['courierId'];
  }
}

```

```

    _selectedStatusId = widget.order['statusId'];
    _loadCouriers();
  }

Future<void> _loadCouriers() async {
  try {
    final snapshot = await FirebaseFirestore.instance.collection('users').where('role', isEqualTo:
'courier').get();
    if (!mounted) return;
    setState() {
      _couriers = snapshot.docs.map((doc) => {'id': doc.id, 'name': doc.data()['displayName'] ??
context.l10n.adminNoName}).toList();
    });
  } catch (_) {
    // Список курьеров останется пустым; назначение можно повторить после перезагрузки.
  }
}

Future<void> _saveChanges() async {
  if ((_selectedStatusId == 'delivering' || _selectedStatusId == 'delivered') && _selectedCourierId == null) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(context.l10n.adminCourierRequired),
        backgroundColor: AppPalette.danger,
        duration: Duration(seconds: 4),
      ),
    );
    return;
  }

  if (_selectedStatusId == 'new' && _selectedCourierId != null) {
    _selectedStatusId = 'assigned';
  }

  try {
    await widget.dbService.updateOrder(
      orderId: widget.order['id'],
      statusId: _selectedStatusId ?? 'new',
      courierId: _selectedCourierId,
    );
    if (mounted) {

```

```

        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.adminOrderUpdated), backgroundColor: AppPalette.primary)
        );
      }
    } catch (_) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.adminOrderUpdateFailed), backgroundColor:
AppPalette.danger)
        );
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: EdgeInsets.only(bottom: 16),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
      child: Padding(
        padding: EdgeInsets.all(20),
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Expanded(
              flex: 2,
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(widget.order['number'] ?? context.l10n.courierOrderDefault, style: TextStyle(fontSize: 18,
fontWeight: FontWeight.bold)),
                  SizedBox(height: 8),
                  Text(context.l10n.adminCustomer('${widget.order['customerName'] ?? ''}')),
                  Text(context.l10n.adminAddress('${widget.order['address'] ?? ''}')),
                  if ((widget.order['customerPhone']?.toString() ?? '').isEmpty)
                    Text(context.l10n.adminPhone('${widget.order['customerPhone'] ?? ''}')),
                  if ((widget.order['deliverySlot']?.toString() ?? '').isEmpty)
                    Text(context.l10n.adminDeliverySlot(deliverySlotLabel(widget.order['deliverySlot'].toString(),
context.l10n))),
                  Text(context.l10n.adminAmount('${widget.order['totalAmount'] ?? ''}), style:
TextStyle(fontWeight: FontWeight.bold, color: AppPalette.primaryDark)),

```



```

        DropdownMenuItem(value: 'cancelled', child: Text(context.l10n.statusCancelled)),
      ],
      onChanged: (val) => setState(() => _selectedStatusId = val),
    ),
  ],
),
),
const SizedBox(width: 20),
ElevatedButton(
  style: ElevatedButton.styleFrom(backgroundColor: AppPalette.primary, foregroundColor:
Colors.white, padding: const EdgeInsets.symmetric(vertical: 20)),
  onPressed: _saveChanges,
  child: Text(context.l10n.save),
),
],
),
),
);
}
}

```

lib/widgets/app_snackbar.dart

```

import 'package:flutter/material.dart';

import '../utils/app_palette.dart';

void showAppSnackBar(
  BuildContext context,
  String message, {
  bool isError = false,
  IconData? icon,
  bool floating = false,
}) {
  final messenger = ScaffoldMessenger.of(context);
  messenger.hideCurrentSnackBar();
  messenger.showSnackBar(
    SnackBar(
      content: Row(
        children: [
          if (icon != null) ...[
            Icon(icon, color: Colors.white),
            const SizedBox(width: 12),

```



```

    ],
    Expanded(child: Text(message)),
  ],
),
backgroundColor: isError ? AppPalette.danger : AppPalette.primary,
duration: const Duration(seconds: 2),
behavior: floating ? SnackBarBehavior.floating : SnackBarBehavior.fixed,
shape: floating ? RoundedRectangleBorder(borderRadius: BorderRadius.circular(15)) : null,
margin: floating ? const EdgeInsets.only(bottom: 20, left: 20, right: 20) : null,
),
);
}

```

lib/widgets/centered_message.dart

```

import 'package:flutter/material.dart';
import '../utils/app_palette.dart';

class CenteredMessage extends StatelessWidget {
  const CenteredMessage(this.message, {super.key, this.icon});

  final String message;
  final IconData? icon;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            if (icon != null) ...[
              Icon(icon, size: 56, color: const Color(0xFFB3B3B3)),
              const SizedBox(height: 12),
            ],
            Text(
              message,
              textAlign: TextAlign.center,
              style: const TextStyle(fontSize: 16, color: AppPalette.textSecondary),
            ),
          ],
        ),
      ),
    );
  }
}

```

```

    ),
  );
}
}

```

lib/widgets/input_dialog.dart

```

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import '../services/database_service.dart';
import '../utils/app_palette.dart';
import '../l10n/l10n.dart';

/// Shows a single-field dialog. The dialog owns its controller, so nothing is
/// disposed while the closing animation is still running. [onSubmit] may throw
/// a [DatabaseOperationException]; its message is shown inside the dialog.
Future<void> showInputDialog(
  BuildContext context, {
  required String title,
  required String hint,
  required Future<void> Function(String value) onSubmit,
  String? Function(String?)? validator,
  TextInputType? keyboardType,
  int? maxLength,
  int maxLines = 1,
  List<TextInputFormatter>? inputFormatters,
}) {
  return showDialog<void>(
    context: context,
    builder: (_) => _InputDialog(
      title: title,
      hint: hint,
      onSubmit: onSubmit,
      validator: validator,
      keyboardType: keyboardType,
      maxLength: maxLength,
      maxLines: maxLines,
      inputFormatters: inputFormatters,
    ),
  );
}

```

```

class _InputDialog extends StatefulWidget {
  const _InputDialog({
    required this.title,
    required this.hint,
    required this.onSubmit,
    this.validator,
    this.keyboardType,
    this.maxLength,
    this.maxLines = 1,
    this.inputFormatters,
  });

  final String title;
  final String hint;
  final Future<void> Function(String value) onSubmit;
  final String? Function(String?)? validator;
  final TextInputType? keyboardType;
  final int? maxLength;
  final int maxLines;
  final List<TextInputFormatter>? inputFormatters;

  @override
  State<_InputDialog> createState() => _InputDialogState();
}

class _InputDialogState extends State<_InputDialog> {
  final TextEditingController _controller = TextEditingController();
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();

  bool _isSaving = false;
  String? _errorText;

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  Future<void> _submit() async {
    if (!_formKey.currentState!.validate()) return;

```

```

setState() {
  _isSaving = true;
  _errorText = null;
});

try {
  await widget.onSubmit(_controller.text);
  if (mounted) Navigator.pop(context);
} on DatabaseOperationException catch (error) {
  if (!mounted) return;
  setState() {
    _isSaving = false;
    _errorText = error.message;
  });
} catch (_) {
  if (!mounted) return;
  setState() {
    _isSaving = false;
    _errorText = context.l10n.saveFailed;
  });
}
}

@override
Widget build(BuildContext context) {
  return AlertDialog(
    title: Text(widget.title),
    content: Form(
      key: _formKey,
      child: TextFormField(
        controller: _controller,
        autofocus: true,
        enabled: !_isSaving,
        keyboardType: widget.keyboardType,
        maxLength: widget.maxLength,
        maxLines: widget.maxLines,
        inputFormatters: widget.inputFormatters,
        validator: widget.validator,
        decoration: InputDecoration(hintText: widget.hint, errorText: _errorText),
      ),
    ),
  ),

```

```

actions: [
  TextButton(
    onPressed: _isSaving ? null : () => Navigator.pop(context),
    child: Text(context.l10n.cancel, style: TextStyle(color: Colors.grey)),
  ),
  TextButton(
    onPressed: _isSaving ? null : _submit,
    child: Text(context.l10n.save, style: TextStyle(color: AppPalette.primary)),
  ),
],
);
}
}

```

lib/widgets/language_switcher.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../l10n/l10n.dart';
import '../providers/locale_provider.dart';
import '../utils/app_palette.dart';

/// Переключатель RU / EN. [onDark] рисует его светлым на темных фонах.
class LanguageSwitcher extends StatelessWidget {
  const LanguageSwitcher({super.key, this.onDark = false});

  final bool onDark;

  @override
  Widget build(BuildContext context) {
    final locale = context.watch<LocaleProvider>();
    final foreground = onDark ? Colors.white : AppPalette.textPrimary;

    Widget option(String label, Locale value, String semanticLabel) {
      final selected = locale.locale.languageCode == value.languageCode;
      return Semantics(
        button: true,
        selected: selected,
        label: semanticLabel,
        child: InkWell(
          borderRadius: BorderRadius.circular(999),
          onTap: () => context.read<LocaleProvider>().setLocale(value),

```

```

        child: Container(
          padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
          decoration: BoxDecoration(
            color: selected ? AppPalette.primary : Colors.transparent,
            borderRadius: BorderRadius.circular(999),
          ),
          child: Text(
            label,
            style: TextStyle(
              color: selected ? Colors.white : foreground,
              fontWeight: FontWeight.w700,
              fontSize: 13,
            ),
          ),
        ),
      );
    }

    return Container(
      padding: const EdgeInsets.all(3),
      decoration: BoxDecoration(
        color: onDark ? Colors.black.withValues(alpha: 0.35) : Colors.white,
        borderRadius: BorderRadius.circular(999),
        border: Border.all(color: onDark ? Colors.white54 : AppPalette.border),
      ),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          option('RU', AppLocale.russian, context.l10n.languageRussian),
          option('EN', AppLocale.english, context.l10n.languageEnglish),
        ],
      ),
    );
  }
}

```

lib/widgets/product_card.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/product_model.dart';

```

```

import '../providers/cart_provider.dart';
import '../providers/favorite_provider.dart';
import '../screens/shop/product_detail_screen.dart';
import '../utils/app_palette.dart';
import 'app_snackbar.dart';
import 'product_image.dart';
import '../l10n/l10n.dart';

class ProductCard extends StatelessWidget {
  const ProductCard({super.key, required this.product, this.width, this.addButtonSize = 45});

  final Product product;
  final double? width;
  final double addButtonSize;

  void _addToCart(BuildContext context) {
    FocusScope.of(context).unfocus();
    final added = context.read<CartProvider>().addProduct(product);
    showAppSnackBar(
      context,
      added ? context.l10n.addToCart(product.nameFor(english: context.isEnglish)) :
context.l10n.noMoreStockNamed(product.nameFor(english: context.isEnglish)),
      icon: added ? Icons.shopping_bag : Icons.info_outline,
      isError: !added,
      floating: true,
    );
  }

  @override
  Widget build(BuildContext context) {
    final isOutOfStock = product.isOutOfStock;
    final isFavorite = context.select<FavoriteProvider, bool>((favorites) => favorites.isFavorite(product.id));
    final titleColor = isOutOfStock ? Colors.grey : AppPalette.textPrimary;

    return GestureDetector(
      onTap: () {
        FocusScope.of(context).unfocus();
        Navigator.push(
          context,
          MaterialPageRoute(builder: (_) => ProductDetailScreen(product: product)),
        );
      }
    );
  }
}

```

```

},
child: Container(
  width: width,
  padding: const EdgeInsets.only(top: 15, left: 15, right: 15, bottom: 10),
  decoration: BoxDecoration(
    border: Border.all(color: const Color(0xFFE2E2E2)),
    borderRadius: BorderRadius.circular(18),
    color: Colors.white,
  ),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Expanded(
        child: Stack(
          children: [
            Center(child: ProductImage(url: product.imageUrl, dimmed: isOutOfStock)),
            if (isOutOfStock)
              Center(
                child: Container(
                  padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
                  decoration: BoxDecoration(
                    color: Colors.black54,
                    borderRadius: BorderRadius.circular(8),
                  ),
                  child: Text(
                    context.l10n.outOfStock,
                    style: TextStyle(color: Colors.white, fontSize: 10, fontWeight: FontWeight.bold),
                  ),
                ),
              ),
            if (isFavorite)
              const Positioned(
                top: 0,
                right: 0,
                child: Icon(Icons.favorite, color: AppPalette.primary, size: 20),
              ),
          ],
        ),
      const SizedBox(height: 15),
      Text(

```



```

        product.nameFor(english: context.isEnglish),
        style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold, color: titleColor),
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
    ),
    const SizedBox(height: 5),
    Text(
        product.qtyFor(english: context.isEnglish),
        style: const TextStyle(fontSize: 14, color: AppPalette.textSecondary),
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
    ),
    const SizedBox(height: 15),
    Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
            Expanded(
                child: Text(
                    '${product.price.toStringAsFixed(0)} P',
                    style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold, color: titleColor),
                    overflow: TextOverflow.ellipsis,
                ),
            ),
            GestureDetector(
                onTap: isOutOfStock ? null : () => _addToCart(context),
                child: Container(
                    width: addButtonSize,
                    height: addButtonSize,
                    decoration: BoxDecoration(
                        color: isOutOfStock ? const Color(0xFFE2E2E2) : AppPalette.primary,
                        borderRadius: BorderRadius.circular(addButtonSize * 0.38),
                    ),
                    child: Icon(Icons.add, color: isOutOfStock ? Colors.grey : Colors.white, size: 24),
                ),
            ),
        ],
    ),
],
),
],
);

```

```

    }
  }
}

lib/widgets/product_filters_sheet.dart

import 'dart:math' as math;

import 'package:flutter/material.dart';

import '../models/product_model.dart';
import '../utils/app_palette.dart';
import '../utils/product_filters.dart';
import '../l10n/l10n.dart';

Future<ProductFilters?> showProductFiltersSheet(
  BuildContext context, {
  required List<Product> products,
  required ProductFilters current,
}) {
  return showModalBottomSheet<ProductFilters>(
    context: context,
    isScrollControlled: true,
    backgroundColor: Colors.white,
    shape: const RoundedRectangleBorder(borderRadius: BorderRadius.vertical(top: Radius.circular(28))),
    builder: (_) => _ProductFiltersSheet(products: products, current: current),
  );
}

class _ProductFiltersSheet extends StatefulWidget {
  const _ProductFiltersSheet({required this.products, required this.current});

  final List<Product> products;
  final ProductFilters current;

  @override
  State<_ProductFiltersSheet> createState() => _ProductFiltersSheetState();
}

class _ProductFiltersSheetState extends State<_ProductFiltersSheet> {
  late final double _lowerBound;
  late final double _upperBound;
  late final List<String> _availableBrands;
  late RangeValues _range;

```

```

late Set<String> _brands;
late ProductSort _sort;

@override
void initState() {
  super.initState();
  final prices = widget.products.map((product) => product.price);
  _lowerBound = prices.isEmpty ? 0 : prices.reduce(math.min).floorToDouble();
  _upperBound = prices.isEmpty ? 1 : math.max(prices.reduce(math.max).ceilToDouble(), _lowerBound +
1);
  _availableBrands = widget.products.map((product) => product.brand).where((brand) =>
brand.isNotEmpty).toSet().toList()
    ..sort();

  _range = RangeValues(
    (widget.current.minPrice ?? _lowerBound).clamp(_lowerBound, _upperBound),
    (widget.current.maxPrice ?? _upperBound).clamp(_lowerBound, _upperBound),
  );
  _brands = {...widget.current.brands};
  _sort = widget.current.sort;
}

ProductFilters _buildFilters() {
  return ProductFilters(
    minPrice: _range.start > _lowerBound ? _range.start : null,
    maxPrice: _range.end < _upperBound ? _range.end : null,
    brands: _brands,
    sort: _sort,
  );
}

@override
Widget build(BuildContext context) {
  return SafeArea(
    child: Padding(
      padding: const EdgeInsets.fromLTRB(24, 20, 24, 16),
      child: SingleChildScrollView(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [

```

```

Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    Text(context.l10n.filtersTitle, style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold)),
    IconButton(onPressed: () => Navigator.pop(context), icon: const Icon(Icons.close)),
  ],
),
const SizedBox(height: 8),
Text(
  context.l10n.filterPrice(_range.start.round(), _range.end.round()),
  style: const TextStyle(fontWeight: FontWeight.w700),
),
RangeSlider(
  values: _range,
  min: _lowerBound,
  max: _upperBound,
  activeColor: AppPalette.primary,
  labels: RangeLabels('${_range.start.round()} ₪', '${_range.end.round()} ₪'),
  onChanged: (values) => setState(() => _range = values),
),
if (_availableBrands.isNotEmpty) ...[
  const SizedBox(height: 8),
  Text(context.l10n.filterBrand, style: TextStyle(fontWeight: FontWeight.w700)),
  const SizedBox(height: 8),
  Wrap(
    spacing: 8,
    runSpacing: 4,
    children: [
      for (final brand in _availableBrands)
        FilterChip(
          label: Text(brand),
          selected: _brands.contains(brand),
          selectedColor: AppPalette.lightPrimary,
          onSelect: (selected) => setState(() {
            selected ? _brands.add(brand) : _brands.remove(brand);
          }),
        ),
    ],
  ),
],
const SizedBox(height: 16),

```

```

Text(context.l10n.filterSort, style: TextStyle(fontWeight: FontWeight.w700)),
const SizedBox(height: 8),
Wrap(
  spacing: 8,
  runSpacing: 4,
  children: [
    for (final sort in ProductSort.values)
    ChoiceChip(
      label: Text(sort.label(context.l10n)),
      selected: _sort == sort,
      selectedColor: AppPalette.lightPrimary,
      onSelect: (_) => setState(() => _sort = sort),
    ),
  ],
),
const SizedBox(height: 20),
Row(
  children: [
    Expanded(
      child: OutlinedButton(
        onPressed: () => Navigator.pop(context, const ProductFilters()),
        child: Text(context.l10n.reset),
      ),
    ),
    const SizedBox(width: 12),
    Expanded(
      child: ElevatedButton(
        style: ElevatedButton.styleFrom(backgroundColor: AppPalette.primary, foregroundColor:
Colors.white),
        onPressed: () => Navigator.pop(context, _buildFilters()),
        child: Text(context.l10n.apply),
      ),
    ),
  ],
),
],
),
);
}

```

```

}

lib/widgets/product_grid.dart

import 'package:flutter/material.dart';

import '../models/product_model.dart';
import 'product_card.dart';

class ProductGrid extends StatelessWidget {
  const ProductGrid({
    super.key,
    required this.products,
    this.padding = const EdgeInsets.symmetric(horizontal: 20, vertical: 10),
  });

  final List<Product> products;
  final EdgeInsetsGeometry padding;

  @override
  Widget build(BuildContext context) {
    return GridView.builder(
      padding: padding,
      gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: 2,
        childAspectRatio: 0.70,
        crossAxisSpacing: 15,
        mainAxisSpacing: 15,
      ),
      itemCount: products.length,
      itemBuilder: (context, index) => ProductCard(product: products[index], addButtonSize: 40),
    );
  }
}

```

```

lib/widgets/product_image.dart

import 'package:flutter/material.dart';

class ProductImage extends StatelessWidget {
  const ProductImage({
    super.key,
    required this.url,
    this.fit = BoxFit.contain,

```

```

        this.iconSize = 50,
        this.dimmed = false,
    });

    final String url;
    final BoxFit fit;
    final double iconSize;
    final bool dimmed;

    @override
    Widget build(BuildContext context) {
        final placeholder = Icon(Icons.image_not_supported, size: iconSize, color: Colors.grey);
        if (url.isEmpty) {
            return placeholder;
        }

        return Image.network(
            url,
            fit: fit,
            color: dimmed ? Colors.white.withValues(alpha: 0.5) : null,
            colorBlendMode: dimmed ? BlendMode.modulate : null,
            errorBuilder: (context, error, stackTrace) => placeholder,
        );
    }
}

```

web/index.html

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8" />
  <meta http-equiv="X-UA-Compatible" content="IE=edge" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta
    name="description"
    content="Nectar Admin — панель управления магазином и доставкой."
  />
  <title>Nectar Admin</title>
  <link rel="icon" type="image/png" href="/favicon.png" />
  <link rel="manifest" href="/manifest.json" />
  <link rel="preconnect" href="https://fonts.googleapis.com" />
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />

```

```

<link
  href="https://fonts.googleapis.com/css2?
family=Golos+Text:wght@400;500;600;700&family=Unbounded:wght@500;700&display=swap"
  rel="stylesheet"
/>
<link rel="stylesheet" href="/admin.css" />
</head>
<body>
<div id="app">
  <section id="login-view" class="screen active">
    <div class="auth-card">
      <div class="brand-badge">Nectar Admin</div>
      <h1>Панель управления магазином</h1>
      <p class="muted">
        Вход доступен только ролям «Администратор» и «Менеджер».
      </p>
      <form id="login-form" class="form-grid">
        <label>
          <span>Email</span>
          <input id="login-email" type="email" required maxlength="100" autocomplete="username"
placeholder="admin@company.com" />
        </label>
        <label>
          <span>Пароль</span>
          <input id="login-password" type="password" required maxlength="128" autocomplete="current-
password" placeholder="Введите пароль" />
        </label>
        <button type="submit" class="primary-button">Войти</button>
      </form>
      <p id="login-error" class="error-text"></p>
    </div>
  </section>

  <section id="admin-view" class="screen">
    <header class="topbar">
      <div class="topbar__brand">
        <div class="topbar__logo">N</div>
        <div>
          <strong>Nectar</strong>
          <div class="muted small">Store Control Center</div>
        </div>
      </div>
    </header>
  </section>

```



```

</div>
<nav id="nav-tabs" class="nav-tabs"></nav>
<div class="topbar__actions">
  <div>
    <div id="current-user-name" class="user-name"></div>
    <div id="current-user-role" class="muted small"></div>
  </div>
  <button id="password-button" class="ghost-button" type="button">
    Сменить пароль
  </button>
  <button id="refresh-button" class="ghost-button" type="button">
    Обновить
  </button>
  <button id="logout-button" class="danger-button" type="button">
    Выйти
  </button>
</div>
</header>

<main class="content">
  <section id="dashboard-section" class="panel-section"></section>
  <section id="products-section" class="panel-section hidden"></section>
  <section id="categories-section" class="panel-section hidden"></section>
  <section id="manufacturers-section" class="panel-section hidden"></section>
  <section id="suppliers-section" class="panel-section hidden"></section>
  <section id="promoCodes-section" class="panel-section hidden"></section>
  <section id="orders-section" class="panel-section hidden"></section>
  <section id="users-section" class="panel-section hidden"></section>
</main>
</section>
</div>

<div id="modal-root"></div>
<div id="toast-root"></div>

<script type="module" src="./admin.js"></script>
</body>
</html>

web/admin.js

import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.5/firebase-app.js";
import {

```

```

EmailAuthProvider,
connectAuthEmulator,
getAuth,
onAuthStateChanged,
reauthenticateWithCredential,
signInWithEmailAndPassword,
signOut,
updatePassword,
} from "https://www.gstatic.com/firebasejs/10.12.5/firebase-auth.js";
import {
  collection,
  connectFirestoreEmulator,
  deleteDoc,
  deleteField,
  doc,
  getDoc,
  getDocs,
  getFirestore,
  increment,
  runTransaction,
  serverTimestamp,
  setDoc,
  updateDoc,
  writeBatch,
} from "https://www.gstatic.com/firebasejs/10.12.5/firebase-firestore.js";

import { categoryTranslations, productTranslations } from "./catalog-translations.js";
import { getLanguage, initLocalization, t } from "./i18n.js";
import {
  UserError,
  computeDashboard,
  endOfDay,
  esc,
  friendlyError,
  isFinishedStatus,
  stockRestorations,
  toDateInputValue,
  validateEntityForm,
  validateNewPassword,
  validateOrderChange,
  validateProductForm,

```

```

} from "./validation.js";

const firebaseConfig = {
  apiKey: "AIzaSyCfdsjRPSj_d6rqnv0VkTYqozj9moggUwbE",
  appId: "1:70969291340:web:464aa7206ce3067331fae9",
  messagingSenderId: "70969291340",
  projectId: "nectar-41848",
  authDomain: "nectar-41848.firebaseio.com",
  storageBucket: "nectar-41848.firebaseio.com",
  measurementId: "G-TR33KSG6WL",
};

// Локальная отладка: http://localhost:8080/?emulator=1 подключает панель к эмуляторам
// Firebase (tool/rules-test) и не трогает боевую базу. На других адресах параметр игнорируется.
const useEmulator =
  ["localhost", "127.0.0.1"].includes(window.location.hostname) &&
  new URLSearchParams(window.location.search).has("emulator");

const app = initializeApp(useEmulator ? { apiKey: "demo-key", projectId: firebaseConfig.projectId } :
  firebaseConfig);
const auth = getAuth(app);
const db = getFirestore(app);
if (useEmulator) {
  connectAuthEmulator(auth, "http://127.0.0.1:9099", { disableWarnings: true });
  connectFirestoreEmulator(db, "127.0.0.1", 8080);
}

const state = {
  user: null,
  role: null,
  section: "dashboard",
  products: [],
  categories: [],
  manufacturers: [],
  suppliers: [],
  promoCodes: [],
  users: [],
  orders: [],
  productSearch: "",
  showDisabledUsers: false,
};

```

```
const sectionLabels = {  
  dashboard: "Дашборд",  
  products: "Товары",  
  categories: "Категории",  
  manufacturers: "Производители",  
  suppliers: "Поставщики",  
  promoCodes: "Промокоды",  
  orders: "Заказы",  
  users: "Пользователи",  
};
```

```
const roleLabels = {  
  admin: "Администратор",  
  manager: "Менеджер",  
  courier: "Курьер",  
  buyer: "Покупатель",  
};
```

```
const statusNames = {  
  ru: {  
    new: "Новый",  
    processing: "В сборке",  
    assigned: "Передан курьеру",  
    delivering: "В пути",  
    delivered: "Доставлен",  
    cancelled: "Отменен",  
  },  
  en: {  
    new: "New",  
    processing: "Being packed",  
    assigned: "Handed to courier",  
    delivering: "On the way",  
    delivered: "Delivered",  
    cancelled: "Cancelled",  
  },  
};
```

```
const collectionConfigs = {  
  categories: {  
    collectionName: "categories",
```

```

    title: "Категории",
    fields: [
      { key: "name", label: "Название", required: true, maxLength: 100 },
      { key: "nameEn", label: "Название (English)", maxLength: 100 },
      { key: "description", label: "Описание", multiline: true, maxLength: 500 },
    ],
  },
  manufacturers: {
    collectionName: "manufacturers",
    title: "Производители",
    fields: [
      { key: "name", label: "Название", required: true, maxLength: 100 },
      { key: "country", label: "Страна", required: true, maxLength: 100 },
      { key: "contactPhone", label: "Телефон", required: true, type: "tel", maxLength: 30 },
    ],
  },
  suppliers: {
    collectionName: "suppliers",
    title: "Поставщики",
    fields: [
      { key: "name", label: "Название", required: true, maxLength: 100 },
      { key: "taxId", label: "ИНН", required: true, maxLength: 12 },
      { key: "address", label: "Адрес", required: true, multiline: true, maxLength: 300 },
      { key: "contactPhone", label: "Телефон", required: true, type: "tel", maxLength: 30 },
      { key: "email", label: "Email", type: "email", maxLength: 100 },
    ],
  },
  promoCodes: {
    collectionName: "promoCodes",
    title: "Промокоды",
    fields: [
      { key: "code", label: "Код", required: true, maxLength: 30 },
      { key: "discountPercent", label: "Скидка, %", required: true, type: "number", min: 1, max: 99, step: 1 },
      { key: "expiresAt", label: "Срок действия", required: true, type: "date" },
    ],
  },
};

```

```

const loginView = document.getElementById("login-view");
const adminView = document.getElementById("admin-view");
const loginForm = document.getElementById("login-form");

```

```

const loginError = document.getElementById("login-error");
const navTabs = document.getElementById("nav-tabs");
const currentUser_name = document.getElementById("current-user-name");
const currentUserRole = document.getElementById("current-user-role");
const modalRoot = document.getElementById("modal-root");
const toastRoot = document.getElementById("toast-root");

initLocalization();

const isAdmin = () => state.role === "admin";

function localizedField(item, field = "name") {
  const value = item?.[field];
  if (getLanguage() === "en" && item?.[`${field}En`]) return item[`${field}En`];
  return value;
}

function getStatusName(statusId, language = "ru") {
  return statusNames[language][statusId] || statusNames[language].processing;
}

/** Легаси-роль customer показывается как покупатель. */
function normalizeRole(role) {
  return role === "customer" ? "buyer" : role || "buyer";
}

function userLabel(user) {
  return user?.displayName || user?.email || "-";
}

function orderTime(order) {
  return order.createdAt?.seconds || 0;
}

/**
 * Выполняет действие пользователя: блокирует кнопку на время работы, показывает
 * понятное сообщение об ошибке и не оставляет необработанных исключений.
 */
async function runGuarded(action, { button = null, success = null } = {}) {
  if (button?.disabled) return;
  if (button) button.disabled = true;

```

```

try {
  const result = await action();
  if (success) showToast(success, "success");
  return result;
} catch (error) {
  console.error(error);
  showToast(friendlyError(error), "error");
} finally {
  if (button) button.disabled = false;
}
}

function confirmAction(message) {
  return window.confirm(t(message));
}

// ----- Авторизация -----

loginForm.addEventListener("submit", handleLogin);
document.getElementById("logout-button").addEventListener("click", () => signOut(auth));
document.getElementById("password-button").addEventListener("click", openChangePasswordModal);
document.getElementById("refresh-button").addEventListener("click", (event) => {
  runGuarded(
    async () => {
      await reloadAndRender();
    },
    { button: event.currentTarget, success: "Данные обновлены." }
  );
});

let authRun = 0;

onAuthStateChanged(auth, async (user) => {
  const run = ++authRun;
  if (!user) {
    resetSession();
    return;
  }

  try {
    const profileDoc = await getDoc(doc(db, "users", user.uid));

```

```

if (run !== authRun) return;

const profileData = profileDoc.exists() ? profileDoc.data() : null;
if (profileData?.isDeleted === true) {
  await denyAccess("Учетная запись отключена. Обратитесь к администратору.");
  return;
}
const role = profileData?.role || "buyer";
if (!["admin", "manager"].includes(role)) {
  await denyAccess("Недостаточно прав для входа в web-панель.");
  return;
}

state.user = {
  uid: user.uid,
  email: user.email,
  displayName: profileData.displayName || user.displayName || "Пользователь",
};
state.role = role;
currentUserName.textContent = state.user.displayName;
currentUserRole.textContent = roleLabels[role];
loginView.classList.remove("active");
adminView.classList.add("active");
buildNavigation();
await loadAllData();
if (run !== authRun) return;
renderCurrentSection();
} catch (error) {
  console.error(error);
  await denyAccess(friendlyError(error));
}
});

async function denyAccess(message) {
  await signOut(auth);
  loginError.textContent = t(message);
  showToast(message, "error");
}

function resetSession() {
  state.user = null;

```



```

state.role = null;
state.section = "dashboard";
Object.assign(state, {
  products: [],
  categories: [],
  manufacturers: [],
  suppliers: [],
  promoCodes: [],
  users: [],
  orders: [],
  productSearch: "",
  showDisabledUsers: false,
});
closeModal();
document.querySelectorAll(".panel-section").forEach((section) => {
  section.innerHTML = "";
});
loginForm.reset();
loginView.classList.add("active");
adminView.classList.remove("active");
}

async function handleLogin(event) {
  event.preventDefault();
  const submitButton = loginForm.querySelector("button[type=submit]");
  loginError.textContent = "";
  const email = document.getElementById("login-email").value.trim();
  const password = document.getElementById("login-password").value;
  if (!email || !password) {
    loginError.textContent = t("Введите email и пароль.");
    return;
  }

  submitButton.disabled = true;
  try {
    await signInWithEmailAndPassword(auth, email, password);
  } catch (error) {
    const wrongCredentials = [
      "auth/invalid-credential",
      "auth/wrong-password",
      "auth/user-not-found",
    ];

```

```

    "auth/invalid-email",
  ],includes(error.code);
  const message = wrongCredentials ? "Не удалось выполнить вход. Проверьте email и пароль." :
friendlyError(error);
  loginError.textContent = t(message);
} finally {
  submitButton.disabled = false;
}
}

```

// ----- Навигация и загрузка -----

```

function buildNavigation() {
  navTabs.innerHTML = "";
  Object.keys(sectionLabels).forEach((section) => {
    const button = document.createElement("button");
    button.type = "button";
    button.textContent = sectionLabels[section];
    button.className = state.section === section ? "active" : "";
    if (state.section === section) button.setAttribute("aria-current", "page");
    button.addEventListener("click", () => {
      state.section = section;
      buildNavigation();
      renderCurrentSection();
    });
    navTabs.appendChild(button);
  });
}

```

```

async function loadAllData() {
  const [products, categories, manufacturers, suppliers, promoCodes, users, orders] = await Promise.all([
    readCollection("products"),
    readCollection("categories"),
    readCollection("manufacturers"),
    readCollection("suppliers"),
    readCollection("promoCodes"),
    readCollection("users"),
    readCollection("orders"),
  ]);

```

```

state.products = products;

```

```

state.categories = sortByName(categories);
state.manufacturers = sortByName(manufacturers);
state.suppliers = sortByName(suppliers);
state.promoCodes = promoCodes.sort((left, right) => String(left.code || "").localeCompare(String(right.code || "")));
state.users = users;
state.orders = orders.sort((left, right) => orderTime(right) - orderTime(left));
}

function sortByName(items) {
  return items.sort((left, right) => String(left.name || "").localeCompare(String(right.name || ""), "ru"));
}

async function readCollection(collectionName) {
  const snapshot = await getDocs(collection(db, collectionName));
  return snapshot.docs.map((documentSnapshot) => ({
    id: documentSnapshot.id,
    ...documentSnapshot.data(),
  }));
}

async function reloadAndRender() {
  await loadAllData();
  renderCurrentSection();
}

const activeUsers = () => state.users.filter((user) => user.isDeleted !== true);

function renderCurrentSection() {
  document.querySelectorAll(".panel-section").forEach((sectionElement) => {
    sectionElement.classList.add("hidden");
  });
  const currentSection = document.getElementById(`${state.section}-section`);
  if (!currentSection) return;
  currentSection.classList.remove("hidden");

  if (state.section === "dashboard") renderDashboard(currentSection);
  if (state.section === "products") renderProducts(currentSection);
  if (state.section === "orders") renderOrders(currentSection);
  if (state.section === "users") renderUsers(currentSection);
  if (["categories", "manufacturers", "suppliers", "promoCodes"].includes(state.section)) {

```

```

    renderGenericCollection(currentSection, state.section);
  }
}

// ----- Дашборд -----

function renderDashboard(container) {
  const dashboard = computeDashboard({
    users: activeUsers(),
    products: state.products,
    orders: state.orders,
    catalogLabel: (product) => localizedField(product),
  });

  container.innerHTML = `
    <div class="section-header">
      <div>
        <h2>Дашборд</h2>
        <p class="muted">Мониторинг клиентов, заказов и товаров магазина.</p>
      </div>
      <div class="section-actions">
        ${isAdmin() ? `<button class="ghost-button" type="button" id="sync-translations-
button">Дополнить английские переводы каталога</button>` : ""}
      </div>
    </div>
    <div class="metrics-grid">
      <article class="metric-card">
        <div class="metric-card__label">Покупатели</div>
        <div class="metric-card__value">${dashboard.buyersCount}</div>
      </article>
      <article class="metric-card">
        <div class="metric-card__label">Активные заказы</div>
        <div class="metric-card__value">${dashboard.activeOrdersCount}</div>
      </article>
      <article class="metric-card">
        <div class="metric-card__label">Товары с низким остатком</div>
        <div class="metric-card__value">${dashboard.lowStockCount}</div>
      </article>
      <article class="metric-card">
        <div class="metric-card__label">Общая выручка</div>
        <div class="metric-card__value">₽${dashboard.totalSales.toFixed(2)}</div>
      </article>
    </div>
  `;
}

```

```

</article>
</div>
<div class="stats-grid">
  <article class="chart-card">
    <h3>Продажи по месяцам</h3>
    <div class="chart-bars">
      ${renderBars(dashboard.monthlySales)}
    </div>
  </article>
  <article class="chart-card">
    <h3>ТОП-5 товаров</h3>
    <div class="chart-bars">
      ${renderBars(dashboard.topProducts)}
    </div>
  </article>
</div>
`;

```

```

const syncButton = document.getElementById("sync-translations-button");
syncButton?.addEventListener("click", () => runGuarded(syncCatalogTranslations, { button:
syncButton }));
}

```

```

async function commitInChunks(operations) {
  const chunkSize = 400;
  for (let start = 0; start < operations.length; start += chunkSize) {
    const batch = writeBatch(db);
    operations.slice(start, start + chunkSize).forEach((apply) => apply(batch));
    await batch.commit();
  }
}

```

```

async function syncCatalogTranslations() {
  const operations = [];

  state.categories.forEach((category) => {
    const translation = categoryTranslations[category.id];
    if (translation && !category.nameEn) {
      operations.push((batch) =>
        batch.update(doc(db, "categories", category.id), {
          nameEn: translation.name,

```

```

        descriptionEn: translation.description,
        updatedAt: serverTimestamp(),
      })
    );
  }
});

state.products.forEach((product) => {
  const translation = productTranslations[product.id];
  if (translation && !product.nameEn) {
    operations.push((batch) =>
      batch.update(doc(db, "products", product.id), {
        nameEn: translation.name,
        descriptionEn: translation.description,
        compositionEn: translation.composition,
        categoryNameEn: categoryTranslations[product.categoryId]?.name || "",
        updatedAt: serverTimestamp(),
      })
    );
  }
});

if (operations.length > 0) {
  await commitInChunks(operations);
  await reloadAndRender();
}

showToast(operations.length > 0 ? `Обновлено записей: ${operations.length}` : "Переводы уже
загружены.", "success");
}

function renderBars(entries) {
  if (!entries.length) {
    return `<p class="muted">Пока недостаточно данных.</p>`;
  }

  const maxValue = Math.max(...entries.map((entry) => Number(entry[1] || 0)), 1);
  return entries
    .map(([label, value]) => {
      const width = Math.max(8, (Number(value) / maxValue) * 100);
      return `
        <div class="bar-row">

```

```

    <span>${esc(label)}</span>
    <div class="bar-track">
      <div class="bar-fill" style="width:${width}%"></div>
    </div>
    <strong>${Number(value).toFixed(0)}</strong>
  </div>
  `;
})
.join("");
}

// ----- Товары -----

function renderProducts(container) {
  container.innerHTML = `
    <div class="section-header">
      <div>
        <h2>Товары</h2>
        <p class="muted">Создание, редактирование и мягкое удаление товаров.</p>
      </div>
      <div class="section-actions">
        <button class="primary-button" type="button" id="add-product-button">Добавить товар</button>
      </div>
      <div class="table-card">
        <div class="toolbar">
          <input id="product-search" type="search" placeholder="Поиск по названию товара" value="${
            {esc(state.productSearch)}}" />
        </div>
        <table>
          <thead>
            <tr>
              <th>Название</th>
              <th>Категория</th>
              <th>Цена</th>
              <th>Остаток</th>
              <th>Статус</th>
              <th>Действия</th>
            </tr>
          </thead>
          <tbody id="products-body">${renderProductRows(filterProducts())}</tbody>

```

```

    </table>
  </div>
`
`;

document.getElementById("add-product-button").addEventListener("click", () => openProductModal());
document.getElementById("product-search").addEventListener("input", (event) => {
  state.productSearch = event.target.value;
  document.getElementById("products-body").innerHTML = renderProductRows(filterProducts());
  bindProductActions();
});
bindProductActions();
}

function filterProducts() {
  const query = state.productSearch.trim().toLowerCase();
  if (!query) return state.products;
  return state.products.filter((product) => `${product.name} || ""} ${product.nameEn} || ""}`.toLowerCase().includes(query));
}

function categoryLabel(product) {
  const category = state.categories.find((item) => item.id === product.categoryId);
  return (category ? localizedField(category) : product.categoryName) || "-";
}

function renderProductRows(products) {
  if (!products.length) {
    return `<tr><td colspan="6" class="muted">Товары не найдены.</td></tr>`;
  }
  return products
    .map((product) => {
      const statusBadge = product.isDeleted
        ? `<span class="badge warning">Скрыт</span>`
        : `<span class="badge">Активен</span>`;
      const toggleButton = product.isDeleted
        ? `<button class="ghost-button" type="button" data-restore-product="${
          { esc(product.id) } }">Восстановить</button>`
        : `<button class="danger-button" type="button"
data-delete-product="${
          { esc(product.id) } }">Удалить</button>`;
      return `
    <tr>

```



```

        <td>${esc(localizedField(product) || "-")}</td>
        <td>${esc(categoryLabel(product))}</td>
        <td>₽${Number(product.price || 0).toFixed(2)}</td>
        <td>${Number(product.stockQuantity || 0)}</td>
        <td>${statusBadge}</td>
        <td>
            <div class="table-actions">
                <button class="ghost-button" type="button" data-edit-product="$
{esc(product.id)}">Редактировать</button>
                ${toggleButton}
            </div>
        </td>
    </tr>
</tr>
`;
})
.join("");
}

function bindProductActions() {
    document.querySelectorAll("[data-edit-product]").forEach((button) => {
        button.addEventListener("click", () => {
            const product = state.products.find((item) => item.id === button.dataset.editProduct);
            if (product) openProductModal(product);
        });
    });

    document.querySelectorAll("[data-delete-product]").forEach((button) => {
        button.addEventListener("click", () => {
            if (!confirmAction("Скрыть товар из каталога? Его можно будет восстановить.")) return;
            setProductHidden(button, button.dataset.deleteProduct, true, "Товар скрыт из каталога.");
        });
    });

    document.querySelectorAll("[data-restore-product]").forEach((button) => {
        button.addEventListener("click", () => {
            setProductHidden(button, button.dataset.restoreProduct, false, "Товар снова доступен в каталоге.");
        });
    });
}

function setProductHidden(button, productId, isDeleted, successMessage) {

```

```

return runGuarded(
  async () => {
    await updateDoc(doc(db, "products", productId), { isDeleted, updatedAt: serverTimestamp() });
    await reloadAndRender();
  },
  { button, success: successMessage }
);
}

// ----- Справочники -----

function renderGenericCollection(container, section) {
  const config = collectionConfigs[section];
  const items = state[section];
  const canEdit = !(section === "suppliers" && !isAdmin());
  const title = config.title;

  container.innerHTML = `
    <div class="section-header">
      <div>
        <h2>${title}</h2>
        <p class="muted">Справочник коллекции ${title.toLowerCase()}</p>
      </div>
      <div class="section-actions">
        ${canEdit ? `<button class="primary-button" type="button"
id="add-${section}-button">Добавить</button>` : ""}
      </div>
    </div>
    <div class="table-card">
      <table>
        <thead>
          <tr>
            ${config.fields.map((field) => `<th>${field.label}</th>`).join("")}
            <th>Действия</th>
          </tr>
        </thead>
        <tbody>
          ${
            items.length
              ? items.map((item) => renderGenericRow(section, config, item, canEdit)).join("")
              : `<tr><td colspan="${config.fields.length + 1}" class="muted">Записей пока нет.</td></tr>`
          }
        </tbody>
      </table>
    </div>
  `
}

```

```

    }
  </tbody>
</table>
</div>
`;

document.getElementById(`add-${section}-button`).addEventListener("click", () =>
openGenericModal(section));

container.querySelectorAll("[data-edit-entity]").forEach((button) => {
  button.addEventListener("click", () => {
    const item = state[button.dataset.editEntity].find((entry) => entry.id === button.dataset.id);
    if (item) openGenericModal(button.dataset.editEntity, item);
  });
});

container.querySelectorAll("[data-delete-entity]").forEach((button) => {
  button.addEventListener("click", () => {
    if (!confirmAction("Удалить запись? Это действие нельзя отменить.")) return;
    runGuarded(
      async () => {
        await deleteEntity(button.dataset.deleteEntity, button.dataset.id);
        await reloadAndRender();
      },
      { button, success: "Запись удалена." }
    );
  });
});
}

function renderGenericRow(section, config, item, canEdit) {
  const cells = config.fields
    .map((field) => {
      let value = item[field.key] || "-";
      if (field.type === "date") {
        value = item[field.key]?.seconds ? new Date(item[field.key].seconds * 1000).toLocaleDateString("ru-RU") : "-";
      }
      if (field.key === "discountPercent") value = `${item[field.key] ?? "-"}%`;
      return `<td>${esc(value)}</td>`;
    })
}

```

```

.join("");

const actions = canEdit
? `

262


```

```

    await deleteDoc(doc(db, collectionConfigs[section].collectionName, itemId));
  }

function openGenericModal(section, existingItem = null) {
  const config = collectionConfigs[section];
  const fieldsMarkup = config.fields
    .map((field) => {
      const rawValue = existingItem?.[field.key];
      let value = rawValue ?? "";
      if (field.type === "date") value = toDateInputValue(rawValue);
      const attributes = [
        `data-field="${field.key}"`,
        field.required ? "required" : "",
        field.maxLength ? `maxlength="${field.maxLength}"` : "",
        field.min !== undefined ? `min="${field.min}"` : "",
        field.max !== undefined ? `max="${field.max}"` : "",
        field.step !== undefined ? `step="${field.step}"` : "",
      ]
      .filter(Boolean)
      .join(" ");
      if (field.multiline) {
        return `
        <label class="full">
          <span>${field.label}</span>
          <textarea ${attributes}>${esc(value)}</textarea>
        </label>
        `;
      }
      return `
        <label>
          <span>${field.label}</span>
          <input ${attributes} type="${field.type || "text"}" value="${esc(value)}" />
        </label>
        `;
    })
    .join("");

  openModal({
    title: existingItem ? `Редактировать: ${config.title}` : `Добавить: ${config.title}`,
    body: `<div class="modal-grid">${fieldsMarkup}</div>`,
  });
}

```

```

onSubmit: async (modalElement) => {
  const values = {};
  config.fields.forEach((field) => {
    values[field.key] = modalElement.querySelector(`[data-field="${field.key}"]`).value.trim();
  });

  const error = validateEntityForm(section, values);
  if (error) throw new UserError(error);
  assertUnique(section, values, existingItem);

  const payload = buildEntityPayload(section, values, existingItem);
  payload.updatedAt = serverTimestamp();
  payload.createdAt = existingItem?.createdAt || serverTimestamp();

  await setDoc(doc(db, config.collectionName, existingItem?.id || doc(collection(db,
config.collectionName)).id), payload, {
    merge: true,
  });
  if (existingItem) await propagateRename(section, existingItem, payload);
  await reloadAndRender();
  showToast("Изменения сохранены.", "success");
},
});
}

function buildEntityPayload(section, values, existingItem) {
  switch (section) {
    case "categories":
      return {
        name: values.name,
        nameNormalized: values.name.toLowerCase(),
        nameEn: values.nameEn,
        description: values.description,
      };
    case "manufacturers":
      return {
        name: values.name,
        nameNormalized: values.name.toLowerCase(),
        country: values.country,
        contactPhone: values.contactPhone,
      };
  }
}

```

```

case "suppliers":
  return {
    name: values.name,
    nameNormalized: values.name.toLowerCase(),
    taxId: values.taxId,
    address: values.address,
    contactPhone: values.contactPhone,
    email: values.email.toLowerCase(),
  };
case "promoCodes": {
  const code = values.code.toUpperCase();
  return {
    code,
    codeNormalized: code,
    discountPercent: Number(values.discountPercent),
    expiresAt: endOfDay(values.expiresAt),
    isActive: existingItem?.isActive ?? true,
    usageCount: existingItem?.usageCount ?? 0,
  };
}
default:
  return { ...values };
}
}

const uniqueMessages = {
  categories: "Категория с таким названием уже существует.",
  manufacturers: "Производитель с таким названием уже существует.",
  suppliers: "Поставщик с таким названием уже существует.",
  promoCodes: "Промокод с таким кодом уже существует.",
};

function assertUnique(section, values, existingItem) {
  const key = section === "promoCodes" ? "code" : "name";
  const wanted = values[key].trim().toLowerCase();
  const duplicate = state[section].some(
    (item) => item.id !== existingItem?.id && String(item[key] || "").trim().toLowerCase() === wanted
  );
  if (duplicate) throw new UserError(uniqueMessages[section]);
}

```

```

/** Название справочника дублируется в товарах: при переименовании обновляем и их. */
async function propagateRename(section, existingItem, payload) {
  const rename = {
    categories: { idField: "categoryId", changes: { categoryName: payload.name, categoryNameEn:
payload.nameEn || "" } },
    manufacturers: { idField: "manufacturerId", changes: { manufacturerName: payload.name } },
    suppliers: { idField: "supplierId", changes: { supplierName: payload.name } },
  }[section];
  if (!rename || existingItem.name === payload.name) return;

  const operations = state.products
    .filter((product) => product[rename.idField] === existingItem.id)
    .map((product) => (batch) =>
      batch.update(doc(db, "products", product.id), { ...rename.changes, updatedAt: serverTimestamp() })
    );
  await commitInChunks(operations);
}

// ----- Заказы -----

function courierOptionsFor(order) {
  const couriers = activeUsers().filter((user) => user.role === "courier");
  const options = couriers.map((user) => ({ id: user.id, label: userLabel(user) }));
  if (order.courierId && !options.some((option) => option.id === order.courierId)) {
    options.push({ id: order.courierId, label: order.courierName || order.courierId });
  }
  return options
    .map(
      (option) =>
        `<option value="${esc(option.id)}" ${option.id === order.courierId ? "selected" : ""}>${
        {esc(option.label)}</option>`
    )
    .join("");
}

function renderOrders(container) {
  const statusOptions = (order) =>
    Object.entries(statusNames.ru)
      .map(([id, name]) => `<option value="${id}" ${order.statusId === id ? "selected" :
      ""}>${name}</option>`)
      .join("");
}

```



```

container.innerHTML = `
<div class="section-header">
  <div>
    <h2>Заказы</h2>
    <p class="muted">Смена статуса и назначение курьера.</p>
  </div>
</div>
<div class="table-card">
  <table>
    <thead>
      <tr>
        <th>Номер</th>
        <th>Клиент</th>
        <th>Сумма</th>
        <th>Статус</th>
        <th>Курьер</th>
        <th>Действия</th>
      </tr>
    </thead>
    <tbody>
      ${
        state.orders.length
        ? state.orders
          .map((order) => {
            const locked = isFinishedStatus(order.statusId);
            return `
          <tr>
            <td>${esc(order.number || order.id)}</td>
            <td>${esc(order.customerName || order.customerEmail || "-")}</td>
            <td>₽${Number(order.totalAmount || 0).toFixed(2)}</td>
            <td>
              <select data-order-status="${esc(order.id)}" ${locked ? "disabled" : ""}>${
                statusOptions(order)
              }</select>
            </td>
            <td>
              <select data-order-courier="${esc(order.id)}" ${locked ? "disabled" : ""}>
                <option value="">Не назначен</option>
                ${courierOptionsFor(order)}
              </select>
            </td>
          `;
          })
        : ''
      }
    </tbody>
  </table>
</div>
`

```

```

        <td>
          ${
            locked
            ? `<span class="muted">Заказ завершен</span>`
            : `<button class="primary-button" type="button" data-save-order="$
{esc(order.id)}">Сохранить</button>`
          }
        </td>
      </tr>
    `;
  })
  .join("")
  : `<tr><td colspan="6" class="muted">Заказов пока нет.</td></tr>`
}
</tbody>
</table>
</div>
`;

```

```

container.querySelectorAll("[data-save-order]").forEach((button) => {
  button.addEventListener("click", () => {
    const orderId = button.dataset.saveOrder;
    const statusId = container.querySelector(`[data-order-status="${orderId}"]`).value;
    const courierId = container.querySelector(`[data-order-courier="${orderId}"]`).value;
    if (statusId === "cancelled" && !confirmAction("Отменить заказ? Товары вернутся на склад, статус
нельзя будет изменить.")) return;
    runGuarded(() => saveOrder(orderId, { statusId, courierId }, { button }));
  });
});
}

```

/**

- * Сохраняет заказ в транзакции: статус, курьер, уведомление покупателю и (при отмене)
- * возврат товаров на склад. Устаревшие данные на экране не могут перезаписать чужие
- * изменения, а остатки не вернутся дважды.

*/

```

async function saveOrder(orderId, { statusId, courierId }) {
  const orderRef = doc(db, "orders", orderId);
  const courier = activeUsers().find((user) => user.id === courierId);
  let outcome = "saved";

```

```

await runTransaction(db, async (transaction) => {
  const orderSnapshot = await transaction.get(orderRef);
  if (!orderSnapshot.exists()) throw new UserError("Заказ не найден. Обновите данные.");
  const order = { id: orderSnapshot.id, ...orderSnapshot.data() };

  const error = validateOrderChange(order, { statusId, courierId });
  if (error) throw new UserError(error);

  const statusChanged = order.statusId !== statusId;
  const courierChanged = (order.courierId || null) !== (courierId || null);
  if (!statusChanged && !courierChanged) {
    outcome = "unchanged";
    return;
  }

  const restorations = statusChanged && statusId === "cancelled" ? stockRestorations(order) : [];
  const productRefs = restorations.map(({ id }) => doc(db, "products", id));
  const productSnapshots = await Promise.all(productRefs.map((ref) => transaction.get(ref)));

  if (courierChanged && courierId && !courier) throw new UserError("Курьер не найден или отключен. Обновите данные.");

  const courierUnchanged = !courierChanged;
  transaction.update(orderRef, {
    statusId,
    statusName: getStatusName(statusId),
    courierId: courierId || null,
    courierName: courierId ? (courier ? userLabel(courier) : courierUnchanged ? order.courierName || null : null) : null,
    courierPhone: courierId ? (courier ? courier.phoneNumber || null : courierUnchanged ? order.courierPhone || null : null) : null,
    statusChangedAt: statusChanged ? serverTimestamp() : order.statusChangedAt || null,
    statusChangedBy: statusChanged ? auth.currentUser.uid : order.statusChangedBy || null,
    updatedAt: serverTimestamp(),
  });

  restorations.forEach(({ quantity }, index) => {
    if (productSnapshots[index].exists()) {
      transaction.update(productRefs[index], { stockQuantity: increment(quantity), updatedAt: serverTimestamp() });
    }
  })
}

```

```

});

if (courierChanged && courierId) {
  const number = order.number || orderId;
  transaction.set(doc(collection(db, "notifications")), {
    userId: courierId,
    text: `Вам назначен заказ №${number}`,
    textEn: `Order #${number} has been assigned to you`,
    createdAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  });
}

if (statusChanged && order.userId) {
  const number = order.number || orderId;
  transaction.set(doc(collection(db, "notifications")), {
    userId: order.userId,
    text: `Заказ №${number}: статус изменен на «${getStatusName(statusId)}»,`,
    textEn: `Order #${number}: status changed to “${getStatusName(statusId, "en")}”`,
    createdAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  });
}

}

});

await reloadAndRender();
showToast(outcome === "unchanged" ? "Изменений нет." : "Заказ обновлен.", "success");
}

// ----- Пользователи -----

function hasActiveOrders(user) {
  const field = user.role === "courier" ? "courierId" : "userId";
  return state.orders.some((order) => order[field] === user.id && !isFinishedStatus(order.statusId));
}

function renderUsers(container) {
  const canManage = isAdmin();
  const users = state.users
    .filter((user) => state.showDisabledUsers || user.isDeleted !== true)
    .sort((left, right) => String(left.displayName || "").localeCompare(String(right.displayName || ""), "ru"));

```

```

const roleCell = (user) => {
  const role = normalizeRole(user.role);
  if (!canManage || user.id === state.user.uid) return esc(roleLabels[role] || role);
  const known = Object.keys(roleLabels);
  const options = (known.includes(role) ? known : [...known, role])
    .map((id) => `<option value="${esc(id)}" ${role === id ? "selected" : ""}>${esc(roleLabels[id] ||
id)}</option>`)
    .join("");
  return `<select data-user-role="${esc(user.id)}">${options}</select>`;
};

const actionCell = (user) => {
  if (!canManage) return `<span class="muted">Только просмотр</span>`;
  if (user.id === state.user.uid) return `<span class="muted">Это вы</span>`;
  if (user.isDeleted === true) {
    return `<button class="ghost-button" type="button" data-restore-user="${
esc(user.id)}">Восстановить</button>`;
  }
  return `<div class="table-actions">
    <button class="primary-button" type="button" data-save-user="${esc(user.id)}">Сохранить
роль</button>
    <button class="danger-button" type="button" data-delete-user="${esc(user.id)}">Отключить</button>
  </div>`;
};

container.innerHTML = `
<div class="section-header">
  <div>
    <h2>Пользователи</h2>
    <p class="muted">Просмотр карточек пользователей и смена ролей.</p>
  </div>
  ${
    canManage
      ? `<div class="section-actions">
        <label class="inline-check"><input type="checkbox" id="show-disabled-users" $
{state.showDisabledUsers ? "checked" : ""} /> <span>Показывать отключенных</span></label>
      </div>`
      : ""
  }
</div>

```

```

<div class="table-card">
  <table>
    <thead>
      <tr>
        <th>Имя</th>
        <th>Логин</th>
        <th>Email</th>
        <th>Телефон</th>
        <th>Роль</th>
        <th>Действия</th>
      </tr>
    </thead>
    <tbody>
      ${
        users.length
        ? users
          .map(
            (user) => `
              <tr class="${user.isDeleted === true ? "row-disabled" : ""}">
                <td>${esc(user.displayName || "-")}</td>
                <td>${esc(user.login || "-")}</td>
                <td>${esc(user.email || "-")}</td>
                <td>${esc(user.phoneNumber || "-")}</td>
                <td>${roleCell(user)}</td>
                <td>${actionCell(user)}</td>
              </tr>`
            )
          .join("")
        : `<tr><td colspan="6" class="muted">Пользователей нет.</td></tr>`
      }
    </tbody>
  </table>
</div>
`;

```

```

document.getElementById("show-disabled-users")?.addEventListener("change", (event) => {
  state.showDisabledUsers = event.target.checked;
  renderUsers(container);
});

```

```

container.querySelectorAll("[data-save-user]").forEach((button) => {

```

```

button.addEventListener("click", () => {
  const user = state.users.find((item) => item.id === button.dataset.saveUser);
  const select = container.querySelector(`[data-user-role="${button.dataset.saveUser}"]`);
  runGuarded(() => changeUserRole(user, select.value), { button });
});
});

container.querySelectorAll("[data-delete-user]").forEach((button) => {
  button.addEventListener("click", () => {
    const user = state.users.find((item) => item.id === button.dataset.deleteUser);
    if (!confirmAction("Отключить пользователя? Он не сможет войти в приложение.")) return;
    runGuarded(() => disableUser(user), { button });
  });
});

container.querySelectorAll("[data-restore-user]").forEach((button) => {
  button.addEventListener("click", () => {
    runGuarded(
      async () => {
        await updateDoc(doc(db, "users", button.dataset.restoreUser), {
          isDeleted: false,
          deletedBy: deleteField(),
          deletedAt: deleteField(),
          updatedAt: serverTimestamp(),
        });
        await reloadAndRender();
      },
      { button, success: "Пользователь восстановлен." }
    );
  });
});
}

function otherActiveAdmins(user) {
  return activeUsers().filter((item) => item.role === "admin" && item.id !== user.id).length;
}

async function changeUserRole(user, role) {
  if (!user) throw new UserError("Пользователь не найден. Обновите данные.");
  if (user.id === state.user.uid) throw new UserError("Нельзя изменить собственную роль.");
  if (normalizeRole(user.role) === role) {

```

```

    showToast("Изменений нет.", "success");
    return;
  }
  if (user.role === "admin" && otherActiveAdmins(user) === 0) {
    throw new UserError("Нельзя снять роль с последнего администратора.");
  }
  if (hasActiveOrders(user)) {
    throw new UserError("У пользователя есть незавершенные заказы: смена роли сейчас невозможна.");
  }
  await updateDoc(doc(db, "users", user.id), { role, updatedAt: serverTimestamp() });
  await reloadAndRender();
  showToast("Роль пользователя обновлена.", "success");
}

async function disableUser(user) {
  if (!user) throw new UserError("Пользователь не найден. Обновите данные.");
  if (user.id === state.user.uid) throw new UserError("Нельзя отключить собственную учетную запись.");
  if (user.role === "admin" && otherActiveAdmins(user) === 0) {
    throw new UserError("Нельзя отключить последнего администратора.");
  }
  if (hasActiveOrders(user)) {
    throw new UserError("У пользователя есть незавершенные заказы: отключение сейчас
невозможно.");
  }
  await updateDoc(doc(db, "users", user.id), {
    isDeleted: true,
    deletedBy: auth.currentUser.uid,
    deletedAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  });
  await reloadAndRender();
  showToast("Пользователь отключен.", "success");
}

// ----- Модальные окна -----

function openProductModal(existingProduct = null) {
  const selectOptions = (items, selectedId, labelOf) =>
    items
      .map((item) => `<option value="${esc(item.id)}" ${selectedId === item.id ? "selected" : ""}>$
{esc(labelOf(item))}</option>`)

```



```

.join("");

const categoryOptions = selectOptions(state.categories, existingProduct?.categoryId, localizedField);
const manufacturerOptions = selectOptions(state.manufacturers, existingProduct?.manufacturerId, (item)
=> item.name);
const supplierOptions = `<option value="">Не выбран</option>${selectOptions(state.suppliers,
existingProduct?.supplierId, (item) => item.name)}`;
const stock = existingProduct?.stockQuantity ?? 0;

openModal({
  title: existingProduct ? "Редактировать товар" : "Добавить товар",
  body: `
    <div class="modal-grid">
      <label><span>Название</span><input data-field="name" maxlength="255" value="$
{esc(existingProduct?.name)}" required /></label>
      <label><span>Цена</span><input data-field="price" type="number" min="0.01" max="1000000"
step="0.01" value="${esc(existingProduct?.price)}" required /></label>
      <label><span>Количество</span><input data-field="stockQuantity" type="number" min="0"
max="9999" step="1" value="${esc(stock)}" required /></label>
      <label><span>Единица</span><input data-field="qty" maxlength="50" value="$
{esc(existingProduct?.qty || "1 шт")}" required /></label>
      <label><span>Категория</span><select data-field="categoryId"
required>${categoryOptions}</select></label>
      <label><span>Производитель</span><select data-field="manufacturerId" required>${
manufacturerOptions}</select></label>
      <label><span>Поставщик</span><select data-field="supplierId">${supplierOptions}</select></label>
      <label><span>Ссылка на изображение</span><input data-field="imageUrl" type="url"
maxlength="2000" value="${esc(existingProduct?.imageUrl)}" /></label>
      <label><span>Страна происхождения</span><input data-field="country" maxlength="100" value="$
{esc(existingProduct?.country)}" /></label>
      <label><span>Срок годности до</span><input data-field="expiryDate" type="date" value="$
{esc(toDateInputValue(existingProduct?.expiryDate))}" /></label>
      <label><span>Калорийность, ккал / 100 г</span><input data-field="calories" type="number"
min="0" max="10000" step="0.1" value="${esc(existingProduct?.calories ?? 0)}" /></label>
      <label><span>Белки, г</span><input data-field="proteins" type="number" min="0" max="10000"
step="0.1" value="${esc(existingProduct?.proteins ?? 0)}" /></label>
      <label><span>Жиры, г</span><input data-field="fats" type="number" min="0" max="10000"
step="0.1" value="${esc(existingProduct?.fats ?? 0)}" /></label>
      <label><span>Углеводы, г</span><input data-field="carbs" type="number" min="0" max="10000"
step="0.1" value="${esc(existingProduct?.carbs ?? 0)}" /></label>

```

```

    <label class="full"><span>Состав</span><textarea data-field="composition" maxlength="2000">${
    {esc(existingProduct?.composition)}}</textarea></label>

    <label class="full"><span>Описание</span><textarea data-field="description" maxlength="2000">${
    {esc(existingProduct?.description)}}</textarea></label>

    <label><span>Название (English)</span><input data-field="nameEn" maxlength="255" value="${
    {esc(existingProduct?.nameEn)}}" /></label>

    <label class="full"><span>Состав (English)</span><textarea data-field="compositionEn"
    maxlength="2000">${{esc(existingProduct?.compositionEn)}}</textarea></label>

    <label class="full"><span>Описание (English)</span><textarea data-field="descriptionEn"
    maxlength="2000">${{esc(existingProduct?.descriptionEn)}}</textarea></label>
  </div>
  `,
  onSubmit: async (modalElement) => {
    const readField = (key) => modalElement.querySelector(`[data-field="${key}"]`).value.trim();
    const values = Object.fromEntries(
      [
        "name", "price", "stockQuantity", "qty", "categoryId", "manufacturerId", "supplierId", "imageUrl",
        "country",
        "expiryDate", "calories", "proteins", "fats", "carbs", "composition", "description", "nameEn",
        "compositionEn", "descriptionEn",
      ].map((key) => [key, readField(key)])
    );

    const error = validateProductForm(values);
    if (error) throw new UserError(error);

    const category = state.categories.find((item) => item.id === values.categoryId);
    const manufacturer = state.manufacturers.find((item) => item.id === values.manufacturerId);
    const supplier = state.suppliers.find((item) => item.id === values.supplierId);
    if (!category) throw new UserError("Выберите категорию товара.");
    if (!manufacturer) throw new UserError("Выберите производителя товара.");

    const payload = {
      name: values.name,
      nameNormalized: values.name.toLowerCase(),
      nameEn: values.nameEn,
      price: Number(values.price.replace(",", ".")),
      stockQuantity: Number(values.stockQuantity),
      qty: values.qty,
      description: values.description,
      descriptionEn: values.descriptionEn,
    };
  },

```

```

    composition: values.composition,
    compositionEn: values.compositionEn,
    country: values.country,
    imageUrl: values.imageUrl,
    expiryDate: values.expiryDate ? endOfDay(values.expiryDate) : null,
    calories: Number(values.calories.replace(",", ".") || 0),
    proteins: Number(values.proteins.replace(",", ".") || 0),
    fats: Number(values.fats.replace(",", ".") || 0),
    carbs: Number(values.carbs.replace(",", ".") || 0),
    categoryId: category.id,
    categoryName: category.name || "",
    categoryNameEn: category.nameEn || "",
    manufacturerId: manufacturer.id,
    manufacturerName: manufacturer.name || "",
    supplierId: supplier?.id || null,
    supplierName: supplier?.name || null,
    isDeleted: existingProduct?.isDeleted || false,
    updatedAt: serverTimestamp(),
    createdAt: existingProduct?.createdAt || serverTimestamp(),
  };
  if (!existingProduct) {
    payload.minStock = 5;
    payload.popularity = 0;
  }

  await setDoc(doc(db, "products", existingProduct?.id || doc(collection(db, "products")).id), payload,
{ merge: true });
  await reloadAndRender();
  showToast("Товар сохранен.", "success");
},
});
}

function openChangePasswordModal() {
  openModal({
    title: "Смена пароля",
    body: `
    <div class="modal-grid">
      <label class="full"><span>Текущий пароль</span><input data-field="currentPassword"
type="password" autocomplete="current-password" required /></label>

```

```

    <label class="full"><span>Новый пароль (минимум 6 символов, буквы и цифры)</span><input
data-field="newPassword" type="password" autocomplete="new-password" minlength="6"
maxlength="128" required /></label>

```

```

    <label class="full"><span>Повторите новый пароль</span><input data-field="confirmPassword"
type="password" autocomplete="new-password" required /></label>

```

```

</div>

```

```

`

```

```

onSubmit: async (modalElement) => {

```

```

    const readField = (key) => modalElement.querySelector(`[data-field="${key}"]`).value;

```

```

    const currentPassword = readField("currentPassword");

```

```

    const newPassword = readField("newPassword");

```

```

    const error = validateNewPassword(newPassword, currentPassword, readField("confirmPassword"));

```

```

    if (error) throw new UserError(error);

```

```

    try {

```

```

        const credential = EmailAuthProvider.credential(auth.currentUser.email, currentPassword);

```

```

        await reauthenticateWithCredential(auth.currentUser, credential);

```

```

        await updatePassword(auth.currentUser, newPassword);

```

```

    } catch (authError) {

```

```

        if (["auth/wrong-password", "auth/invalid-credential"].includes(authError.code)) {

```

```

            throw new UserError("Неверный текущий пароль.");

```

```

        }

```

```

        throw authError;

```

```

    }

```

```

    showToast("Пароль успешно изменен.", "success");

```

```

},

```

```

});

```

```

}

```

```

let closeActiveModal = () => {};

```

```

function openModal({ title, body, onSubmit }) {

```

```

    closeModal();

```

```

    modalRoot.innerHTML = `

```

```

    <div class="modal-backdrop">

```

```

        <div class="modal-card" role="dialog" aria-modal="true" aria-labelledby="modal-title">

```

```

            <h3 id="modal-title">${title}</h3>

```

```

            <div class="muted" style="margin: 8px 0 20px;">Все изменения сохраняются прямо в Cloud
Firestore.</div>

```

```

            <form id="modal-form">

```

```

    ${body}
    <div class="modal-actions">
      <button type="button" class="ghost-button" id="modal-cancel">Отмена</button>
      <button type="submit" class="primary-button">Сохранить</button>
    </div>
  </form>
</div>
</div>
`;

```

```

const modalForm = document.getElementById("modal-form");
const submitButton = modalForm.querySelector("button[type=submit]");
const onKeyDown = (event) => {
  if (event.key === "Escape") closeModal();
};
document.addEventListener("keydown", onKeyDown);
closeActiveModal = () => document.removeEventListener("keydown", onKeyDown);

```

```

document.getElementById("modal-cancel").addEventListener("click", closeModal);
modalForm.addEventListener("submit", async (event) => {
  event.preventDefault();
  if (submitButton.disabled) return;
  submitButton.disabled = true;
  try {
    await onSubmit(modalForm);
    closeModal();
  } catch (error) {
    if (!(error instanceof UserError)) console.error(error);
    showToast(friendlyError(error), "error");
    submitButton.disabled = false;
  }
});

```

```

modalForm.querySelector("input, select, textarea")?.focus();
}

```

```

function closeModal() {
  closeActiveModal();
  closeActiveModal = () => {};
  modalRoot.innerHTML = "";
}

```

```
function showToast(message, type = "success") {
  const toast = document.createElement("div");
  toast.className = `toast ${type}`;
  toast.setAttribute("role", type === "error" ? "alert" : "status");
  toast.textContent = t(message);
  toastRoot.appendChild(toast);
  setTimeout(() => {
    toast.remove();
  }, type === "error" ? 6000 : 3200);
}
```

web/validation.js

```
// Чистые функции веб-панели: экранирование HTML и проверка данных форм.
// Модуль не зависит от DOM и Firebase, поэтому проверяется юнит-тестами
// (tool/web-test). Все тексты ошибок русские: перевод берется из i18n.js.
```

```
const HTML_ESCAPES = { "&": "&amp;", "<": "&lt;", ">": "&gt;", "\"": "&quot;", "'": "&#39;" };
```

```
/** Экранирует значение для вставки в HTML-текст и в атрибуты. */
```

```
export function esc(value) {
  return String(value ?? "").replace(/[\&<>"]'/g, (char) => HTML_ESCAPES[char]);
}
```

```
export const LIMITS = {
  name: 100,
  productName: 255,
  description: 2000,
  categoryDescription: 500,
  address: 300,
  maxStock: 9999,
  maxPrice: 1_000_000,
  maxNutrition: 10_000,
  minPasswordLength: 6,
  maxPasswordLength: 128,
};
```

```
export function validateText(value, label, { required = false, max = LIMITS.name } = {}) {
  const text = String(value ?? "").trim();
  if (!text) return required ? `Поле «${label}» обязательно.` : null;
  if (text.length > max) return `Поле «${label}» не должно превышать ${max} символов.`;
  return null;
}
```

```
}
```

```
export function validatePhone(value, { required = true } = {}) {  
  const text = String(value ?? "").trim();  
  if (!text) return required ? "Введите телефон." : null;  
  const digits = text.replace(/\D/g, "");  
  if (!/^\+?[\d\s\-\()]+\$/ .test(text) || digits.length < 10 || digits.length > 15) {  
    return "Телефон: от 10 до 15 цифр, допустимы +, пробелы, скобки и дефис.";  
  }  
  return null;  
}
```

```
export function validateEmail(value, { required = false } = {}) {  
  const text = String(value ?? "").trim();  
  if (!text) return required ? "Введите email." : null;  
  if (text.length > 100 || !/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/ .test(text)) {  
    return "Некорректный email.";  
  }  
  return null;  
}
```

```
export function validateTaxId(value) {  
  const text = String(value ?? "").trim();  
  if (!text) return "Введите ИНН.";  
  if (!/^\d{10}(\d{2})?$/ .test(text)) return "ИНН должен содержать 10 или 12 цифр.";  
  return null;  
}
```

```
export function validatePromoCode(value) {  
  const code = String(value ?? "").trim().toUpperCase();  
  if (!code) return "Введите код промокода.";  
  if (code.length > 30) return "Код промокода не должен превышать 30 символов.";  
  if (!/^[A-Z0-9]+$/ .test(code)) return "Код промокода: только латинские буквы и цифры.";  
  return null;  
}
```

```
export function validateDiscount(value) {  
  const number = Number(value);  
  if (String(value ?? "").trim() === "" || !Number.isInteger(number) || number < 1 || number > 99) {  
    return "Скидка должна быть целым числом от 1 до 99.";  
  }  
}
```

```

    return null;
}

export function validateUrl(value, { required = false } = {}) {
    const text = String(value ?? "").trim();
    if (!text) return required ? "Введите ссылку." : null;
    try {
        const url = new URL(text);
        if (!["http:", "https:"].includes(url.protocol)) throw new Error("protocol");
    } catch {
        return "Ссылка на изображение должна начинаться с http:// или https://.";
    }
    if (text.length > 2000) return "Ссылка слишком длинная.";
    return null;
}

/** Разбирает значение <input type="date"> как локальную дату; возвращает null для мусора. */
export function parseDateInput(value) {
    const match = /^(\d{4})-(\d{2})-(\d{2})$/.exec(String(value ?? "").trim());
    if (!match) return null;
    const [, year, month, day] = match.map(Number);
    const date = new Date(year, month - 1, day);
    if (date.getFullYear() !== year || date.getMonth() !== month - 1 || date.getDate() !== day) return null;
    return date;
}

/** Конец выбранного дня по местному времени: промокод действует весь этот день. */
export function endOfDay(value) {
    const date = parseDateInput(value);
    if (!date) return null;
    date.setHours(23, 59, 59, 0);
    return date;
}

/** Значение для <input type="date"> из даты Firestore ({seconds}) или Date, по местному времени. */
export function toDateInputValue(value) {
    const date = value?.seconds ? new Date(value.seconds * 1000) : value instanceof Date ? value : null;
    if (!date || Number.isNaN(date.getTime())) return "";
    const pad = (number) => String(number).padStart(2, "0");
    return `${date.getFullYear()}-${pad(date.getMonth() + 1)}-${pad(date.getDate())}`;
}

```



```

function validateNumber(value, label, { min = 0, max, integer = false, required = false } = {}) {
  const text = String(value ?? "").trim().replace(",", ".");
  if (text === "") return required ? `Поле «${label}» обязательно.` : null;
  const number = Number(text);
  if (!Number.isFinite(number)) return `Поле «${label}» должно быть числом.`;
  if (integer && !Number.isInteger(number)) return `Поле «${label}» должно быть целым числом.`;
  if (number < min || (max !== undefined && number > max)) {
    return max === undefined
      ? `Поле «${label}» не может быть меньше ${min}.`
      : `Поле «${label}» должно быть от ${min} до ${max}.`;
  }
  return null;
}

/** Проверка формы товара; возвращает текст первой ошибки или null. */
export function validateProductForm(values) {
  const price = Number(String(values.price ?? "").trim().replace(",", "."));
  if (String(values.price ?? "").trim() === "" || !Number.isFinite(price) || price <= 0) {
    return "Цена должна быть больше нуля.";
  }
  if (price > LIMITS.maxPrice) return `Цена не может превышать ${LIMITS.maxPrice}.`;

  return (
    validateText(values.name, "Название", { required: true, max: LIMITS.productName }) ||
    validateNumber(values.stockQuantity, "Количество", { min: 0, max: LIMITS.maxStock, integer: true,
required: true }) ||
    validateText(values.qty, "Единица", { required: true, max: 50 }) ||
    (values.categoryId ? null : "Выберите категорию товара.") ||
    (values.manufacturerId ? null : "Выберите производителя товара.") ||
    validateUrl(values.imageUrl) ||
    validateText(values.country, "Страна происхождения", { max: LIMITS.name }) ||
    validateText(values.composition, "Состав", { max: LIMITS.description }) ||
    validateText(values.description, "Описание", { max: LIMITS.description }) ||
    validateText(values.nameEn, "Название (English)", { max: LIMITS.productName }) ||
    validateText(values.compositionEn, "Состав (English)", { max: LIMITS.description }) ||
    validateText(values.descriptionEn, "Описание (English)", { max: LIMITS.description }) ||
    (String(values.expiryDate ?? "").trim() && !parseDateInput(values.expiryDate) ? "Некорректный срок
годности." : null) ||
    validateNumber(values.calories, "Калорийность", { max: LIMITS.maxNutrition }) ||
    validateNumber(values.proteins, "Белки", { max: LIMITS.maxNutrition }) ||

```

```

    validateNumber(values.fats, "Жиры", { max: LIMITS.maxNutrition }) ||
    validateNumber(values.carbs, "Углеводы", { max: LIMITS.maxNutrition })
  );
}

/**
 * Проверка формы справочника (categories, manufacturers, suppliers, promoCodes).
 * `today` передается для тестов.
 */
export function validateEntityForm(section, values, { today = new Date() } = {}) {
  switch (section) {
    case "categories":
      return (
        validateText(values.name, "Название", { required: true }) ||
        validateText(values.nameEn, "Название (English)") ||
        validateText(values.description, "Описание", { max: LIMITS.categoryDescription })
      );
    case "manufacturers":
      return (
        validateText(values.name, "Название", { required: true }) ||
        validateText(values.country, "Страна", { required: true }) ||
        validatePhone(values.contactPhone)
      );
    case "suppliers":
      return (
        validateText(values.name, "Название", { required: true }) ||
        validateTaxId(values.taxId) ||
        validateText(values.address, "Адрес", { required: true, max: LIMITS.address }) ||
        validatePhone(values.contactPhone) ||
        validateEmail(values.email)
      );
    case "promoCodes": {
      const error = validatePromoCode(values.code) || validateDiscount(values.discountPercent);
      if (error) return error;
      const expiresAt = endOfDay(values.expiresAt);
      if (!expiresAt) return "Укажите корректный срок действия.";
      if (expiresAt < today) return "Срок действия промокода уже истек.";
      return null;
    }
    default:
      return null;
  }
}

```

```

    }
}

export function validateNewPassword(newPassword, currentPassword, confirmation) {
  if (newPassword.length < LIMITS.minPasswordLength || !/[A-Za-zA-Яа-яЁё]/.test(newPassword) || !/\d/.test(newPassword)) {
    return "Новый пароль: минимум 6 символов, буквы и цифры.";
  }
  if (newPassword.length > LIMITS.maxPasswordLength) return "Пароль не должен превышать 128 символов.";
  if (/\/.test(newPassword)) return "Пароль не должен содержать пробелы.";
  if (newPassword !== confirmation) return "Пароли не совпадают.";
  if (newPassword === currentPassword) return "Новый пароль совпадает с текущим.";
  return null;
}

const FINISHED_STATUSES = ["delivered", "cancelled"];
const COURIER_STATUSES = ["assigned", "delivering", "delivered"];

export function isFinishedStatus(statusId) {
  return FINISHED_STATUSES.includes(statusId);
}

/** Проверка смены статуса заказа; возвращает текст ошибки или null. */
export function validateOrderChange(order, { statusId, courierId }) {
  if (isFinishedStatus(order.statusId) && (statusId !== order.statusId || (courierId || null) !== (order.courierId || null))) {
    return "Завершенный или отмененный заказ изменить нельзя.";
  }
  if (COURIER_STATUSES.includes(statusId) && !courierId) {
    return "Для этого статуса назначьте курьера.";
  }
  return null;
}

/** Сколько единиц каждого товара вернуть на склад при отмене заказа. */
export function stockRestorations(order) {
  const totals = new Map();
  (Array.isArray(order.items) ? order.items : []).forEach((item) => {
    const quantity = Math.trunc(Number(item?.quantity));
    if (!item?.id || !Number.isFinite(quantity) || quantity <= 0) return;

```

```

    totals.set(item.id, (totals.get(item.id) || 0) + quantity);
  });
  return [...totals].map(([id, quantity]) => ({ id, quantity }));
}

/** Переводит ошибку Firebase в понятное сообщение (без технических деталей). */
export function friendlyError(error) {
  const code = error?.code || "";
  if (code === "permission-denied" || code === "auth/insufficient-permission") {
    return "Недостаточно прав для этого действия.";
  }
  if (["unavailable", "auth/network-request-failed", "deadline-exceeded"].includes(code)) {
    return "Нет соединения с сервером. Проверьте интернет и повторите.";
  }
  if (code === "auth/too-many-requests") return "Слишком много попыток. Попробуйте позже.";
  if (code === "auth/requires-recent-login") return "Войдите заново и повторите действие.";
  if (code === "auth/weak-password") return "Пароль слишком простой.";
  if (code === "not-found") return "Запись не найдена: возможно, ее уже удалили. Обновите данные.";
  return error?.userMessage || "Не удалось выполнить действие. Повторите попытку.";
}

/** Ошибка с заранее подготовленным для пользователя текстом. */
export class UserError extends Error {
  constructor(message) {
    super(message);
    this.userMessage = message;
  }
}

/** Значения для дашборда: выручка без отмененных заказов, месяцы по порядку. */
export function computeDashboard({ users, products, orders, catalogLabel = (item) => item.name }) {
  const active = orders.filter((order) => order.statusId !== "cancelled");
  const monthlySales = {};
  active.forEach((order) => {
    if (!order.createdAt?.seconds) return;
    const date = new Date(order.createdAt.seconds * 1000);
    const key = `${date.getFullYear()}-${String(date.getMonth() + 1).padStart(2, "0")}`;
    monthlySales[key] = (monthlySales[key] || 0) + (Number(order.totalAmount) || 0);
  });

  const popularity = {};

```

```

active.forEach((order) => {
  (order.items || []).forEach((item) => {
    const product = products.find((candidate) => candidate.id === item.id);
    const label = (product && catalogLabel(product)) || item.name || "—";
    popularity[label] = (popularity[label] || 0) + (Number(item.quantity) || 0);
  });
});

return {
  buyersCount: users.filter((user) => ["buyer", "customer"].includes(user.role)).length,
  lowStockCount: products.filter((product) => !product.isDeleted && Number(product.stockQuantity || 0)
<= 5).length,
  activeOrdersCount: orders.filter((order) => !isFinishedStatus(order.statusId)).length,
  totalSales: active.reduce((sum, order) => sum + (Number(order.totalAmount) || 0), 0),
  monthlySales: Object.entries(monthlySales).sort(([left], [right]) => left.localeCompare(right)),
  topProducts: Object.entries(popularity)
    .sort((left, right) => right[1] - left[1])
    .slice(0, 5),
};
}

```

firestore.rules

```

rules_version = '2';

// Роли хранятся в users/{uid}.role: admin, manager, courier, buyer (customer = buyer).
// Правила проверены тестами на эмуляторе: tool/rules-test (npm test).
service cloud.firestore {
  match /databases/{database}/documents {

    function signedIn() {
      return request.auth != null;
    }

    function ownProfile() {
      return get(/databases/{database}/documents/users/{request.auth.uid}).data;
    }

    function hasProfile() {
      return signedIn() && exists(/databases/{database}/documents/users/{request.auth.uid});
    }

    function role() {

```

```

    return hasProfile() ? ownProfile().get('role', 'guest') : 'guest';
}

function isActive() {
    return hasProfile() && ownProfile().get('isDeleted', false) != true;
}

function isAdmin() {
    return isActive() && role() == 'admin';
}

function isStaff() {
    return isActive() && role() in ['admin', 'manager'];
}

function isBuyer() {
    return isActive() && role() in ['buyer', 'customer'];
}

function isCourier() {
    return isActive() && role() == 'courier';
}

function changedKeys() {
    return request.resource.data.diff(resource.data).affectedKeys();
}

// Флаг isDeleted владелец меняет только при самоудалении аккаунта (deletedBy = он сам)
// и при откате самоудаления. Заблокированный администратором пользователь
// (deletedBy = uid администратора) снять блокировку и менять профиль не может.
function deletedFlagChangeAllowed() {
    let before = resource.data.get('isDeleted', false);
    let after = request.resource.data.get('isDeleted', false);
    return (before == false && after == false)
        || (before == false && after == true
            && request.resource.data.get('deletedBy', '') == request.auth.uid)
        || (before == true && after == false
            && resource.data.get('deletedBy', '') == request.auth.uid);
}

function validProduct(data) {

```

```

return data.get('name', 'x') is string
  && data.get('name', 'x').size() > 0
  && data.get('name', 'x').size() <= 255
  && data.get('price', 1) is number
  && data.get('price', 1) > 0
  && data.get('stockQuantity', 0) is number
  && data.get('stockQuantity', 0) >= 0;
}

function validPromo(data) {
  return data.get('discountPercent', 1) is number
    && data.get('discountPercent', 1) >= 1
    && data.get('discountPercent', 1) <= 99;
}

// Скидка в заказе обязана совпадать с промокодом, на который заказ ссылается.
function validOrderDiscount(data) {
  let promoId = data.get('promoCodeId', null);
  return promoId == null
    ? data.get('discountPercent', 0) == 0
    : (promoId is string
      && exists(/databases/$(database)/documents/promoCodes/$(promoId))
      && get(/databases/$(database)/documents/promoCodes/$(promoId)).data.discountPercent
      == data.get('discountPercent', 0));
}

// Свободный заказ: без курьера и еще не в работе у другого.
function isFreeOrder() {
  return resource.data.get('courierId', null) == null
    && resource.data.statusId in ['new', 'processing'];
}

function courierClaimKeys() {
  return ['statusId', 'statusName', 'courierId', 'courierName', 'courierPhone', 'updatedAt'];
}

// ----- Пользователи -----
match /users/{userId} {
  allow read: if signedIn() && (request.auth.uid == userId || isStaff());
  // Самостоятельная регистрация: только роль покупателя и только свой документ.
  allow create: if isAdmin() || (

```

```

signedIn() && request.auth.uid == userId &&
request.resource.data.role in ['buyer', 'customer'] &&
request.resource.data.get('isDeleted', false) == false
);
// Владелец меняет профиль, но не роль; роли и блокировки меняет только администратор.
allow update: if isAdmin() || (
signedIn() && request.auth.uid == userId &&
request.resource.data.get('role', 'buyer') == resource.data.get('role', 'buyer') &&
request.resource.data.get('uid', userId) == resource.data.get('uid', userId) &&
// Аватар хранится в профиле как base64 (Firebase Storage платный).
request.resource.data.get('photoData', "").size() <= 210000 &&
deletedFlagChangeAllowed()
);
allow delete: if false;
}

// Публичный индекс «логин -> email» для входа по логину.
match /loginIndex/{login} {
allow get: if true;
allow list: if false;
// Запись создается только для себя и только со своей почтой из токена.
allow create: if isAdmin() || (
signedIn() && request.resource.data.uid == request.auth.uid &&
request.resource.data.keys().hasOnly(['uid', 'email']) &&
request.resource.data.email is string &&
request.resource.data.email.lower() == request.auth.token.get('email', "").lower()
);
allow update: if isAdmin();
// Владелец освобождает свой логин при удалении аккаунта.
allow delete: if isAdmin() || (signedIn() && resource.data.uid == request.auth.uid);
}

// ----- Каталог -----
match /products/{productId} {
allow read: if signedIn();
allow create: if isStaff() && validProduct(request.resource.data);
allow update: if (isStaff() && validProduct(request.resource.data)) || (
// Покупатель при оформлении заказа может только уменьшить остаток
// (не больше 99 единиц за одну запись, как в корзине).
isBuyer() &&
changedKeys().hasOnly(['stockQuantity', 'updatedAt']) &&

```



```

    request.resource.data.stockQuantity >= 0 &&
    request.resource.data.stockQuantity < resource.data.stockQuantity &&
    resource.data.stockQuantity - request.resource.data.stockQuantity <= 99
  );
  allow delete: if isAdmin();
}

match /categories/{docId} {
  allow read: if signedIn();
  allow write: if isStaff();
}

match /manufacturers/{docId} {
  allow read: if signedIn();
  allow write: if isStaff();
}

// Поставщики: менеджер только просматривает, изменяет администратор.
match /suppliers/{docId} {
  allow read: if isStaff();
  allow write: if isAdmin();
}

match /roles/{docId} {
  allow read: if signedIn();
  allow write: if isAdmin();
}

match /orderStatuses/{docId} {
  allow read: if signedIn();
  allow write: if isAdmin();
}

match /promoCodes/{docId} {
  allow read: if signedIn();
  allow create: if isStaff() && validPromo(request.resource.data);
  allow delete: if isStaff();
  allow update: if (isStaff() && validPromo(request.resource.data)) || (
    // Покупатель при оформлении заказа только увеличивает счетчик использований.
    isBuyer() &&
    changedKeys().hasOnly(['usageCount', 'updatedAt']) &&

```

```

    request.resource.data.usageCount == resource.data.get('usageCount', 0) + 1
  );
}

// ----- Заказы -----
match /orders/{orderId} {
  allow read: if signedIn() && (
    isStaff() ||
    resource.data.userId == request.auth.uid ||
    resource.data.get('courierId', null) == request.auth.uid ||
    // Курьеры видят пул свободных заказов, которые можно взять.
    (isCourier() && isFreeOrder())
  );

  allow create: if isBuyer() &&
    request.resource.data.userId == request.auth.uid &&
    request.resource.data.statusId == 'new' &&
    request.resource.data.get('courierId', null) == null &&
    request.resource.data.totalAmount is number &&
    request.resource.data.totalAmount >= 0 &&
    request.resource.data.items is list &&
    request.resource.data.items.size() > 0 &&
    request.resource.data.items.size() <= 100 &&
    validOrderDiscount(request.resource.data);

  allow update: if isStaff() || (
    // Покупатель подтверждает получение (пока заказ у курьера) и скрывает
    // из своего списка только завершенные заказы.
    isBuyer() && resource.data.userId == request.auth.uid && (
      (
        changedKeys().hasOnly(['userConfirmed', 'updatedAt']) &&
        resource.data.statusId in ['assigned', 'delivering', 'delivered']
      ) || (
        changedKeys().hasOnly(['hiddenByUser', 'updatedAt']) &&
        resource.data.statusId in ['delivered', 'cancelled']
      )
    )
  ) || (
    // Курьер меняет статус только своих активных заказов.
    isCourier() && resource.data.courierId == request.auth.uid &&
    resource.data.statusId in ['new', 'processing', 'assigned', 'delivering'] &&

```

```

changedKeys().hasOnly(courierClaimKeys()) &&
request.resource.data.courierId == request.auth.uid &&
request.resource.data.statusId in ['delivering', 'delivered']
) || (
  // Курьер сам берет свободный заказ: он становится исполнителем, статус «передан курьеру».
  isCourier() && isFreeOrder() &&
  changedKeys().hasOnly(courierClaimKeys()) &&
  request.resource.data.courierId == request.auth.uid &&
  request.resource.data.statusId == 'assigned'
) || (
  // Курьер может вернуть свой заказ в пул, пока не выехал.
  isCourier() && resource.data.get('courierId', null) == request.auth.uid &&
  resource.data.statusId == 'assigned' &&
  changedKeys().hasOnly(courierClaimKeys()) &&
  request.resource.data.get('courierId', null) == null &&
  request.resource.data.statusId == 'processing'
);

// Заказы физически не удаляются (финансовая отчетность).
allow delete: if false;
}

// ----- Уведомления -----
match /notifications/{notificationId} {
  allow read: if signedIn() && (isStaff() || resource.data.userId == request.auth.uid);
  allow create: if (isStaff() || isCourier()) &&
    request.resource.data.keys().hasOnly(['userId', 'text', 'textEn', 'createdAt', 'updatedAt']) &&
    request.resource.data.userId is string &&
    request.resource.data.text is string &&
    request.resource.data.text.size() > 0 &&
    request.resource.data.text.size() <= 500 &&
    request.resource.data.get('textEn', '') is string &&
    request.resource.data.get('textEn', '').size() <= 500;
  allow delete: if signedIn() && resource.data.userId == request.auth.uid;
  allow update: if false;
}
}
}

firebase.json

{
  "flutter": {

```

```

"platforms": {
  "android": {
    "default": {
      "projectId": "nectar-41848",
      "appId": "1:70969291340:android:19cdd928fc446c5931fae9",
      "fileOutput": "android/app/google-services.json"
    }
  },
  "ios": {
    "default": {
      "projectId": "nectar-41848",
      "appId": "1:70969291340:ios:9d4feb3c7863a40931fae9",
      "uploadDebugSymbols": false,
      "fileOutput": "ios/Runner/GoogleService-Info.plist"
    }
  },
  "macos": {
    "default": {
      "projectId": "nectar-41848",
      "appId": "1:70969291340:ios:9d4feb3c7863a40931fae9",
      "uploadDebugSymbols": false,
      "fileOutput": "macos/Runner/GoogleService-Info.plist"
    }
  },
  "dart": {
    "lib/firebase_options.dart": {
      "projectId": "nectar-41848",
      "configurations": {
        "android": "1:70969291340:android:19cdd928fc446c5931fae9",
        "ios": "1:70969291340:ios:9d4feb3c7863a40931fae9",
        "macos": "1:70969291340:ios:9d4feb3c7863a40931fae9",
        "web": "1:70969291340:web:464aa7206ce3067331fae9",
        "windows": "1:70969291340:web:712e450706c8921031fae9"
      }
    }
  }
},
"firestore": {
  "rules": "firestore.rules"
},

```

```

"emulators": {
  "auth": {
    "port": 9099
  },
  "firestore": {
    "port": 8080
  },
  "ui": {
    "enabled": false
  },
  "singleProjectMode": true,
  "hosting": {
    "port": 5000
  }
},
"hosting": {
  "public": "web",
  "ignore": [
    "firebase.json",
    "**/.*",
    "**/node_modules/**"
  ],
  "headers": [
    {
      "source": "**",
      "headers": [
        {
          "key": "X-Content-Type-Options",
          "value": "nosniff"
        },
        {
          "key": "X-Frame-Options",
          "value": "DENY"
        },
        {
          "key": "Referrer-Policy",
          "value": "strict-origin-when-cross-origin"
        }
      ]
    }
  ]
}
]

```

}
}

ПРИЛОЖЕНИЕ Д. СКРИПТ БАЗЫ ДАННЫХ АННОТАЦИЯ

В данном программном документе приведены сведения о скрипте базы данных мобильного приложения и веб-панели «Интернет-магазин продуктов «Нектар».

В разделе «Скрипт базы данных» указано наименование скрипта, область применения скрипта, словарь данных базы данных и описание работы базы данных.

СОДЕРЖАНИЕ

1.	СКРИПТ БАЗЫ ДАННЫХ.....	3
1.1	Наименование скрипта.....	3
1.2	Область применения скрипта.....	3
1.3	Словарь данных базы данных.....	3
1.4	Скрипт.....	7

1. СКРИПТ БАЗЫ ДАННЫХ

1.1. Наименование скрипта

Наименование – не предусмотрено. Требованиями технического задания разработка скрипта базы данных не предусмотрена.

1.2. Область применения скрипта

Скрипт не разрабатывался. Приложение использует облачную документоориентированную базу данных Google Cloud Firestore, которая не имеет жесткой схемы: коллекции и документы создаются автоматически при первой записи данных, поэтому SQL-скрипт создания таблиц не требуется.

1.3. Словарь данных базы данных

В таблице 1 представлен словарь данных для базы данных, с которой взаимодействует разработанное программное решение. Роль первичного ключа (PK) играет идентификатор документа, роль внешнего ключа (FK) – поле со ссылкой на идентификатор документа другой коллекции.

Таблица 1 – Словарь данных.

Ключ	Наименование	Тип данных	Описание	Обязательность заполнения
1	2	3	4	5
Коллекция: users				
PK	id	string	Идентификатор документа (совпадает с uid учетной записи)	Да
	uid	string	Идентификатор пользователя	Да
	login	string	Логин	Да
	loginNormalized	string	Логин в нижнем регистре	Да
	displayName	string	Отображаемое имя	Да
	email	string	Email	Да
	emailNormalized	string	Email в нижнем регистре	Да
	phoneNumber	string	Номер телефона	Да
FK	role	string	Роль (roles): admin, manager, courier, buyer	Да
	photoData	string	Аватар в формате base64 (до 150 КБ)	Нет
	photoUrl	string	Ссылка на фото из Google-аккаунта	Нет

Ключ	Наименование	Тип данных	Описание	Обязательность заполнения
1	2	3	4	5
	addresses	array<string>	Сохраненные адреса доставки	Да
	cards	array<string>	Маски банковских карт	Да
	favorites	array<string>	Идентификаторы избранных товаров	Нет
	notificationSettings	map	Настройки уведомлений (push, sms, email)	Да
	isDeleted	boolean	Признак отключения аккаунта	Да
	deletedBy	string	Кто отключил аккаунт	Нет
	deletedAt	timestamp	Дата отключения	Нет
	createdAt	timestamp	Дата создания	Да
	updatedAt	timestamp	Дата изменения	Да
Коллекция: loginIndex				
PK	id	string	Логин в нижнем регистре	Да
FK	uid	string	Идентификатор пользователя (users)	Да
	email	string	Email пользователя	Да
Коллекция: roles				
PK	id	string	Код роли	Да
	name	string	Наименование роли	Да
Коллекция: categories				
PK	id	string	Идентификатор категории	Да
	name	string	Наименование категории	Да
	nameEn	string	Наименование (English)	Нет
	nameNormalized	string	Наименование в нижнем регистре	Да
	description	string	Описание	Нет
	descriptionEn	string	Описание (English)	Нет
	imageUrl	string	Ссылка на изображение	Нет
	createdAt	timestamp	Дата создания	Да
	updatedAt	timestamp	Дата изменения	Да
Коллекция: manufacturers				
PK	id	string	Идентификатор производителя	Да
	name	string	Наименование	Да
	nameNormalized	string	Наименование в нижнем регистре	Да
	country	string	Страна	Да
	contactPhone	string	Контактный телефон	Да
	createdAt	timestamp	Дата создания	Да
	updatedAt	timestamp	Дата изменения	Да
Коллекция: suppliers				
PK	id	string	Идентификатор поставщика	Да

Ключ	Наименование	Тип данных	Описание	Обязательность заполнения
1	2	3	4	5
	name	string	Наименование	Да
	nameNormalized	string	Наименование в нижнем регистре	Да
	taxId	string	ИНН (10 или 12 цифр)	Да
	address	string	Адрес	Да
	contactPhone	string	Контактный телефон	Да
	email	string	Email	Нет
	createdAt	timestamp	Дата создания	Да
	updatedAt	timestamp	Дата изменения	Да
Коллекция: products				
PK	id	string	Идентификатор товара	Да
	name	string	Наименование	Да
	nameEn	string	Наименование (English)	Нет
	nameNormalized	string	Наименование в нижнем регистре	Да
	price	number	Цена, Р (больше 0)	Да
	qty	string	Фасовка (например, «1 кг»)	Да
	unit	string	Единица измерения	Нет
	stockQuantity	number	Остаток на складе (0–9999)	Да
	minStock	number	Порог низкого остатка	Да
	imageUrl	string	Ссылка на изображение	Нет
FK	categoryId	string	Категория (categories)	Да
	categoryName	string	Название категории (копия)	Да
FK	manufacturerId	string	Производитель (manufacturers)	Да
	manufacturerName	string	Название производителя (копия)	Да
FK	supplierId	string	Поставщик (suppliers)	Нет
	supplierName	string	Название поставщика (копия)	Нет
	description	string	Описание (до 2000 символов)	Нет
	descriptionEn	string	Описание (English)	Нет
	composition	string	Состав	Нет
	compositionEn	string	Состав (English)	Нет
	country	string	Страна происхождения	Нет
	calories	number	Калорийность на 100 г	Нет
	proteins	number	Белки на 100 г	Нет
	fats	number	Жиры на 100 г	Нет
	carbs	number	Углеводы на 100 г	Нет
	shelfLifeDays	number	Срок годности, дней	Нет
	expiryDate	timestamp	Срок годности до	Нет
	popularity	number	Популярность	Да
	isDeleted	boolean	Признак скрытия товара	Да
	createdAt	timestamp	Дата создания	Да

Ключ	Наименование	Тип данных	Описание	Обязательность заполнения
1	2	3	4	5
	updatedAt	timestamp	Дата изменения	Да
Коллекция: promoCodes				
PK	id	string	Идентификатор промокода	Да
	code	string	Код (латиница и цифры)	Да
	codeNormalized	string	Код в верхнем регистре	Да
	discountPercent	number	Скидка, % (1–99)	Да
	expiresAt	timestamp	Срок действия	Да
	isActive	boolean	Признак активности	Да
	usageCount	number	Количество использований	Нет
	createdAt	timestamp	Дата создания	Да
	updatedAt	timestamp	Дата изменения	Да
Коллекция: orderStatuses				
PK	id	string	Код статуса	Да
	code	string	Код статуса	Да
	name	string	Наименование статуса	Да
Коллекция: orders				
PK	id	string	Идентификатор заказа	Да
	number	string	Номер заказа (ORD-...)	Да
FK	userId	string	Покупатель (users)	Да
	customerName	string	Имя покупателя (копия)	Да
	customerEmail	string	Email покупателя (копия)	Нет
	customerPhone	string	Телефон покупателя (копия)	Нет
	address	string	Адрес доставки	Да
	paymentMethod	string	Способ оплаты (Онлайн / Наличными)	Да
	paymentCard	string	Маска карты	Нет
	deliverySlot	string	Интервал доставки	Нет
FK	promoCodeId	string	Промокод (promoCodes)	Нет
	promoCode	string	Код промокода (копия)	Нет
	discountPercent	number	Скидка, %	Да
	discountAmount	number	Сумма скидки	Да
	subtotalAmount	number	Стоимость товаров	Да
	deliveryAmount	number	Стоимость доставки	Да
	totalAmount	number	Итоговая стоимость	Да
FK	statusId	string	Статус (orderStatuses)	Да
	statusName	string	Название статуса (копия)	Да
FK	courierId	string	Курьер (users)	Нет
	courierName	string	Имя курьера (копия)	Нет
	courierPhone	string	Телефон курьера (копия)	Нет
	userConfirmed	boolean	Покупатель подтвердил получение	Нет
	hiddenByUser	boolean	Заказ скрыт покупателем	Нет
	items	array<map>	Позиции заказа	Да

Ключ	Наименование	Тип данных	Описание	Обязательность заполнения
1	2	3	4	5
	createdAt	timestamp	Дата создания	Да
	updatedAt	timestamp	Дата изменения	Да
Вложенный массив: orders.items (позиция заказа)				
FK	id	string	Товар (products)	Да
	name	string	Наименование на момент покупки	Да
	nameEn	string	Наименование (English)	Нет
	price	number	Цена на момент покупки	Да
	quantity	number	Количество	Да
	imageUrl	string	Ссылка на изображение	Нет
	categoryName	string	Название категории	Нет
Коллекция: notifications				
PK	id	string	Идентификатор уведомления	Да
FK	userId	string	Получатель (users)	Да
	text	string	Текст на русском (до 500 символов)	Да
	textEn	string	Текст на английском	Нет
	createdAt	timestamp	Дата создания	Да
	updatedAt	timestamp	Дата изменения	Да

1.4. Скрипт

Скрипт создания базы данных требованиями технического задания не предусмотрен. Ниже описана работа базы данных.

Данные хранятся в проекте Firebase «nectar-41848» в виде коллекций документов. Мобильное приложение (Flutter) и веб-панель (JavaScript) обращаются к базе данных напрямую через Firebase SDK; собственного серверного кода нет. Идентификатор документа пользователя совпадает с идентификатором учетной записи Firebase Authentication.

Оформление заказа выполняется одной транзакцией: приложение проверяет наличие и остатки товаров, берет актуальные цены из базы, уменьшает остатки, создает заказ со статусом «new» и увеличивает счетчик использования промокода; при любой ошибке ни одно изменение не сохраняется. Так же транзакцией выполняются взятие свободного заказа курьером (два курьера не могут взять один заказ) и смена статуса заказа в

веб-панели (при отмене товары возвращаются на склад, покупателю и курьеру создаются уведомления).

Жизненный цикл заказа: new (Новый) → processing (В сборке) → assigned (Передан курьеру) → delivering (В пути) → delivered (Доставлен); дополнительно возможен статус cancelled (Отменен). Завершенные заказы изменять нельзя, заказы физически не удаляются.

Права доступа задаются правилами безопасности Firestore (файл firestore.rules), которые проверяют роль пользователя (users/{uid}.role) при каждом обращении. Сводка правил приведена в таблице 2.

Таблица 2 – Правила доступа к коллекциям.

Коллекция	Чтение	Создание	Изменение	Удаление
1	2	3	4	5
users	Владелец, менеджер, администратор	Сам пользователь (роль buyer) или администратор	Владелец (без смены роли; isDeleted – только самоудаление) или администратор	Запрещено
loginIndex	Документ по логину – любой; список запрещен	Пользователь для себя и своей почты; администратор	Администратор	Владелец или администратор
products	Авторизованные	Администратор и менеджер	Администратор и менеджер; покупатель – только уменьшение остатка при заказе	Администратор
categories, manufacturers	Авторизованные	Администратор и менеджер	Администратор и менеджер	Администратор и менеджер
suppliers	Администратор и менеджер	Администратор	Администратор	Администратор
roles, orderStatuses	Авторизованные	Администратор	Администратор	Администратор
promoCodes	Авторизованные	Администратор и менеджер	Администратор и менеджер; покупатель – только usageCount + 1	Администратор и менеджер
orders	Сотрудники,	Покупатель	Сотрудники;	Запрещено

Коллекция	Чтение	Создание	Изменение	Удаление
1	2	3	4	5
	покупатель-владелец, назначенный курьер, курьеры (свободные заказы)	(статус new, без курьера, 1–100 позиций)	покупатель – userConfirmed, hiddenByUser; курьер – взять, вернуть, начать и завершить доставку	
notifications	Владелец, сотрудники	Сотрудники и курьер (до 500 символов)	Запрещено	Владелец

Правила загружаются командой «firebase deploy --only firestore:rules» и проверяются автоматическими тестами на локальном эмуляторе (каталог tool/rules-test, команда «npm test»). Начальное наполнение выполняется из приложения администратором (DatabaseService.reseedCatalog: 9 категорий и 69 товаров); для отладки используется скрипт tool/rules-test/seed-emulator.mjs. Составные индексы не требуются: запросы используют только условия равенства и вхождения в список, сортировка выполняется на стороне клиента.

ПРИЛОЖЕНИЕ Е. ВХОДНЫЕ И ВЫХОДНЫЕ ДАННЫЕ

АННОТАЦИЯ

В данном программном документе приведены входные и выходные данные информационной системы «Нектар» (мобильного приложения и веб-панели управления интернет-магазином продуктов).

СОДЕРЖАНИЕ

1.	ВХОДНЫЕ И ВЫХОДНЫЕ ДАННЫЕ.....	3
1.1	Входные данные.....	3
1.2	Выходные данные.....	7

1. ВХОДНЫЕ И ВЫХОДНЫЕ ДАННЫЕ

1.1. Входные данные

Входные данные информационной системы (мобильного приложения и веб-панели) представлены в таблице 1.

Таблица 1 – Входные данные

Поле	Ограничение	Тип	Описание
1	2	3	4
Экран «Онбординг» и переключатель языка			
Язык интерфейса	RU EN	Строка	Переключатель языка приложения (сохраняется на устройстве)
Экран «Вход»			
Логин или email	[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]+ (не более 100 символов) или [a-zA-Z0-9._-]{3,50}	Строка	Поле ввода логина или адреса электронной почты
Пароль	Не пустое значение	Строка	Поле ввода пароля (скрытый ввод)
Экран «Регистрация»			
Логин	[a-zA-Z0-9._-]{3,50}	Строка	Поле ввода уникального логина
Имя	[a-zA-Za-яA-ЯёЁ -]{2,50}	Строка	Поле ввода имени пользователя
Телефон	\+[0-9 ()-]+ (10–15 цифр)	Строка	Поле ввода номера телефона
Email	[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]+ (не более 100 символов)	Строка	Поле ввода адреса электронной почты
Пароль	Без пробелов, 6–128 символов, минимум одна цифра	Строка	Поле ввода пароля (скрытый ввод)
Подтверждение пароля	Совпадает со значением поля «Пароль»	Строка	Поле повторного ввода пароля
Экран «Восстановление пароля»			
Email	[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]+	Строка	Поле ввода адреса, на который отправляется письмо для сброса пароля
Экраны «Магазин» и «Каталог»			
Поиск	Не предусмотрено	Строка	Поиск товара по

Поле	Ограничение	Тип	Описание
1	2	3	4
			названию
Цена «от»	[0-9]+([.],[0-9]+)?	Число с плавающей точкой	Минимальная цена для фильтрации товаров
Цена «до»	[0-9]+([.],[0-9]+)?	Число с плавающей точкой	Максимальная цена для фильтрации товаров
Бренд (производитель)	Не предусмотрено	Множество строк	Выбранные производители для фильтрации товаров
Сортировка	По умолчанию Сначала свежие По убыванию цены По возрастанию цены	Перечисление	Выбор способа сортировки списка товаров
Экраны «Карточка товара» и «Корзина»			
Количество товара (+ / -)	$1 \leq \text{значение} \leq \min(\text{остаток}, 99)$	Целое число	Количество единиц товара для добавления в корзину
Промокод	[A-Za-z0-9]+	Строка	Поле ввода промокода на скидку
Экран «Оформление заказа»			
Адрес доставки	Выбор из сохраненных адресов	Строка	Адрес, на который будет доставлен заказ
Время доставки	Один из 7 интервалов	Строка	Выбор интервала доставки («Как можно скорее», 09:00–11:00 ... 19:00–21:00)
Способ оплаты	Онлайн Наличными	Строка	Переключатель способа оплаты
Карта	Выбор из сохраненных карт	Строка	Карта для онлайн-оплаты (обязательна при оплате «Онлайн»)
Раздел «Профиль»			
Полное имя	[a-zA-Za-яА-ЯёЁ -]{2,50}	Строка	Поле ввода нового имени пользователя
Номер телефона	\+[0-9 ()-]+ (10–15 цифр)	Строка	Поле ввода нового номера телефона

Поле	Ограничение	Тип	Описание
1	2	3	4
Аватар профиля	Форматы JPEG, PNG; после сжатия не более 150 КБ	Файл	Изображение профиля пользователя
Адрес доставки	1–300 символов	Строка	Поле ввода нового адреса (улица, дом, квартира)
Номер карты	[0-9]{16}	Строка	Поле ввода номера банковской карты (в базе хранится только маска)
Текущий пароль	Не пустое значение	Строка	Поле ввода текущего пароля (скрытый ввод)
Новый пароль	Без пробелов, 6–128 символов, минимум одна цифра, не совпадает с текущим	Строка	Поле ввода нового пароля (скрытый ввод)
Подтверждение нового пароля	Совпадает со значением поля «Новый пароль»	Строка	Поле повторного ввода нового пароля
Push / SMS / Email уведомления	Вкл. Выкл.	Булево	Переключатели настроек уведомлений
Экран курьера «Доставка»			
Действие над заказом	Взять заказ Вернуть в пул Начать доставку Подтвердить доставку	Перечисление	Кнопки перевода заказа между статусами
Веб-панель. Вход и смена пароля			
Email	[a-zA-Z0-9._%+~]+@[a- zA-Z0-9.-]+\.[a-zA-Z]+ (не более 100 символов)	Строка	Поле ввода адреса электронной почты сотрудника
Пароль	Не более 128 символов	Строка	Поле ввода пароля сотрудника (скрытый ввод)
Веб-панель. Форма товара			
Название	1–255 символов	Строка	Наименование товара
Цена	0 < значение ≤ 1 000 000	Число с плавающей точкой	Цена товара в рублях
Количество	0–9999	Целое число	Остаток товара на складе
Единица	1–50 символов	Строка	Фасовка товара

Поле	Ограничение	Тип	Описание
1	2	3	4
			(например, «1 кг»)
Категория, Производитель, Поставщик	Выбор из справочника (поставщик необязателен)	Строка	Связи товара со справочниками
Ссылка на изображение	URL http:// или https://, до 2000 символов	Строка	Ссылка на фотографию товара
Страна происхождения	До 100 символов	Строка	Страна-производитель
Срок годности до	Дата	Дата	Дата окончания срока годности
Калорийность, белки, жиры, углеводы	0–10 000	Число с плавающей точкой	Пищевая ценность на 100 г
Состав, описание (RU/EN)	До 2000 символов	Строка	Многострочные поля описания и состава
Название (English)	До 255 символов	Строка	Английское название товара
Веб-панель. Справочники и промокоды			
Категория. Название / Описание	До 100 / до 500 символов	Строка	Поля категории товаров
Производитель. Название, страна, телефон	До 100 символов; телефон 10–15 цифр	Строка	Поля производителя
Поставщик. Название, ИНН, адрес, телефон, email	ИНН [0-9]{10}([0-9]{2})?; адрес до 300 символов	Строка	Поля поставщика
Промокод. Код	[A-Z0-9]{1,30}	Строка	Код скидки
Промокод. Скидка, %	1–99	Целое число	Размер скидки в процентах
Промокод. Срок действия	Не раньше текущей даты	Дата	Последний день действия промокода
Веб-панель. Заказы и пользователи			
Статус заказа	Новый В сборке Передан курьеру В пути Доставлен Отменен	Строка	Выбор нового статуса заказа из списка
Курьер	Обязателен для статусов «Передан курьеру», «В пути», «Доставлен»	Строка	Выбор курьера, назначенного на заказ
Роль	Администратор	Строка	Выбор роли учетной

Поле	Ограничение	Тип	Описание
1	2	3	4
пользователя	Менеджер Курьер Покупатель		записи

1.2. Выходные данные

Выходные данные информационной системы (мобильного приложения и веб-панели) представлены в таблице 2.

Таблица 2 – Выходные данные

Поле	Ограничение	Тип	Описание
1	2	3	4
Системные интерфейсы и экран «Магазин»			
Системные уведомления	Не предусмотрено	Строка	Всплывающее сообщение (SnackBar / Toast) с текстом ошибки или успеха
Акционный баннер	Не предусмотрено	Строка	Блок «Скидка 20% на первый заказ» с промокодом
Популярное	Не предусмотрено	Горизонтальный список карточек	Элемент, куда выводятся популярные товары
Разделы по категориям	Не предусмотрено	Горизонтальный список карточек	Элемент, куда выводятся товары каждой категории
Экраны «Каталог» и «Карточка товара»			
Плитки категорий	Не предусмотрено	Сетка элементов	Элемент, куда выводятся категории товаров
Карточка товара. Изображение	Не предусмотрено	Изображение	Фотография товара
Карточка товара. Название и фасовка	Не предусмотрено	Строка	Название на языке интерфейса и единица измерения
Карточка товара. Цена	Не предусмотрено	Строка	Цена товара в рублях
Остаток на складе	Не предусмотрено	Строка	Надпись «Осталось: N шт», «Нет в наличии» или «Распродано»
Характеристики	Не предусмотрено	Таблица «ключ – значение»	Производитель, страна, срок годности, состав
Пищевая ценность	Не	Числа с	Калорийность, белки,

Поле	Ограничение	Тип	Описание
1	2	3	4
	предусмотрено	плавающей точкой	жиры, углеводы на 100 г
Экраны «Избранное» и «Корзина»			
Список избранного	Не предусмотрено	Сетка элементов	Товары, отмеченные значком «сердце»
Позиция корзины	Не предусмотрено	Строка	Миниатюра, название, фасовка, количество и стоимость позиции
Подытог, Скидка, Доставка, Итого	Не предусмотрено	Строка	Автоматически рассчитанные суммы (доставка бесплатна от 1500 Р)
Экраны «Заказы» и «Профиль»			
Номер заказа	Не предусмотрено	Строка	Уникальный номер вида ORD-<время>
Статус заказа	Не предусмотрено	Строка	Новый, В сборке, Передан курьеру, В пути, Доставлен, Отменен
Ход выполнения заказа	Не предусмотрено	Шаги	Шаги «Создан – У курьера – Доставлен»
Дата заказа	Не предусмотрено	Дата	Дата оформления заказа
Данные курьера	Не предусмотрено	Строка	Имя и телефон курьера, назначенного на заказ
Имя, email и роль пользователя	Не предусмотрено	Строка	Шапка экрана «Профиль»
Аватар	Не предусмотрено	Изображение	Фотография профиля
История уведомлений	Не предусмотрено	Список строк	Сообщения об изменении статуса заказа и назначении курьера
Доступные промокоды	Не предусмотрено	Список строк	Код, размер скидки и срок действия
Экран курьера «Доставка»			
Свободные заказы	Не предусмотрено	Список карточек	Заказы без курьера со статусом «Новый» или «В сборке»
Активные заказы	Не предусмотрено	Список карточек	Адрес, время, клиент, телефон, состав, сумма и сумма к получению

Поле	Ограничение	Тип	Описание
1	2	3	4
			наличными
История доставок	Не предусмотрено	Список карточек	Доставленные и отмененные заказы курьера
Веб-панель управления			
Показатели дашборда	Не предусмотрено	Числа	Покупатели, активные заказы, товары с низким остатком, общая выручка
Продажи по месяцам, ТОП-5 товаров	Не предусмотрено	Столбчатая диаграмма	Графики на странице «Дашборд»
Таблица товаров	Не предусмотрено	Форматированная таблица	Элемент, куда выводятся товары каталога с поиском, скрытием и восстановлением
Таблицы справочников	Не предусмотрено	Форматированная таблица	Категории, производители, поставщики и промокоды
Таблица заказов	Не предусмотрено	Форматированная таблица	Номер, клиент, сумма, статус, курьер и действия
Таблица пользователей	Не предусмотрено	Форматированная таблица	Имя, логин, email, телефон, роль и действия
Письмо для сброса пароля	Не предусмотрено	Электронное письмо	Письмо со ссылкой для создания нового пароля (отправляет Firebase Authentication)

ПРИЛОЖЕНИЕ Ж. ДИАГРАММА КЛАССОВ

АННОТАЦИЯ

В данном программном документе приведена диаграмма классов серверной части и мобильного приложения «Интернет-магазин продуктов «Нектар».

СОДЕРЖАНИЕ

1. ДИАГРАММА КЛАССОВ.....	3
1.1. Диаграмма классов серверной части.....	3
1.2. Диаграмма классов мобильного приложения.....	3

1. ДИАГРАММА КЛАССОВ

1.1. Диаграмма классов серверной части

На рисунке 1 представлена диаграмма классов серверной части. Серверная часть системы построена на Google Firebase (Cloud Firestore и Firebase Authentication) и не содержит собственного программного кода, поэтому на диаграмме показаны документы (сущности) базы данных, их атрибуты и связи между ними. Веб-панель управления написана на языке JavaScript в функциональном стиле и классов не содержит.

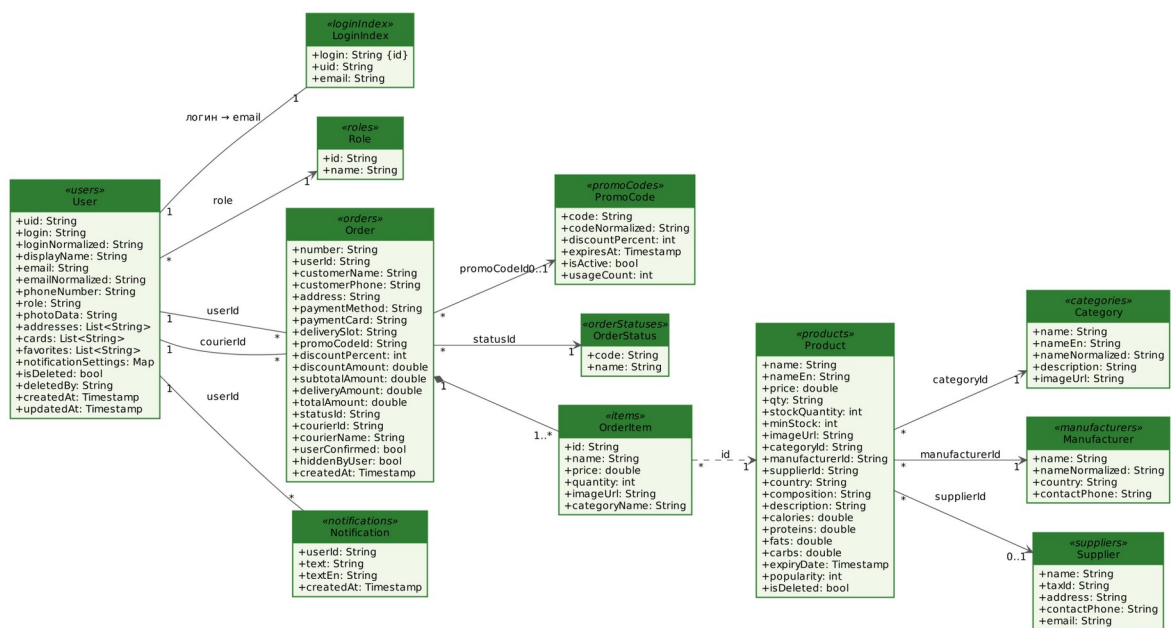


Рисунок 1 – Диаграмма классов серверной части

1.2. Диаграмма классов мобильного приложения

На рисунках 2–7 представлена диаграмма классов мобильного приложения, разработанного на фреймворке Flutter. Диаграмма описывает архитектуру клиентской части, включая основные компоненты пользовательского интерфейса, экраны, модели данных, сервисы и провайдеры состояния.

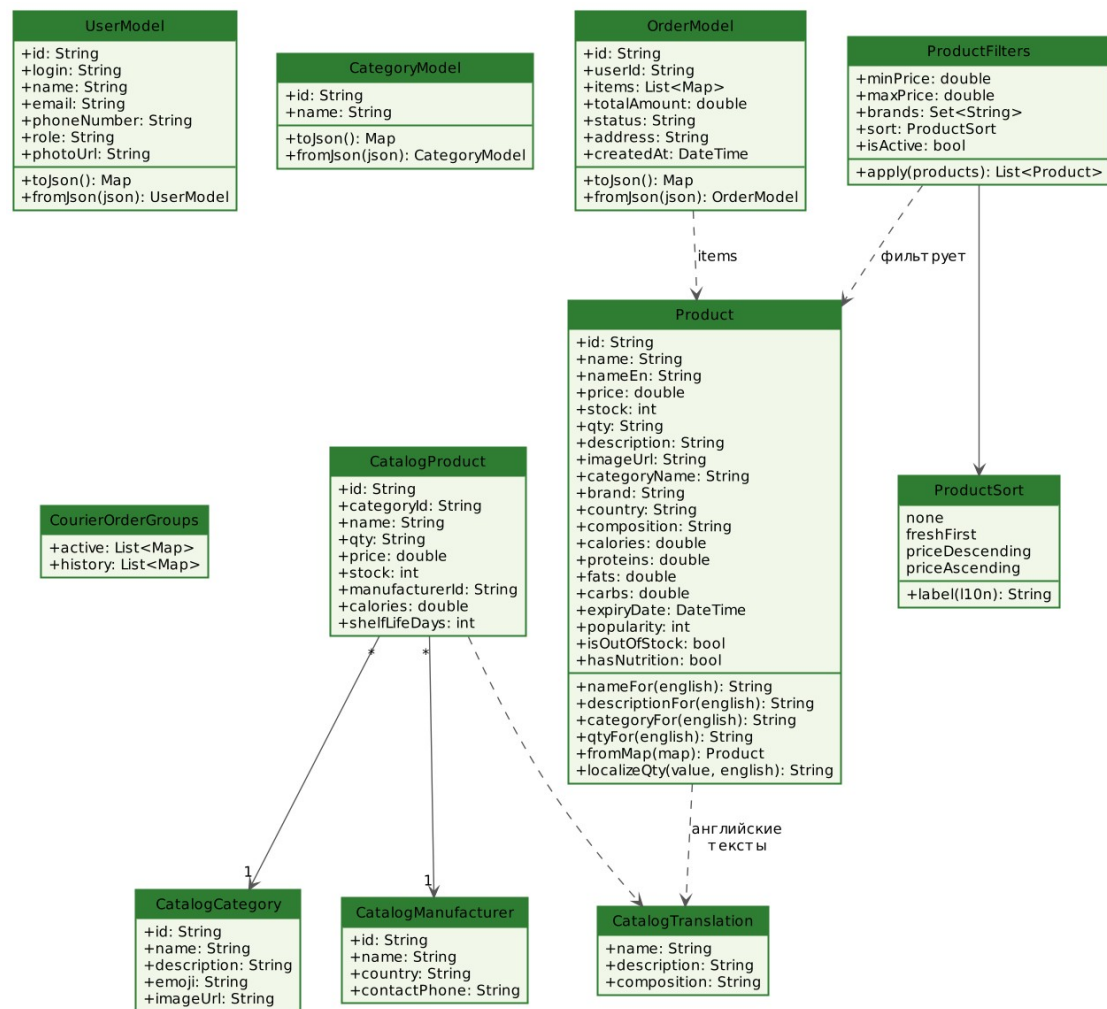


Рисунок 2 – Диаграмма классов мобильного приложения (1). Модели данных и каталог

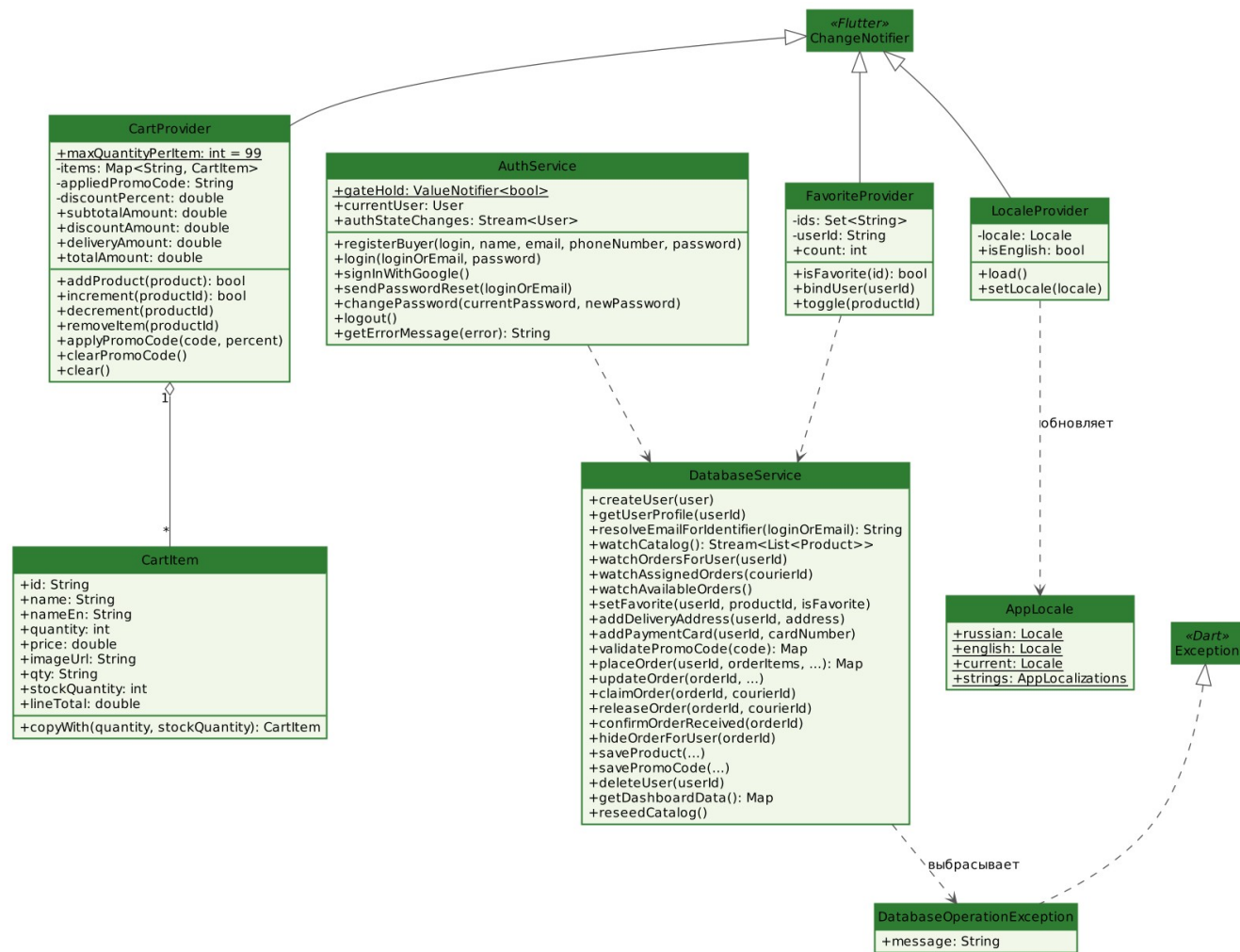


Рисунок 3 – Диаграмма классов мобильного приложения (2). Сервисы и провайдеры состояния

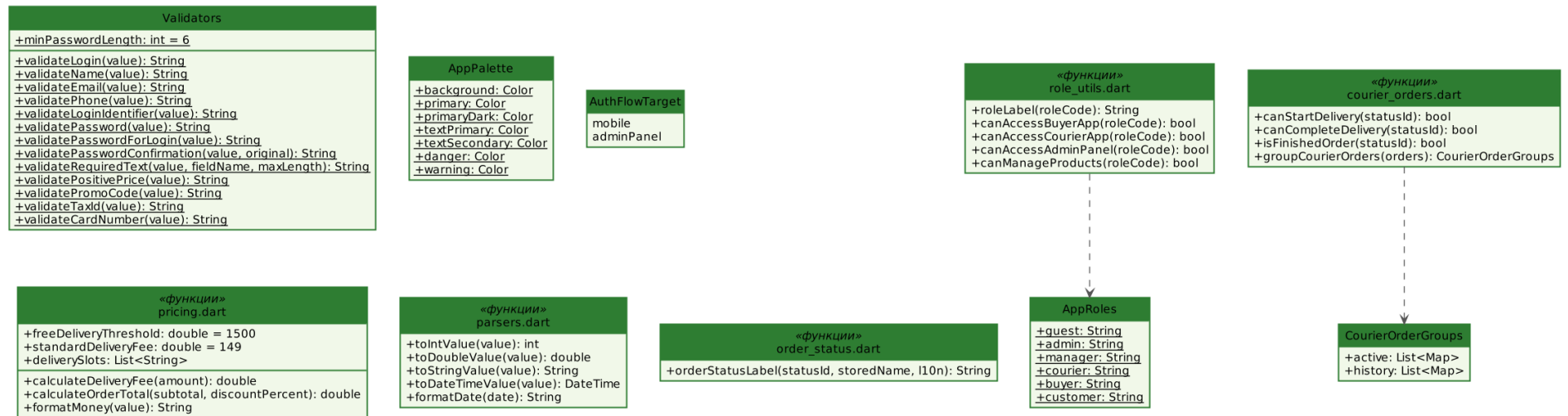


Рисунок 4 – Диаграмма классов мобильного приложения (3). Вспомогательные классы и функции

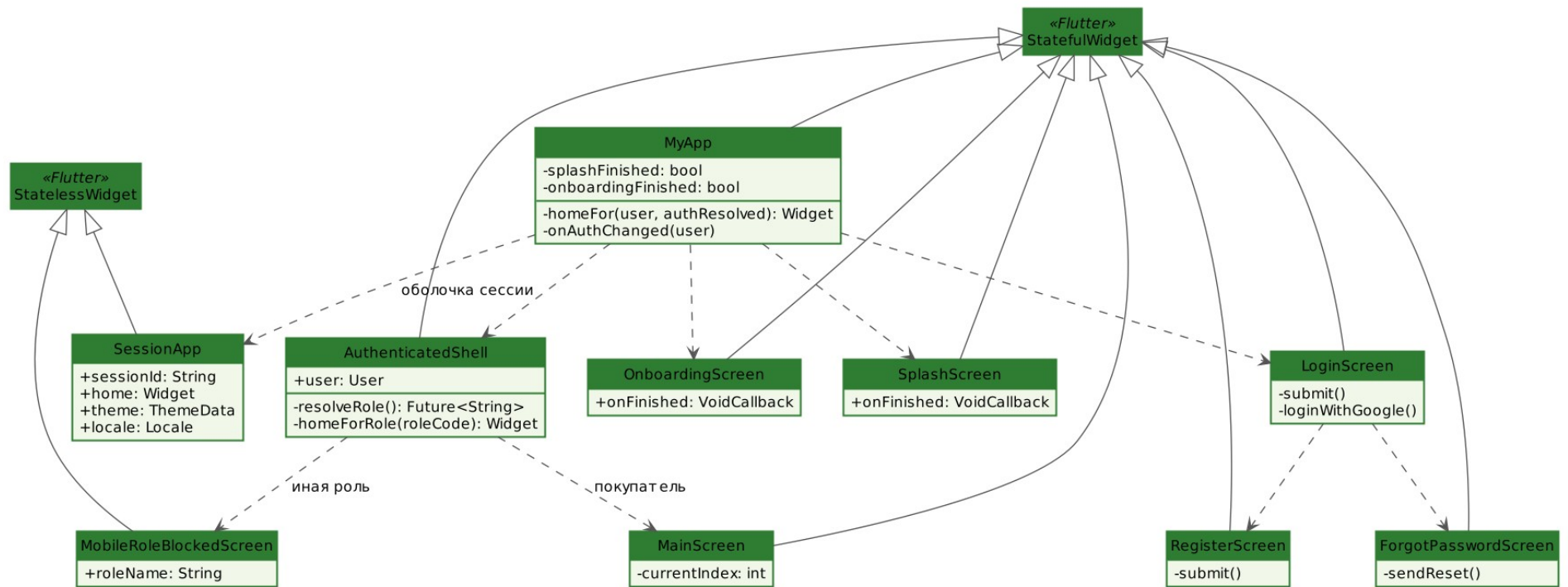


Рисунок 5 – Диаграмма классов мобильного приложения (4). Запуск приложения и авторизация



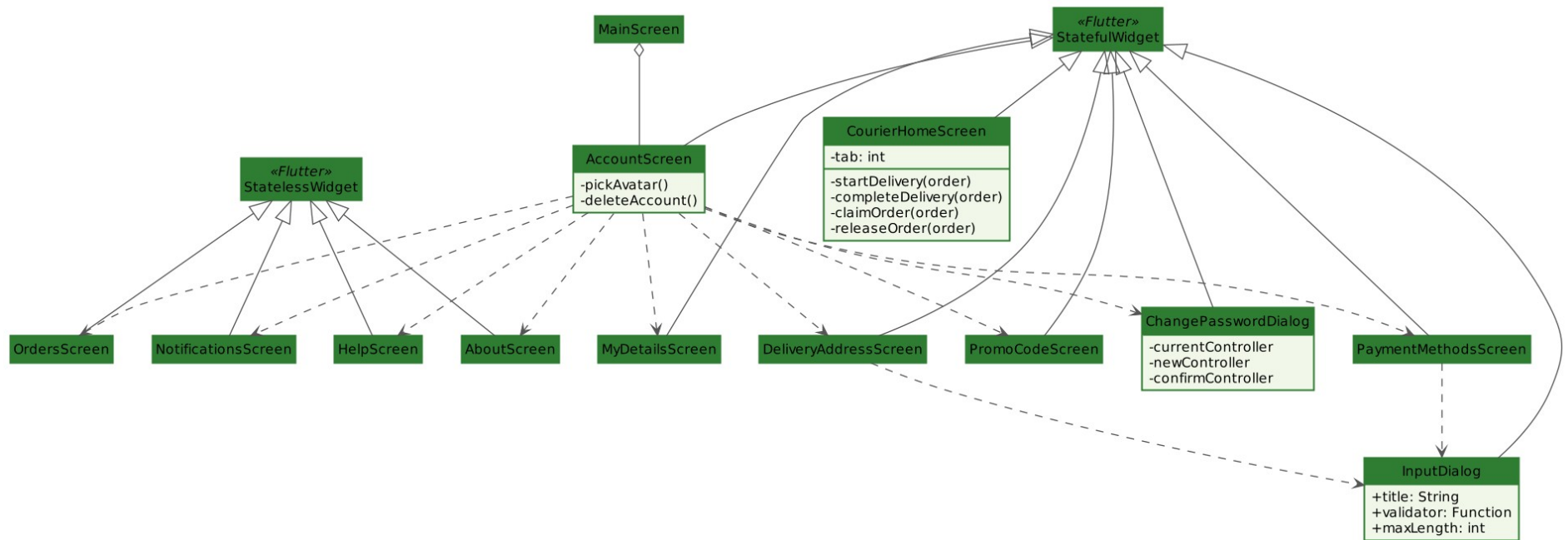


Рисунок 7 – Диаграмма классов мобильного приложения (6). Профиль и курьер

В таблице 1 представлено описание основных классов серверной части.

Таблица 1 – Описание основных классов серверной части

№ п/п	Наименование класса	Тип класса (наследственность)	Описание
1	2	3	4
1	User	Документ Firestore (коллекция users)	Профиль пользователя: логин, имя, email, телефон, роль, аватар (base64), адреса, маски карт, избранное, настройки уведомлений, признак отключения.
2	LoginIndex	Документ Firestore (коллекция loginIndex)	Публичный индекс «логин → uid и email», по которому выполняется вход по логину и обеспечивается уникальность логина.
3	Role	Документ Firestore (коллекция roles)	Справочник ролей пользователей (администратор, менеджер, курьер, покупатель).
4	Category	Документ Firestore (коллекция categories)	Категория товаров с названием на двух языках, описанием и изображением.
5	Manufacturer	Документ Firestore (коллекция manufacturers)	Производитель (бренд) товара: название, страна, контактный телефон.
6	Supplier	Документ Firestore (коллекция suppliers)	Поставщик товаров: название, ИНН, адрес, телефон, email.
7	Product	Документ Firestore (коллекция products)	Товар каталога: название, цена, фасовка, остаток, категория, производитель, пищевая ценность, срок годности, признак скрытия.
8	PromoCode	Документ Firestore (коллекция promoCodes)	Промокод на скидку 1–99 % со сроком действия, признаком активности и счетчиком использований.
9	OrderStatus	Документ Firestore (коллекция orderStatuses)	Справочник статусов заказа (новый, в сборке, передан курьеру, в пути, доставлен, отменен).
10	Order	Документ Firestore (коллекция orders)	Заказ: покупатель, адрес, способ оплаты, время доставки, промокод, суммы, статус, курьер, позиции.
11	OrderItem	Вложенный массив документа Order	Позиция заказа: товар, цена на момент покупки, количество.
12	Notification	Документ Firestore	Уведомление покупателю или

№ п/п	Наименование класса	Тип класса (наследственность)	Описание
1	2	3	4
		(коллекция notifications)	курьеру об изменении заказа (текст на русском и английском).
13	Security Rules	firestore.rules	Правила безопасности: проверка ролей, валидация данных и ограничение допустимых изменений документов для каждой коллекции.
14	Firebase Authentication	Сервис Google Firebase	Регистрация и вход по email и паролю или через Google, восстановление пароля, смена пароля.

В таблице 2 представлено описание основных классов мобильного приложения.

Таблица 2 – Описание основных классов мобильного приложения

№ п/п	Наименование класса	Тип класса (наследственность)	Описание
1	2	3	4
1	MyApp	StatefulWidget	Корневой класс приложения: следит за состоянием авторизации и выбирает стартовый экран (заставка, онбординг, вход, оболочка сессии).
2	SessionApp	StatelessWidget	Оболочка сессии: пересоздает MaterialApp при смене пользователя, задает тему, язык и делегаты локализации.
3	AppPalette	Object	Класс-константа с цветовой палитрой приложения (контраст по WCAG AA).
4	AppLocale	Object	Хранит текущий язык для кода без BuildContext (сервисы, валидаторы) и выдает строки локализации.
5	LocaleProvider	ChangeNotifier	Провайдер языка интерфейса (русский/английский) с сохранением выбора на устройстве.

№ п/п	Наименование класса	Тип класса (наследственность)	Описание
1	2	3	4
6	CartItem	Object	Позиция корзины: товар, цена, количество и доступный остаток.
7	CartProvider	ChangeNotifier	Корзина: добавление, изменение количества, удаление, промокод, расчет скидки, доставки и итога.
8	FavoriteProvider	ChangeNotifier	Провайдер избранного: подписка на массив favorites профиля и переключение товара.
9	AuthService	Object	Сервис авторизации: регистрация покупателя, вход по логину или email, вход через Google, сброс и смена пароля, выход.
10	DatabaseService	Object	Единая точка работы с Cloud Firestore: потоки данных, транзакции заказа, промокоды, справочники, уведомления.
11	DatabaseOperationException	Exception	Исключение с локализованным сообщением об ошибке операции с базой данных.
12	Product	Object	Модель товара с разбором документа Firestore и выбором текста на языке интерфейса.
13	UserModel	Object	Модель профиля пользователя.
14	OrderModel	Object	Модель заказа.
15	CategoryModel	Object	Модель категории товаров.
16	CatalogCategory	Object	Категория стартового каталога.
17	CatalogManufacturer	Object	Производитель стартового каталога.
18	CatalogProduct	Object	Товар стартового каталога (загружается кнопкой перезаливки базы).
19	CatalogTranslation	Object	Английские название, описание и состав товара или категории.
20	ProductFilters	Object	Параметры фильтрации (цена, бренды) и

№ п/п	Наименование класса	Тип класса (наследственность)	Описание
1	2	3	4
			сортировки списка товаров.
21	ProductSort	enum	Перечисление способов сортировки товаров.
22	CourierOrderGroups	Object	Разделение заказов курьера на активные и историю.
23	AppRoles	Object	Коды ролей пользователей.
24	Validators	Object	Проверка полей форм: логин, имя, email, телефон, пароль, промокод, карта, ИНН, цена.
25	AuthFlowTarget	enum	Назначение входа: мобильное приложение или панель администратора.
26	DefaultFirebaseOptions	Object	Параметры подключения Firebase для каждой платформы.
27	SplashScreen	StatefulWidget	Экран загрузки с названием магазина.
28	OnboardingScreen	StatefulWidget	Экран приветствия из трех слайдов с кнопками «Пропустить», «Далее», «Начать».
29	LoginScreen	StatefulWidget	Экран входа по логину или email, вход через Google.
30	RegisterScreen	StatefulWidget	Экран регистрации нового покупателя.
31	ForgotPasswordScreen	StatefulWidget	Экран восстановления пароля по email или логину.
32	AuthenticatedShell	StatefulWidget	Определяет роль вошедшего пользователя и открывает экран покупателя, курьера или сообщение о запрете.
33	MobileRoleBlockedScreen	StatelessWidget	Сообщение для ролей администратора и менеджера о необходимости использовать веб-панель.
34	MainScreen	StatefulWidget	Основной экран покупателя с нижней навигацией (Магазин,

№ п/п	Наименование класса	Тип класса (наследственность)	Описание
1	2	3	4
			Каталог, Корзина, Профиль).
35	ShopScreen	StatefulWidget	Экран «Магазин»: баннер, поиск, популярные товары и товары по категориям.
36	ExploreScreen	StatefulWidget	Экран «Каталог»: плитки категорий, поиск и фильтры.
37	CategoryScreen	StatefulWidget	Список товаров выбранной категории.
38	ProductDetailScreen	StatefulWidget	Карточка товара: описание, характеристики, пищевая ценность, выбор количества, избранное, «Поделиться».
39	FavoritesScreen	StatefulWidget	Список избранных товаров с кнопкой «В корзину всё».
40	CartScreen	StatelessWidget	Корзина: список позиций, промокод, подытог, скидка, доставка, итог.
41	CheckoutBottomSheet	StatefulWidget	Окно оформления заказа: адрес, время доставки, оплата, промокод.
42	AccountScreen	StatefulWidget	Профиль пользователя: аватар, меню разделов, выход и удаление аккаунта.
43	OrdersScreen	StatelessWidget	История заказов покупателя с шагами выполнения и подтверждением получения.
44	MyDetailsScreen	StatefulWidget	Редактирование имени и телефона.
45	DeliveryAddressScreen	StatefulWidget	Список и добавление адресов доставки.
46	PaymentMethodsScreen	StatefulWidget	Список и добавление банковских карт (хранится маска).
47	PromoCodeScreen	StatefulWidget	Ввод промокода и список доступных промокодов.
48	NotificationsScreen	StatelessWidget	Настройки уведомлений и история уведомлений.
49	HelpScreen	StatelessWidget	Часто задаваемые вопросы.

№ п/п	Наименование класса	Тип класса (наследственность)	Описание
1	2	3	4
50	AboutScreen	StatelessWidget	Сведения о приложении.
51	ChangePasswordDialog	StatefulWidget	Диалог смены пароля.
52	CourierHomeScreen	StatefulWidget	Экран курьера: вкладки «Мои заказы» и «Свободные», взятие, возврат, начало и подтверждение доставки.
53	ProductCard	StatelessWidget	Карточка товара в списках: фото, название, цена, избранное, добавление в корзину.
54	ProductGrid	StatelessWidget	Сетка карточек товаров.
55	ProductImage	StatelessWidget	Изображение товара с запасным значком при ошибке загрузки.
56	ProductFiltersSheet	StatefulWidget	Всплывающее окно фильтров и сортировки.
57	InputDialog	StatefulWidget	Универсальный диалог ввода одной строки с проверкой.
58	LanguageSwitcher	StatelessWidget	Переключатель языка RU/EN.
59	CenteredMessage	StatelessWidget	Сообщение по центру экрана (пустой список, ошибка).
60	PublicWebLandingScreen	StatelessWidget	Публичная страница для Flutter Web с ссылкой на вход сотрудников.
61	WebRoleGate	StatefulWidget	Проверка роли пользователя в Flutter Web.
62	AdminHomeScreen	StatefulWidget	Заглушка административной части на Flutter Web (рабочая панель – JavaScript-приложение в каталоге web).
63	WebRoleBlockedScreen	StatelessWidget	Сообщение о запрете доступа к веб-сайту для покупателей и курьеров.

Проверка на уникальность авторского текста курсового проекта

На тему: Разработка мобильного приложения и веб-панели управления для интернет-магазина продуктов «Нектар»

ВЕНИКОВ ИВАН НИКОЛАЕВИЧ
Студент 3 курса группы П-7-23

по специальности 09.02.07 «Информационные системы и программирование»
квалификация: Программист

Уникальность текста проверялась на сайте www.antiplagiat.ru. Процент оригинальности содержимого пояснительной записки составил 90%. Результат представлен на Рисунке 1.

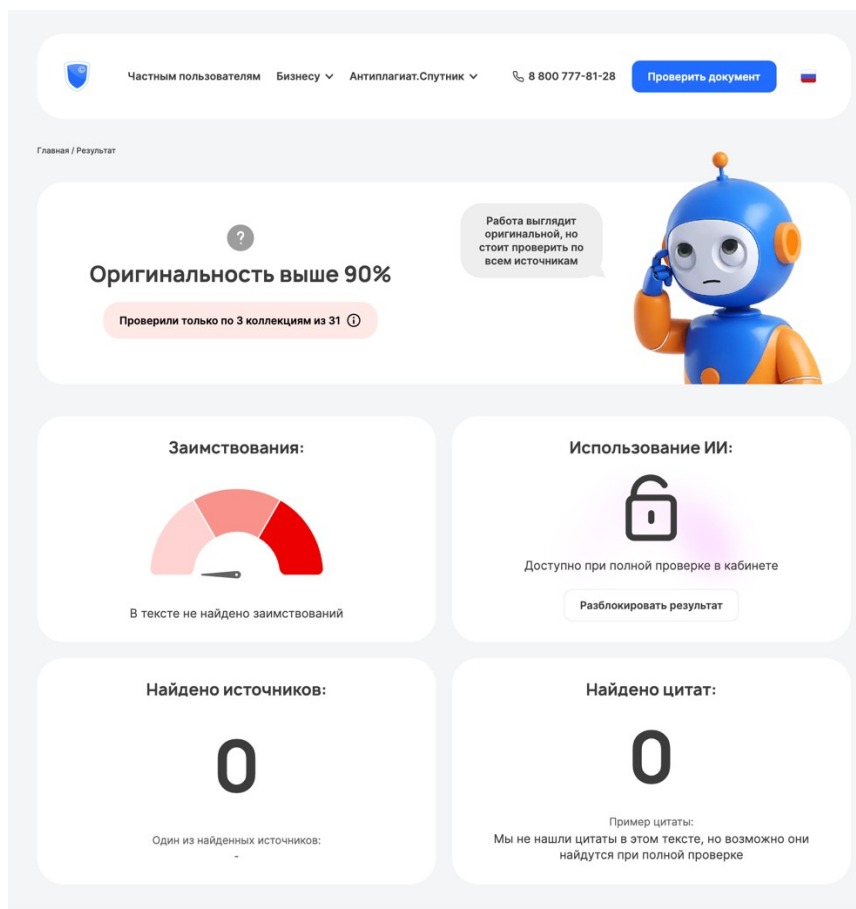


Рисунок 1 - Проверка на уникальность текста пояснительной записки